portage-ng

A declarative reasoning engine for large scale software configuration management

Pieter Van den Abeele

pvdabeel@mac.com

May 2026

Contents

Chapter 1

Introduction

1.1 The growing complexity of software

portage-ng is not a package manager. It is a **reasoning engine** for software configuration management at scale.

To understand why this distinction matters, consider operating systems as examples of complex software systems. An operating system is assembled from thousands of interdependent components — libraries, compilers, language runtimes, desktop environments, system services — each of which evolves independently. The challenge of keeping all those components working together has grown dramatically over the past three decades, and the tools we use to manage that challenge have had to evolve with it.

1.1.1 Binary software distribution

In the earliest model — still the dominant one for distributions like Debian, Red Hat, and Ubuntu — packages arrive as pre-built archives. The distribution maintainers compile each package, test it against a fixed set of other packages, and ship the result. The package manager's job is essentially logistics: download the right set of archives and unpack them into the filesystem. Order barely matters; as long as every required archive is present at the end, inter-package linkage is correct and the system works.

This model is simple and reliable. Because everyone receives the same binaries, configurations are easy to reproduce, and commercial software vendors can build and certify against a known, fixed set of packages. The trade-off is that the user has little control: configuration choices are made upstream, and customisation is limited to what the distribution maintainers decided to provide.

1.1.2 Source-based systems

Before Gentoo, installing software from source on Linux was a manual undertaking. Projects like **Linux From Scratch** (started 1999) documented the process step by step, but every command was the user's responsibility: download, patch, configure, compile, install — by hand, for every package.

The **FreeBSD Ports** system (Jordan Hubbard, 1994) was the first to automate source-based package management. **Gentoo** (originally Enoch, created by Daniel Robbins in 1999, inspired by FreeBSD Ports) brought this idea to Linux. Gentoo 1.0 (March 2002) was the first Linux

distribution to provide fully automated source-based package management as its primary mode of operation.

In its early days, Gentoo targeted a single platform (IA-32) and the dependency problem was tractable: the package manager walked the dependency graph, figured out which packages needed compiling, ordered them, and built them one by one. Compiling from source meant binaries were optimised for the exact hardware — no lowest-common-denominator builds — and the performance advantage was real.

Tools like **Portage** navigated these dependency graphs with an imperative, trial-and-error approach: try a combination, detect conflicts, adjust, and try again. For a single-platform distribution with a moderately sized tree, this approach is adequate.

1.1.3 From packages to knowledge

As Gentoo grew — thousands of packages, a dozen architectures, multiple operating systems via **Gentoo Prefix** — simple graph traversal was no longer sufficient. Each package now has **build-time options** (USE flags) that interact multiplicatively: different CPUs to target, different compilers to use, different versions of those compilers, different optional feature combinations. The space of valid configurations is not merely large — it is combinatorially vast, and every point in that space must be internally consistent.

This is the **metadistribution** concept. Gentoo does not distribute binaries; it distributes **knowledge** — recipes (ebuilds) that describe how to build every component of a system, and configuration parameters (USE flags, keywords, profiles) that let the user tailor the result. The word "package manager" does not do justice to what this entails. Putting pre-built archives together is logistics. Ensuring that thousands of packages, across multiple platforms, with user-specified feature selections and hardware constraints, form an internally consistent system — that is **configuration management**, and it requires more than a graph traversal algorithm.

Yesterday the system worked; today, after a routine update, it does not — and the answer is buried somewhere in the interaction of thousands of constraints across hundreds of packages. A trial-and-error search loop can tell you it *failed*, but not *why*.

1.1.4 From searching to proving

Solving this problem requires more than a better search loop. It requires **reasoning** — the ability to derive consequences from rules, to detect why a configuration is inconsistent, and to explain what must change to make it consistent again.

portage-ng approaches the problem this way. Instead of *searching* for a plan, it *proves* one. Every build plan portage-ng produces is a formal proof — a Prolog term that records, for every package, which rule justified its inclusion and under what constraints. When no fully valid plan exists, portage-ng does not give up: it makes explicit assumptions (flag changes, keyword acceptance, unmasking), proves a plan under those assumptions, and presents the assumptions as actionable suggestions. The proof answers not only "what should I install?" but also "why does this work?" — and when something breaks, "what changed?"

1.1.5 Declarative reasoning

This is inherently a task for **declarative reasoning**. We do not want to prescribe a fixed sequence of imperative steps; we want to state the rules of the domain and let a reasoning engine derive the consequences.

Prolog — an **artificial intelligence** language built on exactly this paradigm — is a natural fit. You specify *what* solution to produce, not *how* to produce it. The runtime — unification, backtracking, and proof search — figures out the "how." Consider a trivial example:

```
os(linux).
os(darwin).
```

```
?- os(Choice).
Choice = linux ;
Choice = darwin.
```

Prolog **automatically** enumerates every valid binding for `Choice`. This built-in **backtracking** — the ability to systematically explore all alternatives — is exactly what a configuration engine needs. Traditional Portage did not originally have backtracking at all; the retry mechanism it acquired later is not designed to enumerate alternatives but to iteratively refine a single solution by accumulating masks across restarts. In Prolog, backtracking over alternatives is not a bolt-on feature — it is a primitive of the language.

The reasoning engine portage-ng implements is not inherently tied to operating system management. We have chosen Gentoo because it captures many of the hardest sub-problems — figuring out the correct USE flag combinations to satisfy user, system, and hardware constraints simultaneously; resolving cyclic dependencies; managing co-installable slots — and any solution that works for Gentoo generalises to simpler domains. But the same proof-based architecture could reason about any domain where entities have capabilities, constraints, and dependencies — from cloud service composition to event-driven automation to hardware design space exploration. Gentoo is the proving ground; the ideas are general.

While source-based configuration management systems like Portage are used by thousands of developers and organisations, the formal concepts behind them — constraint propagation, domain narrowing, proof construction — are understood by only a handful of people. portage-ng aims to push forward the state of the art in this area and, by expressing the resolver as a set of logical rules rather than an opaque imperative algorithm, to make its inner workings more accessible and easier to reason about.

This chapter explains why Gentoo is the right domain, why Prolog is the right language, and how portage-ng's proof-based architecture addresses the problem at a level that imperative package managers cannot reach.

1.2 Why Gentoo?

1.2.1 The metadistribution concept

Most Linux distributions distribute **binaries** — fixed packages with fixed configurations, tested together in a release cycle. Gentoo distributes **knowledge**: recipes (ebuilds) that describe how to build every component of a complete system, and configuration parameters (USE flags, keywords, profiles) that let the user tailor the result to their hardware and requirements.

This is the **metadistribution** concept. We no longer distribute the output of a build process; we distribute the declarative specification of the build process itself. The word "package manager" does not do justice to what this entails. Putting pre-built packages together is logistics. Ensuring that a complex, multi-dimensional configuration space is internally consistent, constructing build plans to realise a chosen configuration, and executing those plans with the right ordering and parallelism — that is **configuration management**.

A single Portage tree contains roughly 32,000 ebuilds. Each ebuild declares:

- **Dependencies** — what it needs at build time, run time, and post-install
- **Use flags** — optional features the user can enable or disable
- **Slots** — multiple versions that can coexist
- **Keywords** — which architectures the package is tested on
- **Use constraints** — restricting valid Use flag combinations

The number of valid configurations is combinatorially enormous. This is not a bug — it is the point. Gentoo's power comes from this configurability. But it also means that reasoning about Gentoo packages is reasoning about a large, richly structured constraint space.

1.2.2 Architectures and keywords

Gentoo was originally built for the IA-32 (x86) architecture. As contributors ported it to other platforms — PowerPC, ARM, SPARC, MIPS, HPPA, and others, often available in 32-bit and 64-bit variants — the project developed the **keyword** system to track per-architecture stability. An ebuild can be marked `amd64` (stable on x86-64), `~arm` (testing on ARM), or carry no keyword for a given architecture (meaning it has not been validated there at all). Keywords turn architecture support into a first-class constraint in the dependency graph: a package that is stable on one architecture may be unstable or unavailable on another, and the resolver must respect those boundaries.

Platforms beyond x86 Linux — such as BSD, Solaris, and others — were handled as regular Gentoo targets with a different kernel and different user-space libraries, using the same Portage machinery and ebuild format. Google's **ChromeOS** is a prominent example of such a different platform delivered and managed entirely by Portage: ChromiumOS maintains a fork of Portage alongside Gentoo-derived overlay repositories (`portage-stable` for unmodified upstream ebuilds, `chromiumos-overlay` for Google-specific packages), and changes flow back to upstream Gentoo regularly.

The **Gentoo Prefix** project (an outgrowth of the Gentoo for Mac OS X effort) addressed a different challenge: installation *within* a pre-built operating system where root binaries cannot

be modified. On platforms like Mac OS X, Prefix installs Portage and all packages into a user-defined offset directory rather than the filesystem root, allowing a full Gentoo-managed software stack to coexist with the host system.

1.2.3 Real-world adoption

Gentoo's approach to source-based configuration management has been adopted well beyond the Gentoo community:

- **ChromiumOS / ChromeOS** (Google). ChromiumOS is the open-source project; ChromeOS is Google's proprietary product shipped on Chromebooks. Both are built using Gentoo's Portage, with overlay repositories (`portage-stable` for unmodified upstream ebuilds, `chromiumos-overlay` for Google-specific packages). In 2025, Google confirmed that ChromeOS and Android are merging into a unified platform (codenamed "Aluminium") for 2026, with Android's kernel as the foundation and ChromeOS's desktop interface layered on top.
- **Container Linux** (CoreOS, later Flatcar). CoreOS Container Linux — a lightweight, container-optimized operating system designed for cloud infrastructure — was built on Gentoo foundations, using Portage and ebuilds for its build system. After CoreOS was discontinued in 2020, **Flatcar Container Linux** continued the Gentoo-based lineage and is deployed at scale by organisations including Adobe (18,000+ nodes), Equinix, and numerous managed Kubernetes providers.

These adoptions are not cosmetic. ChromeOS and Flatcar use the same ebuild format, the same Portage dependency resolver, and the same overlay architecture as upstream Gentoo. The fact that this machinery scales from a single developer's workstation to tens of thousands of production nodes is evidence that Gentoo represents state-of-the-art practice in large-scale software configuration management.

1.2.4 Reasoning about software at scale

When you ask "can I install Firefox with Wayland support on this machine?", you are really asking: "does there exist a consistent assignment of package versions, USE flags, and slot choices across my entire dependency graph such that all constraints are satisfied?" That is a **satisfiability problem** over a structured domain.

portage-ng treats the Portage tree as what it truly is: a **declarative knowledge base**. Ebuilds are not build scripts to execute — they are propositions with preconditions. Dependencies are not edges in a graph to traverse — they are logical implications to prove. Configuration choices are not switches to flip — they are constraints to satisfy.

This shift in perspective — from "searching for a working set of packages" to "proving that a consistent configuration exists" — is what makes portage-ng fundamentally different from Portage, Paludis, and pkgcore — the three existing package managers that operate on the same ebuild base.

1.3 A Prolog primer

If you have never used Prolog, this section gives you enough to follow the rest of the book. If you already know Prolog, skip to Why Prolog?.

1.3.1 Facts and rules

Prolog programs are built from **facts** and **rules**. A fact states something that is true:

```
requires(browser, graphics).
requires(browser, networking).
requires(graphics, fonts).
```

This says: a browser requires graphics and networking; graphics requires fonts. Each line is a fact — something the system knows to be true.

A **rule** says something is true *if* certain conditions hold:

```
needs(X, Y) :- requires(X, Y).
needs(X, Y) :- requires(X, Z), needs(Z, Y).
```

Read `:-` as "if." The first clause says: X needs Y if X directly requires Y. The second says: X needs Y if X requires some intermediate Z, and Z in turn needs Y. Together, these two lines define transitive dependency — if the browser requires graphics and graphics requires fonts, then the browser needs fonts.

1.3.2 Queries and unification

You ask Prolog questions by posing **queries**. Prolog answers by finding values that make the query true:

```
?- needs(browser, What).
What = graphics ;
What = networking ;
What = fonts.
```

Prolog found everything the browser transitively needs. It did this through **unification** — matching the variable `What` against terms in the database — and **backtracking** — systematically trying every possibility.

Unification is more powerful than pattern matching. Two terms unify if there exists a substitution that makes them identical:

```
?- package(Name, stable) = package(editor, Status).
Name = editor, Status = stable.
```

Prolog figured out that `Name` must be `editor` and `Status` must be `stable` for both sides to match. This works in both directions — Prolog does not distinguish between "input" and "output" arguments.

1.3.3 Backtracking

When a Prolog query has multiple solutions, the runtime explores them through **backtracking**. Consider:

```
color(red).
color(green).
color(blue).

?- color(X).
X = red ;
X = green ;
X = blue.
```

Each `;` triggers backtracking: Prolog undoes its last choice and tries the next alternative. This search is built into the language — you do not write a search loop.

1.3.4 Compound terms

Prolog terms can be nested, forming structured data without defining classes or schemas. For example, a package entry might look like:

```
package(editor, version(2, 4, 1), [unicode, spellcheck]).
```

This single term captures a package name, a structured version, and a list of enabled features. Because Prolog comparison (`compare/3`) works structurally on compound terms, two versions can be compared directly — no custom comparator needed. portage-ng uses compound terms extensively to represent versions, dependencies, and proof entries.

1.3.5 Lists and association lists

Prolog lists are linked lists built from `[Head|Tail]`:

```
?- [a, b, c] = [H|T].
H = a, T = [b, c].
```

Looking up a value in a plain list requires walking it from head to tail — O(n) in the worst case. When a proof tree contains thousands of entries, this becomes a bottleneck.

SWI-Prolog provides **association lists** (AVL trees) via `library(assoc)` as an efficient alternative. An AVL tree is a self-balancing binary search tree: keys are kept in order, and the tree is rebalanced after every insertion so that no branch is more than one level deeper than its sibling.

To find a key, we do not scan every element. Instead, we compare the target with the current node and follow the appropriate branch — left if the target is smaller, right if it is larger. The following diagram shows how looking up “ssl” in a tree of seven entries requires only three comparisons:

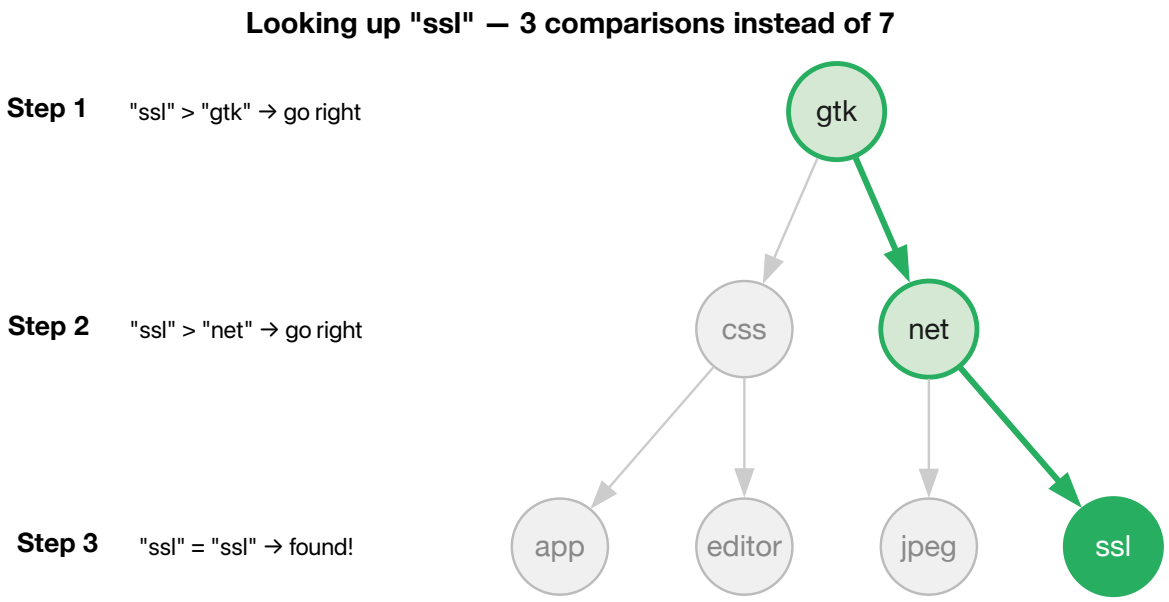


Figure 1 — AVL tree lookup

Because the tree stays balanced, every lookup follows a single path from root to leaf. The length of that path is at most $\log_2(n)$ — with 10,000 entries, an AVL lookup visits at most 14 nodes instead of scanning all 10,000.

portage-ng uses association lists extensively — for the proof, the model, the trigger set, and the constraint store:

```
?- empty_assoc(E),
   put_assoc(editor, E, installed, A1),
   put_assoc(browser, A1, pending, A2),
   get_assoc(editor, A2, Status).
Status = installed.
```

All operations (`get_assoc`, `put_assoc`) are O(log n), which makes them practical for the data structures at the heart of the prover.

1.3.6 Definite Clause Grammars

When portage-ng reads a package's dependency specification, it needs to parse structured text like `>=dev-libs/openssl-1.1:0=` into Prolog terms the prover can reason about. In most languages, writing a parser means writing imperative code — loops, state machines, error handling. In Prolog, you can write the grammar itself as a program.

A **DCG** (Definite Clause Grammar) lets you describe what valid input looks like, declaratively. Prolog takes care of matching the input against the grammar rules. For example, a simple grammar for a greeting:

```
greeting --> [hello], name.
name --> [world].
name --> [prolog].
```

This reads naturally: "a greeting is the word `hello` followed by a name; a name is either `world` or `prolog`." To check whether a sequence matches:

```
?- phrase(greeting, [hello, world]).
true.

?- phrase(greeting, [hello, cat]).
false.
```

The grammar *is* the parser — there is no separate parsing step. portage-ng uses DCGs to parse the **EAPI** (Ebuild API) dependency specification language — EAPI is the versioned interface that defines the syntax and semantics of Gentoo's ebuild format, including version ranges, Use conditionals, slot operators, and choice groups. The result is a parser that reads like a specification of the language it accepts, making it easier to verify, extend, and maintain.

1.3.7 Meta-programming

One of Prolog's most distinctive features is that programs and data are made of the same material: **terms**. A rule like `requires(browser, graphics)` is not just an instruction — it is a data structure that a program can inspect, build, and pass around. This blurs the line between "the program" and "the data it operates on" in a way that is natural in Prolog but awkward in most other languages.

Why does this matter for portage-ng? Because the prover does not just compute a plan — it builds a **proof** that explains why the plan is correct. As the prover works, it constructs a term that records every decision:

```
proof(browser, [
  rule(requires(browser, graphics), [
    proof(graphics, [
      rule(requires(graphics, fonts), [
        proof(fonts, [fact])
      ])
    ])
  ])
]).
```

This proof term says: "the browser is in the plan because it requires graphics, which is in the plan because it requires fonts, which is a base fact." The proof is not a side effect or a log — it is a first-class Prolog term that can be queried, compared, and transformed like any other data.

The same principle applies to assumptions. When the prover cannot satisfy a dependency without, say, accepting a testing keyword, it records that assumption as a term inside the proof:

```
assumed(accept_keywords('~amd64', graphics))
```

At the end, portage-ng can walk the proof, collect all assumptions, and present them to the user as actionable suggestions — precisely because assumptions are data, not scattered side effects.

This capability is called **reification**: turning the process of reasoning into data that can itself be reasoned about. It is what makes the "every plan is a proof" architecture natural in Prolog.

1.4 Why Prolog?

Now that you have seen the basics, here is why Prolog is not just a possible implementation language but the *right* one for building a reasoning engine for software configuration management.

1.4.1 The primitives match the problem

A reasoning engine for configuration management needs a small set of core operations. In Prolog, these operations are built-in primitives rather than library code:

Reasoning need	Prolog primitive
Configuration rules	Horn clauses
Structured data	Compound terms
Search with backtracking	Built-in backward chaining with backtracking
Constraint propagation	Unification and association lists
Specification grammar	Definite Clause Grammars
Proof construction	Terms as data (reification)
Meta-programming	Runtime assertion and introspection

Imperative package managers must re-implement all of these. Portage's `depgraph.py` implements its own retry loop with mask accumulation. Paludis's `decider.cc` implements its own constraint accumulator with exception-driven restart. Both re-invent mechanisms that Prolog provides as proven primitives. By relying on these primitives, the codebase becomes easier to read and understand — developers can focus fully on declaring domain knowledge rather than maintaining search and backtracking machinery.

1.4.2 Meta-level reasoning

Beyond the primitives, Prolog enables **meta-level reasoning** — the ability to reason about the reasoning process itself. For a configuration management engine, this is the decisive advantage.

Reifying assumptions. When the prover cannot satisfy a dependency, it does not throw an error. It records an assumption as a first-class term in the proof tree. The assumption carries structured metadata: why it was made, what alternatives were tried, and what the user can do about it. This is natural in Prolog because proofs are data.

Consider what happens when a package requires a keyword that is not accepted. An imperative resolver would either fail or silently accept the keyword. portage-ng records:

```
rule(assumed(portage://dev-libs/foo-1.0:merge), [])
```

This assumption appears in the proof, flows through the planner, and is presented to the user as: "I assumed dev-libs/foo-1.0 could be merged. To make this work, I will add it for you to package.accept_keywords."

Inspecting proof trees. The explainer module queries the proof AVL to answer "why is this package in the plan?" without re-running the resolver. In an imperative resolver, this would require instrumenting the search loop with logging and replaying it.

Learning from failures. When a proof attempt fails, the prover extracts a learned constraint — a narrowed version domain — from the failure and carries it into the next attempt. This is analogous to CDCL (Conflict-Driven Clause Learning) in SAT solvers, but expressed as Prolog term manipulation rather than boolean clause generation.

1.4.3 Declarative vs. imperative

The difference is not just stylistic. A declarative specification of configuration rules is:

- **Auditable** — the rules can be read as logical statements
- **Testable** — individual rules can be queried in isolation
- **Extensible** — adding a new dependency type means adding clauses, not modifying control flow

In portage-ng, the entire EAPI specification — hundreds of pages of the PMS (Package Manager Specification, the formal document that defines how Gentoo ebuilds, dependencies, USE flags, slots, and all related metadata must be interpreted) — is captured in a set of DCG grammar rules and Prolog clauses. When EAPI 9 added new features, the implementation required adding new grammar rules and clauses. The reasoning engine — prover, planner, and scheduler — did not change at all, because it is domain-agnostic.

1.4.4 Beyond pure Prolog

Prolog's strengths — backtracking, unification, reification — provide the foundation, but building a large-scale reasoning engine also requires structure that pure Prolog does not offer out of the box. portage-ng extends standard Prolog with two mechanisms: **feature terms** for carrying structured metadata through proofs, and **contexts** for organising code into encapsulated, independently stateful components.

1.4.5 Feature logic and feature terms

In classical logic programming, a term is either an atom, a variable, or a compound. That is enough for simple reasoning, but when the prover needs to carry *configuration alongside identity* — "this package, with these USE flags, in this slot, at this proof depth" — a flat compound quickly becomes unwieldy. Feature logic, originally developed by Hassan Aït-Kaci and others for computational linguistics, offers a cleaner model: a **feature term** is a structured record of named attributes (features) whose values may themselves be feature terms. Two feature terms can be **unified** (merged) non-destructively, combining their information while checking for consistency.

Andreas Zeller applied feature logic directly to software configuration management, showing that features and feature unification provide a natural formalism for describing and merging software configurations — capturing version selections, build options, and platform constraints as feature terms rather than ad-hoc data structures. portage-ng builds on this insight: USE flags, slot constraints, version domains, and proof context are all represented as feature terms that the prover unifies as it expands the dependency graph.

portage-ng uses the `?{}` notation to attach a feature term to any literal. The syntax reads as "this literal, *qualified by* these features":

```
portage://app-editors/neovim-0.12.0:run?{[]}
```

Here the feature term `{[]}` is empty — no additional constraints beyond the literal itself. As the prover expands dependencies and resolves USE flags, the feature term accumulates information:

```
portage://app-editors/neovim-0.12.0:run?{[nvimpager, naf(test)]}
```

The feature term `{[nvimpager, naf(test)]}` records that the `nvimpager` USE flag is enabled and `test` is disabled (`naf` stands for "negation as failure" — the standard logic-programming notation for default negation).

Feature unification is the operation that merges two feature terms. When the prover encounters the same package from two different dependency paths, each carrying its own feature term, unification combines them:

- **Plain items** (like USE flags) are collected by union.
- **Constrained sets** `{L}` use intersection semantics — both paths must agree.
- **Keyed values** (`feature:value` pairs) are unified recursively.
- If a term and its negation (`naf(X)` vs. `X`) both appear, unification **fails** — this signals a genuine conflict in the dependency graph.

This mechanism is domain-agnostic: the unifier (`Source/Logic/unify.pl`) does not mention USE flags, slots, or ebuilds. It operates on abstract feature terms. The Gentoo domain layer maps USE flags, slot constraints, and version domains onto feature terms; the unifier simply merges them. A domain hook (`feature_unification:val_hook/3`) allows domain-specific value types (e.g. version domain intersection) to participate in unification without modifying the core.

The result is that every literal in a proof carries a complete, machine-readable description of its resolved configuration. The planner and printer can read this directly — there is no need to re-derive "which USE flags were active" after the fact.

1.4.6 Contextual object-oriented programming

Prolog operates under the **closed-world assumption**: what cannot be derived from the program is considered false. In practice, this means that reasoning about a predicate requires visibility of *all* its clauses — the program is treated as a complete description of the world. For a small program this is manageable, but in a system with tens of thousands of packages, dozens

of repositories, and overlapping configurations, the "world" becomes very large. Not all of it is relevant to every question.

Context-based reasoning addresses this directly: it partitions the closed world into **scoped contexts**, each carrying only the facts and rules that are relevant to a particular component. When reasoning about a repository, only that repository's entries, configuration, and constraints are in scope. The closed-world assumption still holds — but within a well-defined boundary, making reasoning both tractable and modular.

Standard Prolog module systems offer some namespacing, but they were designed for library organisation, not for modelling independent entities that each carry their own state, rules, and encapsulation boundaries. We need a way for each component to have its own context — its own facts, its own rules — with explicit declarations of what is public interface and what is private implementation. Different repositories and different configurations must be able to coexist without interfering with each other's reasoning.

In the object-oriented world, this problem was solved long ago with encapsulation and access control. The challenge is bringing those organisational benefits to Prolog without sacrificing its declarative nature. The rules inside a context must remain ordinary logical clauses — directly translatable into traditional logic — while the context system provides the structure around them.

portage-ng needed object-oriented style programming — classes, instances, encapsulation, access control — but at **runtime**, because repositories and configurations are discovered and instantiated at startup, not known at compile time. No existing Prolog library provided this. **Logtalk**, the best-known approach to object-oriented logic programming, works by compile-time translation: source files are transformed into plain Prolog before execution. That model does not fit a system that creates and composes objects dynamically.

So portage-ng implements its own runtime object system called **context** (implemented in `context.pl`). The syntax is deliberately Logtalk-like — `::-` for method clauses, `::` for message sends, `dpublic`, `dprotected`, `dprivate` for access control — but the underlying mechanism is entirely different: contexts are created, cloned, and composed at runtime through Prolog's own assert/retract machinery. There is no compilation step and no source-to-source transformation.

To illustrate, here is a simplified example of a `person` context:

```
:- module(person, []).
:- class.

:- dpublic([person/1, '~person'/0]).
:- dpublic([get_name/1, set_name/1]).
:- dpublic([get_age/1, set_age/1]).
:- dpublic([add_title/1, remove_title/1]).
:- dprivate(age/1).
:- dprivate(title/1).
:- dprotected(name/1).

person(Name) ::-
  :set_name(Name).
```

```
'~person' ::-
  :this(Context),
  write('Person destructor - '), write(Context), nl.

get_name(Name) ::-
  ::name(Name).

set_name(Name) ::-
  <=name(Name).

set_age(Age) ::-
  <=age(Age).

add_title(Title) ::-
  <+title(Title).

remove_title(Title) ::-
  <-title(Title).

age(Age) ::-
  number(Age), Age > 0.
```

Several things are worth noting. The `:- class` directive declares that this module defines a context class. The `dpublic`, `dprotected`, and `dprivate` directives specify access control — just like in classical OO, public predicates can be called by anyone, protected predicates only by the class and its descendants, and private predicates only within the class itself. The constructor `person/1` initialises the instance by setting its name; the destructor `~person` is called when the instance is destroyed. The `::-` operator (instead of Prolog's standard `:-`) marks instance methods: their clauses are guarded at runtime so that each instance operates on its own state. The `::` prefix reads instance data, `<=` assigns it (replacing any previous value), `<+` adds a fact to the instance, and `<-` removes one — as shown by `add_title` and `remove_title`, which allow a person to accumulate multiple titles. The `:` prefix calls other methods on the same instance.

Using this class is straightforward:

```
?- pieter:newinstance(person).
true.

?- pieter:person('Pieter').
true.

?- pieter:set_age(40).
true.

?- pieter:add_title('Dr.').
true.

?- pieter:add_title('Prof.').
true.

?- pieter:get_age(Age).
Age = 40.
```

```
?- pieter:get_title(Title).
Title = 'Dr.' ;
Title = 'Prof.'.
```

The `newinstance` call creates an instance named `pieter` from the `person` class, and the constructor is invoked with `pieter:person('Pieter')`. From that point on, `pieter` is a live context with its own state. Setting the age and adding titles modifies that instance's private data. Querying titles backtracks over all titles that were added — this is Prolog's backtracking working naturally within the context system.

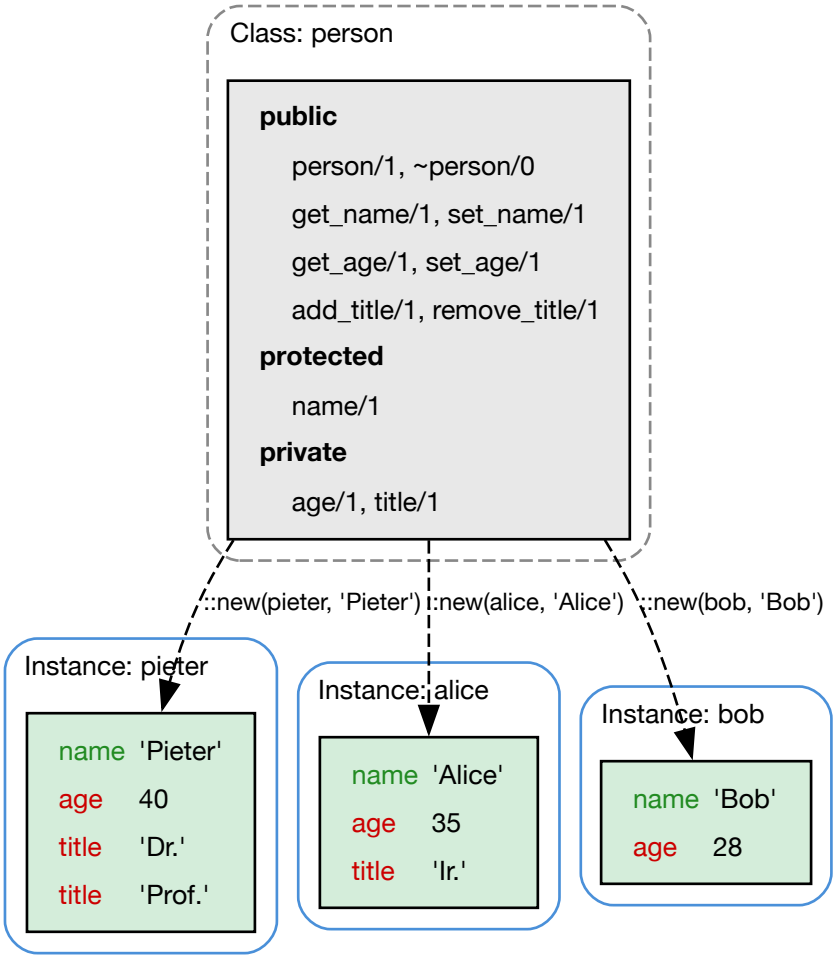


Figure 2 — Person class and instances

Each instance carries its own state: `pieter` has two titles and age 40, `alice` has one title and age 35, `bob` has no titles and age 28. The class defines the shape; each instance fills it independently.

In portage-ng, repositories are context objects: each has a name, a set of entries, and operations like `sync`, `graph`, and `query` that are dispatched through the context. The prover does not need to know which repository it is reasoning about — it receives a context and operates through it.

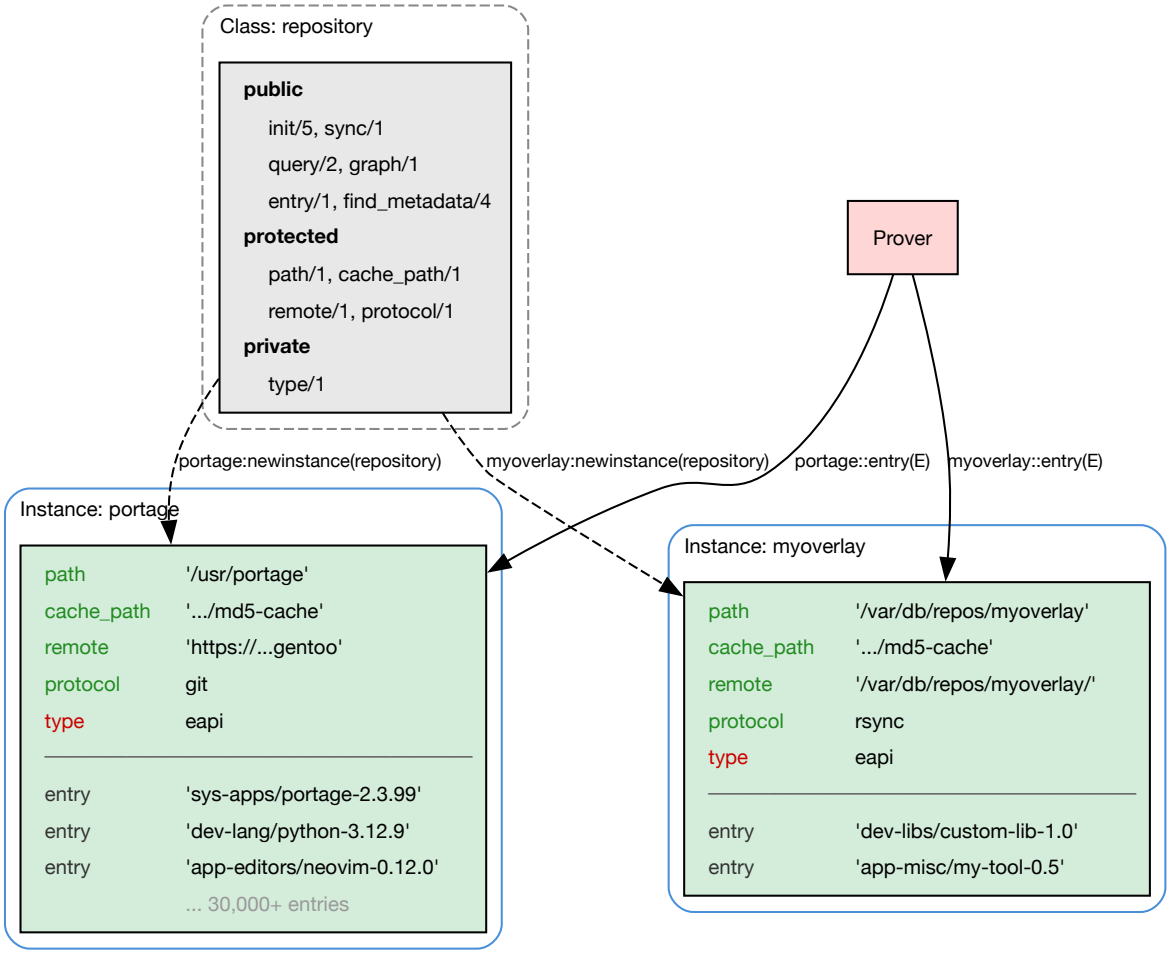


Figure 3 — Repository class and instances

The `repository` class defines the interface — `init`, `sync`, `entry`, `query` — while each instance holds its own path, protocol, and entries. The prover queries `portage:entry(E)` and `myoverlay:entry(E)` through the same module-qualified call; the context system dispatches each call to the right instance's private data.

This approach brings encapsulation, polymorphism, and modularity to Prolog while preserving its declarative core. The rules inside a context are ordinary Prolog clauses; the context system simply controls visibility and dispatches calls to the right instance. The full design is covered in chapters 19 and 20.

1.4.7 Feature logic meets contextual logic

There is a natural synergy between feature logic and contextual logic programming that is worth noting. A context — with its named attributes, access control levels, and instance-specific state — can be viewed as a special kind of feature term: a structured record of features (name, age, title, path, protocol, …) augmented with meta-level annotations (public, protected, private) that control which features are visible to which parts of the program. Conversely, feature unification can be seen as a restricted form of context composition: merging two feature

terms is analogous to combining two contexts, with consistency checking playing the role of access control.

This perspective suggests that feature logic and contextual logic programming are two views of the same underlying formalism — one emphasising data (feature terms as structured records) and the other emphasising behaviour (contexts as encapsulated reasoning units). A unified framework that captures both would be a natural extension: feature terms with access-controlled attributes, or contexts whose state is described and merged via feature unification. This remains an open area for further exploration.

1.4.8 Pengines: contexts over the network

SWI-Prolog provides **Pengines** (Prolog Engines) — lightweight, sandboxed Prolog instances that can be created, queried, and destroyed over HTTP. Each Pengine is an isolated reasoning environment with its own clause store, running inside a host server process. Clients on remote machines can create a Pengine, send it a query, and receive results — all over a standard HTTPS connection.

The connection to contextual logic programming is direct: a Pengine is, in essence, a **remote context**. Just as a local context encapsulates its own state and exposes a public interface, a Pengine encapsulates a Prolog environment and exposes it over the network. The access control that contexts provide locally (public, protected, private) is mirrored by the Pengine sandbox, which restricts which predicates the remote client may call.

portage-ng uses Pengines in its server mode. The server hosts the knowledge base and exposes it through a Pengine application: clients and workers create Pengines on the server, submit proving goals, and receive plan results — all without needing a local copy of the knowledge base. From the client's perspective, the interaction looks like a local Prolog query; the network transport is transparent. This is what makes client–server mode practical for embedded and resource-constrained devices: the full reasoning context lives on the server, and the client merely drives it.

1.5 The "every plan is a proof" philosophy

These ideas come together in portage-ng's central insight: a build plan should not be the output of a search algorithm that happens to terminate. It should be a **proof object** — a term that records, for every package, which rule justified its inclusion and under what constraints.

This gives three properties that traditional resolvers lack:

1. **Completeness.** If a valid plan exists, the prover finds it. If no valid plan exists, the prover completes with explicit assumptions that document exactly where the specification is unsatisfiable.
2. **Explainability.** Every package in the plan can be traced back through the proof tree to the user's original target. "Why is this package here?" is answered by inspecting the proof, not by re-running the resolver.

3. **Reproducibility.** The proof is a first-class Prolog term. Given the same Portage tree, VDB, and configuration, the same proof is produced every time.

1.6 How portage-ng relates to other resolvers

portage-ng is not a rewrite of Portage in Prolog. It is a fundamentally different approach to the same problem:

	Portage	**Paludis**	**pkgcore**	**portage-ng**
Language	Python	C++	Python	SWI-Prolog
Model	Greedy graph + retry	Constraint accumulator + restart	Same as Portage	Inductive proof search
Conflicts	Retries with mask accumulation	Restarts with fresh state	Same as Portage	Iterative refinement with learned domains
Completeness	Sometimes fails	May exhaust restarts	Sometimes fails	Always produces a plan
Guarantees	None	None	None	Every plan is a proof

For a detailed comparison of the reasoning models, see Chapter 21: Resolver Comparison.

1.7 A brief history

The author's involvement with Gentoo began in 2002 as the founder of the first architecture port — PowerPC. That work contributed to the keyword system described above and to expanding the range of platforms where Gentoo could run. In 2003, a formal top-level management structure was implemented for the Gentoo project (GLEP 4). Under this structure, the author served as a senior manager for Gentoo with both strategic and operational responsibility for three areas: Gentoo on alternative operating systems and LiveCD technology, developer tools, and package manager research (Portage). That experience — porting across architectures, managing the resulting configuration complexity, and researching the limits of Portage's imperative resolver — motivated the portage-ng project.

portage-ng began in 2005 as an experiment in applying logic programming to software configuration management. The initial question was simple: could Prolog's built-in search and backtracking replace the hand-written solver in Portage?

The answer turned out to be deeper than expected. Prolog did not just replace the solver — it changed what was possible. The ability to reify proofs meant build plans became inspectable objects. The ability to record assumptions meant the resolver never had to give up. The ability to parse grammars with DCGs meant the EAPI specification could be expressed directly as code.

Over two decades of development, portage-ng has evolved from a proof-of-concept into a full-featured configuration management front-end with PMS 9 / EAPI 9 compliance, distributed

proving, LLM-assisted plan explanation, and measured correctness against Portage across the entire Gentoo tree.

1.8 Further reading

- Chapter 2: Installation and Quick Start — getting portage-ng running on your machine
- Chapter 4: Architecture Overview — how the six pipeline stages fit together
- Chapter 8: The Prover — the inductive proof engine in detail
- Chapter 21: Resolver Comparison — deep dive into how portage-ng compares with Portage, Paludis, and pkgcore

Chapter 2

Installation and Quick Start

2.1 Prerequisites

2.1.1 Required

The following tools must be present on every system that runs portage-ng. SWI-Prolog is the runtime; the others are used during repository syncing, metadata extraction, and distfile verification.

Dependency	Minimum version	Purpose
SWI-Prolog	10.0.0	Runtime interpreter. Must be built with SSL, PCRE, editline, HTTP, crypto, and pengines support.
bash	5	Metadata extraction via `ebuild-depend.sh` and helper scripts.
git	any	Repository syncing (`--sync` with git protocol), version display.
curl	any	Mirror/distfile downloads, HTTP-based repository sync.
openssl CLI	any	Distfile hash verification (`openssl dgst`), TLS certificate generation for client–server encryption.
Gentoo Portage tree	—	A full Portage tree (ebuilds + md5-cache). portage-ng reads the md5-cache for dependency resolution and requires the ebuilds for building.

On most Gentoo systems these are already installed. On non-Gentoo hosts (e.g. macOS), SWI-Prolog and bash are the only items you may need to install manually.

2.1.2 Required for specific features

Some portage-ng features require additional tools. These are only needed for specific commands.

Dependency	Feature	Notes
Graphviz (>= 11)	`--graph`	The `dot` command generates interactive SVG dependency graphs.
dns-sd	Distributed mode	mDNS/Bonjour service discovery. Built-in on macOS; use `avahi-browse` on Linux.
ebuild	`--merge` / `--build`	Actual package building delegates to Portage's ebuild infrastructure. Not needed for `--pretend`.
rsync	`--sync` (rsync)	Only when using rsync-based repository sync.
tar	`--sync` (HTTP)	Only when using tarball-based repository sync.

2.1.3 Optional

The following are convenient but not required for core operation.

Dependency	Purpose
Python 3	Timeout watchdog in the dev wrapper.
make / **cmake**	Used by build helper scripts for packages that need them.
aha / **perl**	Pretty-print HTML output generation.
pv	Progress bars during batch graph generation.
Ollama	Local LLM inference and vector embeddings for `--search` / `--explain`.

2.1.4 Prolog build requirements

When compiling SWI-Prolog from source, ensure the following optional components are enabled (they are usually built by default):

- **OpenSSL** — required for `library(crypto)`, `library(ssl)`, `library(http/http_ssl_plugin)`
- **PCRE** — required for `library(pcre)` (used in EAPI parsing)
- **GNU Readline / Editline** — required for `library(editline)` (interactive shell)
- **libgmp** — required for arbitrary-precision arithmetic
- **zlib** — required for qcompiled file support (`Knowledge/kb.qlf`)

2.2 Building

From the project root:

```
make check    # verify SWI-Prolog is installed
make build    # create the portage-ng binary
make install  # install to /usr/local/bin (requires sudo)
```

The `build` target uses `swipl --stand_alone=true` to produce a self-contained binary.

2.3 First run

2.3.1 Pretend (dry-run)

Generate a build plan without executing it:

```
portage-ng --pretend app-editors/neovim
```

portage-ng proves a dependency graph, plans it into parallel steps, and presents the result:

```
>>> Emerging : portage://app-editors/neovim-0.12.0:run?{[]}

These are the packages that would be merged, in order:

Calculating dependencies... done!

 └─ step 1 ─┤ download portage://dev-python/tree-sitter-0.25.2
            │           └─ file ─┤ 170.27 Kb  tree-sitter-0.25.2.gh.tar.gz
            │ download portage://dev-lua/mpack-1.0.13
            │           └─ file ─┤ 16.17 Kb   mpack-1.0.13.tar.gz

 └─ step 2 ─┤ install  portage://dev-lang/lua-5.1.5-r200
            │           └─ conf ─┤ USE = "readline deprecated"
            │                    │ SLOT = "5.1"
            │ install  portage://dev-libs/msgpack-6.0.0-r1
            │           └─ conf ─┤ USE = "-doc -examples -test"
            │                    │ SLOT = "0/2-c"
            ...

 └─ step 7 ─┤ install  portage://app-editors/neovim-0.12.0
            │           └─ conf ─┤ USE = "nvimpager -test"
            │                    │ LUA_SINGLE_TARGET = "luajit -lua5-1"

 └─ step 8 ─┤ run      portage://app-editors/neovim-0.12.0

Total: 59 actions (20 downloads, 19 installs, 1 update, 19 runs),
       grouped into 8 steps.
       18.82 Mb to be downloaded.
```

Actions within the same step can execute in parallel. The plan distinguishes download, install, update, and run phases. Each package shows its resolved configuration (Use flags, slot, target selection).

If portage-ng had to make assumptions during proving, they are reported at the end with suggested fixes and draft bug reports.

2.3.2 Interactive shell

Drop into a Prolog shell with the full knowledge base loaded:

```
portage-ng --shell
```

The shell provides direct access to the knowledge base. The built-in `query:search/2` predicate offers a readable way to explore it.

Search for packages by name:

```
?- query:search([name(neovim), description(D)], Repository://Entry).
D = "Vim-fork focused on extensibility and agility",
Repository = portage,
Entry = 'app-editors/neovim-9999'.
```

Press `;` to see the next result, or `.` to stop. Prolog backtracks through all matching ebuilds automatically.

Look up slot and keywords:

```
?- query:search([name(neovim), slot(S), keywords(K)], Repository://Entry).
S = '0',
K = unstable(amd64),
Repository = portage,
Entry = 'app-editors/neovim-0.12.0'.
```

Search across repositories:

```
?- query:search([name(firefox), description(D)], Repository://Entry).
D = "Firefox Web Browser",
Repository = portage,
Entry = 'www-client/firefox-149.0'.
```

Count all ebuilds:

```
?- aggregate_all(count, portage:entry(_), Total).
Total = 31535.
```

Read a single metadata field:

```
?- cache:entry_metadata(portage, 'app-editors/neovim-0.12.0', description, D).
D = "Vim-fork focused on extensibility and agility".
```

The full cache schema and query language are documented in Chapter 6: Knowledge Base.

2.3.3 Sync the Portage tree

Sync the repository and regenerate the knowledge base cache:

```
portage-ng --sync
```

The sync performs three phases for each registered repository:

1. **Repository sync** — pulls the latest Portage tree (via git, rsync, or HTTP tarball depending on configuration).
2. **Metadata sync** — reads the md5-cache files and, if configured, regenerates cache entries for ebuilds that have changed.
3. **Knowledge base sync** — parses all cache entries into Prolog facts (the `cache:entry`, `cache:entry_metadata`, `cache:manifest`, etc. predicates) and saves the compiled knowledge base to disk.

```
>>> Syncing 1 registered repository

--- Syncing repository "portage" ---

 Syncing repository ... ok
 Syncing metadata   ... Ebuild: sys-apps/portage-2.3.99-r1
                        Ebuild: dev-lang/python-3.13.3
                        Ebuild: sys-libs/glibc-2.41
                        ...
                        Updated metadata.
 Syncing kb         ... Ebuild: acct-group/abrt-0
                        Ebuild: acct-group/adm-0
                        Ebuild: acct-group/audio-0
                        ...
                        Manifest: app-accessibility/at-spi2-core
                        Manifest: app-accessibility/brltty
                        ...
                        Updated prolog knowledgebase.

--- Syncing profile ---

 Saving knowledge base ... ok
```

During the knowledge base sync, every ebuild's metadata — dependencies, Use flags, keywords, slots, descriptions, manifests — is parsed and asserted as Prolog facts. The entire Gentoo repository (over 30,000 ebuilds) is held in memory as a native Prolog database, enabling lightning-fast lookups without any disk I/O during reasoning.

SWI-Prolog's just-in-time (JIT) indexing further accelerates these lookups. When a predicate like `cache:entry_metadata(portage, 'app-editors/neovim-0.12.0', description, D)` is first called, the runtime automatically builds hash indices on the arguments that are bound. Subsequent calls with the same argument pattern jump straight to matching clauses instead of scanning all 30,000+ entries linearly. This indexing is created on demand and updated transparently as facts are asserted or retracted — no manual index declarations are needed.

Once syncing completes, the knowledge base is saved to disk using SWI-Prolog's qcompile mechanism (`Knowledge/kb.qlf`). qcompile serializes Prolog clauses into a compact binary

format that can be loaded back in a fraction of the time it takes to parse the original source. On subsequent runs, portage-ng loads the `.qlf` file directly, making startup near-instantaneous — even for a repository with tens of thousands of ebuilds.

2.4 Running tests

```
make test            # PLUnit tests
make test-overlay    # Overlay regression tests (80 scenarios)
```

See Chapter 23: Testing and Regression for details.

2.5 Further reading

- Chapter 3: Configuration — setting up Portage tree paths, `/etc/portage`, and profiles
- Chapter 14: Command-Line Interface — full CLI reference
- `portage-ng(1)` manpage — exhaustive option reference

Chapter 3

Configuration

Dependency resolution only makes sense in *your* environment: which profile you use, which USE flags you set, which packages are already on disk, and which extra trees you layered on top of Gentoo. portage-ng is designed so you do not pay a "migration tax" to express any of that. It reads the same files and databases as traditional Portage.

Configuration, in this chapter's sense, is the act of **telling portage-ng where your machine keeps that truth** (paths, repositories, profile strategy) and **which Gentoo-side files to honour** —so the prover plans against the world you actually run, not a generic default.

This chapter starts with the central configuration file (`config.pl`), then shows how to register repositories and sync them into the knowledge base, and finally covers the `/etc/portage/` files that control policy (USE flags, masks, keywords).

3.1 The configuration file

The central configuration file is `Source/config.pl`. It is a plain Prolog source file — every setting is a Prolog fact or rule that you can read, query, or override. The file is organised into logical sections; the most important ones for getting started are summarised below.

3.1.1 General

Setting	Default	Purpose
`config:name/1`	`'portage-ng-dev'`	Program name shown in banners and logs.
`config:hostname/1`	(auto-detected)	Current hostname, used to select per-machine configuration.
`config:installation_dir/1`	(from Prolog flag)	Root of the portage-ng source tree. The knowledge base, certificates, and config files are resolved relative to this path.
`config:number_of_cpus/1`	(auto-detected)	Parallelism level for parsing, proving, and building.
`config:verbosity/1`	`debug`	Verbosity level for runtime messages.

3.1.2 Repository and metadata

Setting	Default	Purpose
`config:trust_metadata/1`	`true`	When `true`, trust the repository-shipped md5-cache. When `false`, regenerate cache entries locally for every ebuild — expensive, but useful for overlay development.
`config:write_metadata/1`	`true`	Write on-disk cache entries for locally changed or new ebuilds during sync.

3.1.3 Gentoo profile

Setting	Default	Purpose
`config:gentoo_profile/1`	`'default/linux/amd64/23.0/...'`	The Gentoo profile path relative to the Portage tree's `profiles/` directory. This must match the profile symlink on your Gentoo system.

3.1.4 Profile loading strategy

Setting	Default	Purpose
`config:profile_loading/2`	`standalone → live`	Controls whether profile data is parsed from the Portage tree on every startup (`live`) or loaded from a pre-serialized cache (`cached`). Set per mode: standalone, daemon, worker, client, server.

See Profile loading strategy for details on generating and using the profile cache.

3.1.5 Paths

Setting	Purpose
`config:portage_confdir/1`	Path to the `/etc/portage` directory (or a development copy). Determines where `make.conf`, `package.use`, `package.mask`, etc. are read from. Comment out to use built-in fallback defaults.
`config:pkg_directory/2`	Per-hostname path to the VDB directory (`/var/db/pkg` on a standard Gentoo system).
`config:world_file/1`	Path to the world set file (auto-resolved from hostname).
`config:graph_directory/2`	Per-hostname output directory for generated dependency graphs and `.merge` files.
`config:build_root/1`	Root directory for build work (equivalent to Portage's `PORTAGE_TMPDIR`).
`config:build_log_dir/1`	Directory for per-package build logs.

3.1.6 Machine selection

Setting	Purpose
`config:systemconfig/1`	Resolves the machine-specific configuration file. Looks for `Source/Config/<hostname>.local.pl`; falls back to `Source/Config/default.pl` if not found.

The machine config file is where repositories are created and registered — covered in the next section.

3.1.7 Proving

Setting	Default	Purpose
`config:time_limit/1`	`300` (seconds)	Maximum time for a single proof/plan computation before aborting.
`config:proving_target/1`	`run`	Proof depth: `install` for compile-time dependencies only, `run` to include runtime dependencies.
`config:reprove_max_retries/1`	`120`	Maximum iterative learn-and-restart retries when the prover encounters conflicts.
`config:avoid_reinstall/1`	`false`	When `true`, verify already-installed packages instead of re-merging them.

3.1.8 Output

Setting	Default	Purpose
`config:default_printing_styl`	`'fancy'`	Plan output style: `short`, `column`, or `fancy` (tree-structured with Unicode box drawing).
`config:color_output/0`	asserted	ANSI colour in terminal output. Retract to disable.
`config:color_palette/1`	`full`	Use flag colouring: `easy` (classic Portage red/blue) or `full` (reason-based, showing where each flag came from).

3.1.9 Network

Setting	Default	Purpose
`config:server_host/1`	`'mac-pro.local'`	Server hostname for client-server mode.
`config:server_port/1`	`4000`	HTTPS port for the Pengine server.
`config:bonjour_service/1`	`'_portage-ng._tcp.'`	mDNS service name for automatic server/worker discovery.

3.1.10 LLM integration (optional)

LLM integration is entirely optional. If you do not need `--explain`, `--chat`, or semantic search, the LLM modules can be removed from the load graph without affecting core functionality (proving, planning, building).

Setting	Default	Purpose
`config:llm_default/1`	`claude`	Default LLM service for `--explain` and `--chat`.
`config:llm_model/2`	(per service)	Model version for each LLM provider (ChatGPT, Claude, Gemini, Ollama, etc.).
`config:llm_use_tools/1`	`true`	Whether the LLM may execute Prolog code locally during a conversation.

Most settings have sensible defaults. For a typical Gentoo system, the main items to configure are `config:gentoo_profile/1`, `config:portage_confdir/1`, and the repository definitions in the machine config file.

3.2 Configuring repositories

Not every literal in a proof refers to the same backing store. portage-ng models **several repository kinds** so the resolver can combine "what exists upstream", "what is already installed", and "what I added locally" without conflating them:

Name	Role
`portage`	The main Gentoo tree, backed by md5-cache — the canonical source of buildable versions.
`pkg`	Installed packages (the VDB under `/var/db/pkg/`). Ground truth for what is already on the machine.
`overlay`	Additional ebuild trees (user overlays, testing repos, local layers), each with their own cache and sync rules.

Each repository is a named **OO instance** created with a directive like:

```
:- portage:newinstance(repository).
```

The name before `:newinstance` is the name you choose for the repository — `portage` here, but it could be anything: `:- myoverlay:newinstance(repository).` would create a repository called `myoverlay`. Each instance is specialised with a location, cache path, remote URL, protocol, and type. This uses the same **context-based OO** machinery introduced in Chapter 1 (`Source/Logic/context.pl`): a repository is an object that responds to `sync`, `find_metadata`, `find_vdb_entry`, and so on, not a loose bag of paths.

Multiple repositories coexist in one proof. If you registered both `portage` and `myoverlay`, a dependency chain can span both: a `portage://` literal may pull in a `myoverlay://` literal when only the overlay carries a needed version. Installed packages participate too — a `pkg://` literal satisfies a runtime dependency without planning a fresh merge. Keeping repositories separate avoids conflating "what is available upstream", "what I added locally", and "what is already installed", while still allowing the prover to relate them during resolution.

Machine files in `Source/Config/` decide which of these instances exist on your host; see Machine configuration.

3.3 Machine configuration

Each machine has a configuration file under `Source/Config/` that creates and registers repository instances. portage-ng looks for `Source/Config/<hostname>.local.pl` first; if not found, it falls back to `Source/Config/default.pl`.

A machine config file creates one or more repositories using `newinstance`, initialises each with paths and a sync protocol, and registers it with the knowledge base.

The five arguments to `init` are:

1. **Local path** — where the repository lives on disk.
2. **Cache path** — the md5-cache directory inside the repository.
3. **Remote URL** — the upstream to sync from.
4. **Protocol** — how to sync: `git`, `rsync`, or `http` (tarball download).
5. **Type** — the repository format: `eapi` for standard Gentoo ebuild trees.

The examples below show the supported options.

Portage tree via git (the most common setup):

```
:- portage:newinstance(repository).
:- portage:init('/usr/portage',
                '/usr/portage/metadata/md5-cache',
                'https://github.com/gentoo-mirror/gentoo',
                'git', 'eapi').
:- kb:register(portage).
```

Portage tree via rsync:

```
:- portage:newinstance(repository).
:- portage:init('/usr/portage',
                '/usr/portage/metadata/md5-cache',
                'rsync://rsync.gentoo.org/gentoo-portage',
                'rsync', 'eapi').
:- kb:register(portage).
```

Portage tree via HTTP snapshot:

```
:- portage:newinstance(repository).
:- portage:init('/usr/portage',
```

```
                '/usr/portage/metadata/md5-cache',
                'http://distfiles.gentoo.org/releases/snapshots/current/portage-
latest.tar.bz2',
                'http', 'eapi').
:- kb:register(portage).
```

User overlay (a second ebuild tree layered on top):

```
:- myoverlay:newinstance(repository).
:- myoverlay:init('/var/db/repos/myoverlay',
                  '/var/db/repos/myoverlay/metadata/md5-cache',
                  '/var/db/repos/myoverlay/',
                  'rsync', 'eapi').
:- kb:register(myoverlay).
```

Local distfiles directory:

```
:- distfiles:newinstance(repository).
:- distfiles:init('/usr/portage/distfiles',
                  '', '', 'local', 'distfiles').
:- kb:register(distfiles).
```

Multiple repositories can be registered in the same file. During proving, the resolver queries all registered repositories and distinguishes them by their `portage://`, `pkg://`, or overlay prefix.

3.4 Syncing the tree

portage-ng does not crawl ebuild directories on every pretend merge. At runtime it works from a **compiled Prolog knowledge base** built from the same **md5-cache** files Portage uses: precomputed metadata blobs (dependencies, slots, USE defaults, and so on) produced by sourcing each ebuild through bash and extracting its declared variables — a process traditionally driven by Gentoo's `egencache`, which writes the results under the repository's cache directory. Treat the Portage tree, for resolver purposes, as **a directory of cache files** plus ebuilds; the cache is what makes bulk queries feasible.

```
portage-ng --sync
```

`--sync` is the umbrella operation that brings that picture up to date: it syncs registered repositories (via git, rsync, or snapshot download), regenerates on-disk metadata where configured, **reloads** md5-cache into dynamic Prolog facts (according to the structure defined in `cache.pl`), and **persists** the result to disk so subsequent runs start near-instantaneously.

```
portage-ng --regen
```

`--regen` (alias `--metadata`) addresses a narrower problem: **refresh the on-disk md5-cache** without performing a network sync. portage-ng can generate the md5-cache entirely on its own — it does not need traditional Portage or `egencache` for this step. Each ebuild is sourced through bash and its metadata extracted in incremental, parallel passes (see

`repository:sync(metadata)` and `config:trust_metadata/1` in the source). Traditional Portage is only needed for the actual *building* of packages, since portage-ng's current focus is on the reasoning and planning side. Note that `--regen` is not a substitute for loading facts into Prolog: after regenerating the cache, run **`--sync`** again so `Knowledge/kb.qlf` matches the updated on-disk cache.

You can also sync a single repository by name:

```
portage-ng --sync myoverlay
```

This syncs only the `myoverlay` repository (and saves the knowledge base afterwards). Useful when you have changed an overlay but the main Gentoo tree is still up to date.

3.4.1 Repositories

In portage-ng's architecture, all repositories are registered with a central **knowledge base** (`knowledgebase.pl`). The command-line interface talks to the knowledge base, which delegates sync operations to each registered repository. After syncing the repositories, the knowledge base also triggers a **profile sync** — this reads the Gentoo profile directory (the chain of `make.defaults`, `package.mask`, `use.mask`, etc. that define your system's baseline policy) and the `/etc/portage/` user configuration files, loading them into `preference` facts that the prover consults during resolution.

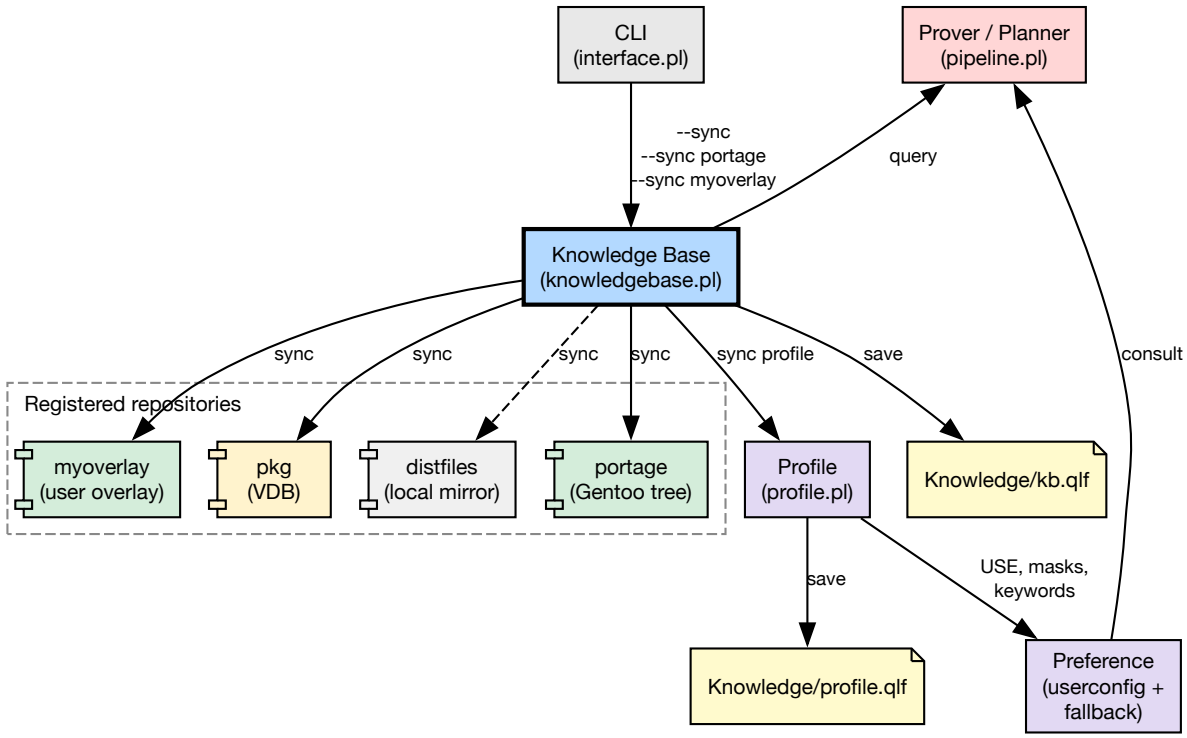


Figure 4 — Sync architecture

The result is two serialised cache files:

- **`Knowledge/kb.qlf`** — all repository and cache facts (ebuilds, metadata, manifests).

- **`Knowledge/profile.qlf`** — all profile-derived data (USE terms, masks, per-package USE, license groups).

See Profile loading strategy for details on live vs. cached profile loading.

3.4.2 Installed packages

To reason about what is already on the machine, portage-ng needs to know which packages have been installed. Portage records this in the **VDB** (Var DataBase), a directory tree at `/var/db/pkg/` with one subdirectory per installed `category/package-version`. Each subdirectory contains metadata files that capture the state at install time: dependency declarations (`DEPEND`, `RDEPEND`, `PDEPEND`), the active `USE` flags, `SLOT`, `KEYWORDS`, compiler flags, a file manifest (`CONTENTS`), and bookkeeping fields like `BUILD_TIME` and `SIZE`.

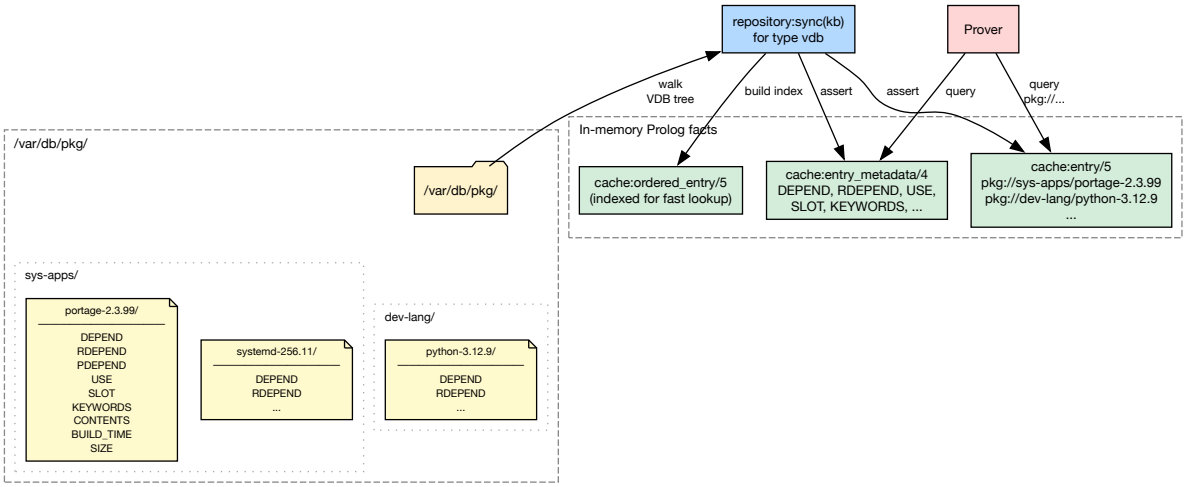


Figure 5 — VDB architecture

When `--sync` runs, the knowledge base syncs the `pkg` repository by walking the VDB tree and loading each installed package into the same in-memory fact structure used for available ebuilds. From that point on, the prover queries installed and available packages through the same interface — the only difference is the prefix: `pkg://` for installed packages, `portage://` for available ones.

This uniform representation means that during resolution, an already-installed package can satisfy a dependency directly without planning a fresh merge. In the plan output, these appear as `[nomerge]` — the prover verified the dependency is met by what is already on disk.

3.5 Gentoo configuration

Gentoo users already curate policy in `/etc/portage/`: USE overrides, masks, licences, and keywords. portage-ng **reuses that investment** — it reads Gentoo's standard `/etc/portage/` configuration files, making it a drop-in replacement for dependency resolution and plan computation from a *policy* perspective.

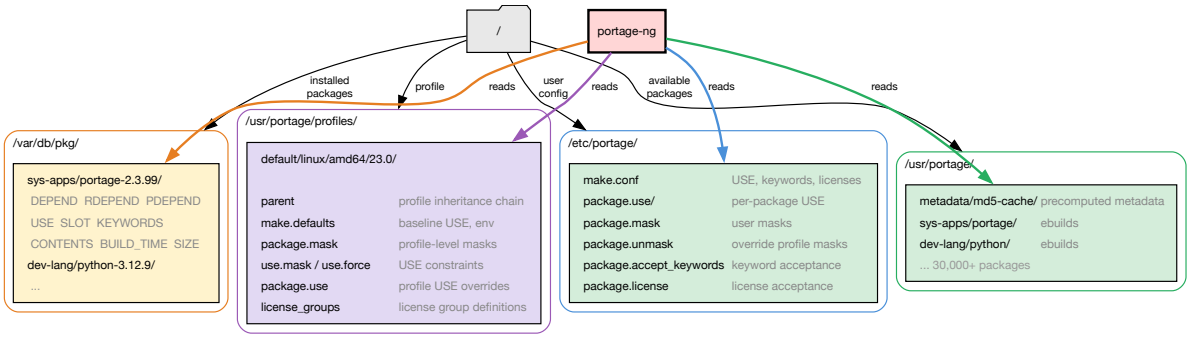


Figure 6 — Gentoo on-disk files read by portage-ng

The diagram shows the four on-disk sources portage-ng consults: user configuration under `/etc/portage/`, the profile chain under the Portage tree's `profiles/` directory, the installed-package database (VDB) under `/var/db/pkg/`, and the Portage tree itself with its ebuilds and md5-cache.

3.5.1 Supported files

portage-ng recognises the following standard Gentoo configuration files. Set `config:portage_confdir/1` in `Source/config.pl` to point at your `/etc/portage` directory (or use the bundled templates under `Source/Config/Gentoo/` during development).

File	Purpose
`make.conf`	Global environment variables (USE flags, keywords, licenses, etc.).
`package.use`	Per-package USE flag overrides.
`package.mask`	User package masks.
`package.unmask`	Overrides profile-level masks.
`package.accept_keywords`	Per-package keyword acceptance.
`package.license`	Per-package license acceptance.

All files are read from the directory set by `config:portage_confdir/1` (typically `/etc/portage/`).

These files use standard Gentoo syntax, so existing `/etc/portage/` directories work without modification.

3.5.2 File format

All files follow standard Gentoo syntax:

- Lines starting with `#` are comments
- Empty lines are ignored
- Inline `#` comments are stripped

make.conf

Bash-style `KEY="value"` assignments. Parsed by the same engine that reads profile `make.defaults` files (`profile:make_defaults_kv/2`).

```
USE="X alsa dbus -systemd"
ACCEPT_KEYWORDS="~amd64"
VIDEO_CARDS="intel"
```

package.use / package.accept_keywords / package.license

One entry per line: a package atom followed by space-separated values.

```
# package.use
app-editors/vim        -X
>=sys-libs/gdbm-1.26   berkdb

# package.accept_keywords
=sys-apps/portage-3.0  ~amd64
dev-util/pkgdev        **

# package.license
app-text/calibre       BSD
```

package.mask / package.unmask

One package atom per line (simple `cat/pkg` or versioned like `>=cat/pkg-1.0`).

```
sys-apps/systemd
>=dev-lang/python-3.13
```

3.5.3 Directory layout

All `package.*` files support both single-file and directory layouts, matching Portage's convention:

```
/etc/portage/package.use           ← single file
/etc/portage/package.use/          ← directory
/etc/portage/package.use/custom    ← files read in sorted order
/etc/portage/package.use/gaming
```

When the path is a directory, all non-hidden files in it are read in sorted (lexicographic) order.

3.5.4 Template files

For development without a real Gentoo system, portage-ng ships template configuration files in `Source/Config/Gentoo/`:

```
Source/Config/Gentoo/
  ├── make.conf
  ├── package.use
```

```
├── package.mask
├── package.unmask
├── package.accept_keywords
└── package.license
```

These contain commented examples that mirror a typical Gentoo setup. On a real Gentoo system, point `config:portage_confdir/1` directly at `/etc/portage` instead.

3.6 Precedence

When the same setting appears in more than one place, portage-ng resolves it by checking sources from most specific to most general. The first match wins. For environment-like settings such as use flags, keywords, licenses, etc., the lookup order is:

1. **Command-line environment variables** — values passed on the command line override everything else.
2. **make.conf** — your `/etc/portage/make.conf` settings come next.
3. **Configuration templates** — defaults provided by the portage-ng configuration templates (under `Source/Config/Gentoo/`). These serve as a development baseline when no real `/etc/portage/` is configured.
4. **Built-in defaults** — hard-coded baseline values when nothing else is specified.

Package masks and per-package USE overrides follow a similar layering. Gentoo's profile tree is applied first (the chain of `package.mask`, `package.use`, `use.mask`, and `use.force` files that define baseline policy for your chosen profile). Your `/etc/portage/` files are applied on top, so they can override profile-level decisions. Finally, fallback defaults fill in anything left unspecified.

In practice this means your `/etc/portage/` customisations always take priority over profile defaults, and anything you pass on the command line takes priority over both.

3.7 Profile loading strategy

Profile data (USE flags, masks, per-package USE, license groups) can be loaded in two ways each time portage-ng starts:

- **Live** — the Gentoo profile tree is parsed from disk on every startup. This is the most accurate option because it always reflects the latest state of the profile, but it takes a moment longer to start.
- **Cached** — profile data is loaded from a pre-serialized cache file (`Knowledge/profile.qlf`). This makes startup near-instantaneous, but the cache must be regenerated (via `--sync`) whenever the profile changes.

The strategy is set per operating mode in `config.pl`. portage-ng supports several modes of operation (standalone, daemon, worker, client, server — see Chapter 14: Command-Line Interface for details). Each mode can use a different loading strategy:

```
config:profile_loading(standalone, live).
config:profile_loading(daemon,     cached).
config:profile_loading(worker,     cached).
config:profile_loading(client,     live).
config:profile_loading(server,     cached).
```

If the cached strategy is set but `Knowledge/profile.qlf` does not exist yet, portage-ng falls back to live loading automatically.

3.7.1 Generating the profile cache

The `--sync` command generates `Knowledge/profile.qlf` automatically:

```
portage-ng --sync
```

After syncing all repositories, portage-ng walks the profile tree once and serializes all profile-derived data to disk. Subsequent runs that use the `cached` strategy load this file instead of re-parsing the profile tree.

3.7.2 What gets cached

The profile cache captures the following data so it does not need to be re-derived from the profile tree on each startup:

Data	Source files
USE flag defaults	`make.defaults` along the profile chain
USE masks and forced flags	`use.mask`, `use.force`
Package masks	`package.mask`, `package.unmask`
Per-package USE overrides	`package.use`
Per-package USE masks and forced flags	`package.use.mask`, `package.use.force`
License groups	`license_groups`

3.8 World sets

portage-ng maintains world sets — the list of packages explicitly requested by the user — under `Source/Knowledge/Sets/world/`. Each machine can have its own `.local` world set file. The `@world` target resolves to all packages in the active world set. The format is the same as Gentoo's `/var/lib/portage/world`, so you can point portage-ng at your Gentoo system's world file and use Portage and portage-ng side by side.

World set management is handled through the `set.pl` module, which supports `world(Atom):register` and `world(Atom):unregister` proof literals to add/remove packages during `--merge` operations.

After you change world membership or sync new tree data, rely on the same **sync workflow** described above: a standalone `--sync` refreshes `Knowledge/kb.qlf` (and the profile cache) so resolution sees an up-to-date union of tree, VDB, and world-related facts.

3.9 Further reading

- Chapter 2: Installation and Quick Start — prerequisites and first run
- Chapter 6: Knowledge Base and Cache — how the Portage tree is loaded into Prolog facts
- Chapter 14: Command-Line Interface — CLI options that interact with configuration

Chapter 4

Architecture Overview

Reasoning about software configurations is not a single algorithm you "run once." It is a chain of transformations: turn repository facts into a logical problem, search for a proof that explains *why* each step is needed, turn that proof into an ordered plan, and finally render or execute it. portage-ng is structured as a **pipeline** because that sequence is the natural shape of the work. Each stage has a clean role—parse facts, prove a plan, schedule it, print (or build) it—and can be characterized in isolation: the reader produces a fixed vocabulary of literals; the prover produces a justified partial order of dependencies; the planner and scheduler refine that into something a human or a build system can follow. Treating the system as a pipeline is therefore a **design decision**: it keeps stages testable, replaceable, and easier to reason about than a monolith where parsing, search, ordering, and output are tangled together.

4.1 The pipeline

portage-ng processes a user request through a linear pipeline of six stages:

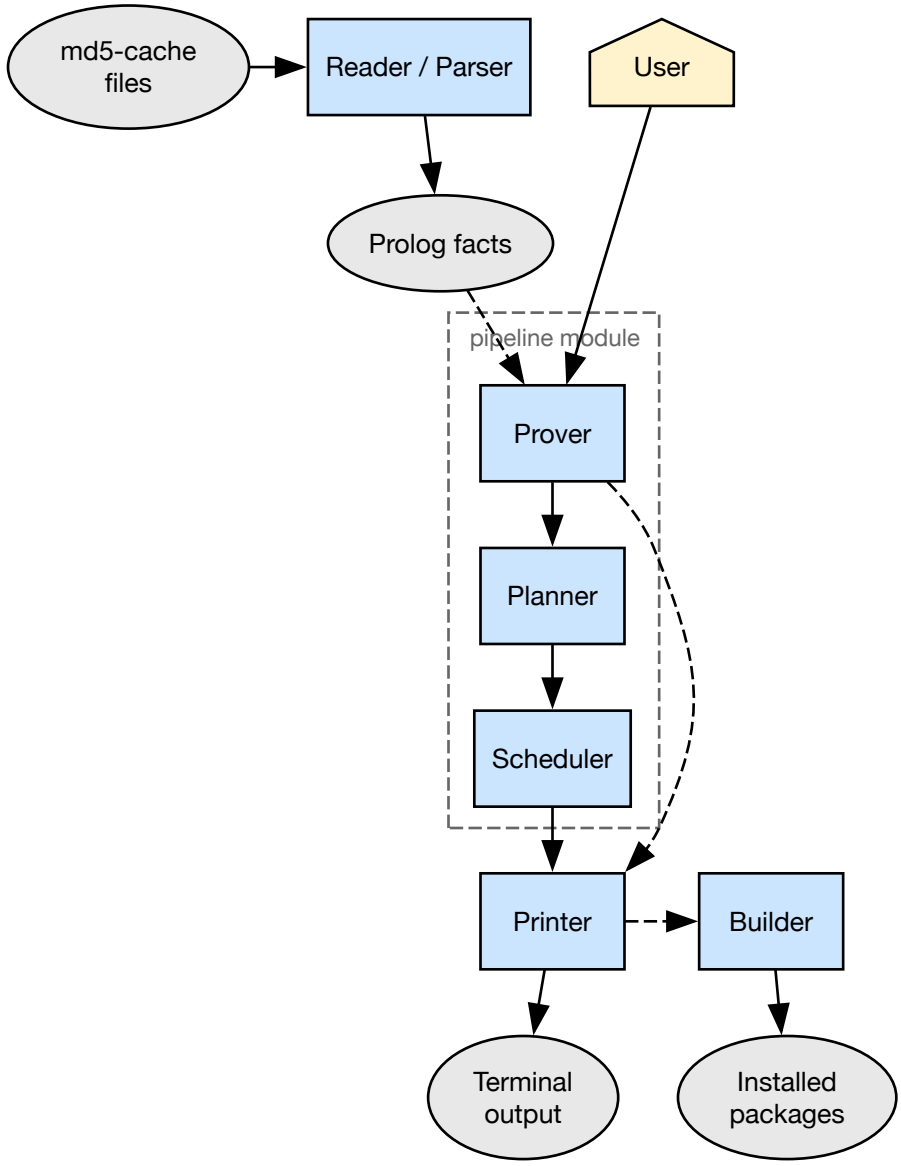


Figure 7 — Pipeline overview

```
reader/parser  →  prover  →  planner  →  scheduler  →  printer  →  builder
                  └──────── pipeline ────────┘
```

The prover produces four AVL trees — **Proof**, **Model**, **Constraints**, and **Triggers** — that flow through the rest of the pipeline. Together they capture *why* each literal was accepted, *what* is known, *what restrictions* must hold, and *who depends on whom*. Section Data structures describes each one in detail.

The prover, planner, and scheduler together form the `pipeline` module. Two canonical entry points share the same 5-tier committed-choice progressive relaxation (strict, keyword_acceptance, blockers, unmask, keyword_unmask):

```
pipeline:prove_plan_with_fallback(Goals, Proof, Model, Plan, Triggers)
pipeline:prove_with_fallback(Goals, Proof, Model, Triggers)
```

The first runs the full pipeline (prove + plan + schedule) and is used by all production paths. The second runs the prover only and is used by layered tests and `--bugs`.

Stage	Module	Input	Output
Reader / Parser	`reader.pl`, `parser.pl`, `eapi.pl`	Ebuild md5-cache files	Prolog facts (`cache:entry/5`)
Prover	`prover.pl`	Goal literals (from user)	Proof, Model, Constraints, Triggers
Planner	`planner.pl`, `kahn.pl`	Proof, Triggers	Plan + Remainder
Scheduler	`scheduler.pl`	Plan, Remainder	Plan (with SCC merge-sets)
Printer	`printer.pl`, `Printer/`	Proof, Model, Plan	Terminal output, `.merge` files
Builder	`builder.pl`, `Builder/`	Plan	Ebuild phase execution

4.2 Operating modes

portage-ng can run in several modes, each tailored to a different deployment scenario. The mode determines which modules are loaded, how the knowledge base is accessed, and whether proving happens locally or is distributed across machines. The mode is selected with `--mode` on the command line (e.g. `portage-ng --mode server`). When no mode is specified, standalone is used.

4.2.1 Standalone

The default and most common mode. A single process on a single machine loads the full knowledge base, runs the complete pipeline (prover, planner, scheduler, printer, builder), and produces results locally. This is what you use for day-to-day `--pretend`, `--merge`, `--shell`, and `--sync`.

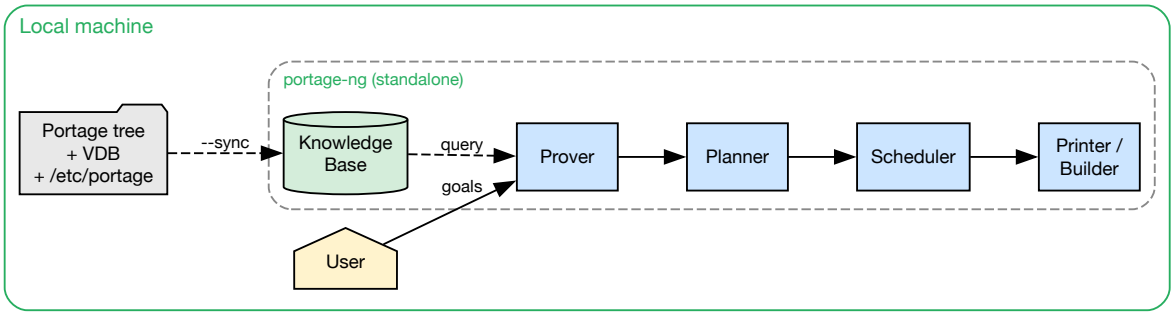


Figure 8 — Standalone mode

Everything happens in one process: the Portage tree, VDB, and `/etc/portage/` configuration are synced into the knowledge base, and the user's goal literals are proven, planned, and printed — all on the same machine.

4.2.2 Client and server

In client–server mode, the reasoning happens on a powerful server while a lightweight client submits requests and displays results. The client and server communicate over TCP/IP with SSL encryption (HTTPS), so they can run on different machines — potentially on different networks.

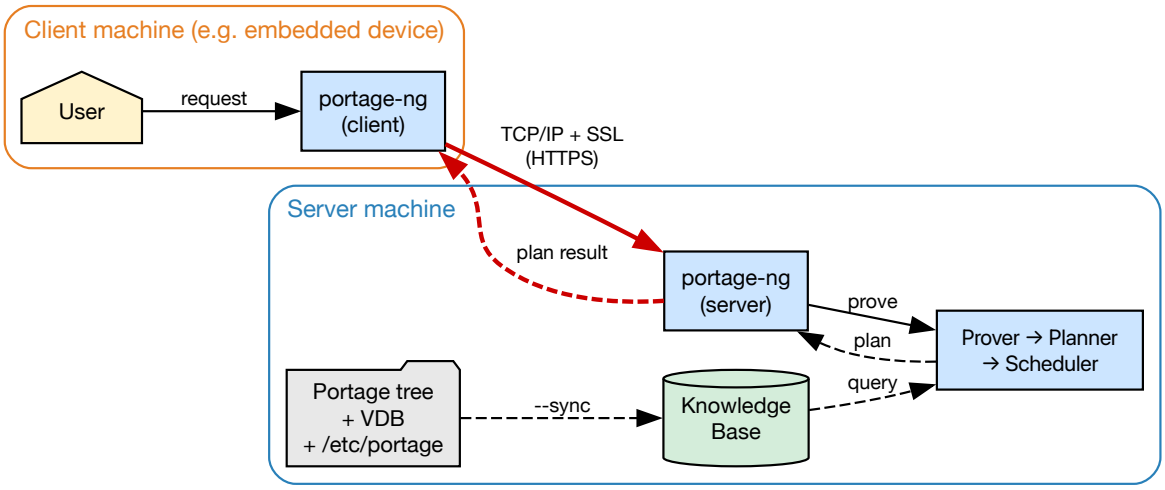


Figure 9 — Client–server mode

The server hosts the knowledge base and runs the full pipeline. The client needs only the thin slice of printing and pipeline glue required to render output. This makes client–server mode ideal for **embedded systems** and resource-constrained devices: the client binary is small, uses minimal memory, and delegates all proving to the server. Queries return in milliseconds because the knowledge base is already loaded and indexed on the server side.

4.2.3 Daemon / IPC

Daemon mode is similar to standalone, but the process stays resident and listens on a Unix socket for commands from local processes. Both the daemon and its clients run on the **same machine**.

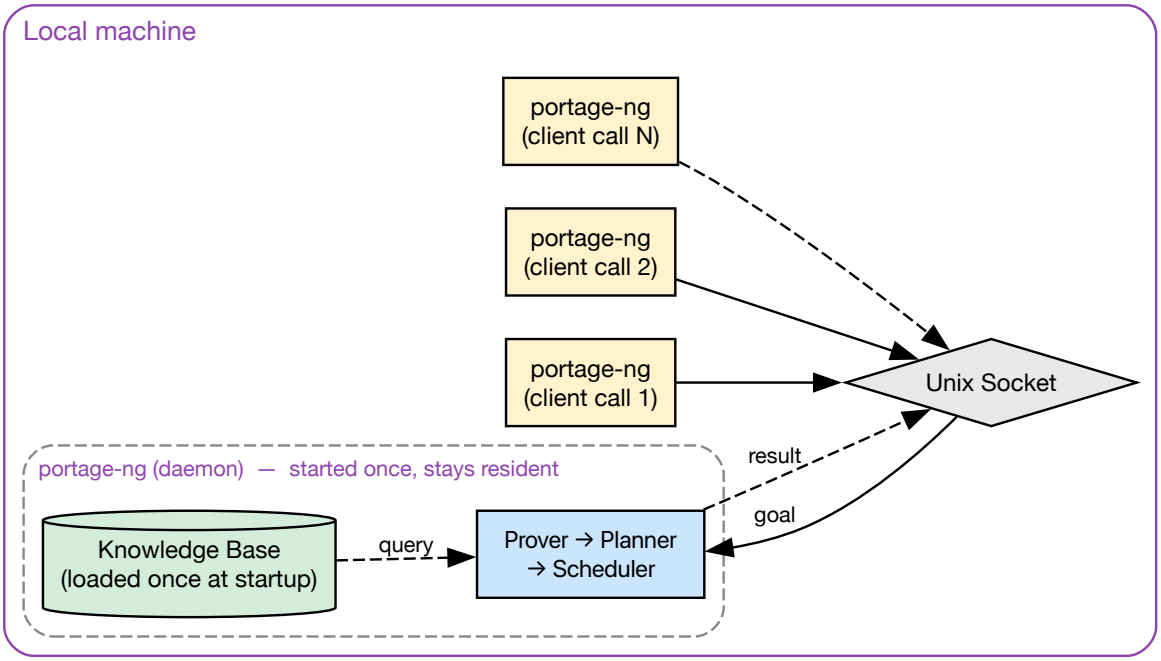


Figure 10 — Daemon / IPC mode

The key advantage is **startup performance**. In standalone mode, every invocation loads the full knowledge base from disk — tens of thousands of Prolog facts — before it can answer a single query. In daemon mode, the knowledge base is loaded **once** when the daemon starts and stays in memory. Subsequent queries arrive over the Unix socket and are answered in **milliseconds**, because there is no parsing, no qcompile loading, no JIT indexing warmup — just a direct query against the already-loaded, already-indexed knowledge base. This makes daemon mode well suited for interactive tooling, editor integrations, and scripts that issue many small queries in quick succession.

4.2.4 Workers

Worker mode enables **distributed proving** across multiple machines. A central server advertises itself via **Bonjour** (mDNS/DNS-SD), and workers on the local network automatically discover it without manual configuration.

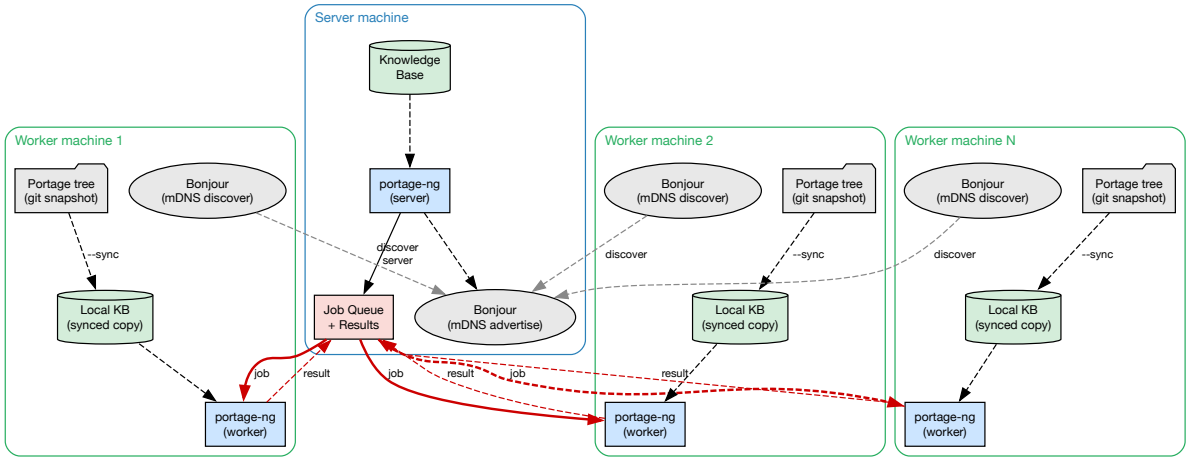


Figure 11 — Worker mode

Each worker machine maintains its own local copy of the Portage tree (typically via a **git snapshot**) and runs `--sync` locally to build its own knowledge base. This ensures all workers reason against the same set of ebuilds — tree synchronisation is a prerequisite for consistent results across the cluster.

Once a worker discovers the server, it polls the job queue for proving tasks: the server breaks a large proof (e.g. `@world`) into independent sub-goals, distributes them to available workers, and collects the results. Each worker runs the full pipeline locally (prover, planner, scheduler), so proving scales horizontally — adding more worker machines reduces wall-clock time for large proof sets.

See Chapter 14: Command-Line Interface for the full mode reference and Chapter 17: Distributed Proving for TLS certificate setup and cluster configuration.

4.3 Module load order

Each mode loads only the modules it needs. This keeps startup time, memory footprint, and failure modes appropriate to the deployment:

The load order is defined in `Source/loader.pl`. Each operating mode loads a different subset of modules:

```
load_common_modules        — SWI-Prolog libraries, OO context, config, OS,
                             interface, EAPI, reader, subprocess, bonjour,
                             feature unification, daemon

load_standalone_modules    — Full pipeline: KB (cache, repository, query),
                             Gentoo domain (version, rules, ebuild, VDB,
                             preference), prover, planner, scheduler,
                             printer, builder, grapher, writer, test

load_server_modules        — HTTP server, Pengines, sandbox

load_client_modules        — HTTP/socket client, subset of printer/pipeline
```

```
load_worker_modules        — Same pipeline as standalone + client + cluster

load_llm_modules           — LLM provider backends, explain, semantic search
```

4.4 Domain-agnostic core vs Gentoo-specific rules

Traditional Portage couples the resolver tightly to Gentoo semantics: USE flags, slots, profiles, and cache layout are not optional details—they are woven through the same code paths as the search strategy. That makes it hard to test "the resolver" without dragging the entire domain along, and hard to experiment with alternative rule sets or other package ecosystems.

portage-ng deliberately **separates** a domain-agnostic core from a Gentoo-specific rules layer. The prover does not know what a USE flag *is*. It sees abstract literals and Horn-style rules; expanding a goal means calling a single hook-shaped interface and continuing the search. The same engine could, in principle, reason about RPM packages, Nix derivations, or Cargo crates —you would supply different `rule/2` implementations and a different knowledge base, not a different prover. That separation is intentional: it isolates *how we search* from *what Gentoo means*, so the core can be exercised and compared without re-implementing Portage wholesale. Packages, USE flags, and slots never appear as primitives in the core; they are interpreted entirely inside the rules layer:

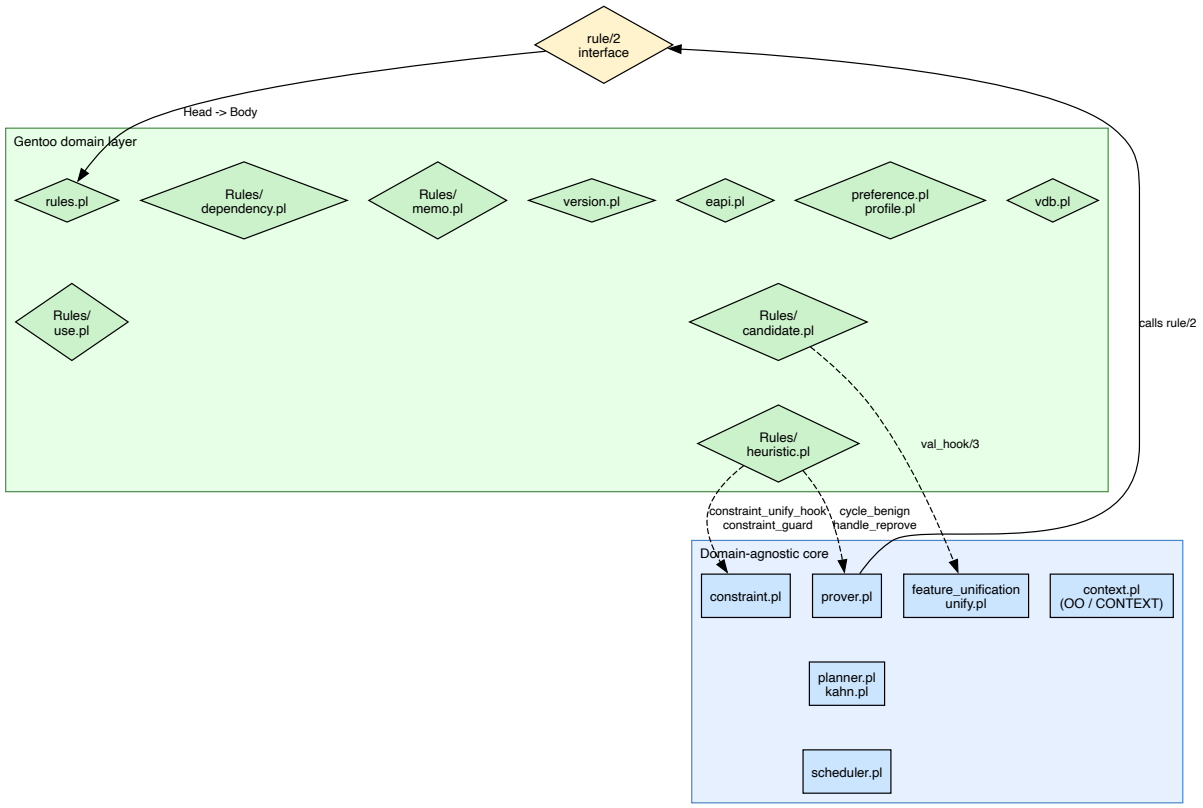


Figure 12 — Layer separation

The **`rule/2` interface** is the contract between the domain-agnostic core and the domain-specific layer. Everything Gentoo-specific—consulting the knowledge base, evaluating USE conditionals, resolving candidates, emitting constraint terms—lives on the far side of that boundary.

```
rules:rule(Head, Body)
```

The prover calls `rule/2` to expand a literal into its dependencies. The rules module implements this by consulting the knowledge base, evaluating USE conditionals, resolving candidates, and emitting constraint terms.

This separation means the same reasoning engine could be applied to a different domain by supplying a different set of rules.

4.5 Data structures

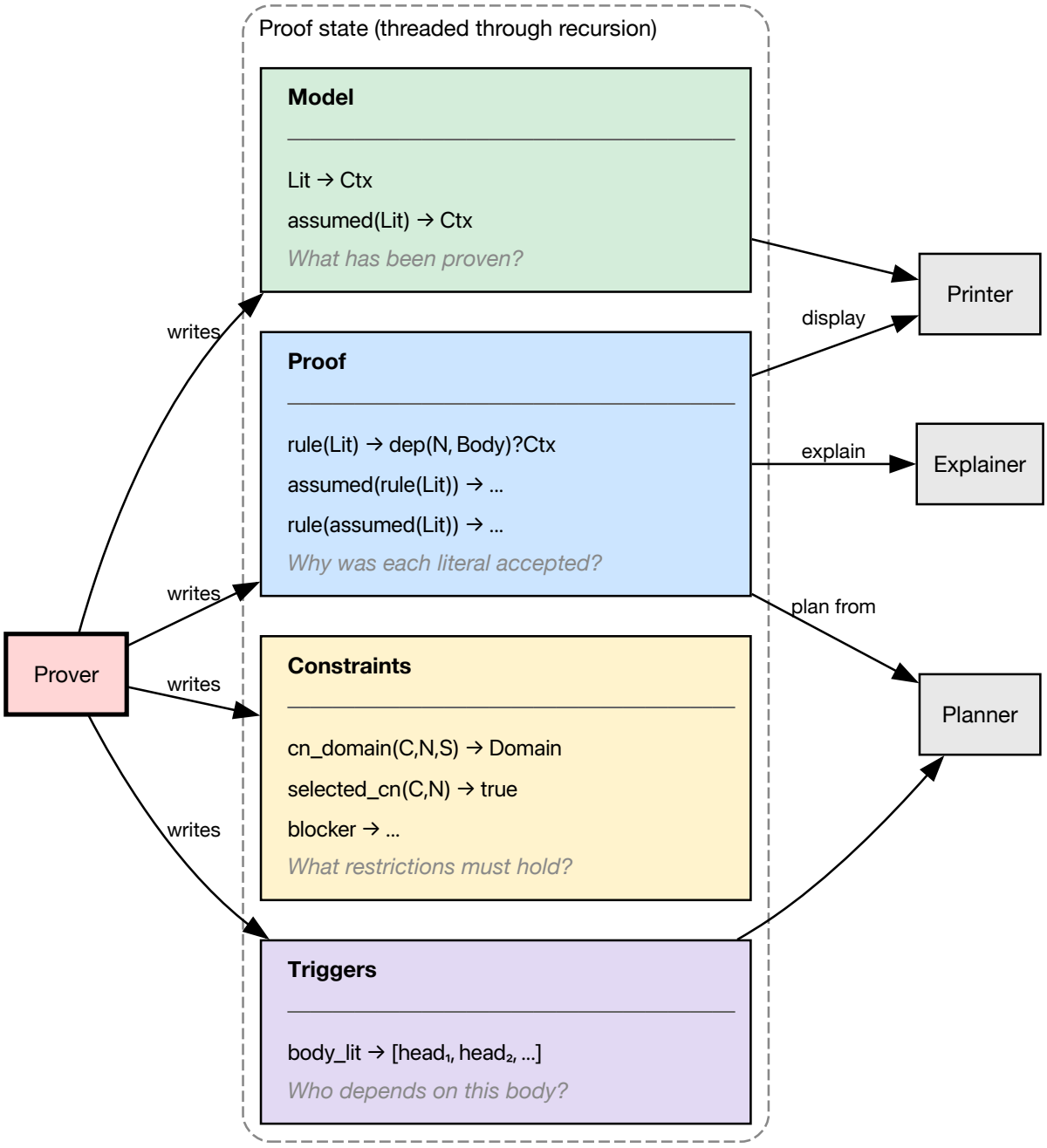


Figure 13 — Data structures

During proof search, the prover must answer four kinds of question at once: *why* was this literal accepted, *what* is already known, *what restrictions* must remain consistent across branches, and *who depends on whom* when context or assumptions change. Four balanced trees (AVL maps via `library(assoc)`) hold exactly those roles. Together they capture the **complete state** of a proof attempt: the prover threads them through recursive expansion without relying on a soup of unrelated global mutable flags for "current model" or "current explanation."

- **Proof** — Records *why* each literal was proven: the justification (which rule instance and body linked the head). Without it, you cannot explain the plan or reconstruct the dependency argument for the user.
- **Model** — Records *what* has been proven: the current state of knowledge (each literal and its proof-term context). This is the structure that memoizes success: the same literal is not re-proved from scratch along every path.
- **Constraints** — Records *restrictions* that must hold: version domains, slot locks, blockers, and similar invariants. They cross-cut the proof tree; they are not local to a single rule application.
- **Triggers** — Records *which heads depend on which bodies*—a reverse-dependency index. When a context changes or delayed work fires, the prover uses triggers to find "who cares" about that body without scanning the entire proof.

The prover maintains these four structures during proof construction:

Structure	Key	Value	Purpose
Proof	`rule(Lit)` or `assumed(rule(Lit))`	`dep(N, Body)`	Which rule and body justified each literal; `N` is the dependency count
Model	`Lit` or `assumed(Lit)`	proven	Every literal that has been established
Constraints	e.g. `cn_domain(dev-libs, openssl, 0)`	`version_domain(...)`	Accumulated invariants: version domains, slot locks (`slot(3)`), blockers
Triggers	body literal	`[head, ...]`	Reverse-dependency index: which heads depend on this body literal

The Proof and Model structures use different key schemes to distinguish normal proofs from assumptions:

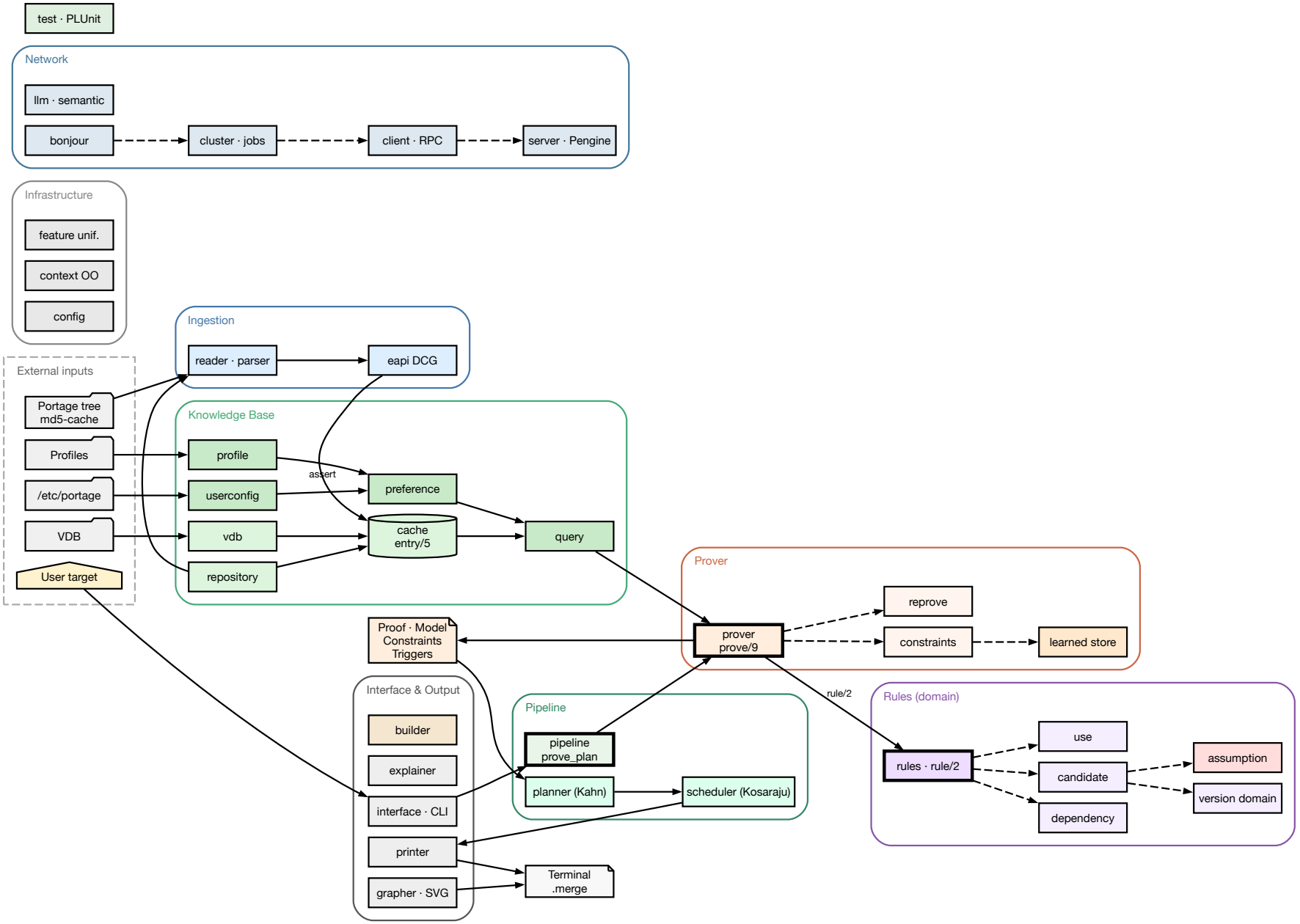
- `rule(Lit)` — normally proven literal
- `assumed(rule(Lit))` — prover cycle-break assumption
- `rule(assumed(Lit))` — domain assumption (dependency cannot be satisfied)

See Chapter 5: Proof Literals for the literal format and Chapter 9: Assumptions for the assumption taxonomy.

4.6 Architecture diagram

The following page shows the full system architecture in landscape orientation, covering all layers from external inputs through the knowledge base, prover, planner, and output pipeline.

portage-ng: Full System Architecture


test · PLUnit
Network
llm · semantic
bonjour
cluster · jobs
client · RPC
server · Pengine
Infrastructure
feature unif.
context OO
config
Ingestion
reader · parser
eapi DCG
External inputs
Portage tree
md5-cache
Profiles
/etc/portage
VDB
User target
Knowledge Base
profile
userconfig
vdb
repository
assert
preference
cache
entry/5
query
Prover
prover
prove/9
reprove
constraints
learned store
Proof · Model
Constraints
Triggers
Interface & Output
builder
explainer
interface · CLI
printer
grapher · SVG
Terminal
.merge
Pipeline
pipeline
prove_plan
planner (Kahn)
scheduler (Kosaraju)
rule/2
Rules (domain)
rules · rule/2
use
candidate
dependency
assumption
version domain

4.7 Further reading

- Chapter 5: Proof Literals — the `Repo://Entry:Action?{Context}` term format
- Chapter 8: The Prover — inductive proof search in detail
- Chapter 12: Planning and Scheduling — wave planning and SCC decomposition

Chapter 5

Proof Literals

5.1 The universal literal format

Every term that flows through the portage-ng pipeline — from rules to prover to planner to printer — uses the same universal format:

```
Repo://Entry:Action?{Context}
```

Each component answers a question that arises at a different stage of the pipeline:

- **`Repo`** — *where* does this fact come from? The **rules** consult different repositories (the Portage tree, the VDB, an overlay) and the repository prefix travels with the literal so the prover never confuses an available package with an installed one.
- **`Entry`** — *what* package version is meant? When the rules expand a dependency, they select a concrete cache entry (`'category/name-version'`). This identifier is the key the **prover** uses to look up and store proof work — two dependency paths that resolve to the same entry share the same proof node.
- **`Action`** — *how* should the pipeline treat this entry? The rules assign an action (`:install`, `:run`, `:download`, `:update`, …) that tells the **planner** which phase of work this literal represents and how to order it relative to others.
- **`Context`** — *why* and *under what conditions* was this literal introduced? As the prover expands the dependency graph, each literal accumulates a feature-term context: which parent introduced it (`self`), which USE flags are required (`build_with_use`), ordering constraints (`after`), slot locks, and so on. At join points where two dependency paths reach the same literal, the prover **merges** their contexts via feature unification. The **printer** reads the final context to display USE flags, slot information, and assumption reasons.

Traditional resolvers scatter this information across separate side structures. portage-ng packs it into the literal itself, making every term **self-describing**: you can inspect a single literal and know its repository, version, phase, and full provenance without consulting external tables.

5.2 Operator precedences

The literal format is defined by three infix operators declared in `Source/Logic/context.pl`. In SWI-Prolog, **higher** precedence means the operator becomes the **principal functor** at that level of the term — i.e. it sits **higher** in the parse tree. The ordering `://` (603) > `?` (602) > `:` (601) was chosen so that the structure lines up with everyday use: you scope by **repository** first,

then attach the **context list** to the **ebuild core** (`Entry:Action`), with **entry** and **action** paired at the innermost level. That makes the common cases — "everything in `portage`", or "this `category/name-version` with this phase" — parse in the way you read them. The `?{Context}` annotation is intentionally the outer wrapper around the core (after `://`) because **context is what changes most often during proof search**; the repository and entry/action spine stay stable while USE, ordering, and constraint features are merged and refined.

Operator	Precedence	Associativity	Parses as
`://`	603	xfx	`Repo :// Rest`
`?`	602	xfx	`Core ? {Context}`
`:`	601	xfx	`Entry : Action`

Because `://` has the highest precedence, a full literal parses as:

```
Repo :// ((Entry : Action) ? {Context})
```

That is: repository scopes the whole term; the ebuild core is `Entry : Action`; the context list attaches to that core.

5.3 `Repo` — the repository

The leftmost component identifies which registered repository the literal belongs to. It is an atom — the same atom used when registering the repository with the knowledge base:

```
:- portage:newinstance(repository).
:- kb:register(portage).
```

Common repository atoms:

Atom	Meaning
`portage`	The main Gentoo Portage tree
`pkg`	The VDB (installed packages database)
`overlay`	A user or test overlay

Literals from different repositories can coexist in the same proof. For example, a `portage://...` literal might depend on a `pkg://...` literal when an installed package satisfies a dependency.

5.4 `Entry` — the cache entry

The middle component is the cache entry identifier — a quoted atom in the format `'category/name-version'`:

```
'sys-apps/portage-3.0.77-r3'
'dev-lang/python-3.13.2'
```

This atom maps directly to the second argument of `cache:entry/5`:

```
cache:entry(portage, 'sys-apps/portage-3.0.77-r3', 'sys-apps', 'portage',
            version([3,0,77],'',...)).
```

The category, name, and version are also available as separate fields in the cache, but the combined atom serves as the unique key for lookup.

5.5 `Action` — the phase

The component after the entry (inside the `Entry : Action` pair) specifies what operation the literal represents. Actions fall into three categories:

5.5.1 Ebuild actions

These apply to `Repo://Entry` literals:

Action	Meaning
`run`	Full installation + runtime availability
`install`	Build and install (DEPEND + BDEPEND + RDEPEND)
`download`	Fetch source archives
`fetchonly`	Fetch only, do not build
`reinstall`	Reinstall an already-installed package
`update`	Update to a newer version
`downgrade`	Downgrade to an older version
`upgrade`	Upgrade (used in VDB context)
`depclean`	Remove an unneeded package
`uninstall`	Uninstall a package

5.5.2 Dependency and validation actions

Before the prover can expand a package's full dependency tree, it first needs to answer two questions: *which dependencies are actually active* given the package's USE flags, and *is the USE configuration itself consistent*? These questions are answered in two dedicated phases — `:config` and `:validate` — that run **before** the main `:install` / `:run` expansion.

The `:config` phase — computing the dependency model

When portage-ng resolves a package, it does not immediately try to prove every dependency listed in the ebuild's metadata. Instead, it first builds a **dependency model**: a stable snapshot of which dependencies are active under the current USE flag configuration.

An ebuild's metadata contains conditional dependencies guarded by USE flags. For example, `dev-lang/python` might declare:

```
RDEPEND="ssl? ( dev-libs/openssl )
         readline? ( sys-libs/readline )
         !readline? ( sys-libs/libedit )"
```

The `:config` phase evaluates each dependency term against the effective USE flags and retains only the **active** dependencies. In the example above, if `ssl` is enabled and `readline` is disabled, the model will contain `dev-libs/openssl` and `sys-libs/libedit` — the `sys-libs/readline` dependency is dropped because its USE guard is not satisfied. Same-slot self-references (a package listing itself as a build dependency in the same slot) are treated as bootstrap dependencies and checked against the VDB. Cross-slot self-references (same category and name but a different slot) are resolved normally as regular dependencies.

When a choice group or constraint forces a decision, the prover may also **assume** a flag — for instance, if an `exactly_one_of` group requires at least one member to be enabled and none currently is, the prover picks the most likely candidate and records a domain assumption so the user is informed.

The result is a **model** whose keys are the surviving dependency terms — the ones that actually need resolving.

For choice groups (OR dependencies), the `:config` phase picks one viable alternative:

```
RDEPEND="|| ( dev-db/postgresql dev-db/mariadb dev-db/sqlite )"
```

becomes a `choice_group(Deps):config?{Context}` literal. The rules try each alternative, preferring already-installed packages, and commit to one choice. The chosen dependency enters the model; the others are discarded. This means that by the time the main proof begins, every OR group has been resolved to a single concrete dependency.

Action	Literal head	Meaning
`config`	grouped dependency	Resolve a dependency group under USE flags
`config`	package dependency	Check a single dependency
`config`	choice group	Pick one alternative from an OR group
`config`	USE conditional	Evaluate a USE-guarded block

The `:validate` phase — checking REQUIRED_USE consistency

Ebuilds can declare constraints on which USE flag combinations are valid. For example:

```
REQUIRED_USE="^^ ( python_targets_python3_12 python_targets_python3_13 )"
```

This says "exactly one Python target must be selected." The `^^` operator translates to an `exactly_one_of_group(...)` term. Before expanding the package's dependencies, portage-ng wraps each REQUIRED_USE constraint as a `:validate` literal:

```
exactly_one_of_group([required(python_targets_python3_12),
                      required(python_targets_python3_13)]):validate?{[
  self(portage://'dev-lang/python-3.13.2')
]}
```

The rules check whether the effective USE flags for the package (identified by the `self(...)` context tag) satisfy the constraint. For `exactly_one_of`, the check counts how many of the listed flags are enabled and verifies the count is exactly one. If the constraint is violated, the rules emit a domain assumption recording the conflict:

```
assumed(conflict(required_use, exactly_one_of_group(Deps)))
```

The full set of REQUIRED_USE operators:

Operator	Group term	Constraint
`^^`	`exactly_one_of_group(Deps)`	Exactly one flag enabled
any-of	`any_of_group(Deps)`	At least one flag enabled
`??`	`at_most_one_of_group(Deps)`	At most one flag enabled
(none)	`use_conditional_group(...)`	Conditional: if A then B

Each of these operators is wrapped as a `:validate` literal and checked against the package's effective USE flags:

Action	Literal head	Meaning
`validate`	`exactly_one_of_group(...)`	Check `^^` constraint
`validate`	`any_of_group(...)`	Check any-of constraint
`validate`	`at_most_one_of_group(...)`	Check `??` constraint

5.5.3 Non-ebuild literal heads

Some literals do not follow the `Repo://Entry` pattern:

Literal	Meaning
`world(Atom):register`	Add a package to the @world set
`world(Atom):unregister`	Remove a package from the @world set
`target(Query, Arg):run`	Top-level target resolution
`target(Query, Arg):fetchonly`	Top-level fetch-only target
`target(Query, Arg):uninstall`	Top-level uninstall target

5.6 `Context` — the feature-term list

The context is a Prolog list wrapped in `{}` and attached via the `?` operator. It carries per-literal metadata that records provenance, ordering, constraints, and USE requirements:

```
portage://'dev-lang/python-3.13.2':install?{[
  self(portage://'sys-apps/portage-3.0.77-r3'),
  build_with_use:use_state([ssl, threads], []),
  after(portage://'sys-apps/portage-3.0.77-r3':install)
]}
```

Reading this literal: "install `dev-lang/python-3.13.2` from the portage repository, because `sys-apps/portage` needs it (`self`), with USE flags `ssl` and `threads` enabled (`build_with_use`), and schedule it after the installation of `sys-apps/portage` (`after`)."

The context list is **not** an unstructured bag of annotations. It is the proof-side counterpart of **feature terms** in the sense used by Zeller-style feature logic (see Chapter 20: Context Terms): a structured collection of features that can be **merged** when two dependency paths describe the same package under different conditions. When two paths reach the same literal with different USE requirements or other features, the prover does not arbitrarily pick one path's context — it **combines** them using feature term unification (`sampler:ctx_union/3`), which relies on the same feature machinery as the rest of the context subsystem. That is why context lives in a dedicated suffix of the literal: it is the part that must stay open to **merge** and **refine** as the proof graph grows.

5.6.1 `self` — who introduced this dependency

Every dependency in an ebuild comes from somewhere. The `self(Repo://Entry)` tag records *which package* introduced this literal as a dependency. When the rules expand a package's dependency list, they stamp every child literal with the parent's identity:

```
portage://'dev-libs/openssl-3.4.1':install?{[
  self(portage://'dev-lang/python-3.13.2')
]}
```

This says "openssl is here because python depends on it." The `self` tag serves three purposes:

1. **Provenance tracking.** The printer can show *why* a package appears in the plan — who pulled it in.
2. **USE flag resolution.** When checking whether a USE flag is enabled for a dependency, the rules look up the effective USE flags of the ebuild identified by `self`. This is how the `:validate` phase works: the `self` tag tells the REQUIRED_USE checker which package's USE configuration to consult.
3. **Self-dependency detection.** When a package lists itself as a build dependency in the same slot (which happens for bootstrap packages), the rules recognise this by comparing the dependency target's category, name, and slot to the `self` entry. Same-slot self-deps are checked against installed packages; cross-slot self-deps (e.g. `antlr-tool:4` depending on `antlr-tool:3.5`) are treated as regular dependencies and resolved normally.

At most one `self` tag is present per context. When a literal is stamped, any previous `self` is replaced — the immediate parent is what matters.

5.6.2 `build_with_use` — requirements imposed by parent

Gentoo dependency atoms can carry *bracketed USE requirements*: conditions that must hold on the dependency target. For example, in `sys-apps/portage`'s metadata:

```
RDEPEND="dev-lang/python[ssl,threads]"
```

The brackets `[ssl,threads]` mean "I need python, and it must be built with the `ssl` and `threads` USE flags enabled." The rules translate this into a `build_with_use` context tag:

```
portage://'dev-lang/python-3.13.2':install?{[
  build_with_use:use_state([ssl, threads], [])
]}
```

The `use_state(Enabled, Disabled)` term lists which flags must be on and which must be off. Negative requirements like `[-test]` appear in the disabled list:

```
build_with_use:use_state([], [test])
```

When two dependency paths reach the same package with different USE requirements, the prover merges them via feature unification. If portage requires `python[ssl]` and another package requires `python[xml]`, the merged context becomes:

```
build_with_use:use_state([ssl, xml], [])
```

If two paths disagree — one requires `[debug]` and another requires `[-debug]` — the unification detects the conflict and the constraint system handles it (potentially triggering a reprove with different candidate selection).

The `build_with_use` tag is distinct from the package's own USE flags. A package's USE flags are determined by profile, user configuration, and defaults. The `build_with_use` tag captures what *other packages demand of this package*. The printer reads both to display the final USE flag set, marking flags that were pulled in by dependency requirements.

5.6.3 `after` — ordering constraints

The planner needs to know the order in which actions should be scheduled. The `after(Literal)` tag expresses a hard ordering constraint: "this literal must come after the specified literal in the final plan."

```
portage://'dev-lang/python-3.13.2':download?{[
  after(portage://'sys-apps/portage-3.0.77-r3':install)
]}
```

Ordering constraints arise naturally from the dependency structure. When package A depends on package B, the rules add `after(B:install)` to A's download and dependency contexts. This ensures that B is installed before A starts building.

The `after` tag **propagates**: when it is set on a literal, it is also injected into that literal's own children. If A must come after B, then A's dependencies also implicitly come after B. This transitive propagation ensures that entire subtrees are correctly ordered.

For cases where ordering should *not* propagate, the `after_only` variant exists. This is used primarily for PDEPEND (post-dependencies): a package's post-dependencies must come after the package itself, but the post-dependency's own children should not inherit that ordering constraint.

```
after_only(portage://'app-editors/neovim-0.12.0':run)
```

The planner reads both `after` and `after_only` from every literal's context to build the dependency edges that drive Kahn's topological sort.

5.6.4 Summary of context tags

Tag	Purpose
`self(Repo://Entry)`	The parent ebuild that introduced this dependency
`build_with_use:use_state(En, Dis)`	USE flags that must be enabled/disabled on this package
`after(Literal)`	Must come after this literal; propagates to children
`after_only(Literal)`	Must come after this literal; does not propagate
`slot(C, N, Ss):{Candidate}`	Slot lock from `:=` sub-slot rebuild semantics
`replaces(pkg://Entry)`	Which installed package this action replaces
`assumption_reason(Reason)`	Why a domain assumption was made
`suggestion(Type, Detail)`	Actionable suggestion (keyword, unmask, use change)
`constraint(cn_domain(C,N):{D})`	Inline version domain constraint
`onlydeps_target`	Marks a literal as an `--onlydeps` target
`world_atom(Atom)`	Planning marker for @world set membership

Contexts are merged at join points via feature term unification, which uses Zeller-inspired feature unification. See Chapter 20: Context Terms for full details.

5.7 Canonical decomposition

The prover stores literals in assoc/AVL structures keyed by a **stable** identity. That creates a design tension: during proof search, the **context** is constantly enriched — new `build_with_use` features appear, ordering constraints are propagated, slot locks and learned domains are attached — but the **underlying package and phase** (repository, cache entry, action) are still the same logical goal. If the full term including context were used as the key, every refinement

would look like a **new** node: you would get duplicate entries for "the same" install step, incoherent merging, and broken sharing of proof work.

Canonical decomposition fixes that by splitting each literal into a **core** used for identity (`R://L:A`) and a **context list** carried as associated data. Two encounters of the same core with different contexts collide on the same key; the prover then **merges** contexts (via feature term unification) instead of forking duplicate keys.

Two predicates handle decomposition:

5.7.1 `prover:canon_literal/3`

Strips the context from a literal, returning the core key and context separately:

```
canon_literal(R://(L:A),            R://L:A, {}).
canon_literal(R://(L:A?{Ctx}),      R://L:A, Ctx).
canon_literal(R://(L:A)?{Ctx},      R://L:A, Ctx).
canon_literal(R://(L:A?{C1})?{C2},  R://L:A, Merged).
```

The core `R://L:A` is used as the key in the Model AVL. The context is stored as the value.

5.7.2 `prover:canon_rule/3`

Similarly decomposes a rule head, producing a context-free key for the Proof AVL.

This decomposition ensures that when a literal is re-encountered with a different context, the prover can find the existing proof entry and merge the contexts rather than creating a duplicate.

5.8 How literals flow through the pipeline

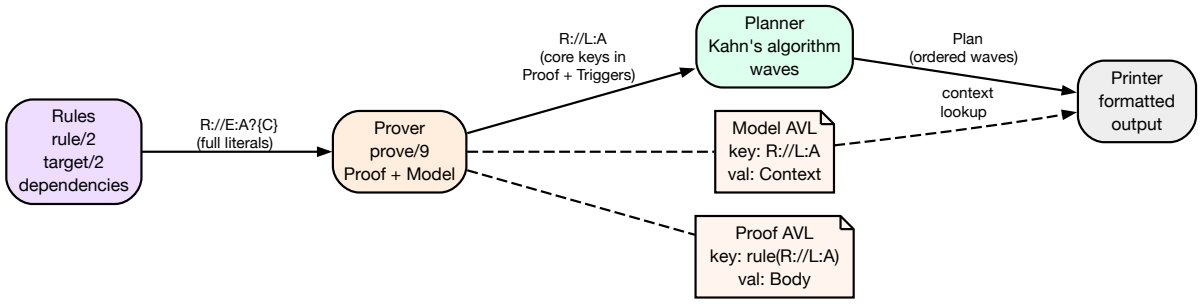


Figure 14 — Literal flow through the pipeline

1. **Rules** produce literals. The `target/2` rule resolves a user query to a `Repo://Entry:run?{Context}` literal. Dependency rules produce further literals with appropriate actions and contexts.
2. **Prover** stores the core literal (`R://L:A`) as the key in the Model AVL and the context as the value. The Proof AVL uses `rule(R://L:A)` as the key, with the rule body and context as the value.

3. **Planner** extracts rule heads from the Proof AVL, using `canon_literal/3` to get core literals. Kahn's algorithm schedules these into concurrent waves based on dependency edges.
4. **Printer** reads the Plan (a list of waves), looks up each literal in the Model AVL to recover its context, and formats the output.

5.9 Worked example

Tracing `target('sys-apps/portage'):run?{[]}` through the pipeline:

```
1. User runs: portage-ng --pretend sys-apps/portage

2. Interface creates goal literal:
   [target('sys-apps'-'portage', []):run?{[]}]

3. Interface invokes the prover with this goal.

4. Prover uses rules to expand target/2:
   rule(target('sys-apps'-'portage', []):run?{[]},
        [portage://'sys-apps/portage-3.0.77-r3':run?{[]}])

5. Prover uses rules to expand :run:
   rule(portage://'sys-apps/portage-3.0.77-r3':run?{[]},
        [portage://'sys-apps/portage-3.0.77-r3':install?{[]},
         ...RDEPEND literals...])

6. Prover uses rules to expand :install:
   rule(portage://'sys-apps/portage-3.0.77-r3':install?{[]},
        [portage://'sys-apps/portage-3.0.77-r3':download?{[]},
         ...DEPEND/BDEPEND literals with self/1, build_with_use, after...])

7. Prover stores each proven literal in the Model AVL:
   Key: portage://'sys-apps/portage-3.0.77-r3':run
   Val: [] (context)

8. Planner places :download in wave 1, :install in wave 2, :run in wave 3

9. Printer outputs:
   [1] portage://sys-apps/portage-3.0.77-r3  download
   [2] portage://sys-apps/portage-3.0.77-r3  install
   [3] portage://sys-apps/portage-3.0.77-r3  run
```

5.10 Further reading

- Chapter 8: The Prover — how the prover uses these literals
- Chapter 11: Rules and Domain Logic — how rules produce literals
- Chapter 20: Context Terms — deep dive into context semantics and feature unification

Chapter 6

Knowledge Base and Cache

6.1 From ebuild to fact: the metadata pipeline

Every package in Gentoo begins life as an **ebuild**: a bash script in the Portage tree that declares metadata (dependencies, USE flags, slot, license, and more). Portage-ng does not run those scripts. Instead, it consumes a pre-digested form of the same information.

`egencache` (or portage-ng's `--regen`) walks the tree and turns ebuilds into **md5-cache** files: flat key–value text blobs that summarize each ebuild's metadata. Portage-ng's reader and parser then loads those files, runs their contents through the **EAPI DCG grammar** (see Chapter 7), and **asserts** `cache:entry/5` (and related) facts into the in-memory knowledge base. For fast startup, those facts are **qcompiled** into `Knowledge/kb.qlf`, so the next session reloads binary bytecode instead of reparsing thousands of text files.

That end-to-end path — ebuild → cache generation → md5-cache → grammar → Prolog facts → QLF — is the **metadata pipeline**. It is deliberate: portage-ng never executes bash for package metadata; it works **entirely from metadata** that has already been extracted and normalized.

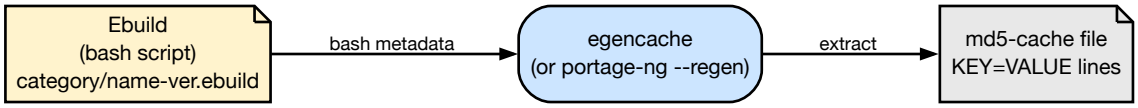


Figure 15 — Cache generation: ebuild to md5-cache

Once the md5-cache files exist on disk, portage-ng's reader parses each one through the EAPI grammar and asserts the resulting terms as Prolog facts:

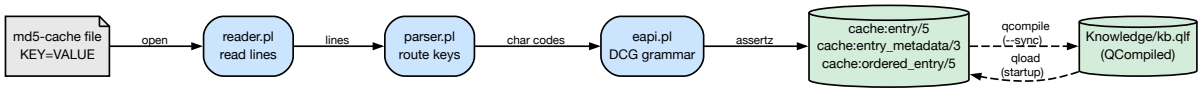


Figure 16 — Ingestion: md5-cache to Prolog facts

The knowledge base is the in-memory representation of the Gentoo Portage tree at the end of that pipeline. It stores every ebuild's metadata as Prolog facts that can be queried in sub-millisecond time.

6.2 The cache data structure

The core data structure is `cache:entry/5`, a dynamic predicate with one fact per ebuild:

```
cache:entry(Repository, Id, Category, Name, Version).
```

Argument	Example	Meaning
`Repository`	`portage`	Registered repository atom
`Id`	`'sys-apps/portage-3.0.77-r3'`	Full category/name-version string
`Category`	`'sys-apps'`	Package category
`Name`	`'portage'`	Package name
`Version`	`version([3,0,77],'',...)`	Parsed version as `version/7` term

Additional cache predicates store per-ebuild metadata:

Predicate	Content
`cache:entry_metadata/3`	EAPI, SLOT, KEYWORDS, LICENSE, etc.
`cache:ordered_entry/5`	Entries ordered by version (for candidate selection)
`cache:provides/3`	Virtual package mappings

6.3 Why this cache design?

The shape of `cache:entry/5` is not arbitrary; it matches how Prolog and the prover actually use the data.

Indexing and lookup. Prolog's strength is pattern matching on structured terms. `cache:entry/5` is arranged so that **first-argument indexing** on `Repository` and **third- and fourth-argument** access on `Category` and `Name` align with the most common query shapes: "in this repo, what entries exist for this category/name?"

Versions as terms. The version is stored as a **`version/7` compound term** so that **standard `compare/3`** on the term gives correct version ordering **without any runtime conversion** to another representation. The prover and rules can treat versions as ordinary Prolog data.

Splitting metadata. Rich fields (EAPI, SLOT, KEYWORDS, LICENSE, …) live in **`cache:entry_metadata/3`** rather than bloating every `entry/5` fact. That keeps the **hot path** — finding candidates by repository, category, and name — **lightweight**, while still allowing full metadata when needed.

Pre-sorted candidates and version preference. **`cache:ordered_entry/5`** holds entries **pre-sorted by version (newest first)** for candidate selection. Building that structure once at load or regen time **avoids repeated sorting during proof search**, where the same name may be considered many times under different contexts.

The ordering is more than an optimisation — it encodes **preference**. Prolog iterates over `ordered_entry/5` clauses in assertion order, so the newest version is tried first. When the prover searches for a candidate that satisfies a dependency, it encounters the highest version before

any older alternative. If that candidate passes all constraint guards, it is selected without ever considering older versions. If it fails (wrong slot, masked, REQUIRED_USE violation), Prolog's backtracking moves to the next clause — the next-highest version — automatically.

This design has a formal counterpart in **ordered logic programs** as studied by Vermeir and Van Nieuwenborgh ("Preferred Answer Sets for Ordered Logic Programs," JELIA 2002). In their framework, when multiple rules can derive conflicting conclusions, a **partial order over rules** determines which one prevails. In portage-ng the "rules" are candidate versions for a given category/name, and the partial order is the version comparison: newer versions have higher priority. Prolog's clause order directly implements this priority — no separate preference layer or scoring function is needed. The result is that the prover naturally gravitates toward the newest compatible version, which matches Gentoo's standard policy, while still falling back to older versions when constraints demand it.

See Chapter 21: Resolver Comparison for more on Vermeir's ordered logic and its role alongside Zeller's feature logic and CDCL-style conflict learning.

6.4 Repositories and the knowledge base registry

Repositories are not just atoms: they are **objects** in the OO context system (`Source/Logic/context.pl`). Each repository has its own **identity**, **paths**, **sync method**, and **cache** partition. The **context** machinery provides **instance creation** (`newinstance`), **method dispatch**, and **visibility guards**, so different repository kinds can share an interface while differing in behavior — for example, `portage:sync` and `overlay:sync` can implement sync differently behind the same method.

Repositories are registered via that OO context system. Each repository is an instance of the `repository` class:

```
:- portage:newinstance(repository).
:- kb:register(portage).
```

The knowledge base module (`knowledgebase.pl`) maintains a registry of all loaded repositories. **`kb:register/1`** records which repositories are **active** so the rest of the system can iterate or dispatch over them. Multiple repositories can be registered simultaneously — for example, the main Portage tree (`portage`), the VDB of installed packages (`pkg`), and user overlays (`overlay`).

Each repository instance manages its own cache facts. The `repository` class provides methods for syncing, loading, and querying:

```
portage:sync.          % Sync from remote
portage:read.          % Read md5-cache into Prolog facts
portage:search(Query). % Search entries
```

6.5 Syncing and cache regeneration

`--sync` performs a full repository synchronization. It is the "wide" end of the metadata pipeline: it brings the tree up to date, then materializes fresh cache facts and QLF artifacts.

1. Fetches the latest Portage tree (via git, rsync, or HTTP)
2. Reads md5-cache files via the EAPI grammar into cache predicates
3. Generates `Knowledge/kb.qlf` (qcompiled facts for fast reload)
4. Generates `Knowledge/profile.qlf` (serialized profile data)

`--regen` regenerates the md5-cache incrementally. It replaces `egencache`: only changed or new ebuilds are re-parsed, and regeneration runs in parallel across available cores.

6.6 Compiling knowledge

On subsequent startups, portage-ng loads `Knowledge/kb.qlf` instead of re-parsing the entire md5-cache directory. `qcompile` files are a SWI-Prolog binary format that loads an order of magnitude faster than parsing text files. That step closes the pipeline opened by ebuilds and md5-cache: the **authoritative** working set for proving is the compiled fact base, not the shell sources.

The raw Prolog facts are also available as `Knowledge/kb.raw` for debugging.

6.7 Query layer

The cache facts described above — `cache:ordered_entry/5`, `cache:entry_metadata/4`, and friends — are ground relational tuples: a flat, indexed collection of facts that describes the known world. In database terminology, this is an **extensional database** (EDB). The query module (`Source/Knowledge/query.pl`) adds an **intensional** layer on top: it defines high-level query predicates that are compiled down to direct lookups over the base relations.

This architecture is closely related to **Datalog**, the declarative query language that sits at the intersection of logic programming and relational databases. In Datalog, ground facts form the base relations and rules define derived views; queries are conjunctive queries over those relations, with guaranteed termination. portage-ng's query layer follows the same pattern: list queries compile into conjunctions of cache lookups (conjunctive queries), every variable is grounded through the EDB (the Datalog safety property), and the query layer itself always terminates. Where the system goes beyond strict Datalog is in its use of compound terms (`version/7`, `slot/1`) rather than flat constants, and in the model queries that invoke the prover — at which point we leave the Datalog fragment and enter full recursive Prolog reasoning.

Rather than interpreting queries at runtime through a generic search function, portage-ng uses SWI-Prolog's **`goal_expansion/2`** — a compile-time macro facility that acts as a Datalog-style query compiler — to rewrite high-level query goals into **direct** calls to indexed cache predicates before the program even runs.

6.7.1 Goal expansion by example

Consider a rule that needs to find all ebuilds named `neovim`:

```
query:search(name(neovim), Repository://Entry).
```

At load time, `goal_expansion/2` rewrites this into:

```
cache:ordered_entry(Repository, Entry, _, neovim, _).
```

The high-level `search` call disappears entirely. What remains is a direct call to the indexed cache predicate, where SWI-Prolog's first-argument indexing on `Repository` and fourth-argument indexing on `Name` make the lookup near-instantaneous. No dispatching, no interpretation — just a pattern match against the fact base.

A conjunctive query expands into a conjunction of cache calls:

```
query:search([name(neovim), category('app-editors')], R://E).
```

becomes:

```
cache:ordered_entry(R, E, 'app-editors', neovim, _).
```

The payoff shows up at scale: **sub-millisecond** query behavior across **tens of thousands** of entries (on the order of 32,000+ in a typical Portage tree), because the hot queries are specialised at compile time.

6.7.2 `query:search` — the main query predicate

`query:search/2` is the primary interface for querying the knowledge base. Its first argument describes what to search for; its second argument binds the matching `Repository://Entry`:

```
query:search(name(neovim), R://E).
query:search([category('dev-libs'), name(openssl)], R://E).
query:search(description(D), portage://'app-editors/neovim-0.12.0').
```

The following search terms are supported:

Search term	Matches
`name(Name)`	Package name
`category(Cat)`	Package category
`entry(Id)`	Full entry atom (`'category/name-version'`)
`repository(Repo)`	Repository atom
`version(Ver)`	Exact version term
`slot(Slot)`	Slot value
`subslot(Sub)`	Sub-slot value
`keyword(KW)`	Architecture keyword
`description(D)`	Package description
`eapi(E)`	EAPI version
`license(L)`	License
`homepage(H)`	Homepage URL
`maintainer(M)`	Package maintainer
`eclass(E)`	Inherited eclass
`iuse(Flag)`	USE flag declared in IUSE
`masked(true/false)`	Whether the package is masked

Search terms can be combined as a list for conjunctive queries. The `all(...)` wrapper collects all matching values, and `latest(...)` returns only the first (highest-version) match.

6.7.3 `query:select` — version and metadata comparison

For queries that need comparison operators (not just equality), portage-ng uses a `select(Key, Comparator, Value)` term inside `search`:

```
query:search(select(version, greaterequal, Ver), R://E).
query:search(select(slot, equal, '3'), R://E).
query:search(select(keyword, wildcard, 'amd*'), R://E).
```

For version comparisons, the `select` clauses expand at compile time into direct `cache:ordered_entry` lookups combined with `eapi:version_compare/3`:

Comparator	Meaning
`equal`	Exact version match
`smaller`	Version strictly less than
`greater`	Version strictly greater than
`smallerequal`	Less than or equal
`greaterequal`	Greater than or equal
`notequal`	Not equal
`wildcard`	Wildcard match (e.g. `3.0*`)
`tilde`	Fuzzy matching (same base version, any revision)

For non-version keys, `select` falls through to `cache:entry_metadata/4` lookups with the appropriate comparison. This keeps the version hot path — which is exercised thousands of times during candidate selection — fully indexed and compiled.

6.7.4 `model(...)` queries — dependency model construction

Beyond simple metadata lookups, `query:search/2` also supports **model queries** that compute derived structures from the raw cache data. The most important of these constructs the **grouped dependency model** for an ebuild:

```
query:search(model(dependency(Merged, install)):config?{Ctx}, R://E).
query:search(model(dependency(Merged, run)):config?{Ctx}, R://E).
query:search(model(dependency(Merged, pdepend)):config?{Ctx}, R://E).
query:search(model(dependency(Merged, fetchonly)):config?{Ctx}, R://E).
```

Each of these is expanded at compile time into a sequence of:

1. **Self-context injection** — ensures `self(R://E)` is present in `Ctx`, so downstream rules can identify circular self-dependencies.
2. **`findall` over raw dependency metadata** — collects all `cache:entry_metadata/4` facts for the relevant dependency keys (BDEPEND, CDEPEND, DEPEND, IDEPEND, RDEPEND, and/or PDEPEND depending on the phase).
3. **`prover:prove_model/6`** — evaluates USE-conditional branches in the dependency specification, producing an AVL model of active dependency literals.
4. **`group_dependencies/2`** — groups the flat dependency list by category/name/slot/phase, producing the `grouped_package_dependency` terms that the rules layer consumes.

The result `Merged` is a list of grouped dependency terms ready for resolution by the rules layer (see Chapter 11).

Why not cached? Model construction depends on mutable proof state beyond the explicit context argument: `build_with_use` varies per dependency path, `prover:assuming` flags change between fallback attempts, and `memo:selected_cn_snap_` evolves during the proof. A cache was attempted but removed because it could not produce a sound cache key that captured all of these dimensions. See the comment at the top of `Source/Knowledge/query.pl` for the full rationale.

6.8 Further reading

- Chapter 7: The EAPI Grammar — how md5-cache files are parsed into cache predicates
- Chapter 3: Configuration — repository path setup
- Chapter 8: The Prover — how the prover queries the knowledge base

Chapter 7

The EAPI Grammar

The Gentoo **Package Manager Specification** (PMS) defines a dependency language that is easy to underestimate on first reading. A single atom can carry a version comparator, a category and package name, a Gentoo version string, slot and sub-slot operators, USE restrictions, and —wrapped around lists of atoms—USE conditionals, choice groups, and blockers. Traditional Portage implements this surface syntax with an ad hoc parser written in Python.

portage-ng uses Prolog's built-in **Definite Clause Grammar** (DCG) notation to encode the same language directly (`Source/Domain/Gentoo/eapi.pl`). The insight is simple: PMS dependency syntax *is* a grammar, so it should be expressed as one. DCG rules describe that grammar directly; the parser is the grammar, not a separate layer that tries to stay in sync with a prose spec. What PMS says and what the executable parser accepts are one artifact—the rules in `eapi.pl`—rather than a specification document drifting away from a pile of regexes and special cases.

The grammar fully implements PMS 9 / EAPI 9.

7.1 What gets parsed

The EAPI grammar is exercised whenever md5-cache metadata is loaded. Each file under the Portage tree's `metadata/md5-cache/` directory holds **one ebuild's** worth of metadata as a flat list of lines, each line a single `KEY=VALUE` pair (PMS 9, §14.3). A typical fragment looks like this:

```
BDEPEND=>=dev-build/cmake-3.16
DEFINED_PHASES=compile configure install prepare test
DEPEND=dev-libs/openssl:= dev-libs/libffi:=
EAPI=8
IUSE=debug doc test
KEYWORDS=~amd64 ~arm64
RDEPEND=dev-libs/openssl:= >=dev-lang/python-3.10[ssl,threads]
REQUIRED_USE=|| ( python_targets_python3_11 python_targets_python3_12 )
SLOT=0
```

The DCG is responsible for turning the *values* of dependency-related keys into structured terms: dependency strings (`DEPEND`, `BDEPEND`, `RDEPEND`, `PDEPEND`, `IDEPEND`), USE-conditional groups, version operators, slot operators, USE dependencies, and `REQUIRED_USE` constraints. Other keys (`EAPI`, `SLOT`, `KEYWORDS`, `DESCRIPTION`, …) use smaller, dedicated value rules in the same module —still DCG-driven, but without the full dependency expression machinery.

7.2 A worked example: one dependency atom

Consider a single atom as it might appear in `RDEPEND` or `DEPEND`:

```
>=dev-libs/openssl-3.0:0/3=[ssl,-test]
```

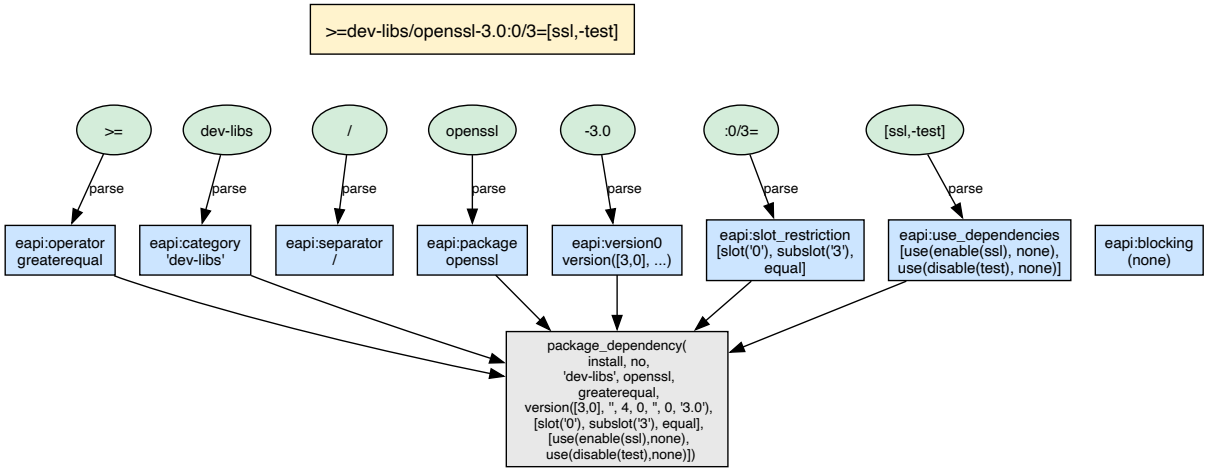


Figure 17 — Dependency atom anatomy

The core DCG rule for a package atom is `eapi:package_dependency/3` in `eapi.pl`. Conceptually it composes the way PMS §8.3 suggests reading the text: optional blocker, optional comparator, `category/package`, optional version, optional slot restriction, optional USE dependency list, with a small helper to merge "`=` + wildcard" into the dedicated `wildcard` operator:

```
eapi:package_dependency(T, _R://_E, Output) -->
  eapi:blocking(B),                                    % optional
  eapi:operator(O),                                    % optional
  eapi:category(C), eapi:separator, !, eapi:package(P),  % required
  eapi:version0(V, W),                                 % optional
  eapi:slot_restriction(S),                            % optional
  eapi:use_dependencies(U),                            % optional
  { eapi:select_operator(O, W, Op),
    Output = package_dependency(T, B, C, P, Op, V, S, U) }.
```

7.2.1 Matching each piece

1. **`blocking`** — No `!` or `!!` prefix, so this clause leaves the blocking marker as "none" (`no` in the concrete term).
2. **`operator`** — The leading `>=` matches the `greaterequal` operator.
3. **`category`, `/`, `package`** — Consumes `dev-libs`, the slash, and `openssl`.
4. **`version0`** — After the hyphen, parses `3.0` into a `version/7` term (and records that there is no `=*`-style wildcard suffix on this atom).
5. **`slot_restriction`** — The `:0/3=` fragment becomes a list describing the main slot, sub-slot, and trailing `=` (rebuild-on-slot change semantics).
6. **`use_dependencies`** — Bracket contents parse as a comma-separated list: `ssl` as an enable requirement, `-test` as a disable requirement.
7. **`select_operator`** — With no wildcard, the final operator remains `greaterequal`.

7.2.2 Intermediate state and difference lists

Operationally, each DCG goal is expanded into an ordinary Prolog predicate with two extra arguments: the **current suffix** of the input code list and the **remaining suffix** after the rule succeeds. That is the standard DCG **difference-list** threading: parsing advances by shortening the difference between "input seen so far" and "still to read."

You can read the parse as a sequence of **remaining input** snapshots (conceptual, not a separate data structure the code prints):

After this part succeeds	Remaining input (conceptually)
(start)	`>=dev-libs/openssl-3.0:0/3=[ssl,-test]`
`operator`	`dev-libs/openssl-3.0:0/3=[ssl,-test]`
`category` + `/` + `package`	`-3.0:0/3=[ssl,-test]`
`version0`	`:0/3=[ssl,-test]`
`slot_restriction`	`[ssl,-test]`
`use_dependencies`	(empty — parse succeeds)

Calling `phrase(Rule, Codes)` wraps that pattern: it requires the rule to consume all of `Codes` (or you supply an explicit remainder). There is no hand-maintained cursor variable in application code—the DCG expansion supplies it.

7.2.3 Final term

For the install-time dependency role (`package_d` in the grammar), parsing the string above yields a `package_dependency/8` term of this shape (as produced by the running grammar; minor formatting may vary):

```
package_dependency(
  install,
  no,
  'dev-libs',
  openssl,
  greaterequal,
  version([3, 0], '', 4, 0, '', 0, '3.0'),
  [slot('0'), subslot('3'), equal],
  [use(enable(ssl), none), use(disable(test), none)])
```

The first argument (`install` / `run` / `compile`) records *which* PMS dependency class is being parsed (`DEPEND` vs `RDEPEND` vs `BDEPEND`), so the same DCG surface syntax feeds slightly different typing in the abstract syntax.

7.3 Why DCG instead of regex or ad hoc code?

The dependency language defined by PMS is recursive: USE conditionals contain dependency lists, which themselves contain atoms, which may carry nested USE restrictions. A regex or hand-coded string scanner can handle flat patterns, but recursive structure calls for a recursive

formalism. Prolog's DCG notation is exactly that formalism, and it brings several practical advantages.

Composition. The package-atom rule is assembled from small, self-contained nonterminals — `blocking`, `operator`, `category`, `separator`, `package`, `version0`, `slot_restriction`, and `use_dependencies` — each of which can be understood and tested in isolation:

```
dep_atom --> blocking, operator, category, '/', package, version_suffix, slot_suffix,
use_deps.
```

Local testing. Every nonterminal is an ordinary Prolog predicate, so you can call `phrase/2` on a single rule without loading the cache or the full pipeline.

Free recursion. USE conditionals and nested choice groups use the same mechanism as every other rule: `eapi:dependencies//3` recurses through lists of `eapi:dependency//3` alternatives, so nested `flag? ( … )` structures need no separate stack machine.

Failure locality. When parsing fails, the failure occurs at a named nonterminal — a clear indication of which part of the input was unexpected, rather than a terse "pattern did not match" from a regex engine.

Graceful evolution. New PMS features tend to introduce new alternatives or new rules (additional operators, value forms, EAPI-9 extensions), not a rewrite of central control flow. Adding a rule to a DCG is a one-clause change; the equivalent in a Python parser is typically a new branch in an `if eapi >= …` ladder spread across several functions.

7.4 DCG grammar design

The grammar is implemented in `Source/Domain/Gentoo/eapi.pl` as a set of DCG rules. DCGs are a natural fit for dependency specifications because:

- Dependency atoms have recursive structure (USE conditionals nest).
- The grammar is context-free at the level PMS defines.
- DCG rules compose as Prolog predicates, so the parser **constructs** Prolog terms while it reads text.

7.4.1 Dependency atoms

The table below summarizes how the surface syntax maps into fields of the abstract atom (illustrated with the same running example):

Component	Example	Parsed as
Version operator	`>=`	Comparator atom (e.g. `greaterequal`)
Category	`dev-libs`	Atom
Name	`openssl`	Atom
Version	`3.0`	`version/7` term
Slot operator	`:0/3=`	Slot + sub-slot + rebuild flag
USE deps	`[ssl,-test]`	Enable/disable (and related) wrappers

7.4.2 USE conditionals

USE-conditional dependency groups use the syntax:

```
flag? ( deps... )      — include deps if flag is enabled
!flag? ( deps... )     — include deps if flag is disabled
```

These are parsed into conditional terms that the rules layer evaluates against the USE model during proof construction.

7.4.3 Choice groups

PMS defines three choice operators for REQUIRED_USE and dependency specs:

- `||` **(any-of)** — `|| ( a b c )` — at least one of the listed items must be satisfied
- `^^` **(exactly-one-of)** — `^^ ( a b c )` — exactly one must be satisfied
- `??` **(at-most-one-of)** — `?? ( a b c )` — at most one may be satisfied

7.5 Reader/parser pipeline

Loading md5-cache is a small pipeline with clear separation of concerns.

The **repository** side (`Source/Knowledge/repository.pl`) knows where the cache lives: under each tree's `metadata/md5-cache/` directory, with one file per `category/package-version` entry (`repository:get_cache_file/2` resolves entry → path). Sync and incremental updates decide *which* entries need work; for each entry that must be read, the repository opens the flat cache file.

`Source/Pipeline/reader.pl` does one job: given a path (or stream), it reads the file line by line into a list of strings—each string is still a raw `KEY=VALUE` line, unchanged.

`Source/Pipeline/parser.pl` walks that list. For each line it converts the string to character codes and runs `phrase(eapi:keyvalue(metadata, …), Codes)`. That single DCG entry point dispatches on the key: dependency keys delegate to the full dependency grammar (`DEPEND`, `BDEPEND`, `RDEPEND`, `PDEPEND`, `IDEPEND`, `REQUIRED_USE`, …); non-dependency keys use lighter value rules (`EAPI`, `SLOT`, `KEYWORDS`, `IUSE`, …).

So the data flow is:

```
md5-cache files  →  reader.pl (lines)  →  parser.pl  →  eapi.pl (DCG)  →  cache
predicates
```

1. **`reader.pl`** reads each md5-cache file into a list of lines (one key/value pair per line).
2. **`parser.pl`** parses every line through `eapi:keyvalue/3`, which routes values to the appropriate DCG subtree.
3. **`eapi.pl`** builds structured Prolog terms (e.g. `depend(D)`, `rdepend(D)`, `slot(S)`, `eapi(E)`).
4. The results are asserted as `cache:entry/5` and related predicates, populating the knowledge base.

The reader supports incremental loading — only new or changed files need to be re-parsed when using `--regen`.

7.6 Parsed output

After parsing, each ebuild is represented by a set of cache predicates. The dependency model for an ebuild is a list of `package_dependency/8` terms:

```
package_dependency(DepType, Blocking, Category, Name, Operator, Version,
                   SlotInfo, UseInfo)
```

These terms are consumed by the rules layer during proof construction. The EAPI grammar handles all PMS 9 / EAPI 9 constructs:

- Version operators: `=`, `>=`, `<=`, `>`, `<`, `~`, `=*` (wildcard)
- Slot operators: `:SLOT`, `:SLOT/SUBSLOT`, `:*`, `:=`
- USE dependencies: `[flag]`, `[-flag]`, `[flag=]`, `[!flag=]`, `[flag(+)]`, `[flag(-)]`
- Blockers: `!cat/pkg` (weak), `!!cat/pkg` (strong)
- All-of groups (implicit conjunction)
- Any-of groups (`|| ( ... )`)
- USE conditionals (`flag? ( ... )`, `!flag? ( ... )`)

7.7 Further reading

- Chapter 6: Knowledge Base and Cache — how parsed data is stored
- Chapter 11: Rules and Domain Logic — how dependency terms are consumed during proof construction
- Chapter 22: Dependency Ordering — PMS dependency type semantics

Chapter 8

The Prover

8.1 Domain independence

The central design insight is easy to miss because portage-ng is *about* Gentoo packages: **the prover does not know what a "package" is.** It works only with abstract literals and rules (Logic). That is deliberate. It means the reasoning core can be exercised and tested without importing the whole Portage domain, and the same engine could — in principle — prove goals in any domain that encodes its constraints as Horn-style expansions behind a single hook.

The prover's only contract with the outside world is this: **given a literal, `rules:rule/2` (or the configured `rule/2` delegate) returns a body — a list of sub-literals that must hold for the head to hold.** Everything that makes Gentoo "Gentoo" — USE flags, slots, version domains, PDEPEND side effects — lives in the rule layer and in proof-term annotations (`?{Context}`), not in the prover's control flow. The prover walks literals; the domain explains what each literal means.

That separation is what keeps the implementation in `Source/Pipeline/prover.pl` readable: backward chaining, cycle handling, context merging, and bookkeeping — not emerge policy.

The prover is the core reasoning engine of portage-ng. Given a list of target literals, it constructs a formal proof that all dependencies can be satisfied — or completes with explicit assumptions documenting exactly where the dependency specification is unsatisfiable.

8.2 Why AVL trees?

The prover maintains its main state in **four AVL trees** from `library(assoc)` (Proof, Model, Constraints, and Triggers — see the module header in `prover.pl`). Plain hash tables would win on raw point lookups, but assoc trees buy a property that matters more here: they are **persistent** (functional). Each `put_assoc/4` produces a *new* tree and leaves the previous one intact.

In Prolog, that lines up with **backtracking**. When the prover must undo a choice, variable bindings revert and the "old" assoc values bound in earlier choice points are still the right snapshots. A mutable hash map would need an explicit save/restore discipline on every failure — the sort of manual undo-stack work traditional Portage does in places, with corresponding risk of subtle inconsistency. Here, the data structures and Prolog's search rule stay aligned.

Complexity is **O(log n)** per update and lookup. For on the order of tens of thousands of literals, that is a small constant number of comparisons (roughly fifteen for 32,000 entries) — more than fast enough compared with the cost of calling into domain rules and unification.

8.3 Inductive proof search

The prover performs inductive proof search via backward chaining. For each literal in the proof queue:

1. **Check the model.** If the literal is already proven (present in the Model AVL), merge contexts via feature term unification and continue.
2. **Check the cycle stack.** If the literal is currently being proved (on the stack), handle the cycle:
   - If `heuristic:cycle_benign/1` succeeds, treat it as already proven (benign cycle — no assumption recorded).
   - Otherwise, record a cycle-break assumption (`assumed(rule(Lit))` in Proof, `assumed(Lit)` in Model).
3. **Expand via `rule/2`.** Call `rules:rule(Lit, Body)` to get the rule body — the list of sub-literals that must be proven to justify `Lit`.
4. **Record in Proof.** Store `rule(Lit) → dep(N, Body)?Ctx` in the Proof AVL, where `N` is the dependency count.
5. **Record in Model.** Store `Lit → Ctx` in the Model AVL.
6. **Update Triggers.** For each body literal, add `Lit` to its trigger list in the Triggers AVL.
7. **Recurse.** Add the body literals to the proof queue and continue.

Steps 1 and 6 are where **prescient proving** and the **reverse-dependency index** connect; the sections below unpack those ideas.

8.4 Proof term structure

The Proof AVL maps rule keys to structured values:

```
rule(Lit) → dep(DepCount, Body)?Ctx
```

Field	Meaning
`rule(Lit)`	The literal that was proven
`DepCount`	Number of dependencies (body length)
`Body`	List of body literals
`Ctx`	Context under which the literal was proven

The dependency count is stored alongside the body because it is used by downstream stages without having to recompute it. The planner uses it to determine the **fan-out** of each node when building topological waves: a literal with many dependencies is heavier to schedule than one with few. The printer uses it to produce indentation and step counts in the plan output. Storing the count once, at proof time, avoids repeated `length/2` calls over the same body list during planning and printing.

Special keys: - `assumed(rule(Lit))` with `dep(-1, Body)?Ctx` — prover cycle-break - `rule(assumed(Lit))` with `dep(0, [])?Ctx` — domain assumption

8.4.1 Concrete Proof AVL entry

The literal itself is still just a term the prover passes through; the `portage://` prefix and atom naming are domain choices. A representative Proof entry after expanding an install goal might look like this (body shortened):

```
rule(portage://'sys-apps/portage-3.0.77-r3':install)
  → dep(5,
        [ portage://'dev-lang/python-3.12.3':install,
          portage://'sys-libs/glibc-2.40-r5':install,
          … ])?{[ self(portage://'sys-apps/portage-3.0.77-r3'),
                … ]}
```

So the Proof AVL answers: **which rule instance was used**, **how many dependencies** it had, **what the body literals are**, and **under which `?{Context}` list** that expansion was valid. The exact features inside `?{…}` are documented in Chapter 5: Proof Literals; the prover treats them as data merged by feature term unification, not as special cases.

8.5 Model construction

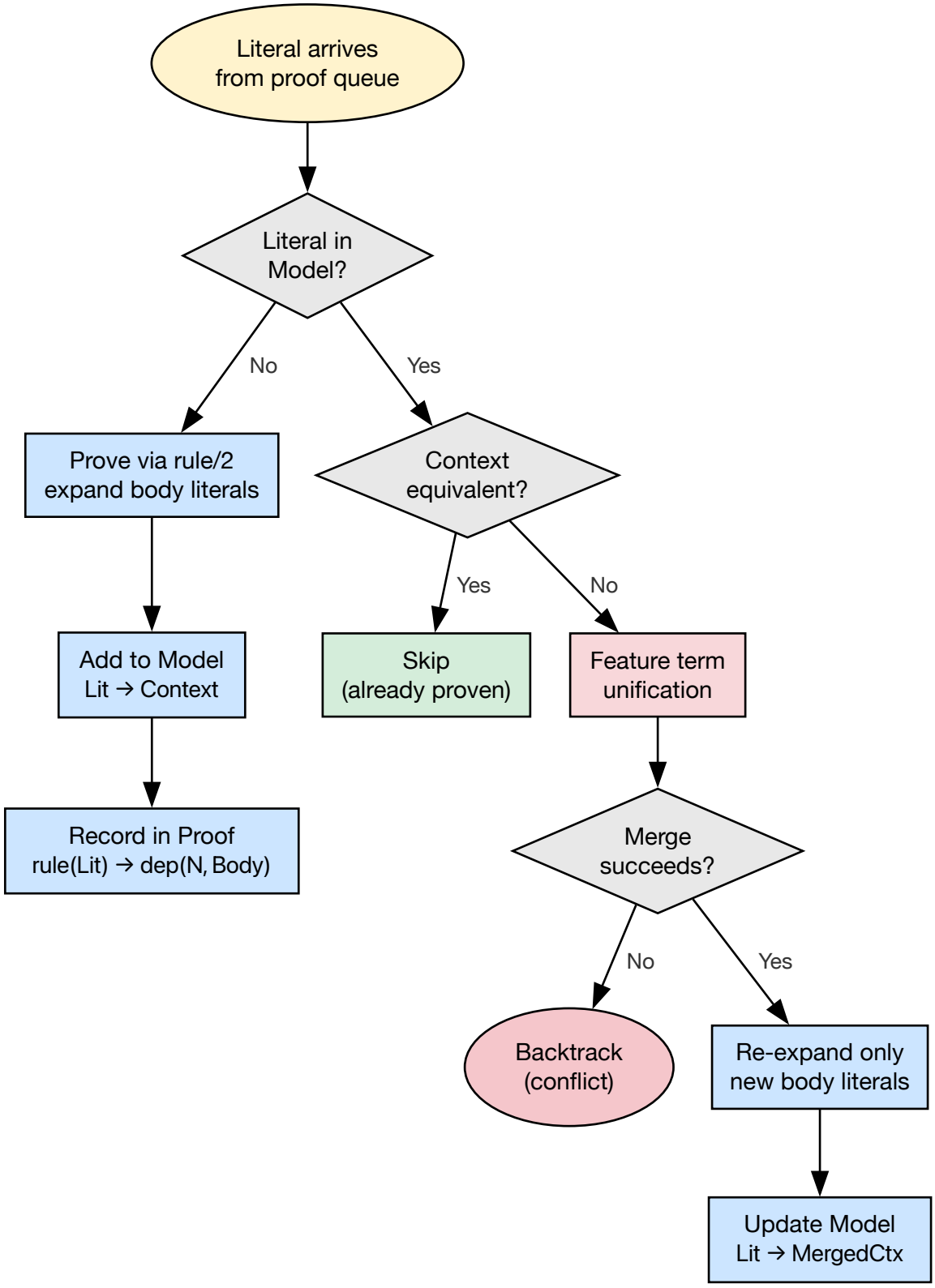


Figure 18 — Model construction flow

The Model AVL records every proven literal with its context. It serves two purposes:

1. **Memoization.** When a literal is encountered again, the prover checks the model first. If found, it merges the new context with the existing one via feature term unification rather than re-proving the literal (when the incoming context is not already equivalent to the stored one — see below).
2. **Plan generation.** The planner reads the model to determine which literals are in the proof and what contexts they carry.

A lightweight variant, `prove_model`, skips Proof and Triggers bookkeeping for internal query-side model construction where only the model is needed.

8.5.1 Concrete Model AVL entry

Model entries are simpler: **literal → context** under which it was last committed to the proof.

```
portage://'dev-libs/openssl-3.3.2':install
  → [ build_with_use:use_state([ssl], []),
      … ]
```

Multiple features can accumulate in that list as different dependency paths impose different requirements; the merge semantics are defined by the domain's feature term unification (`sampler:ctx_union/3`).

8.5.2 Re-encountering a literal: feature term unification

When the queue delivers the same `Lit` again with a **new** `?{Context}`:

1. The prover finds `Lit` in the Model AVL (with stored context `OldCtx`).
2. If the new context is semantically the same as the stored one (`prover:proven/3`), nothing more is done — no second expansion.
3. Otherwise it merges contexts via feature term unification (`sampler:ctx_union/3`). If the merge fails (e.g. conflicting USE enable/disable sets), the goal fails and ordinary Prolog backtracking retracts the choice that led to the clash.
4. If the merge succeeds, the prover may **re-call** `rule/2` on a canonical literal carrying `MergedCtx`, **subtract** the previously proven body from the new body, and prove only the **difference** — updating Proof, Model, and Triggers incrementally and storing `Lit → MergedCtx` in the Model.

So "seen before" does not mean "frozen forever"; it means **accumulate constraints and re-expand only what new information demands.**

8.6 Prescient proving

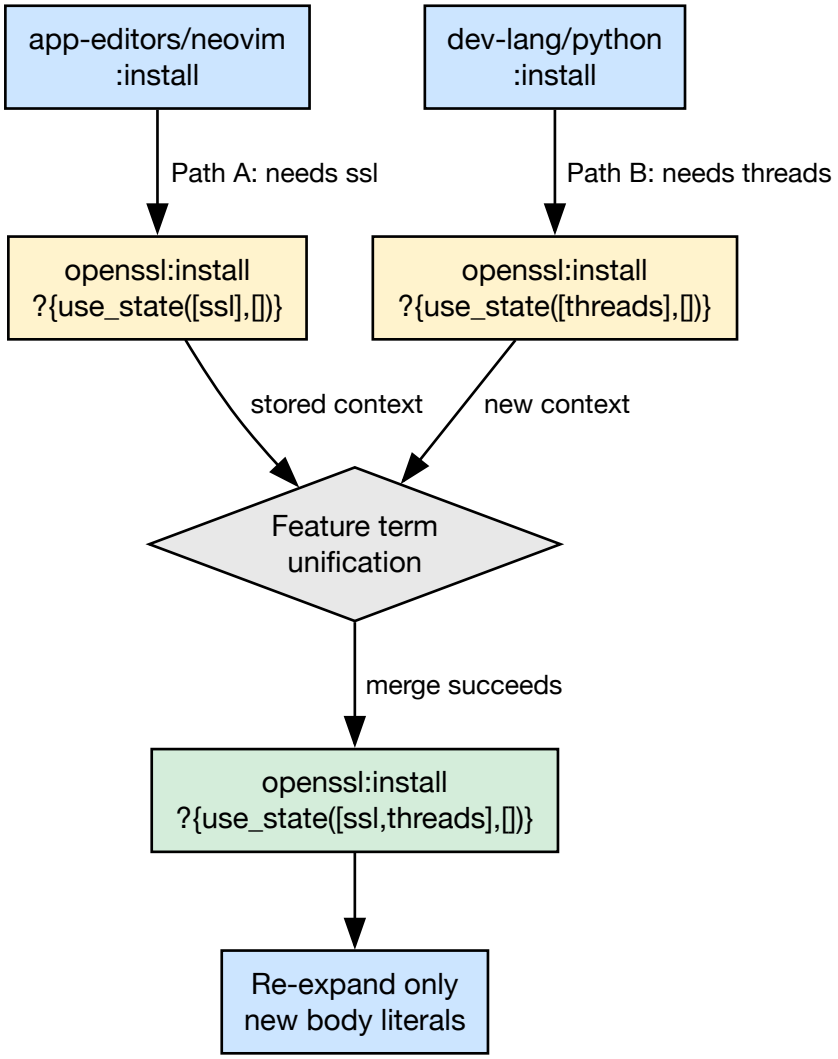


Figure 19 — Prescient proving

When a literal is re-encountered with a changed context (e.g. new USE requirements from a different dependency path), the prover merges contexts via feature-unification and re-expands only the difference. This is called **prescient proving** because knowledge about constraints imposed later in the proof is incorporated into earlier decisions **without** unwinding the whole branch and starting the literal from scratch.

8.6.1 Walkthrough: two paths into `dev-libs/openssl`

Imagine two dependency paths that both need `dev-libs/openssl:install`:

- Path A pulls it in with **USE** `ssl` required in the build set.
- Path B pulls it in with **USE** `threads` required.

Without prescient-style merging, a naive story would be: prove openssl once under path A's context; later, when path B arrives with incompatible or extra requirements, discover that the

earlier proof was too weak and **backtrack far enough to re-prove** openssl under a wider or corrected context — repeating work and thrashing the search.

With prescient proving, the second encounter does not throw away the first. The prover merges the proof-term contexts:

```
First encounter:   openssl:install ?{use_state([ssl],       [])}
Second encounter:  openssl:install ?{use_state([threads],   [])}
After unification: openssl:install ?{use_state([ssl,threads],[])}
```

The merged context commits openssl to satisfying **both** paths at once. The prover then checks whether this wider context is still consistent with every constraint the rules attach to that literal — profile masks, `REQUIRED_USE`, version domains, and so on.

- If the check **succeeds**, no full re-proof is needed. The prover re-expands the rule under the merged context and proves only the **new** body literals that the wider context introduces beyond what was already established.
- If the check **fails** (for example, contradictory flags — a USE flag required both enabled and disabled), the merge is rejected and the prover **backtracks** to try another candidate.

That is the sense in which the prover is "prescient": **later requirements are folded into the context of an earlier proof step** through merging and targeted re-expansion, rather than discovering the conflict only after committing to a too-narrow past choice.

8.7 Triggers and the reverse-dependency index

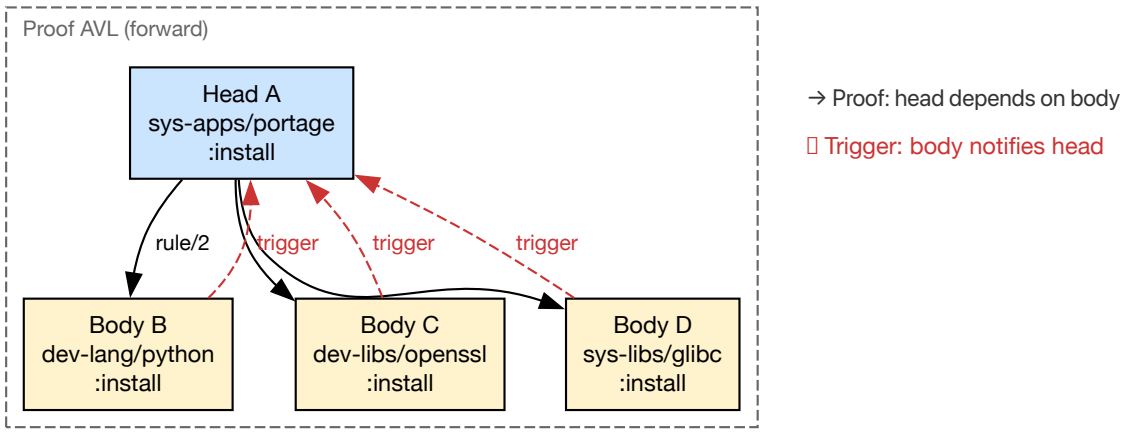


Figure 20 — Triggers reverse-dependency index

The Triggers AVL is the piece that makes prescient updates **addressable**: it records, for each body literal, **which rule heads depend on it**.

8.7.1 Construction

When the prover proves head literal **A** with body **[B, C, D]**, it extends the Triggers AVL with three reverse edges:

```
Proof (forward):     rule(A) → dep(3, [B, C, D])

Triggers (reverse):  B → [A, …]
                     C → [A, …]
                     D → [A, …]
```

In general, for a rule `rule(H, Body)` where `Body = [L₁, L₂, …, Lₙ]`, the operation `add_triggers/4` inserts:

> For each $L_i \in$ Body: Triggers[L_i] := [H | Triggers[L_i]]

Each time a rule is recorded or re-recorded (including after a prescient merge), these reverse edges are extended so the index stays consistent with the current bodies.

8.7.2 Concrete example

Suppose the prover establishes:

```
rule(sys-apps/portage:install) → dep(3, [dev-lang/python:install,
                                         dev-libs/openssl:install,
                                         sys-libs/glibc:install])
```

The Triggers AVL then contains:

```
dev-lang/python:install  → [sys-apps/portage:install, …]
dev-libs/openssl:install → [sys-apps/portage:install, …]
sys-libs/glibc:install   → [sys-apps/portage:install, …]
```

If openssl's context later changes (prescient merge adds a new USE flag), the prover can look up `Triggers[dev-libs/openssl:install]` in O(log n) time and immediately find that `sys-apps/portage:install` needs to be revisited.

8.7.3 Forward vs reverse lookup

The Proof and Triggers AVLs are **duals** of each other:

Direction	AVL	Lookup	Answers
Forward	Proof	`rule(H) → dep(N, Body)`	What does H depend on?
Reverse	Triggers	`L → [H₁, H₂, …]`	Who depends on L?

Together they form a **bidirectional dependency graph**: Proof walks forward from heads to bodies; Triggers walks backward from bodies to heads.

8.7.4 Downstream use

Later phases — planners, schedulers, diagnostics — use Triggers to answer **"if this literal moves, what else moves?"** in logarithmic time per lookup. Without these reverse edges,

nothing in the pipeline could enumerate which install heads depended on a given dependency literal, and the graph carried out of proving would be incomplete for anything that walks dependencies backwards.

8.8 Entry rules and the pipeline

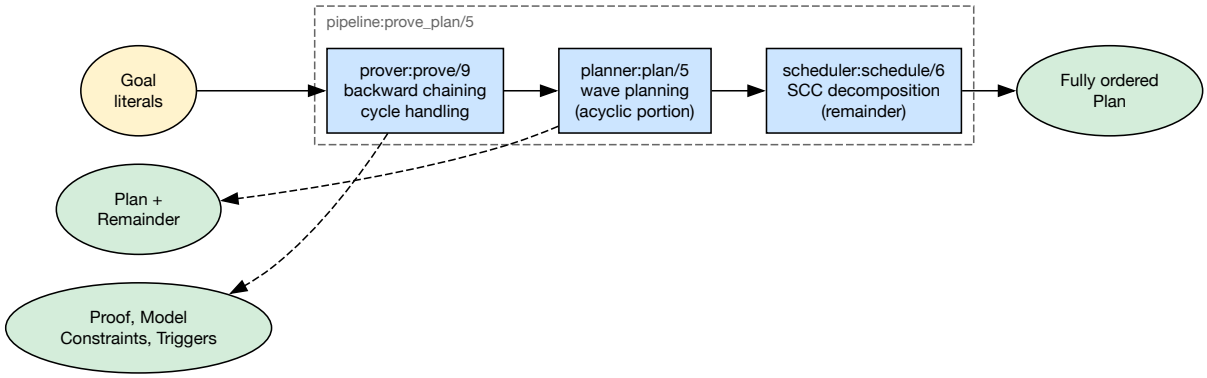


Figure 21 — prove_plan pipeline

The pipeline module provides two canonical entry points, both with the same 5-tier committed-choice progressive relaxation (strict, keyword_acceptance, blockers, unmask, keyword_unmask):

- `pipeline:prove_plan_with_fallback/5` — full pipeline (prove + plan
  - schedule). Used by production paths (`--pretend`, `--graph`, `--build`) and `pipeline:test_stats`.
- `pipeline:prove_with_fallback/4` — prover only. Used by layered tests (`prover:test`, `planner:test`, `scheduler:test`) and `--bugs`. Each test layer adds its own stages on top.

Underneath, `prove_plan_basic/5` chains three stages:

```
pipeline:prove_plan_basic(Goals, ProofAVL, ModelAVL, Plan, TriggersAVL)
```

1. `prover:prove/9` — constructs Proof, Model, Constraints, and Triggers
2. `planner:plan/5` — wave planning for the acyclic portion
3. `scheduler:schedule/6` — SCC/merge-set scheduling for the remainder

The prover is wrapped in `with_reprove_state` which saves and restores the learned constraint store across reprove retries. Inside that, `prove_with_retries` catches `prover_reprove` exceptions and restarts up to `reprove_max_retries` times (default 3).

See Chapter 9: Assumptions and Constraint Learning for the reprove mechanism in detail.

8.9 Multiple stable models

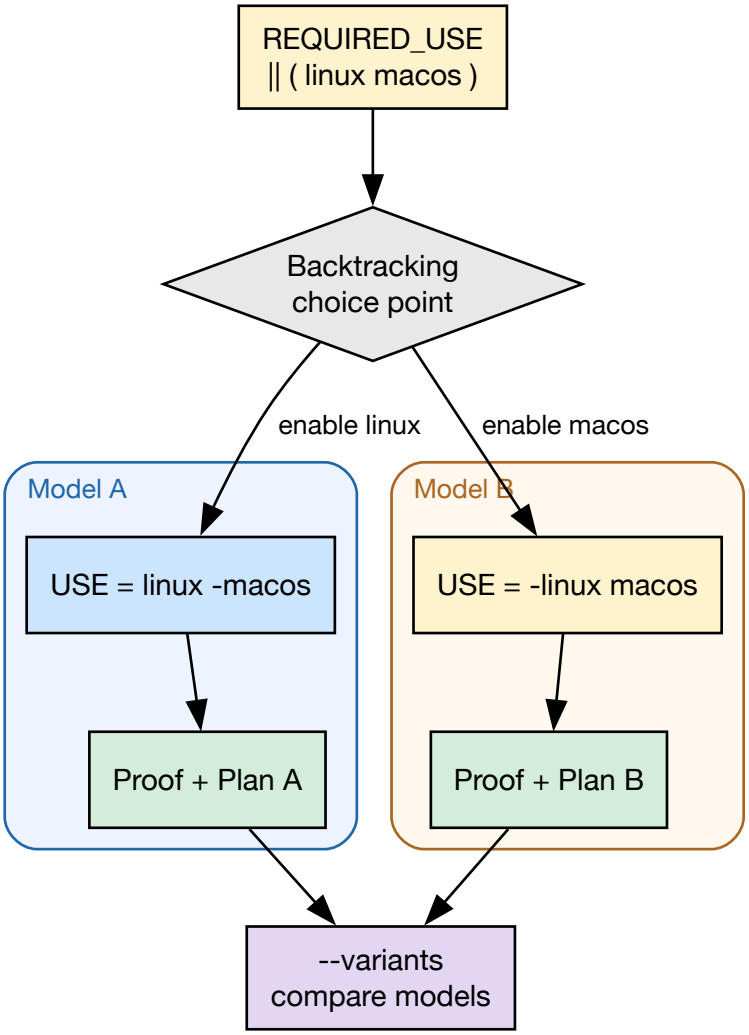


Figure 22 — Multiple stable models

The prover can produce different solutions (stable models) of the USE flag configuration space. Using Prolog's built-in backtracking, different valid configurations of the same target can be explored and compared.

For example, a `REQUIRED_USE="|| ( linux macos )"` constraint yields two stable models:

```
Model A:  USE="linux -macos"     Model B:  USE="-linux macos"
```

The `--variants` CLI option enables this mode, running the prover with different USE flag configurations via `variant:use_override` and `variant:branch_prefer`.

8.10 Proof obligations

After a literal is proven, the prover queries the domain for additional proof obligations via `heuristic:proof_obligation/4`. This lets the domain inject derived obligations — extra literals

to be appended to the proof queue — without the prover understanding domain-specific semantics.

PDEPEND dependencies are handled this way: they are discovered only after a literal is resolved and are injected as proof obligations via `rules:literal_hook/4`.

8.11 Further reading

- Chapter 9: Assumptions and Constraint Learning — the reprove mechanism and constraint learning
- Chapter 5: Proof Literals — the literal format
- Chapter 11: Rules and Domain Logic — how `rule/2` works
- Chapter 4: Architecture Overview — the full pipeline

Chapter 9

Assumptions and Constraint Learning

9.1 Always a proof, never a dead end

Most dependency resolvers stop when they cannot satisfy a constraint: no solution, no plan, and often little more than a terse error. portage-ng takes a different stance. **It does not give up.** When every above-board alternative has been exhausted, the prover records an **assumption** — effectively: "I am proceeding *as if* this dependency could be satisfied" — and continues building the proof.

The outcome is **always** a complete plan. Either the proof is **strict** (no assumptions), or it is a proof **under assumptions**, with the unresolved fragments called out explicitly. Assumptions are not treated as opaque failures; they are **proposals**. They tell you which pieces of configuration or tree state would need to change for the same reasoning chain to become a strict proof. This is the same habit of mind as in mathematics: *"Assuming the Riemann hypothesis, we can prove …"* — the argument is valid *conditional* on the assumptions; make them true, and the condition disappears.

The sections below walk through a concrete missing-dependency example, then explain how suggestion tags turn assumptions into actionable hints. After that, the chapter documents the same mechanisms in technical detail: assumption taxonomy, reprove loop, REQUIRED_USE flow, entry rules, constraint guards, progressive relaxation, assumption printing, and the analogy to conflict-driven clause learning.

9.2 Worked assumption example: missing `dev-libs/foo`

Suppose the user runs:

```
portage-ng --pretend some-package
```

and some package in the graph depends on **`dev-libs/foo`**, which has **no ebuild** in any repository the knowledge base knows about.

What the prover does

1. The **grouped package dependency** rule tries to prove the dependency by enumerating candidates for category `dev-libs` and name `foo`.
2. **Search returns no entries** — there is nothing to install, so every candidate path fails.

3. After **backtracking** exhausts those paths, the **fallback chain** runs (parent narrowing, reprove with learned domains, and so on, as documented below). None of that invents a missing package.
4. The domain layer finally takes the **assumption path**: it builds a condition whose head is `assumed(grouped_package_dependency(…))`, tags the proof-term context with a reason (and optional suggestions), and the catch-all rule `rule(assumed(_), [])` lets the prover close that branch of the proof.

What appears in the Proof AVL

The proof tree stores a **domain assumption** with a key of the form `rule(assumed(Lit))`, where `Lit` is the grouped-dependency literal. For example (category and name as atoms, dependency list abbreviated):

```
rule(assumed(grouped_package_dependency('dev-libs', 'foo', …):config?{Ctx}))
    → dep(0, [])?Ctx
```

The exact `Action` (`:config`, `:install`, `:run`, …) depends on which phase of the grouped dependency is being proved; the important invariant is the **`rule(assumed(...))`** proof key (see Assumption Taxonomy).

What the user sees in the plan output

The printer classifies this as a non-existent dependency and emits a **Domain assumptions** block, along the lines of:

```
Domain assumptions: dev-libs/foo (non-existent)
```

Exit code

When any **domain** assumption is present, the CLI exit code is **`2`** (cycle-break-only assumptions alone yield **`1`**; a fully strict proof yields **`0`**).

How to read it

The message is intentionally operational: **to resolve this assumption, ensure `dev-libs/foo` is available in your repository** (overlay, third-party tree, or corrected package name). The plan is still a single coherent merge order; the assumption marks the gap between what the prover can justify from facts and what you must supply from outside.

9.3 Assumptions as actionable proposals

Many assumptions carry **`suggestion(Type, Detail)`** (and related) tags in the literal's **`?{Context}`** list. These encode **configuration changes** that would move the proof toward strictness — often the same changes the **progressive relaxation** tiers simulate when you widen `assuming/1` flags.

Typical shapes in the codebase include:

Tag (representative)	User-facing intent
`suggestion(keyword, '~amd64')`	Accept the unstable keyword — e.g. add to **`package.accept_keywords`** (in sources: `suggestion(accept_keyword, '~amd64')`)
`suggestion(unmask, …)`	Unmask the package — e.g. **`package.unmask`** (`Repo://Entry` when known)
`suggestion(use, …)`	Adjust USE flags — e.g. **`package.use`** (in sources: `suggestion(use_change, Repo://Entry, Changes)`)

When you run in modes that **apply** suggestions (see the builder's `execute_suggestion/…` hooks), those changes are **already reflected in the plan**; your job is to **review and approve** them in your real `/etc/portage` layout, or to treat the tags as a checklist for manual edits. Progressive Relaxation ties the same ideas to the `assuming` tiers (`keyword_acceptance`, blockers, `unmask`).

9.4 Overview

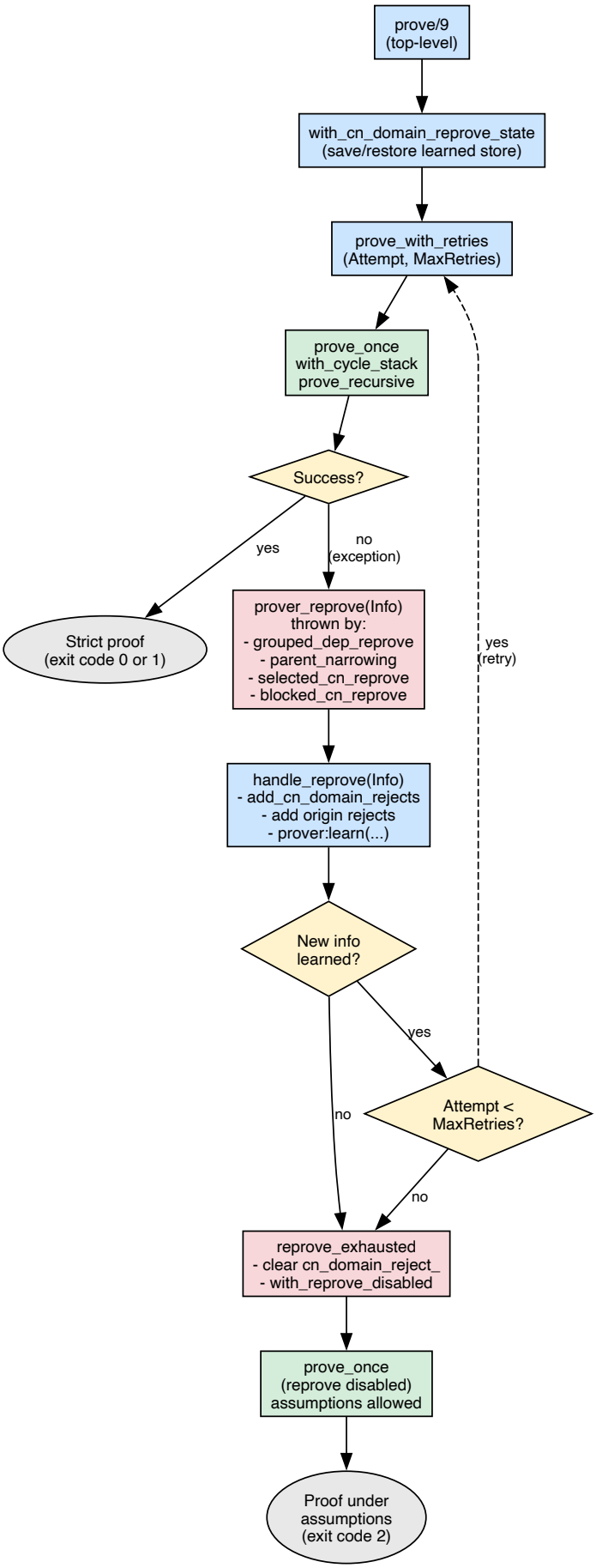


Figure 23 — Reprove mechanism flow

The portage-ng prover builds a formal proof that a set of target packages can be installed. The proof is an AVL tree mapping literals to their justifications. When part of the dependency graph cannot be satisfied, the prover records *assumptions* — lightweight markers that let the proof complete while flagging the unresolved fragment for the user.

Two fundamentally different kinds of assumptions exist, and a bounded reprove mechanism allows the prover to retry the proof with accumulated knowledge before resorting to assumptions.

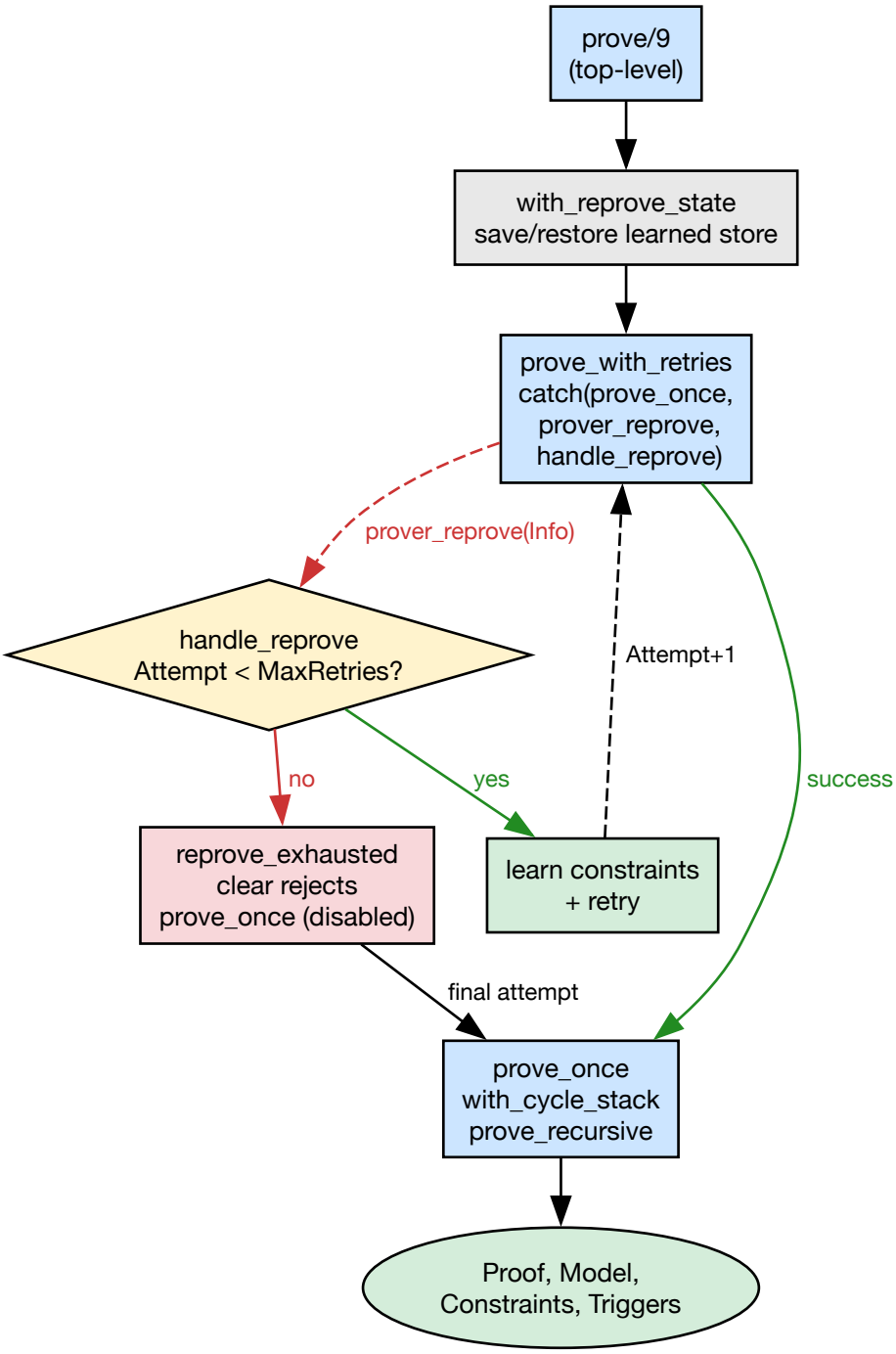


Figure 24 — Reprove loop structure

9.5 Data Structures

The prover maintains four AVL trees during proof construction:

AVL	Key → Value	Purpose
Proof	`rule(Lit)` → `dep(N, Body)?Ctx`	Which rule justified Lit
Model	`Lit` → `Ctx`	Every proven literal + context
Constraints	constraint key → value	Accumulated constraint terms
Triggers	`BodyLit` → `[HeadLit, …]`	Reverse-dependency index

9.6 Assumption Taxonomy

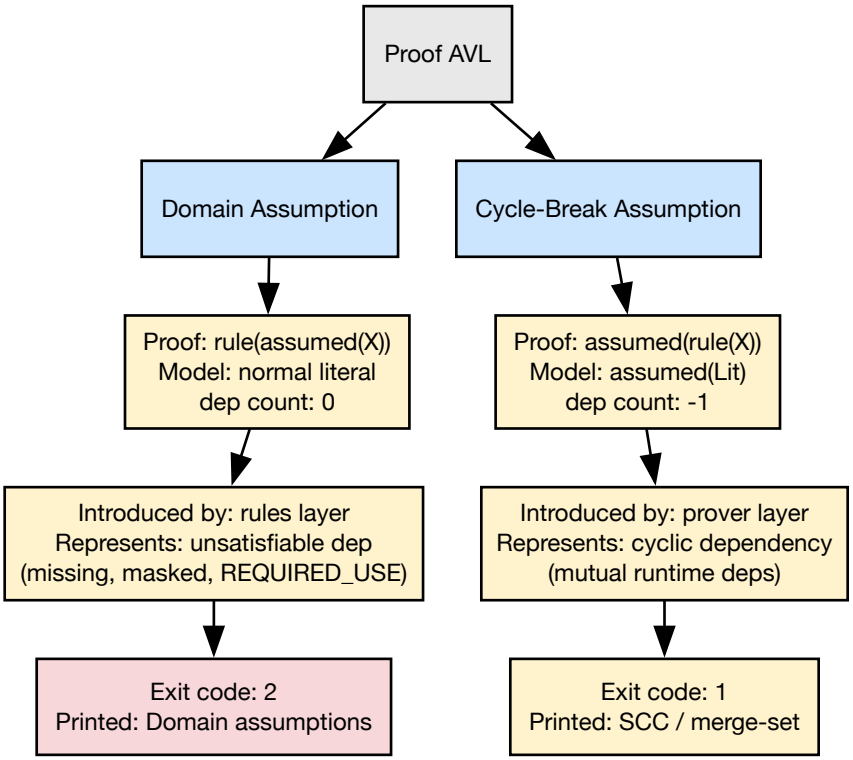


Figure 25 — Assumption taxonomy

The two kinds of assumptions are stored differently in the Proof and Model trees. Confusing them leads to wrong statistics, wrong plan output, or missed warnings.

9.6.1 Domain Assumptions (`rule(assumed(X))`)

Introduced by the **rules layer** when a dependency cannot be satisfied — for example, a package that does not exist in the tree, or a REQUIRED_USE violation that makes every candidate invalid.

How they are created:

The `grouped_package_dependency` rule exhausts all candidates (via Prolog backtracking), then the fallback chain (parent narrowing → reprove → assumption), and finally emits:

```
Conditions = [assumed(grouped_package_dependency(C,N,Deps):Action?{Ctx})]
```

The `assumed(X)` literal in the body is proved by the catch-all rule:

```
rule(assumed(_), []) :- !.
```

This stores `rule(assumed(X))` in the Proof tree.

Where they appear: - Proof: `rule(assumed(X))` → `dep(0, [])?Ctx` - Model: the enclosing literal's entry (normal) - Plan: rendered as "verify" steps + "Domain assumptions" warning block

9.6.2 Prover Cycle-Break Assumptions (`assumed(rule(X))`)

Introduced by the **prover** when it detects a cycle during proof search. If a literal is already on the cycle stack (currently being proved), the prover cannot recurse further without diverging. Instead, it records a cycle-break:

```
put_assoc(assumed(rule(Lit)), Proof, dep(-1, OldBody)?Ctx, Proof1),
put_assoc(assumed(Lit), Model, Ctx, NewModel)
```

Where they appear: - Proof: `assumed(rule(Lit))` → `dep(-1, Body)?Ctx` - Model: `assumed(Lit)` → `Ctx` - Plan: SCC / merge-set scheduling; cycle explanation via `cycle:*`

9.6.3 Summary Table

Property	Domain Assumption	Prover Cycle-Break
Proof key	`rule(assumed(X))`	`assumed(rule(X))`
Model key	(normal literal)	`assumed(Lit)`
dep count	0	−1
Introduced by	rules layer	prover layer
Represents	unsatisfiable dependency	cyclic dependency
Printed as	"Domain assumptions"	cycle break (SCC)
Exit code contribution	2	1

9.7 Reprove Mechanism

When a conflict is detected during proof search, the domain layer does not simply fail — it records what went wrong and requests a retry with refined knowledge.

9.7.1 Triggering Reprove

Several predicates can throw `prover_reprove(Info)`:

Source	When
`maybe_learn_wildcard_domain`	Wildcard dep fails and parent already narrowed or single-version; learns upper-bound `cn_domain` from wildcard
`maybe_learn_parent_narrowing`	Parent introduced a dep that made (C,N) unsatisfiable; learns to exclude parent version
`maybe_request_grouped_dep_reprove`	Effective domain conflicts with selected CN; domain inconsistent; version/slot constraints present
`selected_cn_unique_or_reprove`	CN-domain constraint conflicts with already-selected candidate (constraint guard)
`selected_cn_not_blocked_or_reprove`	Blocker detected via blocked source snapshot

Each throws `prover_reprove(cn_domain(C, N, RejectDomain, Candidates, Reasons))`.

9.7.2 Handling Reprove

When `prove_with_retries` catches a `prover_reprove(Info)` exception, it delegates to `heuristic:handle_reprove/2`, which proceeds in three steps:

1. **Record what went wrong.** The handler extracts the conflicting category, name, domain, and candidate list from `Info` and adds them to a reject set (`memo:cn_domain_reject_`). If the conflict was introduced by a specific parent, that origin is also recorded so the prover avoids the same path on the next attempt.
2. **Decide whether to retry.** If new information was actually learned (`Added = true`) and the attempt count is still below `reprove_max_retries`, the handler restarts the proof from scratch by calling `prove_with_retries` with an incremented attempt counter. The learned rejects carry over, so the prover will not repeat the same conflict.
3. **Give up gracefully.** If nothing new was learned, or the retry budget is exhausted, the handler calls `reprove_exhausted`, which **clears** the reject set so the final attempt runs unbiased. It then invokes `prove_once` with reprove **disabled** — no further `prover_reprove` exceptions can be thrown, so the proof completes with assumptions where necessary.

9.7.3 Learned Constraint Store

The `prover:learn/3` and `prover:learned/2` predicates maintain a key-value store that **persists across reprove retries** within the same top-level `prove/9` invocation. This is distinct from the reject set (which accumulates and is cleared on exhaustion).

The domain uses learned constraints for:

- **Candidate narrowing** — `grouped_dep_effective_domain` intersects the local+context domain with any learned domain.
- **Conflict learning** — constraint guards learn the domain when a conflict is detected.

- **Parent narrowing** — `maybe_learn_parent_narrowing` learns to exclude the parent version when a child dep cannot be satisfied.
- **Wildcard failure learning** — `maybe_learn_wildcard_domain` derives an upper-bound domain from a wildcard constraint (e.g. `=pkg-0.6*` → `< 0.7`) when parent narrowing alone could not resolve the conflict.

9.7.4 Retry Budget

`reprove_max_retries` defaults to 20 (configurable via `config:reprove_max_retries/1`). The final attempt runs with reprove disabled so the proof can complete with assumptions if necessary.

9.8 Use model violation flow

When a parent package forces USE flags on a dependency via bracketed USE deps (e.g. `cat/pkg[feature]`), and the dependency's `REQUIRED_USE` forbids that flag combination, the REQUIRED_USE violation mechanism ensures the prover explores alternatives before assuming.

9.8.1 Step-by-step flow

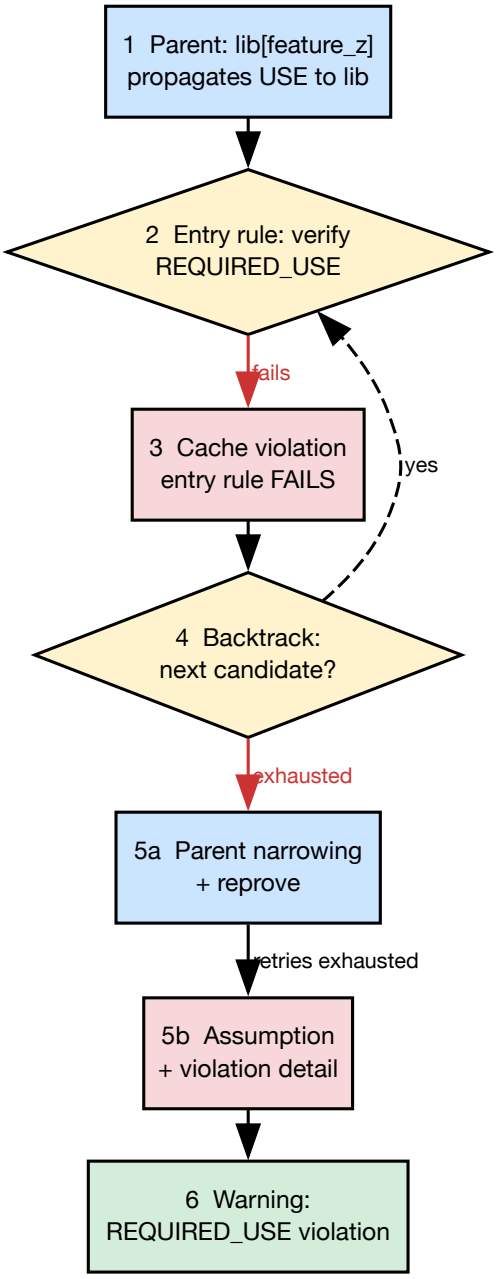


Figure 26 — Use model violation flow

The diagram above shows the six stages the prover walks through when a parent forces a USE flag that violates a dependency's REQUIRED_USE. Each step is explained below.

Step 1 — USE propagation. The parent atom `cat/app` depends on `cat/lib[feature_z]`. The bracketed flag is carried forward as a `build_with_use` context annotation, so every candidate version of `lib` will be evaluated with `feature_z` enabled.

Step 2 — Entry rule verification. When `lib`'s `:install` (or `:run`) entry rule fires, it computes the full USE model and calls `use:verify_required_use_with_bwu` to check whether the resulting flag set satisfies `lib`'s `REQUIRED_USE` expression.

Step 3 — Fail, not assume. If verification fails (e.g. `lib` declares `REQUIRED_USE=!feature_z`), the entry rule caches a structured violation description via `memo:requse_violation_/3` and then **fails**. It does *not* produce an assumption, because doing so would hide the failure from the candidate selection logic and bypass the entire reprove mechanism.

Step 4 — Candidate backtracking. The failure propagates back to `grouped_package_dependency`, which tries the next candidate version of `lib`. A different version may have a different `REQUIRED_USE` that does not conflict with `feature_z`.

Step 5a — Parent narrowing + reprove. When all candidates are exhausted, the fallback chain activates. `maybe_learn_parent_narrowing` learns to exclude the current parent version (`app-1.0`) and throws `prover_reprove`, giving the prover a chance to retry with a different parent that may not force `feature_z`.

Step 5b — Assumption with violation detail. After all reprove retries are exhausted, the prover falls through to the assumption path. `explanation:assumption_reason_for_grouped_dep` retrieves the cached `requse_violation_` info and enriches the assumption context with a `required_use_violation(...)` tag.

Step 6 — Warning output. The printer recognises the enriched context and emits a structured REQUIRED_USE violation warning, showing which flags were forced, what expression they violate, and which parent triggered the conflict.

9.8.2 Memo cache

The violation info is cached via `memo:requse_violation_/3` (thread-local, survives backtracking since `assertz` is side-effecting). It is: - **Asserted** in the entry rule before failing - **Consumed** in the `grouped_package_dependency` assumption path (retracted after enriching the context) - **Cleared** by `memo:clear_caches/0` at the start of each proof run

9.9 Entry rule structure

Every `:install` and `:run` entry rule follows the same layered structure. Understanding this structure is important because it explains *when* the prover fails, *when* it assumes, and *why* the distinction matters.

Gate checks. The rule begins with a deterministic cut (`!`) — there is exactly one entry-rule clause per literal form, so Prolog must not search for alternatives at this level. Candidate alternatives are provided one level up by `grouped_package_dependency`. After the cut, three quick gate checks run in order:

1. **Mask gate** — if the ebuild is masked and the `unmask` relaxation tier is not active, the rule fails immediately.
2. **Keyword gate** — if no accepted keyword exists and `keyword_acceptance` is not active, the rule fails.
3. **Already-installed short-circuit** — if the package is already installed (and `--emptytree` was not requested), the rule succeeds with an empty condition list. Nothing further needs to be proved.

USE model verification. When none of the gates apply, the rule queries the ebuild's metadata and computes the full USE model (combining profile defaults, user overrides, and `build_with_use` annotations from the parent). It then checks the result against the ebuild's `REQUIRED_USE` expression. If the check fails, the violation is cached (`memo:requse_violation_/3`) and the rule fails — *not* assumes — so that backtracking can explore other candidate versions (see section 9.8.1).

Dependency model construction. If the USE model passes, the rule builds the dependency model: it looks up cached or freshly computed dependency lists, orders them, and returns the full condition list (selected CN, constraints, download literal, dependency literals, etc.).

If the dependency model itself cannot be built — for example because every branch of an `any_of_group` is filtered out — the rule produces a domain assumption tagged with `issue_with_model`. This is deliberately an assumption rather than a failure, because the problem is intrinsic to the ebuild metadata, not something that trying a different candidate version would resolve.

9.10 Constraint guards and reprove integration

Every time the prover unifies a new constraint term into the proof, it calls `rules:constraint_guard(Key, Constraints)` to verify that the constraint is consistent with what has already been proved. The guard has three possible outcomes:

- **Succeed silently** — the constraint is compatible and the proof continues normally.
- **Fail** — the constraint conflicts with the current proof state. Prolog's built-in backtracking explores an alternative within the same proof attempt (e.g. a different candidate version).
- **Throw** `prover_reprove(...)` — the conflict cannot be resolved by simple backtracking. The prover catches the exception, records a learned constraint, and restarts the proof from scratch with the new knowledge (see section 9.7).

Three specialised guards in `candidate.pl` cover the most common conflict types:

- `selected_cn_unique_or_reprove` checks that the selected category/name pair is consistent with prior selections. If two dependency paths select different versions of the same package in the same slot, this guard detects the inconsistency and triggers a reprove with a narrowed domain.
- `selected_cn_not_blocked_or_reprove` enforces blocker constraints. When a package is blocked by another package that has already been selected, this guard triggers a reprove so the prover can learn to avoid the blocked combination.
- `maybe_request_cn_domain_reprove` handles remaining domain inconsistencies. If the selected version falls outside the intersection of all accumulated version domains, the guard learns the correct domain and triggers a reprove.

The constraint guards above operate within a single proof attempt. But what happens when the entire proof cannot succeed under strict constraints? Rather than immediately falling through to assumptions, the pipeline offers one more tool: progressive relaxation.

9.11 Progressive Relaxation

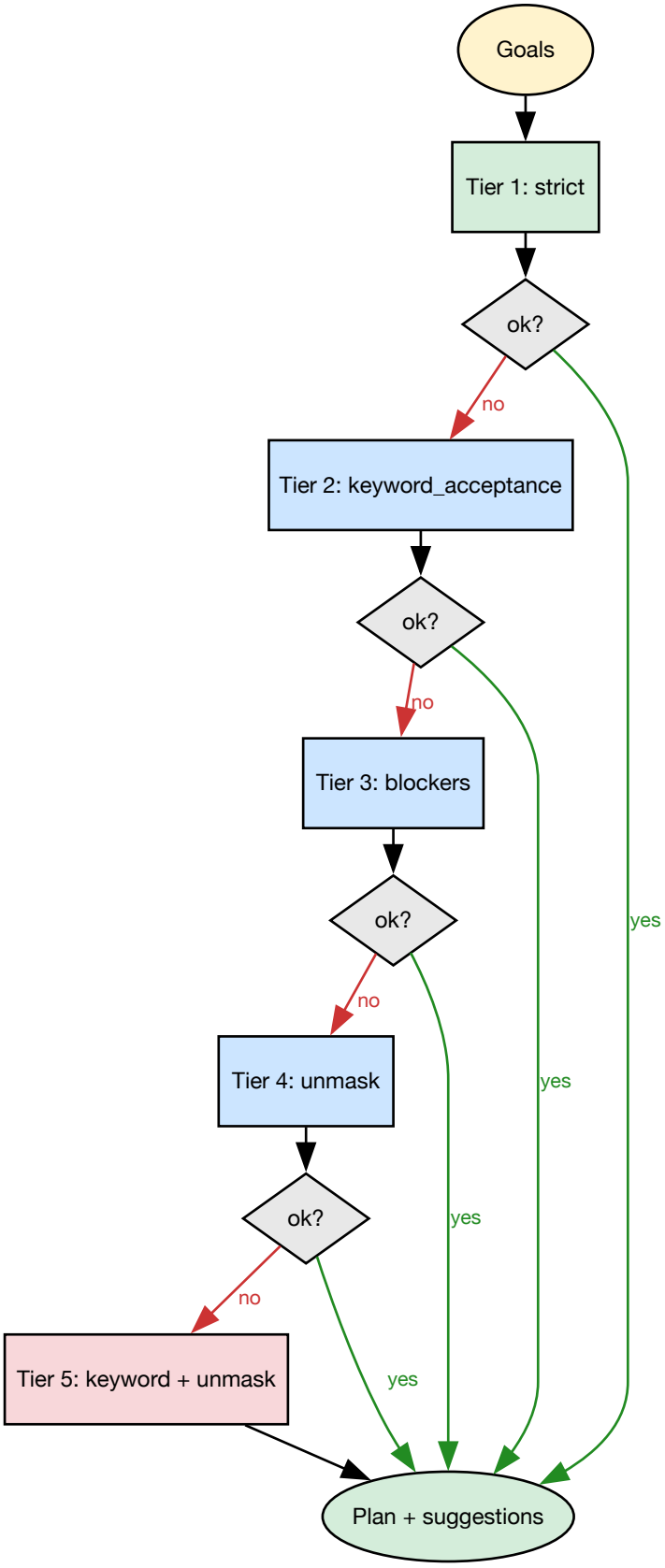


Figure 27 — Progressive relaxation tiers

Not every dependency graph can be satisfied under the strictest interpretation of the repository metadata. A package may exist only with an unstable keyword, or be masked by the profile, or conflict with an already-installed blocker. Rather than giving up at the first such obstacle, the pipeline applies **progressive relaxation**: it re-runs the entire proof under successively weaker constraints until a complete plan emerges.

The mechanism lives in `pipeline:prove_plan_with_fallback/6`. Each tier wraps the prover call inside `prover:assuming/2`, which sets a dynamic flag that the domain rules consult at decision points.

Tier	`assuming` flag	What is relaxed
1 (strict)	none	All masks, keywords, and blockers enforced
2	`keyword_acceptance`	Unstable keywords (`~amd64`) accepted
3	`blockers`	Blocker constraints downgraded to warnings
4	`unmask`	Masked packages unmasked
5	`keyword_acceptance` + `unmask`	Both relaxations combined (last resort)

The tiers are tried in order via Prolog's committed-choice if-then-else (`->` / `;`) — the first tier that succeeds commits and returns a `FallbackUsed` tag (`false`, `keyword_acceptance`, `blockers`, `unmask`, or `keyword_unmask`).

The same 5-tier fallback chain is shared by two canonical entry points in the pipeline module:

- `prove_plan_with_fallback/5` — full pipeline (prove + plan + schedule), used by production paths (`--pretend`, `--graph`, `--build`).
- `prove_with_fallback/4` — prover only (no plan/schedule), used by layered tests (`prover:test`, `planner:test`, `scheduler:test`) and `--bugs`. Each test layer adds its own stages on top.

9.11.1 How `assuming/2` works

`prover:assuming(Flag, Goal)` stores a dynamic flag (`prover_assuming_<Flag>`) for the duration of `Goal`, using `setup_call_cleanup` to guarantee cleanup even on exceptions. Domain predicates test this flag with the zero-argument `prover:assuming(Flag)`:

- **`candidate:eligible/1`** — when `keyword_acceptance` is active, candidates with any keyword are accepted; when `unmask` is active, masked candidates pass.
- **`candidate:accepted_keyword_candidate/7`** — two fallback clauses widen the candidate pool: one for unstable keywords, one for masked packages.
- **`candidate:assume_blockers/0`** — returns `true` when blocker constraints should become warnings instead of hard failures.

9.11.2 Suggestion tags

When a relaxation flag is active and a candidate is admitted under that relaxation, the domain tags the literal's context with a `suggestion/2` term that records exactly which configuration change would eliminate the need for the relaxation:

Suggestion tag	Meaning	Target file
`suggestion(accept_keyword, '~amd64')`	Accept the unstable keyword	`package.accept_keywords`
`suggestion(unmask, R://E)`	Unmask the package	`package.unmask`
`suggestion(use_change, R://E, Changes)`	Adjust USE flags	`package.use`

These tags flow through the proof into the plan output. In builder mode, `builder:dispatch_suggestions/1` can apply the suggestions automatically (writing to `/etc/portage/package.*` files); in pretend mode, they appear as actionable hints in the plan output.

9.11.3 Formal guarantee

Each tier still produces a **complete proof** — the plan is always coherent and fully ordered. The relaxation only widens the candidate pool; it does not skip proof obligations or bypass constraint guards. The suggestion tags make it possible to **trace back** every relaxation to a concrete configuration change, so the weaker proof can be strengthened incrementally.

Once the proof is complete — whether strict or under assumptions — the results must be communicated to the user. The assumption printing pipeline inspects the Proof AVL and translates each assumption into a classified, actionable message.

9.12 Assumption printing pipeline

After the proof is complete, the printer walks the Proof AVL and collects every entry that represents an assumption. Assumptions fall into two families, and the printer handles them differently.

Domain assumptions are stored under `rule(assumed(X))` keys. These represent situations where the prover could not find a real rule and had to accept the literal on faith. When the printer encounters one, it inspects the literal and its context to classify the assumption and produce a meaningful message:

- **REQUIRED_USE violation** — the context contains a `required_use_violation(...)` tag (see section 9.8.1). The printer emits a structured block showing which USE flags were forced, which `REQUIRED_USE` expression they violate, and which parent caused the conflict.
- **Non-existent dependency** — the literal is a `grouped_package_dependency` without context. This means no candidate version exists at all (the category/name is not in the repository).
- **Grouped dependency with reason** — the literal is a `grouped_package_dependency` with an `assumption_reason` tag in its context. The printer extracts the reason label (e.g. "all candidates masked", "blocker conflict") and shows it alongside the dependency.

- **Model unavailable** — the context contains `issue_with_model`. The dependency model could not be built (e.g. all `any_of_group` branches were filtered). The printer reports this as a metadata problem.
- **Generic** — any domain assumption that does not match the above patterns is printed with the raw literal for debugging.

Cycle-break assumptions are stored under `assumed(rule(X))` keys. These mark points where the prover broke a dependency cycle by assuming a literal that was already being proved. The printer delegates to `cycle:print_cycle_explanation`, which reconstructs the cycle path and explains which packages form the loop.

9.12.1 Assumption type classification

The classification logic lives in `assumption.pl`. Given an assumption literal and its context, it returns a type tag that the warning printer uses to select the appropriate output format:

Pattern	Type tag
Context contains `required_use_violation`	`required_use_violation`
`grouped_package_dependency` (no context)	`non_existent_dependency`
`grouped_package_dependency` (with context)	Extracted from `assumption_reason`
`R://E:install`	`assumed_installed`
`R://E:run`	`assumed_running`
Blocker literal	`blocker_assumption`
Context contains `issue_with_model`	`issue_with_model`

The combination of learned constraints, constraint guards, and progressive relaxation is reminiscent of a well-known technique from the SAT solving world.

9.13 Conflict-driven clause learning connection

The learned constraint store is analogous to CDCL (Conflict-Driven Clause Learning) in SAT solvers, but expressed as version domains rather than boolean clauses:

CDCL concept	portage-ng equivalent
Conflict analysis	Constraint guard detecting domain inconsistency
Learned clause	`prover:learn(cn_domain(C,N,S), NarrowedDomain, _)`
Unit propagation	`grouped_dep_effective_domain` applying learned domains
Restart	`prover_reprove` catch-and-retry loop
Decision level	Reprove attempt number

The key difference is granularity: CDCL operates on boolean variables, while portage-ng operates on version domains — structured sets that carry more information per constraint.

With the proving and output mechanisms described, the following sections cover practical aspects: a testing checklist to catch regressions, and a source file map for navigating the codebase.

9.14 Testing learned constraints

When testing changes to the reprove or assumption mechanism, the following checklist helps catch regressions quickly:

- **Exit code** — the process exit code summarises the proof outcome:
  - `0` — no assumptions at all (clean proof)
  - `1` — only prover cycle-break assumptions
  - `2` — at least one domain assumption (e.g. missing dependency)
- **"Total: N actions"** — this line must appear in the output, confirming that the proof completed and a plan was produced.
- **"non-existent" count** — count the lines containing "non-existent" to check how many domain assumptions were made. An unexpected increase signals a regression.
- **No "Unknown message"** — the output should not contain "Unknown message" or unhandled exception traces. These indicate an assumption type that the printer does not recognise.
- **Runtime** — a single-target proof should complete within a few seconds. A significant increase compared to previous runs suggests excessive reprove retries or a learning bug.
- **Test suite** — the overlay and portage test suites (`prover:test_stats/1`) should maintain their previous pass rate. Any drop indicates that a change has broken handling of a known edge case.

9.15 Source File Map

File	Role
`Source/Pipeline/prover.pl`	Core proof engine, reprove retry loop, cycle detection, learned store
`Source/Domain/Gentoo/rules.pl`	Domain rules: entry rules, grouped deps, `rule(assumed(_),[])`
`Source/Domain/Gentoo/Rules/candidate.pl`	Candidate selection, reprove triggers, parent narrowing
`Source/Domain/Gentoo/Rules/heuristic.pl`	Reprove state management, reject accumulation
`Source/Domain/Gentoo/Rules/memo.pl`	Thread-local caches including `requse_violation_/3`
`Source/Domain/Gentoo/Rules/use.pl`	`verify_required_use_with_bwu`, `describe_required_use_violation`
`Source/Pipeline/Prover/explanation.pl`	`assumption_reason_for_grouped_dep` diagnosis
`Source/Pipeline/Prover/explainer.pl`	`term_ctx/2`, "why" queries
`Source/Pipeline/Printer/Plan/assumption.pl`	Assumption type classification
`Source/Pipeline/Printer/Plan/warning.pl`	Assumption detail rendering

9.16 Further reading

- Chapter 8: The Prover — the proof search algorithm
- Chapter 10: Version Domains — domain operations used by constraint learning
- Chapter 11: Rules and Domain Logic — entry rules, fallback chains, and REQUIRED_USE handling
- Chapter 21: Resolver Comparison — Zeller, Vermeir, and CDCL foundations

Chapter 10

Version Domains

Version domains are the mechanism by which portage-ng reasons about version constraints. Every version comparison, every dependency operator (`>=`, `<=`, `~`, `=*`), and every learned constraint is expressed as an operation on version domains.

10.1 Why version domains matter

Picture two packages that both pull in `dev-libs/openssl`, but with different requirements. Package A depends on `>=dev-libs/openssl-3.0`: any OpenSSL from 3.0 upward is acceptable on that path. Package B depends on `<dev-libs/openssl-3.2`: only versions strictly below 3.2 are acceptable there. If both constraints apply to the same install, you are not looking for a single magic number first — you are asking which versions lie in the overlap of two sets. Versions that satisfy both are exactly those in **3.0 ≤ v < 3.2**: the half-open interval **[3.0, 3.2)**.

That overlap is the **intersection** (in domain terms, the **meet**) of two version domains. Version domains represent **sets** of acceptable versions; combining constraints from different dependency paths means intersecting those sets until you obtain the tightest description still compatible with everything seen so far. The rest of this chapter spells out how those sets are stored, compared, and merged in code.

10.2 Version representation

Versions are stored as `version/7` compound terms:

```
version(NumsNorm, Alpha, SuffixRank, SuffixNum, SuffixRest, Rev, Full)
```

Field	Example	Meaning
`NumsNorm`	`[3,0,77]`	Normalized numeric components
`Alpha`	`''` or `'a'`	Alpha suffix (empty atom if none)
`SuffixRank`	`4`	Numeric rank of version suffix (`_alpha`=1, `_beta`=2, `_pre`=3, `_rc`=4, (none)=5, `_p`=6)
`SuffixNum`	`0`	Suffix number (e.g. `3` in `_rc3`)
`SuffixRest`	`''`	Additional suffix components
`Rev`	`3`	Revision number (from `-r3`)
`Full`	`'3.0.77-r3'`	Original version string

Empty or absent versions use the atom `version_none`.

10.3 Version comparison

Versions are compared using Prolog's standard `compare/3` directly on the compound term. No runtime key conversion is needed — the `version/7` structure is designed so that standard term ordering produces correct PMS version ordering:

```
compare(Order, version([3,0,77],...), version([3,1,0],...))
% Order = (<)
```

This works because: - `NumsNorm` is a list of integers (lexicographic list comparison) - `SuffixRank` maps suffixes to integers in PMS order - `Rev` is a plain integer

10.4 Version domain model

A version domain represents a set of acceptable versions for a package. It is stored as:

```
version_domain(Slots, Bounds)
```

Field	Type	Meaning
`Slots`	list or `any`	Acceptable slots (or `any` for unconstrained)
`Bounds`	structured term	Version bounds (upper, lower, exact, wildcard)

The `none` atom represents an unconstrained domain (all versions accepted).

10.5 Domain operations

10.5.1 Domain meet (intersection)

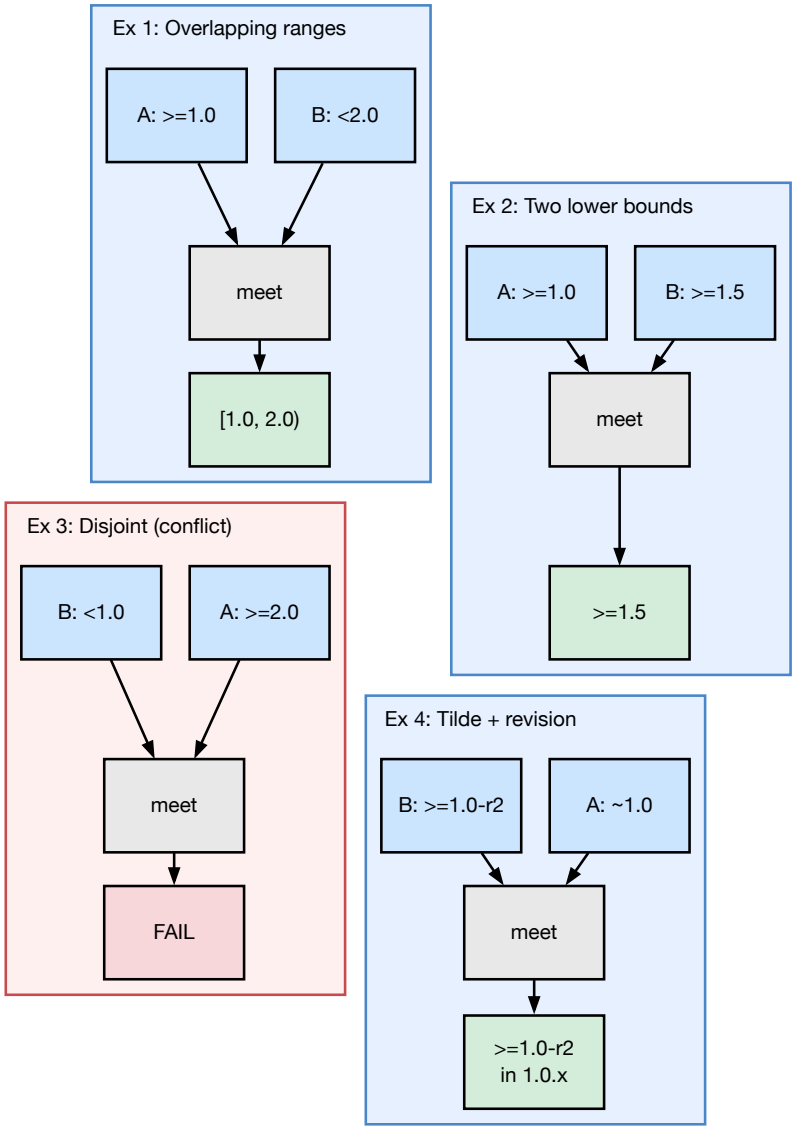


Figure 28 — Domain meet examples

When two dependency paths impose different version constraints on the same package, the domains are intersected:

```
version_domain:domain_meet(Domain1, Domain2, Intersection)
```

The intersection computes the tightest bounds that satisfy both constraints. If the intersection is empty (no version satisfies both), the goal fails: there is no `Intersection` term that stays consistent with the combined bounds (and related checks such as slot compatibility).

10.5.2 Worked examples (meet in practice)

The following sketches use dependency-style wording; internally each side becomes a `version_domain/2` (or `none`) whose bounds are merged by `domain_meet/3`. Intuitively, **meet = AND** over acceptable versions.

Example 1 — overlapping range.

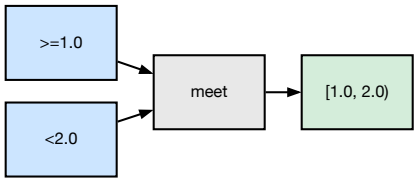


Figure 29 — Meet example 1

Lower bound `>=1.0` and upper bound `<2.0` describe the half-open interval **[1.0, 2.0)**. The meet is that interval: every version that satisfies both operators is exactly a version at least 1.0 and strictly below 2.0.

Example 2 — two lower bounds.

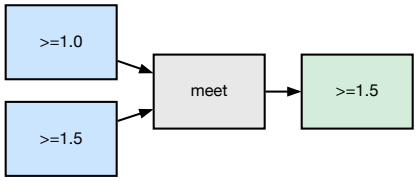


Figure 30 — Meet example 2

`>=1.0` meets `>=1.5`. The stricter requirement wins: the intersection is **`>=1.5`**. Anything below 1.5 was already ruled out by the second constraint.

Example 3 — disjoint constraints (conflict).

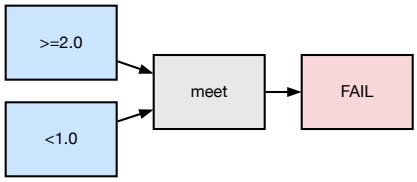


Figure 31 — Meet example 3

`>=2.0` meets `<1.0`. No version can be simultaneously at least 2.0 and below 1.0. `domain_meet/3` **fails**: the normalised domain is empty, and the prover must treat this as an unsatisfiable combined requirement.

Example 4 — tilde vs revision-aware lower bound.

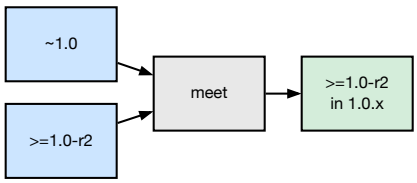


Figure 32 — Meet example 4

`~1.0` (in PMS terms: same main version as `1.0`, any revision) restricts candidates to the **1.0** line. Meeting that with `>=1.0-r2` keeps you on that line but drops revisions below `-r2`: the effective set is **`>=1.0-r2` within the 1.0.x family**, not a broader branch of the package.

In all cases, when the meet succeeds, candidate selection and consistency checks use the **narrower** domain; when it fails, there is no shared non-empty set of versions to choose from.

10.5.3 Consistency check

```
version_domain:domain_consistent(Domain)
```

Verifies that a domain is non-empty — that at least one version in the repository satisfies the bounds.

10.6 Feature logic intuition

In Zeller-style **feature logic**, a *feature* describes a set of objects that share certain properties — not necessarily a single object, but a well-bounded *set* described by those properties. A **version domain** is the same idea at the version level: it is a feature whose extension is the set of versions that satisfy given slot and bound constraints (above a threshold, below a threshold, exact match, tilde range, wildcard prefix, and so on).

Feature unification — meeting two features so that an object must satisfy both — corresponds directly to **domain intersection**. This is not merely a pedagogical analogy: portage-ng wires version domains into the generic unification hook in `feature_unification:val_hook/3`, so that merging domain values follows the same meet operation as `version_domain:domain_meet/3`.

A practical consequence is **monotonic narrowing**: along a resolution path, domains only become *tighter* (fewer acceptable versions), never wider, unless explicitly reset by a broader reprove strategy. Each successful refinement shrinks the search space; that is why successive reprove attempts can be viewed as making measurable progress toward either a concrete choice or a clear conflict.

10.7 Learned domain narrowing

The prover's learned constraint store uses version domains to carry narrowed version information across reprove retries. Each learned constraint is keyed by a category–name–slot triple:

```
cn_domain(Category, Name, Slot)
```

10.7.1 Conflict detection and learning

When two dependency edges impose incompatible version requirements on the same package, the constraint guard detects an empty intersection and records a narrowed domain for future attempts.

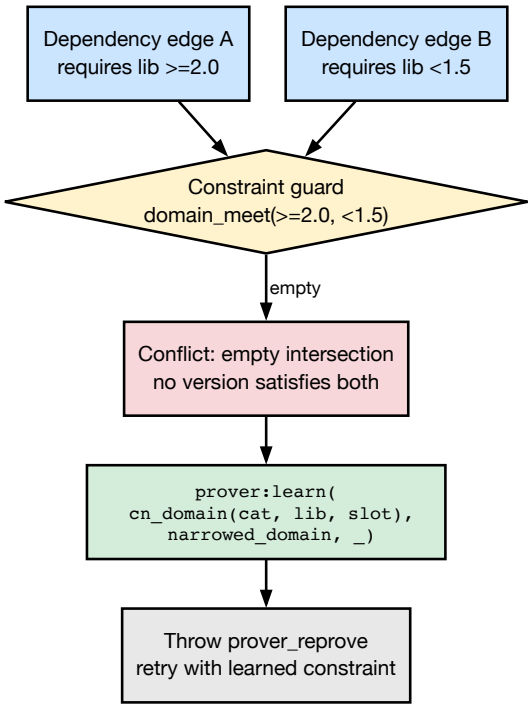


Figure 33 — Conflict detection and domain learning

The guard calls `domain_meet/3` on the two incoming domains. When the result is empty (no version satisfies both), the guard stores the narrowed domain via `prover:learn/3` and throws `prover_reprove` to restart the proof with this new knowledge.

10.7.2 Applying learned domains on reprove

On the next reprove attempt, `grouped_dep_effective_domain` intersects the learned domain with the local domain coming from the current proof context, *before* candidate selection begins.

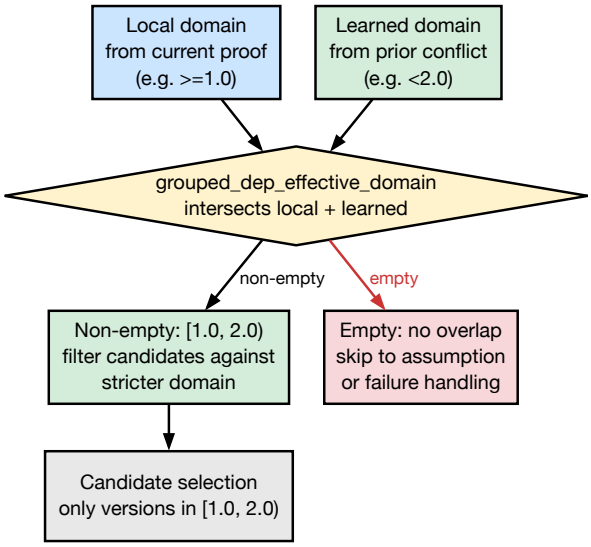


Figure 34 — Applying learned domain on reprove

If the intersection is **non-empty**, candidates are filtered against the stricter combined domain — avoiding the same dead-end choice that caused the previous conflict. If the intersection is **empty**, there is no compatible overlap left: the prover can skip directly to assumption or failure handling instead of selecting candidates from an empty domain.

Chapter 9 walks through the reprove and learning mechanics in full; Chapter 11 ties domains into rule evaluation and candidate generation.

10.7.3 Wildcard domain learning

When a wildcard dependency like `=dev-python/gast-0.6*` cannot be satisfied, `candidate:maybe_learn_wildcard_domain/4` derives an upper-bound domain from the wildcard: the last numeric component is incremented to produce an exclusive upper bound (e.g. `0.6*` → `< 0.7`, `1.2.3*` → `< 1.2.4`). The resulting `version_domain(any, [bound(smaller, UpperVer)])` is learned via `prover:learn/3` and triggers a reprove.

This mechanism is guarded: it only fires when the parent has already been narrowed by a prior `maybe_learn_parent_narrowing` attempt (or when the parent is a single-version package, making parent narrowing futile). The guard ensures parent narrowing gets priority for multi-version parents, correctly handling cross-package wildcard conflicts (e.g. two packages requiring different wildcard ranges of the same dependency).

10.7.4 Connection to feature logic

This mechanism is inspired by Zeller's feature logic: version sets are identified by feature terms and refined by incrementally narrowing the set until each component resolves to a single version. Each successful `learn` call tightens the feature, and each `grouped_dep_effective_domain` call applies the accumulated tightening — a direct realisation of monotonic domain narrowing.

10.8 Version operators

The EAPI grammar supports the following version operators, each producing a different domain constraint:

Operator	Syntax	Domain meaning
`>=`	`>=cat/pkg-1.0`	Lower bound: version >= 1.0
`<=`	`<=cat/pkg-2.0`	Upper bound: version <= 2.0
`>`	`>cat/pkg-1.0`	Strict lower bound
`<`	`<cat/pkg-2.0`	Strict upper bound
`=`	`=cat/pkg-1.0`	Exact version match
`~`	`~cat/pkg-1.0`	Version match ignoring revision (any `-rN`)
`=*`	`=cat/pkg-1*`	Wildcard: any version starting with `1`

10.9 Further reading

- Chapter 9: Assumptions and Constraint Learning — how domains interact with the reprove mechanism
- Chapter 11: Rules and Domain Logic — how version domains feed into candidate selection
- Chapter 21: Resolver Comparison — Zeller's feature logic and CDCL connections

Chapter 11

Rules and Domain Logic

11.1 How we resolve dependencies

The prover works with abstract literals and rules. It does not know what "install" means for Gentoo — it only knows how to find a matching rule and prove its body. The **rules layer** is the bridge between that abstract proof search and the concrete world of ebuilds, USE flags, and version constraints.

When the prover encounters a literal like `portage://'sys-apps/portage-3.0.77-r3':install`, it is the rules layer that answers questions such as:

- What are this ebuild's dependency lists (DEPEND, BDEPEND, RDEPEND)?
- Which USE flags are active for this particular path through the dependency graph?
- Which repository entries are valid candidates, given version, slot, and keyword constraints?
- Should a USE-conditional dependency like `ssl? ( dev-libs/openssl )` be included or skipped?
- Are there blockers, REQUIRED_USE constraints, or ordering requirements?

All of these decisions are encoded as rule bodies and proof-term context annotations (`?{...}` lists). The sections below follow one resolution from user target to installed graph, then cover cycles, USE semantics, and what happens when the rules layer must record an assumption.

11.2 How dependency resolution works (end-to-end)

A typical run starts with a user query like `sys-apps/portage`. The rules layer turns this into a `target/2` literal and resolves it to the best eligible candidate — the newest version that is not masked, has an accepted keyword, and satisfies any slot constraints. This resolution produces sub-literals that drive the rest of the proof.

The resolution then branches depending on the action:

- **`:run`** resolves runtime dependencies (RDEPEND). PDEPEND is handled in the same pass through the prover's literal hook (see Hooks).
- **`:install`** resolves build-time dependencies (DEPEND and BDEPEND) and attaches ordering constraints (`after/1`) that express which packages must be installed before others.

Each dependency atom from the metadata becomes a `grouped_package_dependency` literal. The candidate selection machinery then applies version ranges, slot operators, keyword and mask policy, and any learned constraints from prior reprove attempts (see Chapter 9 and Chapter 10).

USE-conditional dependencies are included only when the condition holds in the effective USE set for that ebuild and path. For example, `ssl? ( dev-libs/openssl )` adds `dev-libs/openssl` to the body only if `ssl` is enabled; otherwise the branch is skipped entirely. When a parent requires particular flags on a child, those requirements propagate via `build_with_use` in the proof-term context (see USE flags in depth).

The prover walks this structure depth-first: each successful rule expansion adds literals to the proof and updates the model. When a rule fails, Prolog backtracks to try an alternative candidate or, ultimately, records an assumption.

11.3 The `rule/2` interface

The single entry point between the prover and the domain logic is:

```
rules:rule(+Head, -Body)
```

The prover passes a literal as `Head`, and the rules layer returns a list of sub-literals `Body` that must be proved in order to justify it. The prover never interprets what the literals mean — all Gentoo-specific knowledge is encapsulated inside the rule clauses.

The table below lists the main head patterns. Target rules translate a user query into a concrete ebuild. Action rules (`:install`, `:run`, `:download`) expand an ebuild into its dependency obligations. Dependency rules resolve individual atoms to candidates. Validation rules enforce REQUIRED_USE constraints. The catch-all `assumed(X)` clause handles domain assumptions when no real rule applies.

Head pattern	Purpose
`target(Q, Arg):run`	Resolve a user target to a candidate ebuild
`target(Q, Arg):fetchonly`	Fetch-only target resolution
`target(Q, Arg):uninstall`	Uninstall target resolution
`Repo://Ebuild:install`	Build and install an ebuild (DEPEND + BDEPEND)
`Repo://Ebuild:run`	Runtime availability (RDEPEND)
`Repo://Ebuild:download`	Fetch source archives
`Repo://Ebuild:fetchonly`	Fetch only
`Repo://Ebuild:depclean`	Remove unneeded package
`grouped_package_dependency(...):Action`	Resolve a grouped dependency
`package_dependency(...):config`	Configure a single dependency
`exactly_one_of_group(...):validate`	Validate REQUIRED_USE `^^`
`any_of_group(...):validate`	Validate REQUIRED_USE any-of
`at_most_one_of_group(...):validate`	Validate REQUIRED_USE `??`
`assumed(X)`	Catch-all for domain assumptions

11.4 Candidate resolution

When the rules layer encounters a dependency, it must choose a concrete version of the target package. This process has three stages: eligibility filtering, version-ordered selection, and a fallback chain for when no candidate works.

11.4.1 Eligibility filtering

Before a candidate version is considered, `candidate:eligible/1` checks three things:

- **Masking** — is the ebuild masked by the profile or user configuration?
- **Keyword acceptance** — does the ebuild have an accepted keyword for the current architecture?
- **Installed status** — is the package already installed (checked against the VDB)?

If a candidate fails these checks and no relaxation tier is active (see Chapter 9, Progressive Relaxation), the entry rule fails and Prolog backtracks to try the next candidate.

11.4.2 Version-ordered selection

`candidate:resolve/2` resolves a query to a specific `Repository://Ebuild` pair. Candidates are tried newest-first via `cache:ordered_entry/5`, so the prover naturally prefers the latest eligible version.

11.4.3 Dependency ordering within a group

Before proving the dependencies of a package, `candidate:dep_priority/2` sorts them so that tightly constrained siblings are proved first. This reduces greedy conflicts where an unconstrained sibling selects a version that later clashes with a tighter constraint:

BaseK	Constraint type	Example
1	Tight upper bound (range)	`>=1.0 <2.0`
4	Tilde constraint	`~dev-ruby/railties-8.1.1`
8	Wildcard constraint	`=dev-python/gast-0.6*`
999	Unconstrained	`dev-libs/openssl`

Lower keys are proved first. Within each tier, slot specificity further refines the order. The effect is that tilde and wildcard dependencies lock their `selected_cn` before unconstrained siblings pick a potentially conflicting version.

11.4.4 Self-dependencies and cross-slot handling

When a package lists itself as a build dependency (e.g. `antlr-tool:4` needing `antlr-tool:3.5` to bootstrap), the rules layer distinguishes **same-slot self-deps** from **cross-slot self-deps**.

Same-slot self-deps (same category, name, and slot as the parent) are treated as bootstrap dependencies: if the package is already installed, the dependency is satisfied; otherwise the rule fails so that backtracking can reach a bootstrap alternative.

Cross-slot self-deps (same category and name but a *different* slot) are treated as **regular dependencies** and resolved normally. This prevents model build failures when the cross-slot version is not yet installed.

11.4.5 Fallback chain

When every candidate for a grouped dependency has been tried and none succeeded, the rules layer activates a fallback chain before giving up:

- **Wildcard domain learning** — `maybe_learn_wildcard_domain` fires when a wildcard dependency (e.g. `=dev-python/gast-0.6*`) fails resolution and the parent has already been narrowed by a prior parent-narrowing attempt, or the parent is a single-version package (where parent narrowing would be futile). It derives an upper-bound `cn_domain` from the wildcard constraint (e.g. `< 0.7`) and learns it via `prover:learn/3`, then throws `prover_reprove`.
- **Parent narrowing** — `maybe_learn_parent_narrowing` records that the current parent version led to a dead end and throws `prover_reprove`, so the prover can retry with a different parent.
- **Domain reprove** — `maybe_request_grouped_dep_reprove` checks whether domain or constraint conflicts exist and, if so, triggers a reprove with learned constraints.
- **Domain assumption** — as a last resort, the rules layer emits `assumed(grouped_package_dependency(...))`. This records the failure as a domain assumption so the proof can still complete.

11.5 Cycles and how portage-ng handles them

Circular dependencies are a fact of life in the Portage tree. A language runtime may be packaged with tooling that itself depends on that runtime, creating a loop. The prover detects these cycles during its depth-first proof search: it keeps track of which literals are currently being proved, and if the same literal appears again while it is still on the stack, a cycle has been found.

Before breaking a cycle with an assumption, the prover asks the domain whether the cycle is **benign**. The hook `heuristic:cycle_benign/2` inspects the repeating literal and the cycle path. If the hook succeeds, the literal is treated as already justified and added to the model without a cycle-break assumption. If the hook fails, the prover records a cycle-break assumption (`assumed(rule(Lit))` in the proof, `assumed(Lit)` in the model). This is separate from domain assumptions introduced by `rule(assumed(X), [])`.

The benign classification is conservative and pattern-based. For example, cycles that pass through `:run` (RDEPEND paths) are often treated as ordering-style cycles rather than hard failures — mirroring how traditional resolvers tolerate certain cyclic patterns.

After the proof is complete, cyclic portions of the `:run` side of the graph are grouped into **strongly connected components (SCCs)** by the scheduler, so that the merge ordering respects the cycle structure. For more on proof search and assumptions, see Chapter 8 and Chapter 9.

11.6 USE flags in depth

USE flags play a central role in dependency resolution. They determine which dependency branches exist, which packages are eligible, and whether REQUIRED_USE constraints are satisfied.

11.6.1 Effective USE and conditionals

For each ebuild the rules layer computes an **effective USE set** — the final set of flags that are active for this particular proof path. USE-conditional dependencies like `ssl? ( dev-libs/openssl )` are evaluated against this set: if the flag is active, the dependency is included; otherwise it is skipped.

The key predicates are `use:effective_use/3` (computes the full effective USE set for an ebuild) and `use:evaluate_conditional/3` (evaluates a single flag condition).

11.6.2 `build_with_use`

When a parent dependency requires specific USE flags on a child (e.g. `dev-libs/openssl[threads]`), those requirements travel through the proof as `build_with_use` context annotations. They influence how the child's effective USE set is computed, ensuring that parent requirements are not silently ignored.

11.6.3 `REQUIRED_USE`

Gentoo's REQUIRED_USE expressions (e.g. `^^ ( gtk qt5 )` meaning "exactly one of gtk or qt5") are enforced through dedicated validation literals. If the active USE set violates a REQUIRED_USE expression, the rule fails and the prover backtracks to try another candidate or records an assumption (see Chapter 9, section 9.8).

11.6.4 Priority order

USE flags are resolved in priority order, highest priority first:

1. `build_with_use` from the parent's dependency context
2. **User configuration** (`/etc/portage/package.use`)
3. **Profile defaults**
4. **Ebuild IUSE defaults**

The most important consequence is that context wins over profile defaults: a `build_with_use` requirement from the parent can force or forbid a flag regardless of what the profile would normally choose. This is why two proofs for the same package can produce different USE sets — they arrive through different dependency paths with different context annotations.

11.6.5 Conflicts and backtracking

When USE-derived constraints conflict — for example, REQUIRED_USE fails, a conditional branch does not apply as expected, or an eligibility check fails — the relevant rule fails. The prover then backtracks: it tries another candidate version, another slot, or another branch of

the search tree. If no alternative succeeds, the candidate layer records a domain assumption, often tagged with a suggestion for which `package.use` change would resolve the conflict (see Assumptions as proposals).

11.7 Choice groups

Gentoo's PMS defines three choice-group operators that constrain how many members of a set may be active at the same time. The rules layer maps each operator to a dedicated validation literal that the prover must satisfy as part of the proof:

Operator	Rule clause	Semantics
any-of (a b c)	`any_of_group(Deps):validate`	At least one must be satisfied
`^^ ( a b c )`	`exactly_one_of_group(Deps):validate`	Exactly one must be satisfied
`?? ( a b c )`	`at_most_one_of_group(Deps):validate`	At most one may be satisfied

If the validation literal fails (e.g. two members of an `exactly_one_of` group are both active), the prover backtracks to try a different USE configuration or candidate version.

11.8 Slot operators

Dependency atoms can carry a slot operator that tells the rules layer how to handle multi-slot packages. A package like `dev-lang/python` may offer several slots (e.g. `3.11`, `3.12`), and the slot operator determines which slots are acceptable and whether a sub-slot change should trigger a rebuild of the dependent package.

Operator	Meaning	Context effect
`:SLOT`	Depend on a specific slot	Filters candidates to that slot
`:*`	Any slot is acceptable	No slot constraint applied
`:=`	Sub-slot rebuild trigger	Records the selected sub-slot; a change triggers rebuild
`:SLOT=`	Specific slot + rebuild	Combines slot filter with rebuild tracking

11.9 Blockers

A blocker dependency says that two packages cannot coexist. Gentoo distinguishes two strengths:

Type	Syntax	Behaviour
Weak blocker	`!cat/pkg`	The blocked package should not be present; resolved at plan time
Strong blocker	`!!cat/pkg`	The blocked package must not be present; the constraint guard fires immediately

Internally, blockers produce `blocked_cn` constraint terms. These are checked against `selected_cn` constraints by `selected_cn_not_blocked_or_reprove`: if the blocked package has already been selected elsewhere in the proof, the guard triggers a reprove so the prover can learn to avoid the conflicting combination (see Chapter 9, section 9.10).

11.10 Hooks

PDEPEND (post-dependencies) represent packages that should be present at runtime but are not required at build time. Unlike DEPEND and RDEPEND, they do not block the build — they are installed afterwards.

In portage-ng, PDEPEND is handled in a single pass inside the prover via the `rules:literal_hook/4` hook. Whenever a literal is successfully proved, the hook checks whether the corresponding ebuild has PDEPEND entries. If it does, those entries are injected as additional proof obligations on the spot. This avoids a separate PDEPEND resolution pass and ensures that post-dependencies are part of the same proof and plan.

11.11 Assumptions as proposals

When strict resolution cannot satisfy every dependency, the rules layer records a **domain assumption** rather than giving up. From a user perspective, an assumption is not a dead end — it is a **proposal** for a configuration change.

The literal's proof-term context is annotated with **suggestion** tags that spell out exactly what to change. Common suggestions include:

- `suggestion(unmask, ...)` — unmask a package
- `suggestion(accept_keyword, ...)` — accept an unstable keyword
- `suggestion(use_change, ..., Changes)` — adjust USE flags

The printer collects these tags and shows them next to the assumption, so you can see which `/etc/portage` file to edit and what to put in it. The plan is still constructed as if the change had already been applied: the merge list is coherent under the stated proposal, and the output tells you which configuration changes would make it real.

For the full story on assumptions and constraint learning, see Chapter 9.

11.12 Rules submodules

The rules layer is not a single monolithic file. It is split across several focused submodules under `Source/Domain/Gentoo/Rules/`, each handling a distinct concern:

Module	File	Purpose
`memo`	`memo.pl`	Thread-local caches for selected candidates and violations
`use`	`use.pl`	USE flag evaluation and REQUIRED_USE checking
`candidate`	`candidate.pl`	Candidate selection, eligibility, and reprove triggers
`heuristic`	`heuristic.pl`	Reprove state, retry budgets, and cycle benignity checks
`dependency`	`dependency.pl`	Dependency model construction and context threading
`target`	`target.pl`	Target resolution (translating a query to a candidate)
`featureterm`	`featureterm.pl`	Context stripping for memoisation keys

11.13 Further reading

- Chapter 8: The Prover — how the prover calls `rule/2` and builds the proof
- Chapter 9: Assumptions — the fallback chain, reprove mechanism, and progressive relaxation
- Chapter 10: Version Domains — how version constraints feed into candidate selection

Chapter 12

Planning and Scheduling

12.1 Why parallel planning?

Traditional package managers (Portage, apt, and similar) typically expose a **sequential** plan to the user: install *A*, then *B*, then *C*. Even when the underlying resolver knows that *B* does not depend on *A*, the presented order is often a single linear timeline.

portage-ng takes a different stance: it produces **parallel** plans from the start. Wave 1 might download *A*, *B*, and *C* concurrently; wave 2 might install *A* while *D* is still downloading; wave 3 might install *B* and *C* together, and so on. This is **not** a post-processing optimization layered on top of a linear schedule. Parallelism is computed **during** planning, as a natural consequence of Kahn's topological sort: at each step, every literal whose prerequisites are satisfied is eligible at once, and that set is exactly one parallel wave.

On a multi-core machine with fast I/O, overlapping work this way can dramatically reduce wall-clock time compared to a strictly sequential narrative.

Planning is also at the **action** level, not the package level. The same logical package may appear as separate literals for download, install, run, and so on. Those actions can therefore land in different waves: one package can still be downloading while another is already installing, whenever the dependency graph allows it.

After the prover completes, the proof must be converted into an executable plan — an ordered sequence of actions with maximal parallelism. This is done in two stages: wave planning for the acyclic portion, and SCC scheduling for any cyclic remainder.

12.2 Dependency types and scheduling

Gentoo's dependency classes do not all impose the same ordering strength. The rules layer records that distinction in **proof-term context** (`?{…}` on each literal): dependency edges and ordering hints are derived from markers such as `after/1` and `after_only/1`. The planner turns those into edges in its dependency graph (including ordering-only constraints where appropriate).

Roughly:

- **DEPEND** and **BDEPEND** — build-time dependencies. They must be satisfied before the build can start, so they contribute **hard ordering**: the consumer's build-related actions wait on the resolved dependencies.

- **RDEPEND** — runtime dependencies. They must be satisfied before the package can be *used* at runtime. The ordering is **looser** than pure build ordering: the prover and feature-term layer can represent this with `after_only/1` so that runtime ordering is enforced where needed without treating every runtime edge like a build blocker.
- **PDEPEND** — post-install dependencies. They are resolved **after** the main proof pass (via hooks in the rules layer). That late binding can introduce or surface **cycles** that wave planning alone cannot schedule; those literals become **remainder** work for the SCC scheduler.
- **IDEPEND** — install-time dependencies (EAPI 8+). They constrain ordering around the install phase specifically, again flowing through the same context and planner machinery as the other classes.

For the exact mapping from PMS ordering to internal edges and constraints, see Chapter 22: Dependency Ordering. The implementation detail lives in the rules and `featureterm` helpers: `after/1` propagates as a real dependency relation, while `after_only/1` can be lowered to ordering constraints (for example `constraint(order_after(…))`) that the planner respects without overstating build-time blocking.

12.3 Wave planning (Kahn's algorithm)

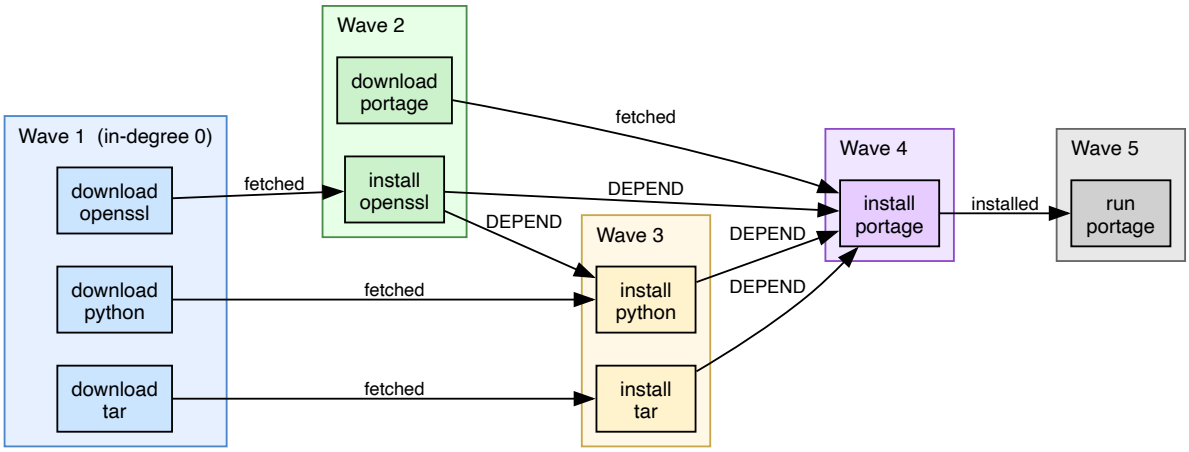


Figure 35 — Wave planning example

The planner (`Source/Pipeline/planner.pl`) uses Kahn's algorithm to produce a topological ordering of the proof graph with parallelism computed from the start.

12.3.1 Why Kahn's algorithm?

Kahn's algorithm is simple, correct for DAGs, and **naturally** exposes parallelism. At each iteration it collects every node whose **in-degree** has dropped to zero — that set is precisely the set of actions that may run **concurrently** at that stage. No second pass is required to "discover" parallel groups.

A common alternative is a **DFS-based** topological sort. That yields a **single** linear ordering (one valid sequence), but it does **not** identify which steps are independent: you get *a* order,

not *all* maximal parallel layers. For a build planner that wants explicit waves, Kahn's layer-by-layer behavior is the better fit.

12.3.2 How it works

1. **Build dependency counts.** For each rule in the Proof AVL, count how many of its body literals are "real" dependencies (not already installed, not assumed).
2. **Initialize the ready queue.** Literals with zero dependencies form the first wave — they can be executed immediately.
3. **Process waves.** For each wave:
   - Remove all ready literals from the graph.
   - Decrement dependency counts for all heads that depended on them.
   - Literals whose count reaches zero join the next wave.
4. **Repeat** until no more literals can be scheduled.

The result is a list of waves, where all literals within a wave can be executed concurrently:

```
Wave 1: [download(A), download(B), download(C)]
Wave 2: [install(A), download(D)]
Wave 3: [install(B), install(C)]
Wave 4: [install(D), run(A)]
```

12.3.3 Parallelism

Actions within a wave are independent and can run in parallel. The planner computes the maximum parallelism at each wave, enabling the printer to show concurrent execution groups and the builder to schedule actual parallel builds.

12.3.4 Remainder

Literals that are part of cycles cannot be scheduled by Kahn's algorithm (their dependency counts never reach zero). These are returned as the **remainder** for the scheduler to handle.

12.4 SCC decomposition (Kosaraju)

12.4.1 From planner remainder to scheduler

The wave planner (section 12.3) processes the acyclic portion of the proof graph and produces a parallel plan. But not every literal can be scheduled this way. When the dependency graph contains cycles — for example, Python depends on setuptools at runtime, and setuptools depends on Python — Kahn's algorithm can never reduce their in-degree to zero: they keep waiting for each other. These unscheduled literals are returned as the **remainder**.

The scheduler (`Source/Pipeline/scheduler.pl`) picks up where the planner left off. Its job is to decompose the remainder into groups of mutually dependent literals and decide how to handle each group. The tool it uses for this is **strongly connected component (SCC) decomposition**.

12.4.2 What is a strongly connected component?

A **strongly connected component** is a maximal set of nodes in a directed graph where every node can reach every other node by following directed edges. In dependency terms, an SCC is a group of packages that all depend on each other, directly or indirectly — a dependency cycle.

A single-node SCC (a node with no self-loop) simply means that node is not part of any cycle and can be scheduled on its own. A multi-node SCC is a genuine cycle that must be handled as a group.

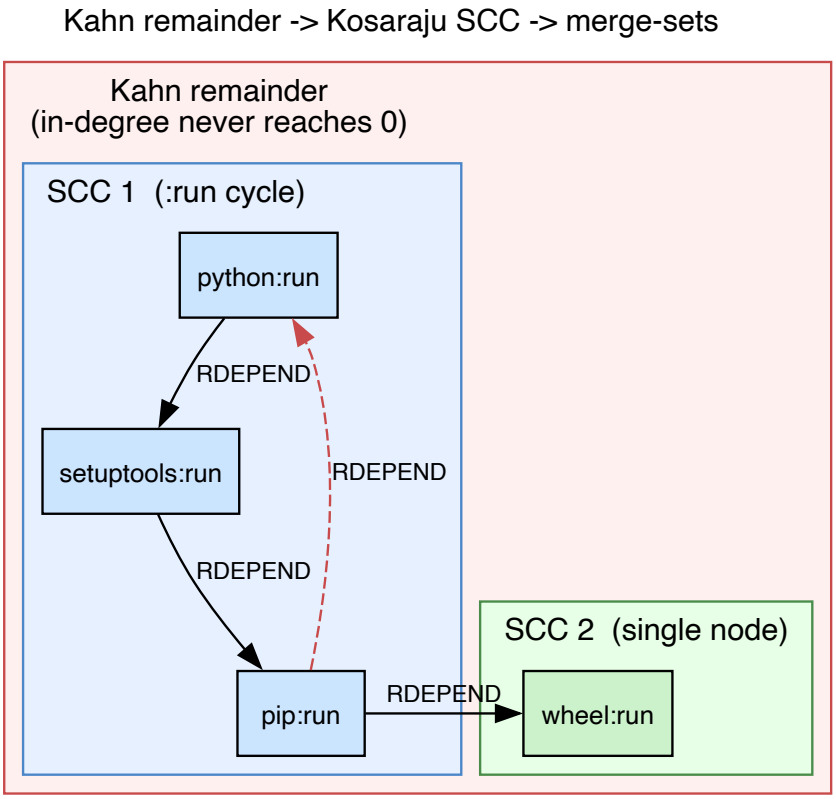
12.4.3 Why Kosaraju?

The two best-known algorithms for SCC decomposition are **Tarjan's algorithm** and **Kosaraju's algorithm**. Both run in linear time (O(V + E)), so performance is not the deciding factor. The choice comes down to implementation characteristics:

- **Tarjan's algorithm** uses a single DFS pass with an explicit stack and "lowlink" bookkeeping. It is compact but harder to implement correctly — the lowlink update rules are subtle, and a small mistake can silently produce wrong components.
- **Kosaraju's algorithm** uses two straightforward DFS passes: one on the original graph to compute a finish order, and one on the transposed graph (edges reversed) to extract SCCs. Each pass is a plain DFS with no extra bookkeeping beyond a visited set.

portage-ng uses Kosaraju because its two-pass structure is easier to verify, easier to debug, and maps naturally to Prolog's depth-first search: each pass is a standard recursive traversal with no mutable lowlink state. The transposed graph is cheap to build since the dependency edges are already stored as Prolog facts.

12.4.4 How the scheduler works



1. DFS forward pass (record finish order)

2. DFS on transposed graph (reverse finish order)

3. Each DFS tree = one SCC; :run SCCs become merge-sets

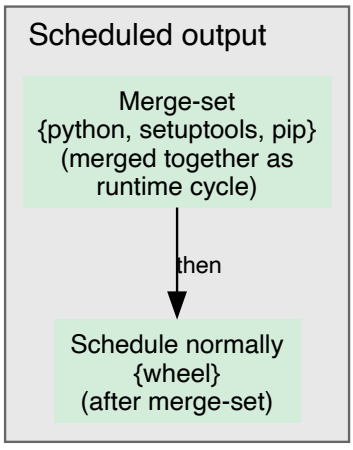


Figure 36 — SCC scheduling example

The scheduler uses Kosaraju's algorithm in four steps:

1. **Build the dependency graph** from the remainder rules — the literals the planner could not schedule and the edges between them.
2. **First DFS pass** — traverse the original graph depth-first, recording the order in which nodes finish (all children explored).
3. **Second DFS pass** — traverse the **transposed** graph (all edges reversed) in reverse finish order. Each DFS tree discovered in this pass is one SCC.
4. **Classify each SCC:**
   - **Single-node SCCs** are scheduled directly as regular plan entries.
   - **Multi-node SCCs** are examined for merge-set eligibility (see below).

12.4.5 Merge-sets

When a multi-node SCC consists entirely of `:run` (runtime) edges, the scheduler produces a **merge-set** — a group of packages that must be treated as available together. This matches how Gentoo's PMS handles runtime dependency cycles: the packages are merged as a group, and the cycle is not considered an ordering error.

The merge-set appears in the plan as a special group entry. The printer renders it with a cycle explanation so the user can see which packages form the loop and why they are grouped together.

12.5 Plan output

The final plan is a list of entries, each annotated with:

- **Wave number** — which parallel wave it belongs to
- **Action** — download, install, run, etc.
- **Literal** — the full `Repo://Entry:Action?{Context}` term
- **Group** — for merge-sets, which SCC group it belongs to

The plan is consumed by the printer for terminal output and by the builder for execution.

12.6 Further reading

- Chapter 8: The Prover — how the Proof AVL is constructed
- Chapter 13: Output and Visualization — how the plan is rendered
- Chapter 15: Building and Execution — how the plan is executed
- Chapter 22: Dependency Ordering — PMS ordering semantics

Chapter 13

Output and Visualization

After the prover completes and the planner produces a parallel plan, the next step is to present the result to the user. This happens before any building — even a `--pretend` run produces the full plan output. portage-ng offers several output formats: a colour-coded terminal plan, `.merge` files for regression testing, interactive SVG dependency graphs, Gantt charts, and structured reports.

13.1 Terminal plan display

The most common output is the terminal plan, which resembles `emerge -vp` but adds parallel wave information and richer detail. The printer (`Source/Pipeline/printer.pl`) orchestrates the display, delegating to submodules under `Source/Pipeline/Printer/`.

13.1.1 Merge list

The plan is rendered as a merge list. Each line represents one action, printed as a numbered step with a coloured **action bubble** and the full package atom.

A typical plan fragment looks like this:

```
└─step 01─┤ install  portage://dev-libs/expat-2.6.4
           │ install  portage://dev-libs/libxml2-2.13.5

└─step 02─┤ install  portage://dev-lang/python-3.12.3
           │ run      portage://dev-lang/python-3.12.3
           │ confirm  portage://dev-lang/python-3.12.3

└─step 03─┤ update   portage://sys-apps/portage-3.0.77-r3
           │          (replaces portage-3.0.65)
           │ download https://.../portage-3.0.77-r3.tar.xz

└─step 04─┤ verify   dev-python/tree-sitter
           │          (non-existent, assumed installed)
```

Actions within the same step can run concurrently — the step number is the wave. Each line shows:

- **Step number** — which parallel wave this action belongs to.
- **Action bubble** — a full word indicating the operation:
  - `install` — new install
  - `update` — version upgrade (shows the replaced version)

  - `downgrade` — version downgrade (shows the replaced version)
  - `reinstall` — reinstall of the same version
  - `run` — runtime dependency check
  - `confirm` — verify that a running dependency is available
  - `download` — fetch source from a mirror
  - `fetchonly` — fetch only, do not build
  - `verify` — assumed dependency that needs manual verification
- **Package atom** — repository, category, name, and version (e.g. `portage://dev-libs/openssl-3.1.4`).
- **Annotations** — contextual notes such as `(replaces ...)` for upgrades/downgrades, `(~amd64)` for keyword-accepted packages, `(USE modified)` for USE flag changes, or `(non-existent, assumed installed)` for unresolvable dependencies.

Target packages — the ones you explicitly asked to prove — appear in **bold green** with a green action bubble. Non-target dependencies use cyan text. Assumed or unresolvable dependencies use yellow or red bubbles to draw attention.

13.1.2 Printing styles

portage-ng supports two printing styles, selectable via `config:printing_style/1`:

- **Bubble style** (default) — a compact visual layout where each action line includes colour-coded indicators and right-edge annotations.
- **Column style** — a tabular layout that aligns version, slot, USE, and repository information in fixed columns for easy scanning.

13.1.3 Pre-action steps

Before the merge list, the printer can show **pre-action steps** — configuration changes that the plan assumes have been applied. These correspond to the `suggestion` tags from the prover (see Chapter 9):

- **Accept keyword** — packages that need `~arch` keyword acceptance.
- **Unmask** — packages that need unmasking.
- **USE changes** — flag changes needed in `package.use`.

Each pre-action step shows the exact line you would add to the corresponding `/etc/portage/package.*` file.

13.1.4 Summary line

At the bottom, a summary line shows the total number of actions (new installs, upgrades, reinstalls, etc.) and the predicted download and disk space usage.

13.2 Assumption and warning output

After the merge list, the printer shows any assumptions the prover had to make. These are grouped into two categories:

Domain assumptions are situations where the prover could not find a real solution and had to accept a literal on faith. Each assumption is printed with:

- The package or dependency that could not be satisfied.
- A classification label (e.g. "non-existent dependency", "REQUIRED_USE violation", "model unavailable").
- An actionable suggestion showing how to resolve the issue.

Cycle breaks are points where the prover broke a dependency cycle. Each cycle break shows the cycle path (which packages form the loop) and an explanation of why the cycle was broken rather than treated as benign.

The assumption type classification is handled by `Printer/Plan/assumption.pl`, and the detailed rendering by `Printer/Plan/warning.pl`. For a full description of the assumption taxonomy, see Chapter 9.

13.3 Writing module

The writer (`Source/Application/Output/writer.pl`) generates `.merge` files — one per target package — containing the portage-ng plan output in a format comparable to `emerge -vp` output. These files are stored in the graph directory configured by `config:graph_directory/2`.

`.merge` files serve two purposes:

- **Regression testing** — by comparing `.merge` files against `.emerge` files (the corresponding `emerge -vp` output for the same target), the compare tooling can detect regressions in dependency resolution accuracy. See Chapter 23 for the comparison workflow.
- **Offline review** — the files provide a persistent record of what portage-ng would do for each target, without needing to rerun the resolver.

13.4 Dependency graph generation

The grapher (`Source/Application/Output/grapher.pl`) produces interactive SVG dependency graphs that let you visually explore the dependency tree for a target.

The generation process has three stages:

1. **Edge extraction** — the proof is traversed to collect all dependency edges (which package depends on which, and through what action).
2. **DOT generation** — a `.dot` file is written with nodes representing packages and edges representing dependencies. Nodes are colour-coded by action type and annotated with version and slot information.
3. **SVG rendering** — the `dot` command (Graphviz) renders the DOT file into an SVG. For large graphs, platform-specific scripts under `Source/Application/System/Scripts/` can run multiple renderings in parallel.

Graph generation is triggered by the `--graph` CLI flag. The resulting SVGs include interactive navigation (zoom, pan, node highlighting) via a built-in JavaScript theme (`navtheme` module).

The screenshot below shows a dependency tree for `app-editors/vim`. The root package appears at the top with its dependencies fanning out below. Nodes are colour-coded: the red-bordered root, grey nodes for already-installed packages, and white nodes for packages that need to be merged. The toolbar at the top allows filtering by dependency type (BDEPEND, DEPEND, RDEPEND, etc.) and switching between graph views.

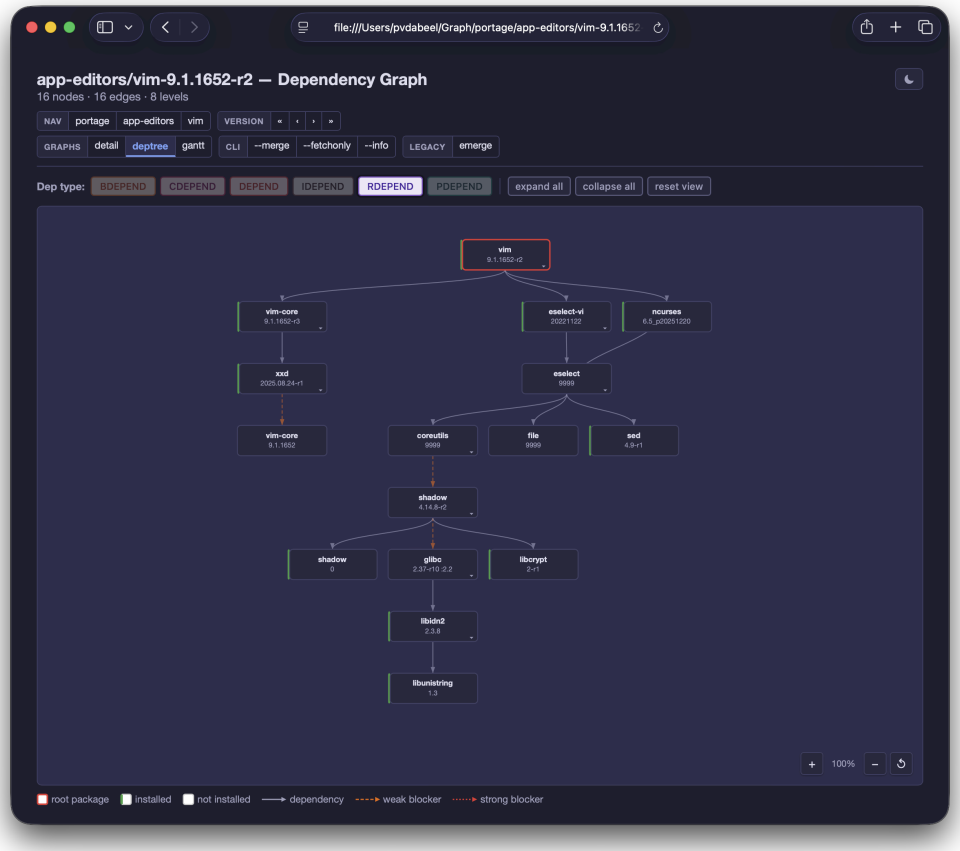


Figure 37 — Dependency graph for app-editors/vim

The **detail view** shows a single package with its full metadata — USE flags (with conditionals), candidate versions, installed status, and dependency atoms. This view is useful for understanding exactly which candidates are available and how USE conditionals affect the dependency tree.

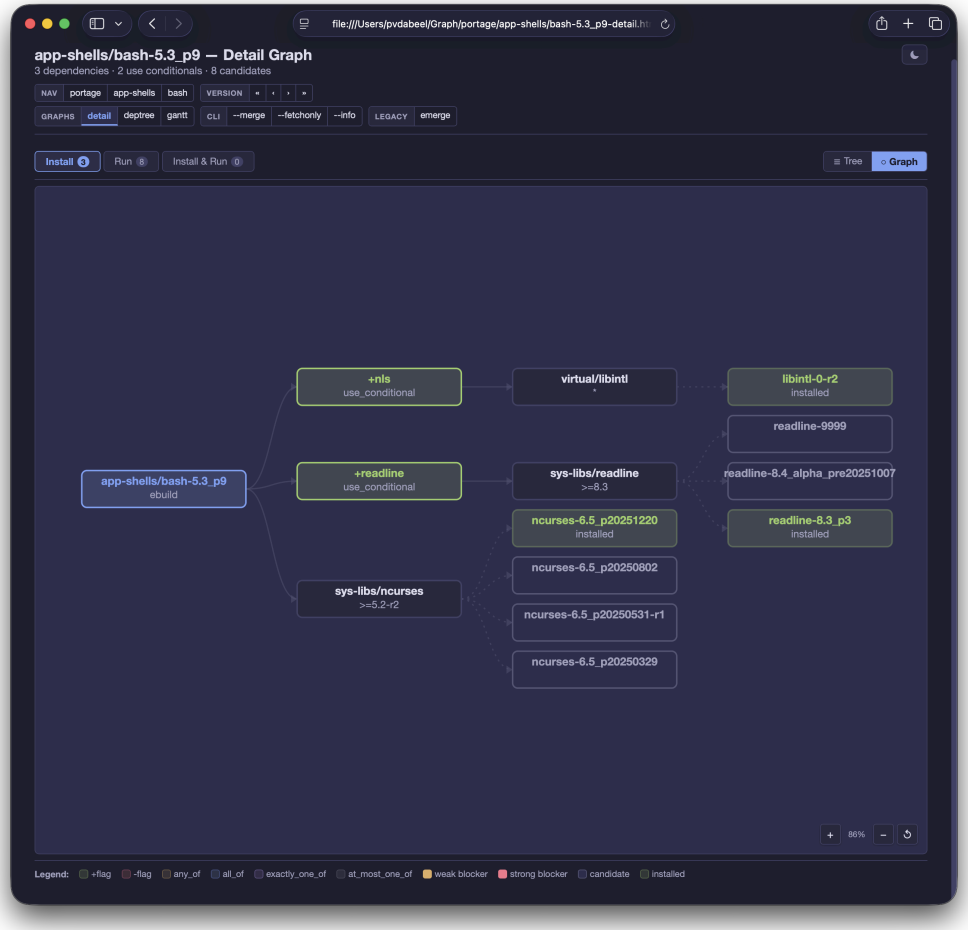


Figure 38 — Detail view for app-shells/bash

13.4.1 Graph submodules

Module	Purpose
`dot`	Graphviz DOT file generation with colour and layout
`deptree`	Hierarchical dependency tree visualisation
`detail`	Detailed single-package view with all metadata
`gantt`	Gantt chart rendering (see below)
`terminal`	Terminal-based ASCII graph rendering
`navtheme`	JavaScript navigation theme for interactive SVGs

13.5 Gantt charts

The `gantt` module produces Gantt charts that visualise the parallel build schedule computed by the planner. Each horizontal bar represents a package, positioned on a timeline according to its wave assignment and estimated build duration.

The chart makes the parallelism visible: packages in the same wave appear side by side, and you can see how downloads, installs, and runtime checks overlap across waves. When build time estimates are available (from VDB sizes or `emerge.log` history), the bar lengths reflect predicted durations.

The screenshot below shows the execution plan for `app-editors/neovim`. Each row is a package; colour-coded blocks show download (blue), install (green), and run (light green) phases across ten steps. Dependency edges are drawn as coloured curves linking the phases — red for DEPEND, blue for BDEPEND, green for RDEPEND — making it easy to see which packages gate others and where parallelism is exploited.

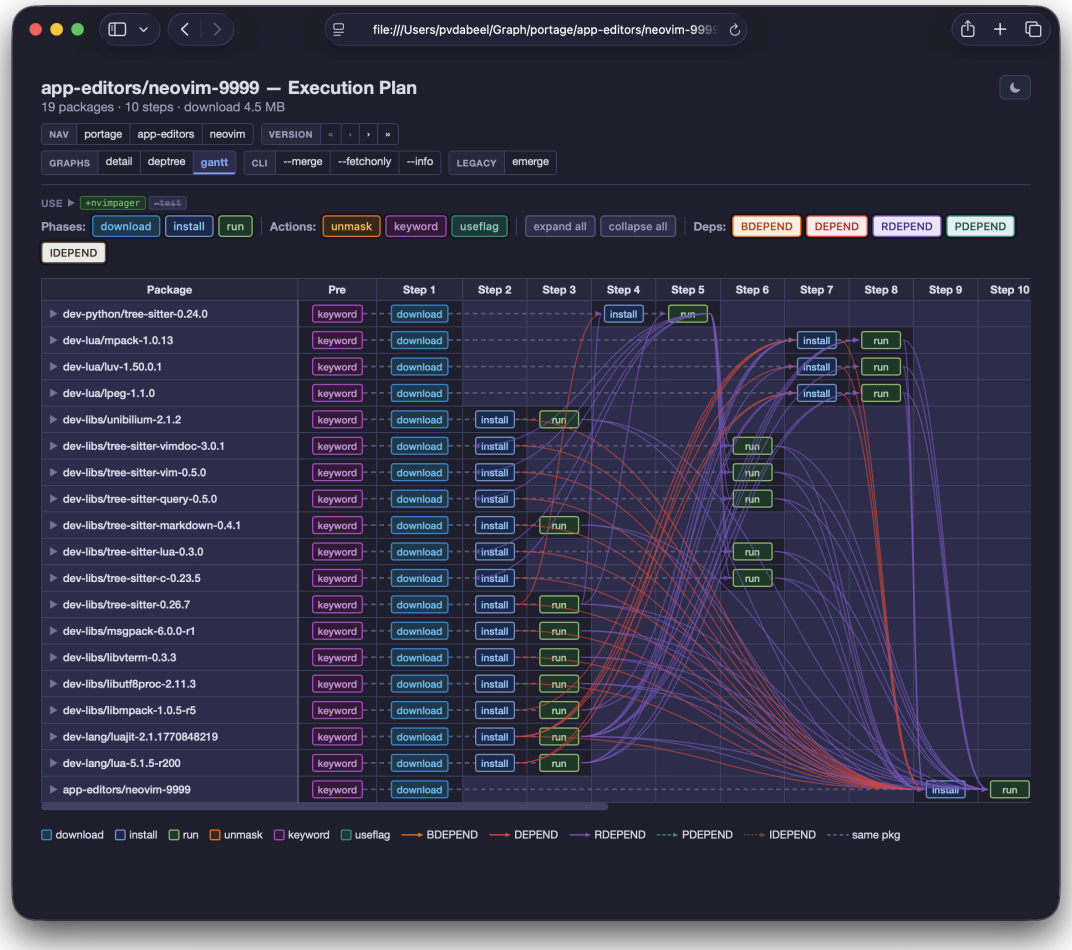


Figure 39 — Gantt chart for app-editors/neovim

13.6 Report generation

The report module (`Source/Application/Output/Report/report.pl`) generates structured reports for analysis, typically as JSON files. Reports can include:

- **Plan summaries** — total actions, waves, parallelism metrics.

- **Assumption breakdowns** — counts and details of domain assumptions, cycle breaks, and blocker conflicts.
- **Performance statistics** — proof time, plan time, cache hit rates, reprove retry counts.
- **Comparison data** — structured output consumed by the tinderbox-ng compare harness (`tinderbox-ng compare` / `tinderbox-ng analyze`), which diffs portage-ng plans against emerge output.

13.7 Printer submodules

The printer pipeline is split across focused submodules:

Module	File	Responsibility
`plan`	`Printer/Plan/plan.pl`	Plan rendering (waves, actions, USE flags)
`assumption`	`Printer/Plan/assumption.pl`	Assumption classification and display
`cycle`	`Printer/Plan/cycle.pl`	Cycle explanation rendering
`warning`	`Printer/Plan/warning.pl`	Assumption detail and warning blocks
`timing`	`Printer/Plan/timing.pl`	Build time display
`index`	`Printer/index.pl`	Package index display
`info`	`Printer/info.pl`	Package info display
`stats`	`Printer/stats.pl`	Statistics display
`state`	`Printer/state.pl`	State tracking during printing
`news`	`Printer/News/news.pl`	Gentoo news item display

13.8 Further reading

- Chapter 12: Planning and Scheduling — how waves and parallelism are computed
- Chapter 14: Command-Line Interface — `--graph`, `--verbose`, `--quiet`, and other output flags
- Chapter 15: Building and Execution — how the plan is executed
- Chapter 23: Testing and Regression — how `.merge` files are used for regression testing

Chapter 14

Command-Line Interface

portage-ng is meant to sit beside Portage, not replace it in name or habit. Many flags will feel immediately familiar: `--pretend`, `--verbose`, `--emptytree`, and the usual resolution switches mirror what you already use with emerge-style workflows. On top of that, a proof-based resolver can expose tools that a traditional dependency solver does not: `--explain` and `--llm` for plan dialogue, `--variants` for USE-sensitive alternatives, and `--search` that can treat a phrase as a natural-language query when structured parsing does not apply.

The CLI is organized around one idea: **every invocation either reasons about packages or acts on them.** Reasoning covers dry-runs, search, similarity, estimates, upstream checks, Bugzilla lookup, and anything that inspects the knowledge base without changing the system. Acting covers merge, unmerge, depclean, fetch-only, and sync-style maintenance. Keeping that distinction in mind makes it easier to choose flags and to script portage-ng safely (often pairing `--pretend` with exploratory options before any real merge).

14.1 Modes

portage-ng operates in one of six modes, selected with `--mode`:

Mode	Description
`standalone`	Full local operation — the default and most common mode
`daemon`	Persistent daemon serving IPC clients via Unix socket
`ipc`	Thin IPC client forwarding requests to a running daemon
`client`	Remote RPC client connecting to a server over HTTPS
`worker`	Compute node for distributed proving (polls server for jobs)
`server`	HTTP + Pengines server with job/result queues

14.1.1 Standalone

The default mode. Loads the full pipeline, knowledge base, LLM modules, and domain logic into a single process. All resolution, planning, graph generation, and building happens locally. Every CLI action (`--pretend`, `--sync`, `--graph`, `--shell`, etc.) is available.

14.1.2 Daemon

Keeps the same in-memory footprint as standalone — full knowledge base, resolver, planner — but listens on a **Unix domain socket** for incoming requests. Use `--background` to fork the daemon into a detached process. The daemon avoids the startup cost of reloading the knowledge base on every invocation, making repeated queries fast.

14.1.3 IPC

A thin front-end that does **not** load the full resolver stack. It connects to a running daemon over the Unix socket, forwards the command-line arguments and environment, streams output back, and exits with the daemon's exit code. If `--background` auto-start is configured and no daemon is listening, the IPC client can launch one automatically. Note that `--shell` is not supported in IPC mode.

14.1.4 Client

A lightweight process that treats a remote **server** as the source of truth for the knowledge base. Local queries are proxied over HTTPS using Pengine RPC (with TLS certificates and digest authentication). The client loads enough of the pipeline to drive the CLI, but proving and KB access happen on the server side. Use `--host` and `--port` to specify the server.

14.1.5 Server

Runs the full standalone pipeline first (local KB, resolver, planner), then adds an HTTPS Pengine server, TLS, and Bonjour service advertisement. The server exposes job and result message queues so that workers can poll for proving tasks. Use `--background` to fork the server process. See Chapter 17: Distributed Proving.

14.1.6 Worker

A compute node that loads the full proving pipeline locally (like standalone) plus an RPC client for server communication. On startup, the worker discovers the server via Bonjour or explicit `--host`/`--port`, syncs its local portage tree to the server's snapshot, registers its CPU count, and spawns one thread per core. Each thread polls the server for jobs, proves them locally, and posts results back. See Chapter 17: Distributed Proving.

14.2 Actions

Actions are grouped by area. Use the tables below as a quick map from flags to behaviour; the sections that follow add context on targets, search, and everyday workflows.

14.2.1 Merge and resolution

Flag	Action
`--pretend`	Generate and display a build plan (dry-run)
`--merge`	Execute the build plan
`--unmerge <target>`	Remove a package
`--depclean`	Remove unneeded packages
`--fetchonly`	Fetch source archives only

14.2.2 Information

Flag	Action
`--search <query>`	Search packages (supports natural-language via embeddings)
`--similar <target>`	Find packages similar to target (vector similarity)
`--info`	Display repository statistics and configuration
`--installed`	List installed packages

14.2.3 Repository management

Flag	Action
`--sync`	Sync the Portage tree and regenerate caches
`--regen`	Regenerate md5-cache incrementally

14.2.4 Visualization

Flag	Action
`--graph`	Generate interactive SVG dependency graphs
`--estimate`	Show build time estimates

14.2.5 Diagnostics

Flag	Action
`--bugs <target>`	Search Gentoo Bugzilla for known issues
`--upstream <target>`	Check upstream versions via Repology
`--explain` / `--llm`	Get AI-assisted plan explanation
`--variants`	Show plan variants with different USE configurations
`--shell`	Drop into an interactive Prolog shell

14.3 Options

14.3.1 Resolution options

Flag	Effect
`--emptytree`	Prove all dependencies from scratch (ignore VDB)
`--onlydeps`	Prove only dependencies, not the target itself
`--deep`	Deep dependency resolution
`--newuse`	Detect USE flag changes requiring rebuilds
`--update`	Update to newest version

14.3.2 Output options

Flag	Effect
`--verbose`	Verbose output (show USE flags, slot info)
`--quiet`	Minimal output
`--ci`	Non-interactive CI mode (exit codes 0/1/2)
`--jobs N`	Number of parallel jobs
`--timeout N`	Kill after N seconds (requires Python 3)

14.4 Target syntax

Targets can be specified in several formats:

Format	Example	Meaning
`cat/pkg`	`sys-apps/portage`	Resolve latest version
`=cat/pkg-ver`	`=sys-apps/portage-3.0.77`	Exact version
`>=cat/pkg-ver`	`>=dev-lang/python-3.10`	Version constraint
`@set`	`@world`	Package set
`pkg`	`portage`	Ambiguous name (searched across categories)

14.5 CI mode

Use `--ci` for non-interactive automation. Exit codes indicate plan quality:

Code	Meaning
0	Plan completed with no assumptions
1	Plan completed with prover cycle-break assumptions only
2	Plan completed with domain assumptions (e.g. missing deps)

Example:

```
portage-ng --ci --pretend sys-apps/portage
echo $?  # 0, 1, or 2
```

By default, portage-ng runs in standalone mode. Other modes (distributed client, server, worker) are covered in the advanced topics chapters.

14.6 The dev wrapper

When running from a source checkout, use the dev wrapper instead of the installed binary:

```
./Source/Application/Wrapper/portage-ng-dev --pretend sys-apps/portage
```

The wrapper sets up the correct load paths, stack limits, and Prolog flags. It also supports `--timeout N` (requires Python 3) to kill the process after N seconds. For reproducible, non-interactive runs, pipe queries via a here-doc:

```
./Source/Application/Wrapper/portage-ng-dev --shell --timeout 60 <<'PL'
prover:test_stats(portage).
halt.
PL
```

14.7 Tips and tricks

Short recipes that match how people actually use the tool:

- **What does portage-ng think about this package?**
  `portage-ng --pretend --verbose cat/pkg` — full plan with enough detail to compare against emerge-style output.
- **Why is this package in my plan?**
  `portage-ng --pretend --explain cat/pkg` — ask the explainer/LLM path to narrate the plan (see Chapter 16: Semantic Search and LLM Integration).
- **What would change if I enabled this USE flag?**
  `portage-ng --pretend --variants cat/pkg` — surface alternative proofs when USE sets differ.
- **Find packages related to *X***
  `portage-ng --search "X"` — natural-language / semantic search when the query is not structured (requires embeddings; same chapter as above). For an exact package name, use a structured atom such as `name=vim` (the same intent as "`name:X`" in prose, but the CLI grammar uses `=` for equality, not a single `name:X` token). Category and other fields work the same way (`category=…`); see Search query language below.
- **Show me similar packages**
  `portage-ng --similar cat/pkg` — vector similarity from the same embedding stack as semantic search.

- **Quick scripted session**
  Here-doc into the Prolog shell so the full load graph matches interactive use:

```
portage-ng --mode standalone --shell <<'PL'
prover:test_stats(portage).
halt.
PL
```

- **CI / automation**
  `portage-ng --ci --pretend cat/pkg` — non-interactive; interpret exit codes: `0` no assumptions, `1` cycle-break assumptions only, `2` domain assumptions present.
- **Estimate build time**
  `portage-ng --estimate cat/pkg` — build-time hints from VDB and history.
- **Check for upstream updates**
  `portage-ng --upstream cat/pkg` — Repology-oriented upstream comparison.
- **Search Bugzilla**
  `portage-ng --bugs cat/pkg` — Bugzilla-oriented diagnostics for the target.

14.8 Search query language

The `--search` flag accepts **structured** queries built from one or more command-line atoms. Each atom is a *key*, a *comparator*, and a *value* (see Fuzzy and wildcard search for the comparators). When the argument list does **not** parse as that structured form, the text is joined and passed to **semantic** (natural-language) search instead.

```
portage-ng --search name=vim category=app-editors
portage-ng --search license=GPL-2 keywords=amd64
portage-ng --search "text editor with syntax highlighting"  # semantic search
```

Semantic search requires Ollama with a loaded embedding model. See Chapter 16: Semantic Search and LLM Integration.

14.8.1 Fuzzy and wildcard search

Structured search uses explicit comparators on the key:

Comparator	Meaning	Example
`=`	Exact match on the value	`name=vim`
`~`	Fuzzy match (approximate / substring-style, key-dependent)	`name~vim`
`:=`	Wildcard match (`*` in the value)	`name:=*vim*`

Exact search — constrain the package name or another field precisely, e.g. `--search name=vim` (exact package name). In documentation you may see this described informally as `name:vim`; on the command line the equality comparator is `=` (`:` introduces the `:=` wildcard operator instead).

Category filter — `category=app-editors` (or combine with other atoms on the same command line).

Natural language — a query that does not parse as structured keys, e.g. `--search "text editor with syntax highlighting"`, uses vector embeddings over the knowledge base (when enabled and indexed).

Wildcard — use `:=` so `*` is interpreted as a glob-style wildcard, e.g. `name:=*vim*` for any package name containing `vim`. Quote the atom if the shell would expand `*` (e.g. `--search 'name:=*vim*'`).

Combined filters — pass several atoms; each narrows the result set, e.g. `category=dev-libs name:=*ssl*`.

14.9 Further reading

- `portage-ng(1)` manpage — exhaustive option reference
- Chapter 2: Installation and Quick Start — first run examples
- Chapter 13: Output and Visualization — what the output looks like

Chapter 15

Building and Execution

portage-ng is a self-contained dependency resolver and planner. It ships its own code for every stage up to the point where source code must actually be compiled:

- **Cache generation** — portage-ng includes its own md5-cache generator, so it does not depend on Portage's `egencache` or any other external tool to produce the cache files it reasons over.
- **Dependency resolution and planning** — the prover, planner, and scheduler are entirely internal (see Chapters 8-12).
- **Downloading** — source archive fetching, mirror selection, hash verification, and resume are handled by portage-ng's own download module (see Download management below).

The only point where Portage is needed is the **execution of ebuild build phases** (unpack, compile, install, qmerge, etc.). These phases rely on Portage's `ebuild` command and its ecosystem of eclasses and phase functions. portage-ng delegates to that infrastructure so that the full ebuild ecosystem works unchanged, but everything before and after the build steps — dependency calculation, plan ordering, downloading, and output — is handled independently.

15.1 Build delegation

When executing a plan (via `--merge` rather than `--pretend`), the builder module invokes the `ebuild` command for each action in the plan. The command is configurable via `config:ebuild_command/1` (default: `ebuild`).

The builder processes the plan wave by wave, respecting the parallelism computed by the planner. Within each wave, independent actions can run concurrently.

15.2 Ebuild phase execution

The `ebuild_exec.pl` module handles the actual invocation of ebuild phases:

Phase	ebuild command	When
`setup`	`ebuild <path> setup`	Before building
`unpack`	`ebuild <path> unpack`	Extract source archives
`prepare`	`ebuild <path> prepare`	Apply patches
`configure`	`ebuild <path> configure`	Run configure scripts
`compile`	`ebuild <path> compile`	Build from source
`install`	`ebuild <path> install`	Install to staging area
`qmerge`	`ebuild <path> qmerge`	Merge to live filesystem

Phases are executed via `process_create` with output captured for logging. The builder uses `sh` to wrap `ebuild` calls with redirection for asynchronous, logged execution.

15.3 Live build display

During a `--merge` run, portage-ng keeps the terminal display up-to-date so you can see exactly where the build process stands at any moment. The static plan that was printed during the `--pretend` phase is reprinted once, and below it a live "Executing" area shows the current state of every active build slot.

The following example shows the pretend output for `sys-kernel/gentoo-sources`. The plan has three steps: download the source tarball plus patches, install the package, and register the runtime phase.

```
>>> Emerging : portage://sys-kernel/gentoo-sources-6.19.11:run?{[]}

These are the packages that would be merged, in order:

Calculating dependencies... done!

 └─[step  1]─┤ download  portage://sys-kernel/gentoo-sources-6.19.11
             │           └ file ─┤ 877.73 Kb   genpatches-6.19-10.base.tar.xz
             │                   │ 4.22 Kb      genpatches-6.19-10.extras.tar.xz
             │                   │ 148.84 Mb    linux-6.19.tar.xz

 └─[step  2]─┤ install   portage://sys-kernel/gentoo-sources-6.19.11
             │           └ conf ─┤ USE = "build symlink -experimental"
             │                   │ SLOT = "6.19.11"

 └─[step  3]─┤ run       portage://sys-kernel/gentoo-sources-6.19.11

Total: 3 actions (1 download, 1 install, 1 run), grouped into 3 steps.
       149.70 Mb to be downloaded.
```

When the same target is merged with `--merge`, the display turns into a live view. Each step gains a phase line showing the individual ebuild phases and their current state. A snapshot mid-build might look like this:

```
These are the packages being merged, in order:

Executing 3 actions, grouped into 3 steps...

 └─[step  1]─┤ download  portage://sys-kernel/gentoo-sources-6.19.11    ✓
             │           └ file ─┤ 877.73 Kb   genpatches-6.19-10.base.tar.xz     ✓
             │                   │ 4.22 Kb      genpatches-6.19-10.extras.tar.xz
✓
             │                   │ 148.84 Mb    linux-6.19.tar.xz
✓

 └─[step  2]─┤ install   portage://sys-kernel/gentoo-sources-6.19.11    ?
             │           └ exec ─┤ ACTION = setup → unpack → prepare   (42%) 2/7
             │                   │ LOG = /var/log/portage/sys-kernel:gentoo-
sources.log

 └─[step  3]─┤ run       portage://sys-kernel/gentoo-sources-6.19.11
```

In this snapshot, step 1 (download) has completed — each file shows a green check mark on the right edge. Step 2 (install) is active: the action line shows a spinning indicator, and the phase line reveals that `setup` and `unpack` have finished (shown in cyan) while `prepare` is the current phase. The right edge displays the accumulated progress (`42%`) and a phase counter (`2/7`). Step 3 (run) is still pending in dark grey, waiting for the install to finish.

15.3.1 Slot states and colours

Each slot in the live display represents one concurrent build. The slot line changes colour and icon as the build progresses:

State	Colour	Indicator
Pending	Dark grey	Waiting for prerequisites
Active	Cyan (action) + green (target)	Spinning indicator on the right edge
Done	Green	Check mark
Failed	Red	Exclamation mark
Stub	Grey	Phase skipped (already satisfied)

15.3.2 Per-ebuild phase tracking

Below each slot line, the display shows the individual ebuild phases (setup, unpack, prepare, configure, compile, install, qmerge) with their current status. Each phase word is coloured independently:

- **Dark grey** — pending (not yet started)
- **Cyan** — active or in progress
- **Green** — completed successfully
- **Red** — failed

The builder tracks phase state through `builder:exec_phase_state/3`, which is updated by a callback from the ebuild execution module as each phase starts, progresses, and finishes.

15.3.3 Progress indicators

portage-ng shows progress at multiple levels:

- **Per-phase percentage** — during long phases like `compile`, the builder polls the build log every 0.5 seconds and computes a progress estimate. This blends two signals: the growth of the log file (bytes written) and historical data from previous builds of the same package (stored in `Knowledge/phase_stats.pl`).
- **Overall progress** — the right edge of the display shows an accumulated percentage and a counter (`Current/Total`) reflecting how many actions have completed out of the total plan. Stub actions (already satisfied) are excluded from the total.
- **Download progress** — for parallel downloads, each file shows a percentage and transfer speed. Git clones show a separate percentage based on the git progress output.

15.3.4 Log file locations

Each build action writes its output to a log file. The path is computed from the build log directory (`config:build_log_dir/1`) and the ebuild name. When `--logs` is enabled, the log path is displayed below the phase line for each slot. If a phase fails, the log path turns red so you can quickly find the relevant output.

15.3.5 Terminal refresh

The live display uses ANSI cursor movement to update individual lines in place: the builder moves the cursor up to the target line, redraws it, and moves back down. This avoids flooding the terminal with repeated full-screen redraws. All display mutations go through a `build_display` mutex to prevent concurrent workers from interleaving their output.

In non-TTY environments (e.g. CI pipelines), cursor movement is disabled and the builder falls back to sparse status lines.

15.4 Build time estimation

The `buildtime.pl` module predicts build duration from two data sources:

1. **VDB sizes** — the installed file sizes from `/var/db/pkg/*/SIZE` correlate with build complexity.
2. **emerge.log history** — historical build times from `/var/log/emerge.log` provide empirical timing data for packages that have been built before.

The `--estimate` CLI option shows predicted build times in the plan output.

15.5 Jobserver

The `jobserver.pl` module manages parallel build execution. It implements a token-based jobserver that limits concurrent builds to the number of available cores (or a user-specified `--jobs` count).

15.6 Download management

The `download.pl` module handles source archive fetching:

- Mirror layout detection via `curl`
- Parallel downloads across multiple mirrors
- Hash verification via `openssl dgst`
- Resume support for interrupted downloads

Downloads are scheduled as early as possible in the plan — the planner treats `:download` actions as the first wave, so packages can download while others are building.

15.7 Snapshot support

Upgrades can go wrong — a new version may fail to compile, introduce regressions, or break other packages. portage-ng's snapshot module (`Source/Pipeline/Builder/snapshot.pl`) lets you freeze the current system state before a merge and roll back to it afterwards.

15.7.1 How a snapshot is created

When a merge begins, portage-ng automatically creates a snapshot identified by a timestamp (e.g. `20260405-143012`). The snapshot directory contains three files:

- **`manifest.pl`** — a Prolog fact file listing every package currently installed in the VDB, with category, name, version, and slot.
- **`world`** — a copy of the current world set file, so the set of explicitly requested packages can be restored exactly.
- **`actions.pl`** — the planned actions for the merge, recorded so that a rollback knows which packages were touched.

15.7.2 Quickpkg: preserving the old version

The key to rollback is preserving the **binary package** of each package that is about to be replaced. Before portage-ng merges a new version, the builder calls `snapshot:quickpkg_old/2`. This runs `ebuild --skip-manifest <old-ebuild> package` with `PKGDIR` pointed at the snapshot's `binpkgs/` directory. The result is a tarball (`.tbz2` or `.gpkg.tar`) that contains the currently installed files of the old version — essentially the same operation that Gentoo's `quickpkg` tool performs.

Because this happens **per package, just before the upgrade**, the snapshot accumulates exactly the set of binary packages needed to reverse the merge. Packages that were not touched are not quickpkg'd; they remain unchanged on the system.

15.7.3 Listing and diffing snapshots

`--snapshot list` shows all available snapshots with their timestamp, installed package count, and the number of binary packages stored:

```
Available snapshots:
  20260405-143012       2026-04-05 14:30:12   1847 pkgs   12 binpkgs
  20260402-091544       2026-04-02 09:15:44   1843 pkgs    5 binpkgs
```

`--snapshot diff <id>` compares a snapshot's manifest against the current VDB and shows what changed — packages installed since the snapshot, packages removed, and packages whose version changed:

```
Diff against snapshot "20260405-143012":

  Installed since snapshot (2):
    + dev-libs/newlib-4.5.0
    + dev-util/newtool-1.0

  Version changed since snapshot (3):
    ~ sys-libs/glibc  2.40-r2 -> 2.41
    ~ dev-lang/python  3.12.8 -> 3.13.1
    ~ app-editors/vim  9.1.1652-r2 -> 9.1.1700

  Summary: +2 -0 ~3 (3 binpkgs available for rollback)
```

15.7.4 Rolling back

`--snapshot rollback <id>` reinstalls the saved binary packages from the snapshot's `binpkgs/` directory and restores the world set file. Each binary package is merged back onto the system via `ebuild <binpkg> merge`, downgrading the affected packages to their pre-upgrade versions. Combined with `--pretend`, the rollback shows what would be reinstalled without actually making changes.

15.7.5 Lifecycle

After the merge completes (whether successfully or not), the snapshot remains on disk so it can be used for rollback at any later time. Old snapshots can be removed with `--snapshot delete <id>` to reclaim disk space.

15.8 Further reading

- Chapter 12: Planning and Scheduling — how the plan is constructed
- Chapter 13: Output and Visualization — plan display and `.merge` file generation
- Chapter 14: Command-Line Interface — `--merge`, `--jobs`, `--estimate` flags

Chapter 16

Semantic Search and LLM Integration

portage-ng integrates with large language models for two purposes: **semantic search** over the knowledge base using vector embeddings, and **plan explanation** using natural-language generation.

16.1 Semantic search

The semantic search module (`Source/Application/Llm/semantic.pl`) enables natural-language queries over the package knowledge base.

16.1.1 How it works

1. Package descriptions are converted to vector embeddings via Ollama's embedding API (default endpoint: `http://localhost:11434`).
2. Embeddings are stored in an in-memory index.
3. At query time, the search query is embedded and compared against all package embeddings using cosine similarity.
4. Results are ranked by similarity score.

16.1.2 Usage

```
# Natural-language search
portage-ng --search "text editor with syntax highlighting"

# Find packages similar to a known package
portage-ng --similar app-editors/neovim
```

On Apple Silicon, Ollama leverages the GPU and Neural Engine for accelerated embedding computation.

16.1.3 Prerequisites

Semantic search requires: - A running Ollama instance - A loaded embedding model (configured via `config:embedding_model/1`)

16.2 LLM-assisted plan explanation

The `--explain` / `--llm` flags send proof artifacts to an LLM for human-readable interpretation of build plans and assumptions.

16.2.1 Provider backends

portage-ng supports multiple LLM providers, each implemented as a separate module in `Source/Application/Llm/`:

Module	Provider	Notes
`ollama.pl`	Ollama	Local inference; also provides embeddings
`claude.pl`	Anthropic Claude	Requires API key
`chatgpt.pl`	OpenAI ChatGPT	Requires API key
`gemini.pl`	Google Gemini	Requires API key
`grok.pl`	xAI Grok	Requires API key

The default provider is set via `config:llm_default/1`. API keys and endpoints are configured in `Source/config.pl` or via `Source/Config/Private/` template files.

16.3 Calling LLMs

portage-ng offers three ways to interact with an LLM.

16.3.1 Interactive chat (`--llm`)

The `--llm` flag opens an interactive chat session with the default (or a named) LLM service. The session maintains conversation history, so follow-up questions have context:

```
# Chat with the default LLM (configured in config.pl)
portage-ng --llm

# Chat with a specific service
portage-ng --llm claude
portage-ng --llm ollama
```

Inside the session, the LLM streams its response word by word to the terminal. Type `quit` or `exit` to leave.

16.3.2 Plan explanation (`--explain`)

The `--explain` flag resolves a target first, then feeds the full build plan to the LLM as structured context. The user can then ask questions about the plan in a conversational loop:

```
# Explain a build plan interactively
portage-ng --pretend --explain dev-libs/openssl

# Single-shot question
portage-ng --pretend --explain "Why is zlib in the plan?" dev-libs/openssl
```

The plan context includes targets, every package with its action type and USE flags, dependency chains traced back to the root, and any assumptions the prover made. The LLM sees

the full picture and can answer questions like "why is this package included?", "what would happen if I disabled this USE flag?", or "which assumptions were made?"

16.3.3 Programmatic access

From the Prolog shell, any LLM can be called directly:

```
% Call a specific service
claude("Explain what dev-libs/openssl does", Response).
ollama("What is a USE flag?", Response).
grok("Compare DEPEND and RDEPEND", Response).

% Or use the generic dispatcher
explainer:call_llm(claude, "Your question here", Response).
```

Each service maintains its own conversation history, so subsequent calls build on previous context.

16.4 Code execution in the sandbox

One of portage-ng's most distinctive LLM features is that an LLM can **execute Prolog code** inside the running system and receive the output. This turns the LLM from a passive question-answerer into an active investigator that can query the knowledge base, inspect proof artifacts, and verify its own answers.

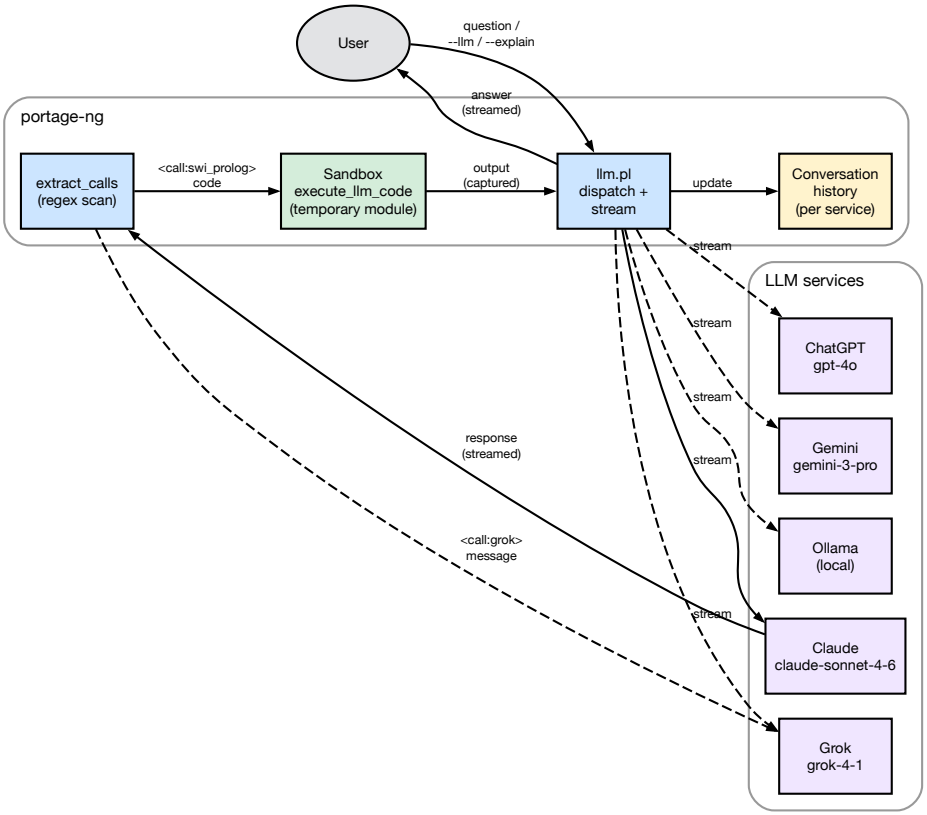


Figure 40 — LLM interaction and code execution

16.4.1 How it works

When an LLM responds, portage-ng scans the response for XML-style call tags. Two kinds are recognized:

- **`<call:swi_prolog>` … `</call:swi_prolog>`** — the enclosed Prolog code is executed in a temporary, sandboxed module. The output (both standard output and error) is captured and sent back to the LLM as a function response.
- **`<call:claude>` / `<call:grok>` / etc.** — the enclosed message is forwarded to another LLM service, and that service's response is sent back. This allows LLMs to consult each other.

The LLM is informed of these capabilities through a system prompt assembled from `config:llm_capability/2` clauses. The prompt explains the available tags and how to use them, without the user needing to know about the mechanism.

16.4.2 Sandbox safety

Code execution is sandboxed by default (`config:llm_sandboxed_execution(true)`). The code runs in a temporary module created by SWI-Prolog's `in_temporary_module/3`, with `sandboxed(true)` ensuring that only safe predicates are accessible. The module is destroyed after execution. This means the LLM can:

- Query the knowledge base (`cache:ordered_entry/5`, `query:search/2`)
- Inspect proof artifacts if they are in scope

- Perform computations and formatting
- Write output that gets captured and returned

But it **cannot** modify the file system, execute shell commands, or alter the running system's state.

16.4.3 Example interaction

The diagram below traces a single round trip. The user asks a question; portage-ng forwards it (with a system prompt listing the available capabilities) to the LLM. The LLM responds with embedded Prolog code wrapped in a `<call:swi_prolog>` tag. portage-ng detects the tag, executes the code in a sandboxed temporary module, captures the output, and sends it back to the LLM as a function result. The LLM then incorporates the verified fact into its final answer, which is streamed back to the user.

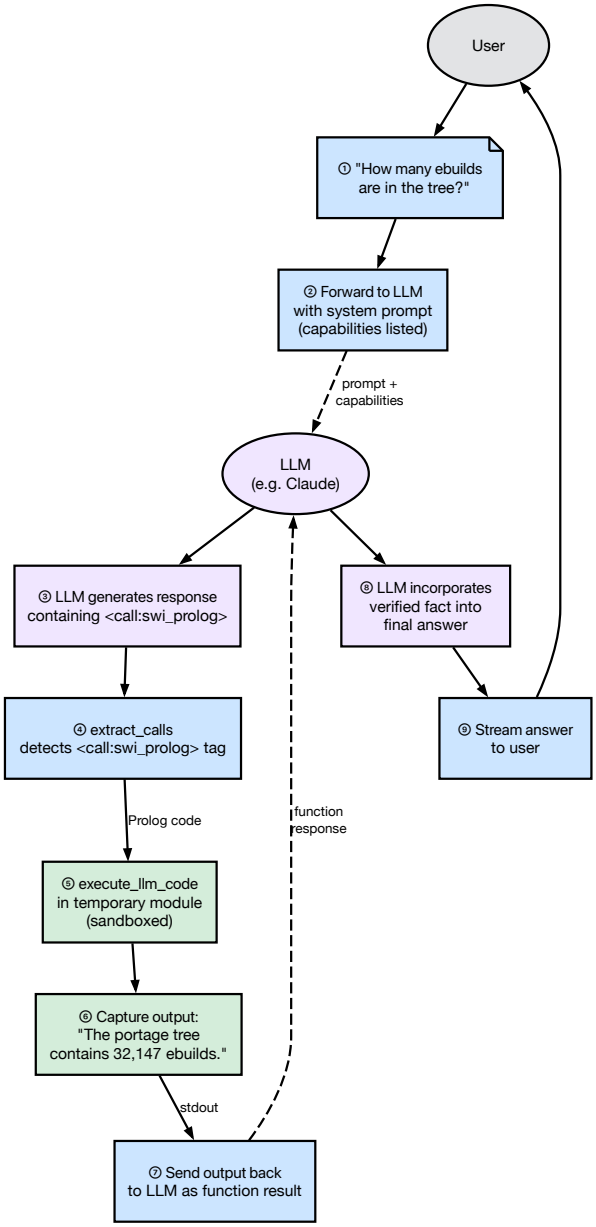


Figure 41 — Code execution round trip

Concretely, a user asks "How many ebuilds are in the tree?" The LLM responds with:

```
<call:swi_prolog>
:- aggregate_all(count, cache:ordered_entry(portage, _, _, _, _), N),
   format('The portage tree contains ~w ebuilds.~n', [N]).
</call:swi_prolog>
```

portage-ng executes this code, captures the output ("The portage tree contains 32,147 ebuilds."), and sends it back to the LLM. The LLM then incorporates this verified fact into its final answer to the user.

16.4.4 Inter-LLM communication

The same tag mechanism allows LLMs to delegate questions to each other. The diagram below shows the flow: the primary LLM (Claude in this example) embeds a `<call:ollama>` tag in its response. portage-ng detects the tag, forwards the enclosed message to Ollama, collects Ollama's response, and sends it back to Claude as a function result. Claude then weaves both perspectives into its final answer.

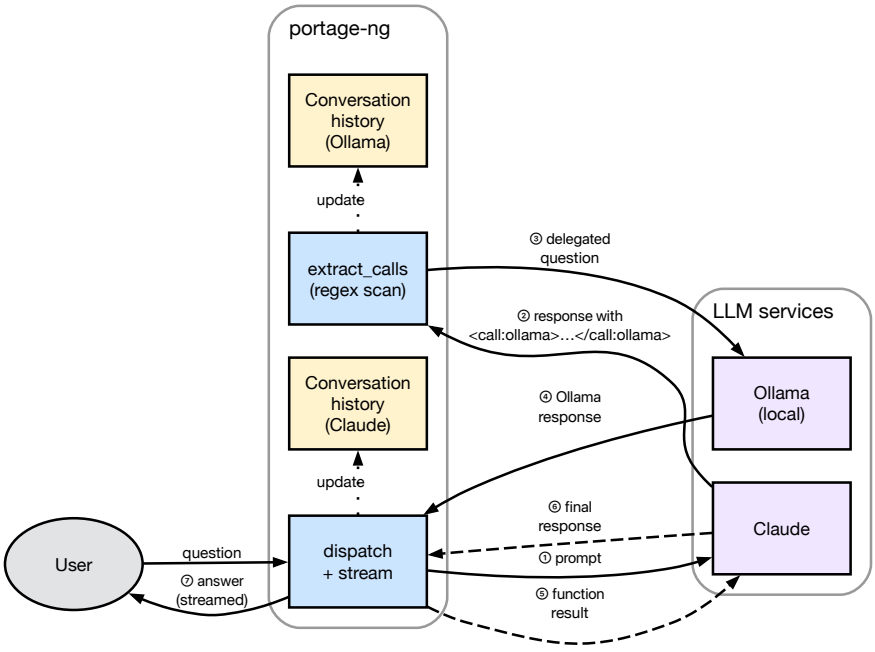


Figure 42 — Inter-LLM communication

For example, a Claude session might embed `<call:ollama>Summarize this dependency chain</call:ollama>` to get a local model's perspective, or `<call:grok>What is the latest version of openssl?</call:grok>` to cross-check information. Each delegated call maintains the target service's own conversation history.

16.5 Suggestions and explanations

Beyond answering questions, the LLM integration supports two practical workflows: **plan explanation** and **failure diagnosis**.

16.5.1 Plan explanation

When `--explain` is active, portage-ng assembles a structured context from the proof artifacts that includes:

- **Targets** — the packages the user requested
- **Plan entries** — every package in the merge plan with its step number, action type (install/run/download), USE flags (with annotations for user-set, profile-forced, and resolver-changed flags), slot information, and dependency chain

- **Reverse dependency paths** — for each package, the chain of dependencies tracing back to the root target ("zlib is needed by openssl, which is needed by python, which is the target")
- **Assumptions** — any domain assumptions or cycle breaks, with classified reasons (missing package, masked, keyword-filtered, slot mismatch, version conflict)

This context allows the LLM to give precise, grounded answers rather than generic advice. When a user asks "why is zlib in the plan?", the LLM can point to the exact dependency chain rather than guessing.

16.5.2 Failure diagnosis

When the resolver cannot satisfy a dependency, it records an `assumption_reason` in the proof context. The explainer's `assumption_reason_for_grouped_dep/6` predicate diagnoses the failure by progressively filtering candidates through existence checks, mask checks, slot restrictions, version constraints, and keyword acceptance. The classified reason (e.g. `missing`, `masked`, `keyword_filtered`, `version_conflict`) is attached to the assumption and included in the LLM context.

When the `--llm` flag is combined with a failing target, portage-ng sends a specialized diagnostic prompt (`config:llm_support/1`) to the LLM, providing the failure details and asking for help identifying the correct package atom or diagnosing the issue.

16.6 Explainer architecture

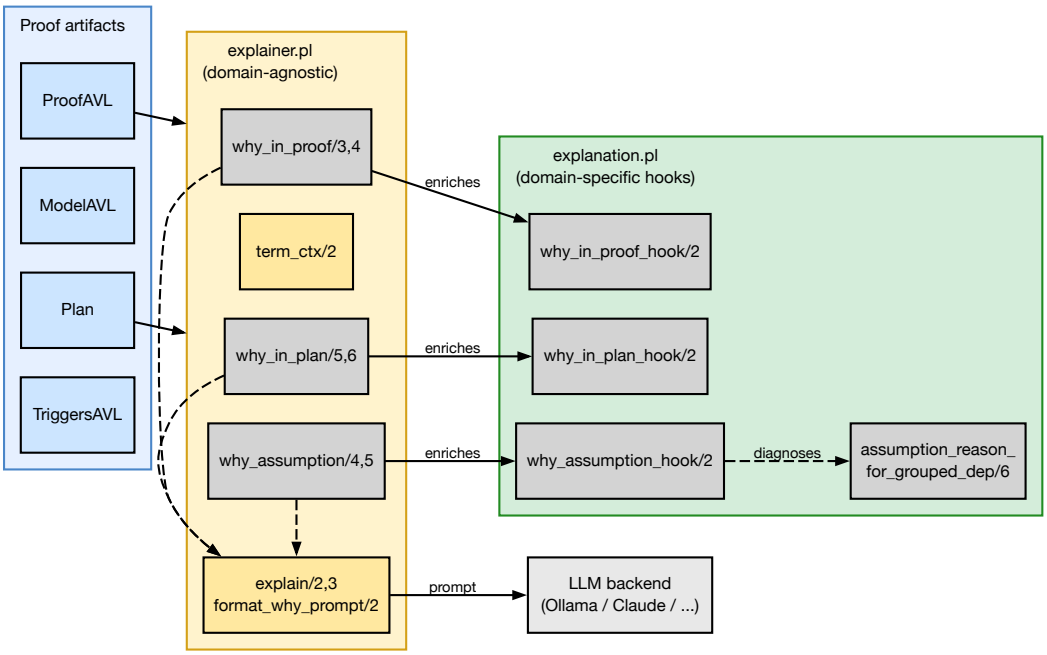


Figure 43 — Explainer architecture

The explainer is split into two modules with clearly separated responsibilities.

16.6.1 `explainer.pl` — domain-agnostic introspection

This module answers "why" questions by inspecting the proof artifacts (ProofAVL, ModelAVL, Plan, TriggersAVL) without knowing anything about Gentoo or Portage. It provides three families of queries:

- **`why_in_proof/3,4`** — given a literal, look it up in the ProofAVL and report how it was proved: via a normal rule, a domain assumption, or a prover cycle-break. Returns the body literals and the proof-term context.
- **`why_in_plan/5,6`** — given a literal and a plan, locate it in the wave schedule and trace a reverse-dependency path (via TriggersAVL) back to a root target. The result explains *why* this package is in the plan (e.g. "required by X, which is required by the target Y").
- **`why_assumption/4,5`** — given an assumption key, classify it (domain assumption, cycle-break, or model-only) and extract any `assumption_reason` tags from the context.

The utility predicate **`term_ctx/2`** extracts the `?{Ctx}` context list from any literal-shaped term. This is how the explainer accesses the structured tags (USE state, slot info, suggestions, assumption reasons) attached to each literal during the proof.

16.6.2 `explanation.pl` — domain-specific hooks

This module provides **enrichment hooks** that inject Gentoo-specific context into the generic Why terms produced by `explainer.pl`. The hooks are multifile predicates, so the domain layer plugs into the generic layer without modifying it:

- **`why_in_proof_hook/2`** — if the proof context contains domain reasons (masking info, keyword filtering, slot constraints), it appends a `domain_reasons(Reasons)` tag to the Why term.
- **`why_in_plan_hook/2`** — reserved for future plan-level Gentoo annotations (currently identity).
- **`why_assumption_hook/2`** — enriches assumption Why terms with domain reasons extracted from the assumption's context.

The module also provides **`assumption_reason_for_grouped_dep/6`**, which diagnoses *why* a grouped dependency resolution failed. It inspects the candidate pool and classifies the failure (missing package, all candidates masked, keyword-filtered, slot mismatch, version conflict, etc.). This diagnosis is cached and feeds the `assumption_reason` tags that appear in the proof context.

16.6.3 LLM dispatch

Both modules produce structured Prolog terms. When the user wants a human-readable explanation, the `explain/2,3` predicates in `explainer.pl` convert the structured Why term into a text prompt via `format_why_prompt/2` (which uses `term_to_atom/2` and prepends a system preamble), then dispatch it to an LLM backend via `call_llm/3`. The LLM backend is configurable (`config:llm_default/1`) and can be Ollama, Claude, or any other service that implements the expected interface.

A separate, richer LLM path exists in `explain.pl` (`Source/Application/Llm/explain.pl`), which builds a multi-section text context from the entire plan (targets, actions, USE flags, assump-

tions) and sends it as a conversational prompt. This is used by `--explain` and the interactive `explain_plan_interactive/5` mode, where the user can ask follow-up questions about the plan.

16.7 Query families

Three families of queries are supported:

- **why_in_proof**: given a literal, find how it was proven (normal rule, domain assumption, or prover cycle-break) and extract its body/deps.
- **why_in_plan**: given a literal and a plan, locate it in the wave-plan and trace a reverse-dependency path (via TriggersAVL) back to a root.
- **why_assumption**: given an assumption key, classify it (domain vs cycle-break vs model-only) and extract any reason tags.

16.8 Usage

All predicates are called with the `explainer:` module prefix.

16.8.1 Step 1: Obtain proof artifacts

Run the pipeline to get the proof, model, plan, and triggers:

```
Goals = [portage://'dev-libs'-'openssl':run?{[]}],
pipeline:prove_plan_with_fallback(Goals, ProofAVL, ModelAVL, Plan, TriggersAVL).
```

Or from a `--shell` session after loading a repository:

```
pipeline:prove_plan_with_fallback([portage://'dev-libs'-'openssl':run?{[]}],
                                  Proof, Model, Plan, Triggers).
```

16.8.2 Step 2: Ask "why" questions

Why is a package in the proof?

```
Target = portage://'dev-libs'-'libffi':install,
explainer:why_in_proof(ProofAVL, Target, Why).
% Why = why_in_proof(
%          portage://'dev-libs'-'libffi':install,
%          proof_key(rule(portage://'dev-libs'-'libffi':install)),
%          depcount(3),
%          body([portage://'sys-devel'-'gcc':install, ...]),
%          ctx([...]),
%          domain_reasons([...]))      % <-- added by explanation hook
```

Why is a package in the plan?

```
Proposal = [portage://'dev-libs'-'openssl':run?{[]}],
explainer:why_in_plan(Proposal, Plan, ProofAVL, TriggersAVL,
```

```
                      portage://'sys-libs'-'zlib':install, Why).
% Why = why_in_plan(
%          portage://'sys-libs'-'zlib':install,
%          location(step(1), portage://'sys-libs'-'zlib'-'1.3.1':install?{...}),
%          required_by(path([portage://'sys-libs'-'zlib':install,
%                            portage://'dev-libs'-'openssl':install,
%                            portage://'dev-libs'-'openssl':run])))
```

Why is something assumed?

```
Key = assumed(portage://'dev-foo'-'bar':install),
explainer:why_assumption(ModelAVL, ProofAVL, Key, Type, Why).
% Type = domain,
% Why  = why_assumption(
%          assumed(portage://'dev-foo'-'bar':install),
%          type(domain),
%          term(portage://'dev-foo'-'bar':install?{[assumption_reason(missing)]}),
%          reason(missing),
%          domain_reasons([...]))      % <-- added by explanation hook
```

16.8.3 Step 3 (optional): Get a human-readable explanation via LLM

```
explainer:why_in_proof(ProofAVL, Target, Why),
explainer:explain(claude, Why, Response).
% Response = "openssl requires libffi as a build dependency because..."

% Or use the default LLM (from config:llm_default/1):
explainer:explain(Why, Response).
```

Available LLM services: claude, grok, chatgpt, gemini, ollama. The default is set via `config:llm_default/1`. See `config.pl` for API keys, models, and endpoints.

16.9 Assumption diagnosis (explanation.pl)

`explanation:assumption_reason_for_grouped_dep/6` is called on the fallback path when no candidate satisfies all constraints. It progressively filters candidates through:

1. Existence check → `missing`
2. Self-hosting restriction → `installed_required`
3. Mask check → `masked`
4. Slot restriction → `slot_unsatisfied`
5. Version constraints → `version_no_candidate(0,V)` / `version_conflict`
6. ACCEPT_KEYWORDS → `keyword_filtered`
7. Fallback → `unsatisfied_constraints`

Example:

```
explanation:assumption_reason_for_grouped_dep(
  install,                                    % Action
```

```
  'dev-libs', 'missing-pkg',                    % Category, Name
  [package_dependency(install,no,'dev-libs','missing-pkg',
                      none,version_none,[],[])],
  [self(portage://'app-misc'-'foo'-'1.0')],     % Context
  Reason).
% Reason = missing
```

16.10 Hook mechanism

The explainer module calls `explanation:why_*_hook(Why0, Why)` after building its generic Why term. If the hook succeeds, the enriched Why replaces the generic one. Each hook extracts `domain_reason(cn_domain(C, N, Tags))` tags from the proof context and appends them as `domain_reasons(Reasons)`.

The hooks are called automatically — no direct invocation needed.

16.11 Further reading

- Chapter 9: Assumptions and Constraint Learning — assumption taxonomy that the explainer queries
- Chapter 8: The Prover — proof artifacts consumed by the explainer
- Chapter 14: Command-Line Interface — `--search`, `--similar`, `--explain` flags

Chapter 17

Distributed Proving

For testing purposes, the full Gentoo tree must be walked through the resolver — tens of thousands of ebuilds — and even on capable hardware that adds up. On a single machine, `prover:test_stats(portage)` typically finishes proving every single ebuild in **under a minute** on a twenty-eight-core workstation — fast enough for day-to-day development, but not the only shape the problem takes.

What if you want the full tree proved **faster** than one box allows, or you want to drive resolution from a **thin client** that does not carry the whole knowledge base? portage-ng answers with a **client–server–worker** architecture: a **server** holds the Portage knowledge base and hands out proof jobs; **workers** pull work, run the proving pipeline, and push results back; **clients** submit targets and collect outcomes over the network. The sections below explain how that stack is wired, how discovery and TLS secure it, and how the same repository abstractions work whether you run standalone, on the server, or on a worker.

17.1 Architecture

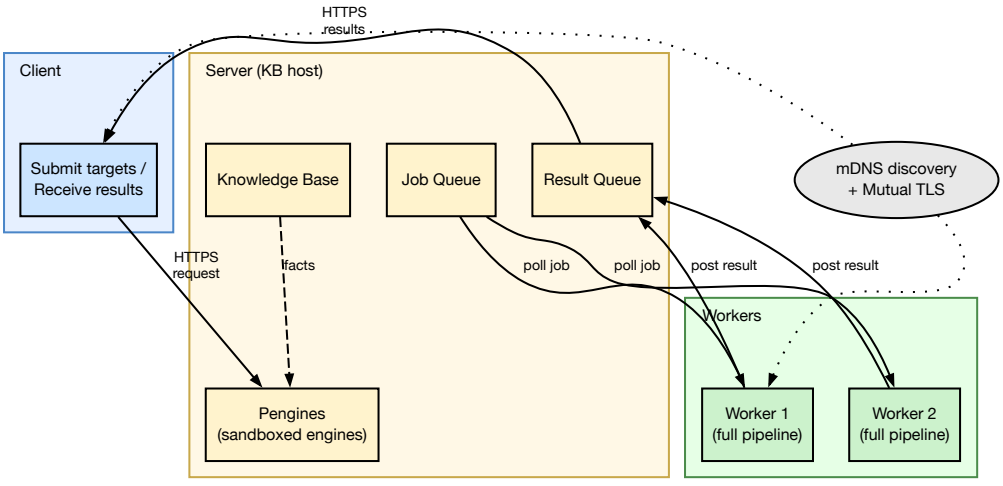


Figure 44 — Cluster architecture

The diagram above shows the three roles. The **server** is the central hub: it holds the in-memory knowledge base, exposes an HTTP interface via Pengines (described below), and multiplexes proof jobs across workers through a job queue and result queue.

Workers are symmetric compute nodes. Each worker carries its own copy of the knowledge base and runs the full proving pipeline locally. You can add as many workers as you need — they scale horizontally, and the server distributes work evenly.

The **client** submits target packages and collects results. It does not need the knowledge base itself; it talks to the server over HTTPS and receives completed proofs as JSON.

17.1.1 Interaction flow

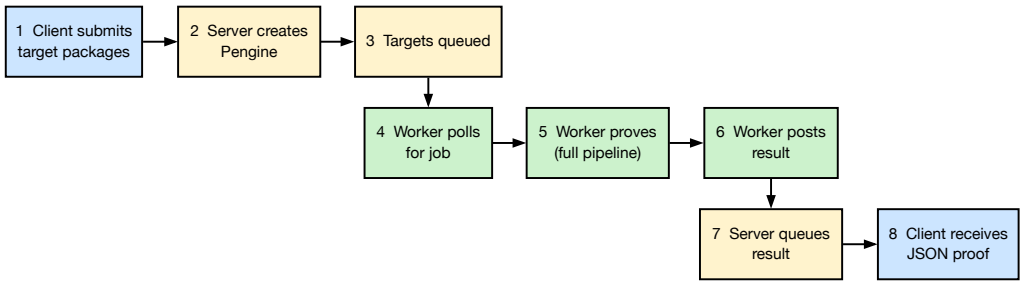


Figure 45 — Cluster interaction flow

The numbered steps show how a proof request travels through the system:

Steps 1-3 (client to server). The client sends a target package (e.g. `sys-apps/portage`) to the server over HTTPS. The server creates a Pengine (a sandboxed Prolog engine) to handle the request and adds the target to its job queue.

Steps 4-6 (worker loop). Workers continuously poll the job queue for work. When a worker picks up a job, it runs the full proving pipeline locally — proof search, planning, and scheduling — using its own copy of the knowledge base. When the proof is complete, the worker posts the result back to the server.

Steps 7-8 (results back to client). The server stores the completed proof in its result queue. The client retrieves the result as a JSON document over HTTPS.

17.1.2 Server

The server is the central coordination point. It runs an HTTP server backed by SWI-Prolog's Pengines library and manages four things: the knowledge base (the full Portage tree loaded in memory), a job queue of proof targets waiting to be processed, a result queue of completed proofs, and the Pengine sandbox that controls what remote callers can execute.

17.1.3 Worker

Each worker is an independent OS process with its own Prolog VM. On startup it loads the knowledge base, then enters a poll loop: it asks the server for the next job, runs the full pipeline (prove, plan, schedule), and posts the result back. Workers are stateless between jobs, so you can add or remove them at any time without affecting other workers or the server.

17.1.4 Client

The client is a thin request layer. It does not carry the knowledge base — it simply submits target packages to the server and collects the completed proofs. This makes it suitable for lightweight machines or scripts that want to drive resolution remotely.

17.1.5 Cluster orchestration

The cluster module sits above the individual roles and provides high-level orchestration: distributing a batch of targets across available workers, collecting results as they arrive, and handling failures (retrying jobs that a worker did not complete).

17.2 Pengines: Prolog as a network service

Pengines ("Prolog engines as a web service") is a library that ships with SWI-Prolog. It turns Prolog query execution into an HTTP-friendly protocol, and portage-ng uses it as the communication layer between clients and the server.

When a client sends a proof request, the server does not run the query in its main thread. Instead, it creates a **Pengine** — a fresh, isolated Prolog engine dedicated to that interaction. The Pengine can read the shared knowledge base (the Portage tree loaded in the server process), but it runs inside a **sandbox** that prevents it from modifying the knowledge base or calling dangerous predicates. Remote callers cannot reshape the server's state.

Answers are streamed back as JSON over HTTP. This means a client does not need to be a full Prolog application — any language that can make HTTP requests and parse JSON can drive proof search. From the outside, portage-ng looks like an ordinary web service that happens to do dependency resolution internally.

17.3 Repository state across modes

An important design question for distributed proving is: how does each mode access the Portage tree? The answer is portage-ng's object-oriented context system (see Chapter 19). Each repository is an instance created through that system, and methods like `portage:read` populate the cache facts.

In **server mode**, repository instances live in the shared server process. All Pengine threads see the same instances and the same loaded knowledge — one tree in memory, many sandboxes reading it.

In **worker mode**, each worker is a separate OS process with its own Prolog VM. It creates its own repository instances and loads its own copy of the knowledge base. Nothing is shared with the server's address space.

The benefit is that the call sites are identical everywhere: the same `portage:read` call appears in standalone, server, and worker code. The object system dispatches to the right backing store depending on the mode, so distributed proving does not require a separate set of data-loading predicates.

17.4 mDNS/Bonjour discovery

Workers and servers discover each other automatically via **mDNS/Bonjour** service advertisement, so there is no need to hand-configure IP addresses.

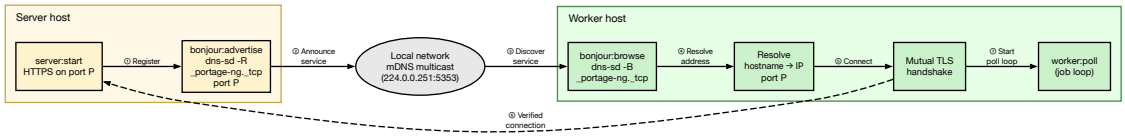


Figure 46 — Bonjour discovery flow

The discovery protocol has three steps:

1. **Register** — when the server starts, it advertises a `_portage-ng._tcp` service on the local network, making its hostname and port visible to any device on the same link.
2. **Browse** — workers browse for that service type and automatically receive the server's address and port.
3. **Connect** — a connection is established from the discovery data, with no manual configuration required.

This is the same zero-configuration networking mechanism used by AirPrint, AirPlay, and many other network services.

17.4.1 Platform support

On **macOS**, the `dns-sd` command ships with the system. The `bonjour.pl` module uses `dns-sd -R` to register the service and `dns-sd -B` to browse for peers.

On **Linux**, the same `dns-sd` command is available through **Avahi** (typically in the `avahi-utils` package). The `bonjour` module hides the platform difference behind a single Prolog interface (`subprocess:dns_sd/...`), so the rest of portage-ng does not need to know which implementation is in use.

After discovery, all traffic is encrypted and mutually authenticated via TLS (see the following sections).

17.5 Sandbox and security

Because Pengines allow remote Prolog execution, the server must control what clients can do. The sandbox module (`Source/Application/Security/sandbox.pl`) enforces a whitelist: only predicates explicitly registered as safe via `sandbox:safe_primitive/1` and `sandbox:safe_meta/2` can be called remotely. Everything else is blocked.

A separate sanitise module (`Source/Application/Security/sanitize.pl`) validates the structure of incoming queries before they reach the sandbox, rejecting malformed or unexpected input early.

17.6 TLS certificates

All communication between server, workers, and clients is encrypted and mutually authenticated using TLS. Mutual authentication means that **both sides** present a certificate during the handshake — the server proves its identity to the client, and the client proves its identity to the server. This prevents unauthorized nodes from joining the cluster.

17.6.1 Certificate hierarchy

portage-ng uses a private Certificate Authority (CA) that acts as the trust root for the entire cluster. Every host-specific certificate is signed by this CA, so any node holding the CA's public certificate can verify any other node's identity.

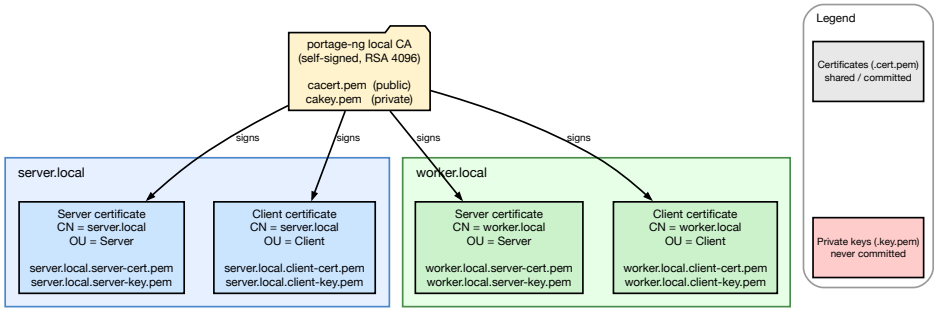


Figure 47 — Certificate hierarchy

The CA is a self-signed RSA 4096-bit certificate valid for 10 years (`CERT_DAYS=3650`). Each host receives two certificates signed by the CA:

- A **server certificate** — presented when the host runs in `--mode server`. The Common Name (CN) is set to the hostname and the Organizational Unit (OU) to "Server".
- A **client certificate** — presented when the host connects to a server as a worker or client. The CN is the local hostname and the OU is "Client".

Both certificates use RSA 2048-bit keys and SHA-256 signatures.

17.6.2 File layout

All certificate files live under the `Certificates/` directory at the project root. The table below shows which files are tracked in git (public certificates) and which are excluded (private keys):

File	Tracked	Description
`cacert.pem`	Yes	CA public certificate (shared trust root)
`cakey.pem`	No	CA private key (never distributed)
`<host>.server-cert.pem`	Yes	Server certificate for `<host>`
`<host>.server-key.pem`	No	Server private key
`<host>.client-cert.pem`	Yes	Client certificate for `<host>`
`<host>.client-key.pem`	No	Client private key
`passwordfile`	Yes	HTTP digest authentication passwords

Private keys and the CA serial file are excluded via `.gitignore`. The public certificates are committed so that nodes can verify each other without manual file copying — only the shared `cacert.pem` needs to be distributed out-of-band.

17.6.3 Mutual TLS handshake

When a worker or client connects to the server, a mutual TLS handshake takes place. Both sides verify the other's certificate against the shared CA before any data is exchanged.

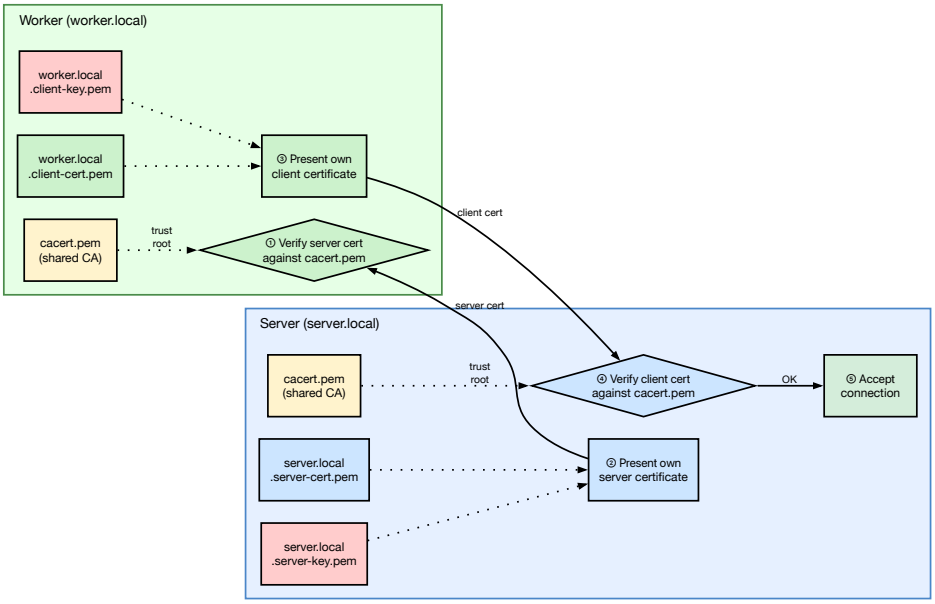


Figure 48 — Mutual TLS handshake

The handshake proceeds in five steps:

1. The **server** presents its server certificate (`server.local.server-cert.pem`), signed by the CA.
2. The **worker** verifies this certificate against its local copy of `cacert.pem`. If verification fails (wrong CA, expired, CN mismatch), the connection is refused.
3. The **worker** presents its client certificate (`worker.local.client-cert.pem`), also signed by the same CA.

4. The **server** verifies this certificate against its own `cacert.pem`. This step is what makes the authentication *mutual* — the server confirms the worker is a legitimate cluster member, not just any TLS client.
5. Both sides accept the connection and begin encrypted communication.

On the server side, the TLS context is configured in `server:start_server` with `peer_cert(true)` to require client certificates, and `cacerts([file(CaCert)])` to set the trust anchor. On the client/worker side, `client:rpc_execute` configures the same CA and presents the local client certificate.

An additional layer of security is provided by **HTTP digest authentication** (`passwordfile`), so even a node with a valid certificate must also know the correct username and password.

17.6.4 How certificates are resolved at runtime

Certificate paths are computed at runtime by `config:certificate/2` and `config:certificate/3`. Given a certificate name like `server-cert.pem`, the predicate prepends the installation directory and `Certificates/` to form the full path. For host-specific certificates, the hostname is prepended to the filename (e.g. `mac-pro.local.server-cert.pem`).

When TLS files are missing, both `server:require_tls_files/4` and `client:require_tls_files/4` print an error message listing the expected file paths and the `make certs` command needed to generate them.

17.7 Generating certificates

To generate a full set of certificates for a host:

```
make certs HOST="$(hostname)"
```

If your hostname includes a `.local` suffix (common on macOS), pass the full name so it matches `config:hostname/1`:

```
make certs HOST="mac-pro.local"
```

This runs `Certificates/Scripts/generate.sh`, which creates three things:

1. A **self-signed CA** (`cacert.pem` + `cakey.pem`) — created only if it does not already exist, so adding a second host reuses the same CA.
2. A **server certificate and key** signed by that CA, with the hostname embedded as the Common Name (CN).
3. A **client certificate and key** signed by the same CA.

17.7.1 Checking and renewing

Certificates are valid for 10 years, but the generation script also supports health checks and renewal:

```
make certs-check                # show expiry status for all hosts
make certs-renew                # renew certs expiring within 30 days
```

The `--check` subcommand prints each certificate's expiry date and flags any that are missing or expiring soon. The `--renew` subcommand regenerates only the certificates that need it, reusing the existing CA and private keys.

17.8 Encrypted two-node cluster: step-by-step

The following walkthrough shows how to set up a minimal cluster with one server and one worker on a local network.

Step 1 — Generate certificates on each machine.

On the server host (e.g. `server.local`):

```
make certs HOST="server.local"
```

On the worker host (e.g. `worker.local`):

```
make certs HOST="worker.local"
```

Each machine now has its own certificate and key, signed by a locally created CA.

Step 2 — Share the trust root.

By default, each machine creates its own CA. For mutual authentication to work, all nodes must trust the same CA. Copy `cacert.pem` from the server to the worker (or designate one machine as the cluster CA and distribute its `cacert.pem` to all nodes). Each node keeps its own host-specific certificate and key.

Step 3 — Start the server.

```
portage-ng --mode server
```

The server loads the knowledge base and begins listening for connections.

Step 4 — Start the worker.

```
portage-ng --mode worker
```

The worker loads its own copy of the knowledge base and begins looking for the server.

Step 5 — Discovery.

If mDNS/Bonjour is available on the network, the worker finds the server automatically via the `_portage-ng._tcp` service advertisement. No IP addresses need to be configured.

Step 6 — Mutual TLS handshake.

When the worker connects, both sides present certificates signed by the shared CA. portage-ng verifies the Common Name and role, so only nodes with valid credentials can join the cluster.

Step 7 — Proving.

The worker polls the server's job queue, runs the full proving pipeline for each target, and posts results back. The server collects completed proofs and makes them available to clients.

17.9 Cluster usage

To run a distributed cluster, every node needs two things:

- A copy of the same `cacert.pem` (the shared trust root).
- Its own host-specific certificate and key pair.

The server is started with `--mode server`, and each worker with `--mode worker`. Discovery happens automatically via mDNS/Bonjour, and TLS ensures that only nodes sharing the same CA can participate. You can add more workers at any time — they will discover the server and start picking up jobs immediately.

17.10 Further reading

- Chapter 2: Installation and Quick Start — dns-sd and openssl prerequisites
- Chapter 14: Command-Line Interface — `--mode` flags
- Chapter 4: Architecture Overview — module load order for different modes

Chapter 18

Upstream and Bug Tracking

portage-ng integrates with external services to check upstream versions and search for known issues, helping users identify outdated packages and known dependency bugs.

18.1 Git repository integration

portage-ng can connect directly to a Git repository and turn Git metadata into Prolog facts. This means it can inspect commit history, changelogs, and file-level changes for any ebuild without relying on separate tools. The Git metadata is ingested alongside the regular cache data, so queries like "when was this ebuild last updated?" or "which ebuilds changed in the last sync?" can be answered from within the resolver.

18.2 Upstream version checking

The upstream module (`Source/Domain/Gentoo/upstream.pl`) checks package versions against upstream releases via the Repology API.

18.2.1 Usage

```
portage-ng --upstream sys-apps/portage
portage-ng --upstream @world
```

18.2.2 How it works

1. For each target package, the module queries the Repology API (`https://repology.org/api/v1/project/<name>`) for version information.
2. The response includes version data across multiple distributions, which is compared against the version in the local Portage tree.
3. Results are categorized:
   - **Up to date** — local version matches or exceeds upstream
   - **Outdated** — a newer upstream version exists
   - **Unknown** — package not tracked by Repology

18.2.3 Output

The upstream check displays a comparison table showing the local version, the latest upstream version, and the status for each package.

18.3 Gentoo Bugzilla integration

The bugs module (`Source/Domain/Gentoo/bugs.pl`) searches Gentoo's Bugzilla instance for known issues related to packages.

18.3.1 Usage

```
portage-ng --bugs sys-apps/portage
```

18.3.2 How it works

1. The module queries Gentoo Bugzilla's REST API for bugs matching the package atom.
2. Results are filtered and displayed with bug number, summary, status, and assignee.

This helps users identify whether a dependency resolution failure is due to a known upstream bug rather than a portage-ng issue.

18.4 Automatic bug report drafts

The issue module (`Source/Domain/Gentoo/issue.pl`) generates structured Gentoo Bugzilla bug report drafts when the prover detects unsatisfiable dependencies.

A generated report includes:

- **Summary** — one-line description of the issue
- **Affected package** — the package atom
- **Unsatisfiable constraints** — the specific dependency that cannot be met
- **Observed state** — what the prover found (missing package, version conflict, REQUIRED_USE violation)
- **Suggested fix** — recommended action (add keyword, unmask, fix dependency)

These drafts can be used as starting points for filing bugs with the Gentoo bug tracker.

18.5 Further reading

- Chapter 14: Command-Line Interface — `--upstream` and `--bugs` flags
- Chapter 9: Assumptions and Constraint Learning — how unsatisfiable dependencies are detected

Chapter 19

Contextual Logic Programming

context is an object-oriented programming paradigm for Prolog, implemented in `Source/Logic/context.pl`. It provides contexts (namespaces), classes, and instances with public, protected, and private access control, multiple inheritance, cloning, and declarative static typing of data members.

19.1 Motivation

Standard Prolog uses a flat global namespace. As applications grow, name collisions, uncontrolled access to dynamic predicates, and lack of modularity become obstacles. **context** addresses this by splitting the global namespace into isolated contexts, each with their own facts and rules.

The key insight is that contexts can be **unified** and can serve as feature terms describing software configurations — directly connecting to Zeller's *Unified Versioning through Feature Logic*. This makes **context** both a software engineering tool and a formal foundation for reasoning about configurations.

In practical terms, this is how portage-ng can treat a Portage tree, an overlay, and a VDB as separate objects that share the same interface but carry independent state — and how dependency contexts can be merged, intersected, and propagated through the proof tree.

19.2 How it differs from Logtalk

The syntax is comparable to Logtalk, but the approach is fundamentally different:

	Logtalk	**context**
Approach	Compile-time translation to plain Prolog	Runtime generation of guarded predicates
Overhead	Source-to-source compilation step	No compilation; contexts created dynamically
Thread safety	Varies by backend	Built-in; tokens are thread-local
Feature unification	Not supported	Contexts unify as feature terms

Because **context** works at runtime, contexts can be created, cloned, and composed dynamically — which portage-ng uses extensively to represent repositories, ebuilds, and configurations as live objects.

19.3 Core concepts

19.3.1 Contexts

A context groups together clauses of a Prolog application. By default, clauses are local to their context and invisible to other contexts unless explicitly exported. Referencing a context is enough to create it (creation ex nihilo).

Context rules are evaluated in the context in which they are defined. An exception are clauses declared as "transparent", which inherit their context from the predicate that is calling them — the same mechanism SWI-Prolog uses for meta-predicates, but applied at the context level.

19.3.2 Classes

A class is a special context that declares public, protected, and private meta-predicates. These declarations control access at three levels:

- **Instantiation** — which predicates are copied into the instance and how they are guarded.
- **Inheritance** — which predicates are visible to subclasses. Private predicates are not inherited; public and protected ones are.
- **Invocation** — which predicates external callers may use. Only public predicates can be called from outside; protected and private predicates throw a `permission_error` if accessed without a valid access token.

A class is declared with `:- class.` (or `:- class([Parent1, Parent2])` for multiple inheritance). Predicate visibility is declared with `:- dpublic(name/1).`, `:- dprotected(age/1).`, and `:- dprivate(secret/1).`

19.3.3 Instances

Instances are dynamically created from a class via `newinstance/1`. The creation process has four stages:

1. **Metadata registration** — the instance is marked as `type(instance(Parent))` in its `$__meta` store.
2. **Predicate inheritance** — all declared predicates from the parent class are copied. Each predicate is wrapped in a **guard** that checks an access token before execution.
3. **Freeze** — the generated predicates are compiled via `compile_predicates/1` for optimal performance (no more interpretation overhead).
4. **Constructor call** — if the class declares a constructor predicate (same name as the class), it is called with the arguments passed to `newinstance/1`.

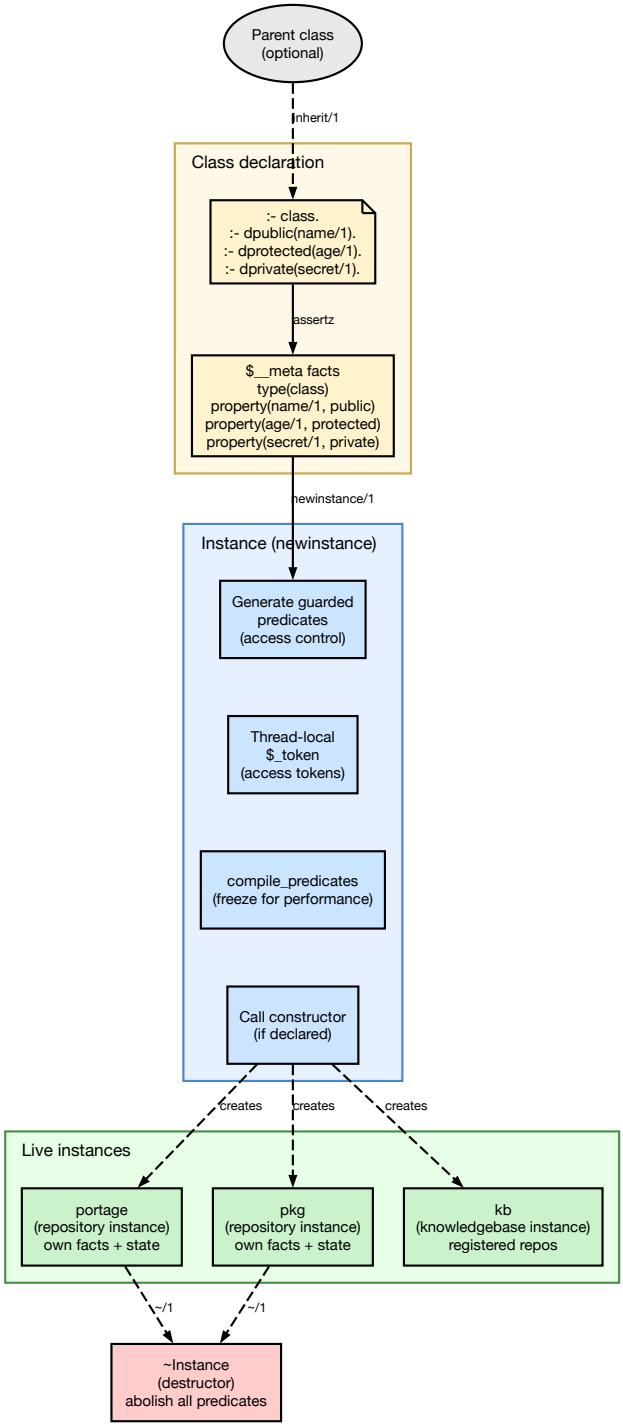


Figure 49 — Class and instance lifecycle

Each instance is a fully independent Prolog module with its own dynamic facts. Multiple instances of the same class coexist without interference — for example, `portage` and `overlay` are both instances of the `repository` class, but each carries its own cached ebuilds, location paths, and sync state.

19.3.4 Destruction

An instance is destroyed by calling `~Instance`. The destructor (if declared) runs first, then all dynamic predicates in the instance module are abolished. Static contexts (built-in Prolog modules) cannot be destroyed — attempting to do so raises a `permission_error`.

19.3.5 Access control and thread safety

The access control mechanism uses **thread-local tokens**. When an external caller invokes a public predicate on an instance, the system asserts a `$_token(thread_access)` fact in the instance's module. The guarded implementations of protected and private predicates check for this token:

- **Public** — the token is asserted before the call and retracted afterwards (via `call_cleanup`). Any code called during the public predicate's execution can access protected and private predicates because the token exists.
- **Protected** — the guard checks that the token exists. If it does (i.e. the call originates from a public predicate on the same instance), execution proceeds. Otherwise, a `permission_error` is thrown.
- **Private** — same check as protected. The difference is conceptual: private predicates are not inherited by subclasses, while protected ones are.
- **Static** — the call is forwarded to the parent class directly, bypassing the instance.

Because the token is `thread_local`, concurrent threads can call public predicates on the same instance without interfering with each other's access grants.

19.4 Operators

context defines several operators for interacting with contexts. The diagram below shows how they relate to an instance's internal structure:

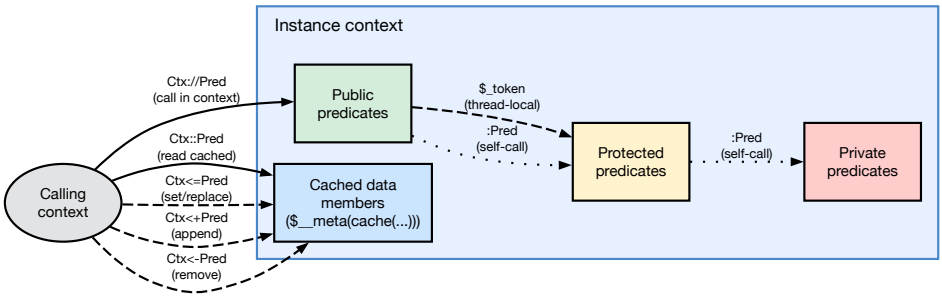


Figure 50 — Context operators

Operator	Meaning
`:Pred`	Call `Pred` in the current context (self-call)
`::Pred`	Read a cached data member
`<=Pred`	Set a data member (evaluate, retract old, assert new)
`<+Pred`	Add a data member (evaluate, assert if not exists)
`<-Pred`	Remove a data member
`Ctx://Pred`	Call `Pred` in a specific context

19.4.1 Data members

The `::`, `<=`, `<+`, and `<-` operators implement data-member-like behaviour. Under the hood, they work with `$__meta(cache(...))` facts:

- **`Ctx::Pred`** — looks up `cache(Pred)` in the instance's metadata. If found, calls the predicate (which succeeds immediately because the cached value has already been evaluated). This is a **read** operation.
- **`Ctx<=Pred`** — evaluates `Pred` in the instance, retracts any existing cache for the same functor/arity, and asserts the new result. This is a **write-replace** operation.
- **`Ctx<+Pred`** — evaluates `Pred` and asserts the result as a new cache entry without removing existing ones. This is an **append** operation, useful for multi-valued data members.
- **`Ctx<-Pred`** — retracts all cache entries matching `Pred`. This is a **delete** operation.

19.4.2 The `://` operator

The `://` operator is the primary way to call a predicate in another context. It is defined simply as `'://'(Context, Predicate) :- Context:Predicate.` — a thin wrapper that resolves the context module and dispatches the call. This operator appears throughout portage-ng in forms like `portage://entry(E)` or `kb://query(Q, R)`.

19.5 Example: a Person class

A complete worked example of a context class — including a constructor, destructor, public/protected/private members, and instance creation — is presented in Chapter 1, section 1.7. The example defines a `person` class with name and age accessors, title management, and demonstrates how instances carry their own state independently.

19.6 How portage-ng uses context

portage-ng uses **context** throughout its architecture. The diagram below shows the two main classes (`repository` and `knowledgebase`), their instances, and how the prover accesses them.

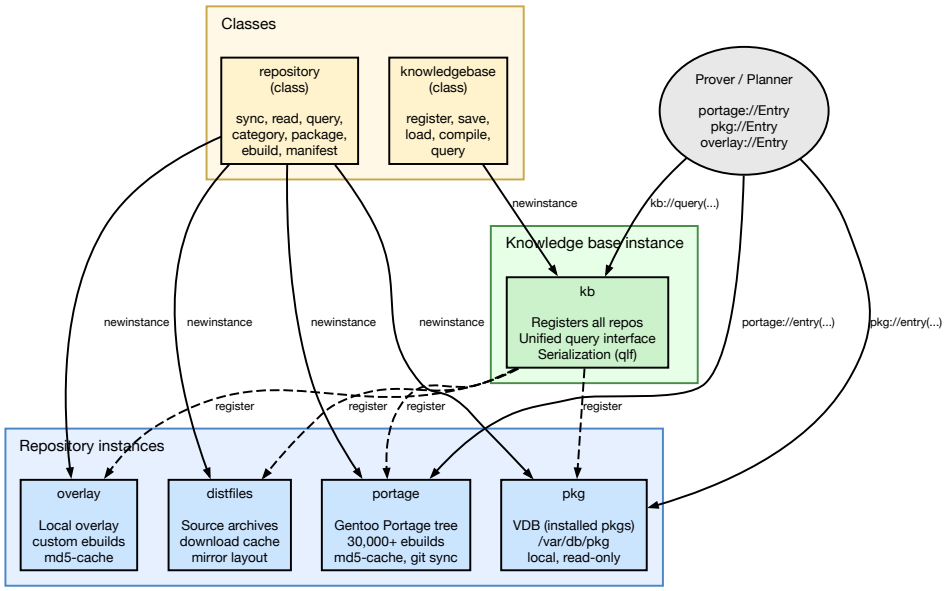


Figure 51 — portage-ng context usage

19.6.1 Repository instances

The `repository` class (`Source/Knowledge/repository.pl`) declares a rich public interface for working with package repositories: syncing from remote sources, reading metadata, querying entries, and generating graphs. Protected members hold configuration (location, cache path, remote URL, sync protocol).

Each repository on disk becomes a named instance:

- **`portage`** — the main Gentoo Portage tree (30,000+ ebuilds), synced via git, with md5-cache metadata.
- **`pkg`** — the VDB (installed packages) at `/var/db/pkg`, a local read-only repository.
- **`overlay`** — a local overlay with custom ebuilds.
- **`distfiles`** — the source archive download cache.

Instance creation and configuration happens in the host-specific config file (e.g. `Source/Config/mac-pro.local.pl`):

```
:- portage:newinstance(repository).
:- portage:init('/Volumes/Storage/Repository/portage-git',
                '/Volumes/Storage/Repository/portage-git/metadata/md5-cache',
                'https://github.com/gentoo/gentoo.git',
                'git', 'portage').

:- pkg:newinstance(repository).
:- pkg:init('/Volumes/Storage/Repository/pkg', '', '', 'local', 'vdb').
```

Because all instances share the same class, the prover does not need to know whether a dependency comes from the main tree, an overlay, or the VDB — the same `Repo://entry(E)` call works for all of them.

19.6.2 Knowledge base instance

The `knowledgebase` class (`Source/Knowledge/knowledgebase.pl`) acts as a registry. Its `register/1` predicate adds repository instances, and its `query/2` predicate searches across all registered repositories. The instance `kb` is created at startup and populated by the configuration file.

In **client mode**, the knowledge base instance is created with a hostname and port: `kb:newinstance(knowledgebase(Host, Port))`. This switches the instance into proxy mode, where queries are forwarded to the remote server via Pengine RPC — but the calling code sees the same `kb://query(Q, R)` interface as in standalone mode.

19.6.3 Context terms in the prover

Beyond the OOP use, the context system's unification semantics flow into the prover. Every literal in the proof tree carries a context list `?{[...]}` that accumulates constraints as the proof deepens. These context terms are merged via feature unification — the same algebraic operation that the context system uses for its data members. This connection is explored in detail in Chapter 20: Context Terms and Feature Unification.

19.7 Serialization

Context instances support serialization through the knowledge base's `save` and `load` predicates. When `kb://save` is called, the cached facts from all registered repository instances are written to a QLC file (`Knowledge/kb.qlf`). On the next startup, `kb://load` reads the compiled file back, restoring all repository instances to their previous state without re-parsing the Portage tree from disk. This is what makes portage-ng's startup fast — the full knowledge base (30,000+ ebuilds with all metadata) loads from the QLC file in under a second.

19.8 Further reading

- A. Zeller, *Unified Versioning through Feature Logic*, 1997
- `Source/Logic/context.pl` — full implementation
- `Documentation/Handbook/20-doc-context-terms.md` — how context terms flow through the prover

Chapter 20

Context Terms in portage-ng

How contexts are created, propagated, and merged across the dependency graph.

20.1 Overview

Every literal in the prover carries a **context** — a list of tagged terms that records provenance, ordering, constraints, and USE requirements as the proof expands through dependencies. The literal format is:

```
Literal:Action?{Context}
```

Contexts are not opaque blobs; they are structured as **feature-term lists** and are merged using a Zeller-inspired feature-unification algorithm. This gives them lattice semantics: merging two contexts produces a well-defined meet that preserves all non-contradictory information from both sides.

20.2 Anatomy of a context

A context is a Prolog list. Each element is either a plain term or a `Feature:Value` pair. The distinction matters for merging:

Form	Example	Merge behaviour
Plain term	`self(R://E)`	Identity match; duplicates dropped
Feature:Value	`bwu:use_state(En,Dis)`	Value-merged by `val_hook/3`
Feature:Compound	`slot(C,N,Ss):{...}`	Compound feature key

For example, `self(portage://sys-apps/portage-3.0.77-r3)` is a plain term that identifies the parent ebuild. `build_with_use:use_state([foo],[bar])` is a Feature:Value pair whose value is merged when two contexts meet. `slot(sys-apps,portage,0/0):{Candidate}` is a compound feature key used for sub-slot rebuild tracking.

20.2.1 Common context tags

The following tags appear most frequently in proof-term contexts. Each tag is a Prolog term added to the context list during rule evaluation.

`self(Repo://Entry)` — set by `dependency:add_self_to_dep_contexts`. Identifies the parent ebuild that introduced this dependency edge.

`build_with_use:use_state(En,Dis)` — set by `dependency:process_build_with_use`. Carries bracketed USE constraints from the dependency atom (e.g. `dev-libs/foo[bar,-baz]`).

`slot(C,N,Ss):{Candidate}` — set by `dependency:process_slot`. Records a slot lock from `:=` (subslot rebuild) semantics.

`after(Literal)` — set by `rules:ctx_add_after`. Ordering constraint: this dependency must come after `Literal` in the plan. Propagates to children.

`after_only(Literal)` — set by `rules:add_after_only_to_dep_contexts`. Ordering constraint that does **not** propagate to children.

`replaces(pkg://Entry)` — set by install/update rules. Records which installed package this action replaces.

`assumption_reason(Reason)` — set by the domain assumption fallback. Records why a domain assumption was made (e.g. `missing`, `masked`, `keyword_filtered`).

`suggestion(Type,Detail)` — set by the relaxation fallback. Records an actionable suggestion (e.g. `accept_keyword`, `unmask`, `use_change`).

`domain_reason(cn_domain(C,N,Tags))` — set by `candidate:add_domain_reason_context`. Diagnostic tags for version domain narrowing.

`constraint(cn_domain(C,N):{Domain})` — set by the constraint system. Carries an inline constraint for domain scoping.

20.3 Context lifecycle

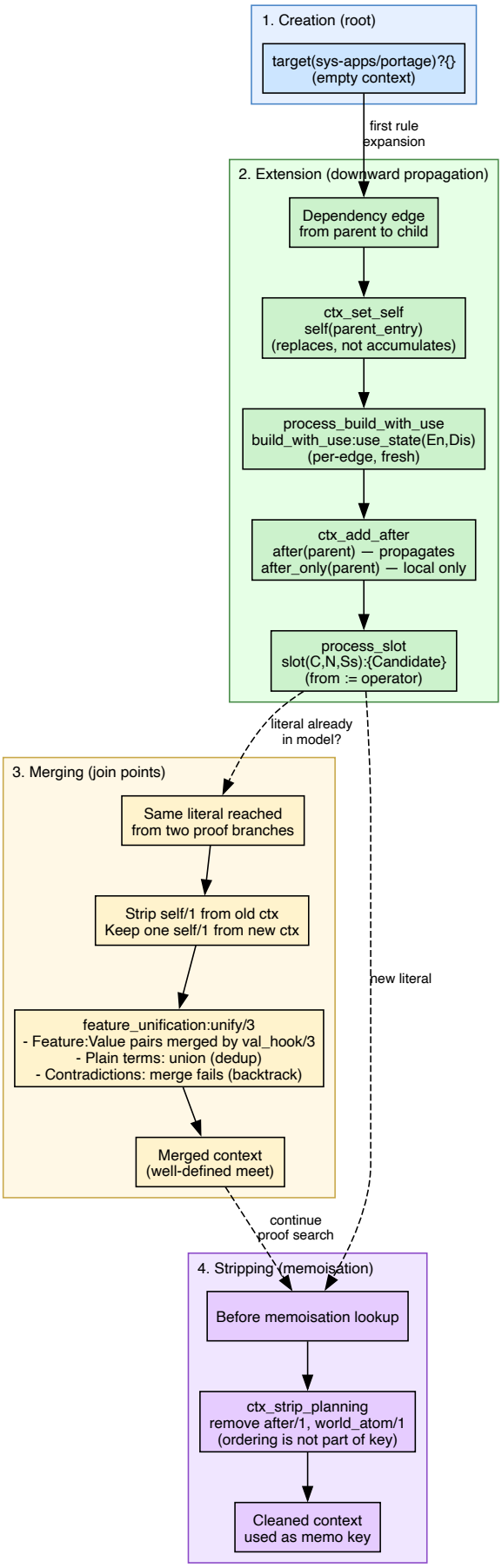


Figure 52 — Context lifecycle

20.3.1 1. Creation (root)

At the top level, the prover starts with an empty context (`{}` or `[]`). The first rule expansion — typically `target/2` → `install` — begins populating it.

20.3.2 2. Extension (downward propagation)

As rules expand dependencies, contexts grow:

```
target(sys-apps/portage)?{}
  └─ install(portage://sys-apps/portage-3.0.77-r3):install?{...}
       ├─ dep(dev-lang/python):install?{self(portage://sys-apps/portage-3.0.77-r3),
       │                                build_with_use:use_state([ssl,threads],[]),
       │                                after(install(portage://...))}
       │    └─ dep(dev-libs/openssl):install?{self(portage://dev-lang/python-3.13),
       │                                     build_with_use:use_state([],[]),
       │                                     after(install(portage://dev-lang/
python-3.13))}
       └─ dep(app-arch/tar):install?{self(portage://sys-apps/portage-3.0.77-r3),
                                    after(install(portage://...))}
```

Key propagation rules:

- **`self/1`** is set to the current ebuild at each dependency edge. It does **not** accumulate — each edge replaces the previous `self`.
- **`build_with_use`** is per-edge: the child gets a fresh `build_with_use` from its dep atom, not the parent's `build_with_use`.
- **`after/1`** propagates transitively (children inherit it).
- **`after_only/1`** does **not** propagate (ordering is local to this edge).
- **`assumption_reason`** and **`build_with_use`** are dropped on PDEPEND edges (via `ctx_drop_build_with_use_and_assumption_reason`).

20.3.3 3. Merging (join points)

When the prover encounters a literal that was already proven with a different context, it merges the old and new contexts via feature term unification:

```
sampler:ctx_union(OldCtx, NewCtx, MergedCtx)
```

The merge algorithm:

1. **Strip** **`self/1`** from the old context entirely.
2. **Extract one** **`self/1`** from the new context (keep it aside).
3. **Unify** the remaining lists via `feature_unification:unify/3`.
4. **Prepend** the extracted `self/1` back onto the result.

This guarantees: - At most one `self/1` in the merged result (from the new/incoming side). - Feature:Value pairs with the same key are merged by `val_hook/3`. - Plain terms present in either side appear in the result (union semantics).

20.3.4 4. Stripping for memoisation

Before checking whether a literal has already been proven, planning markers are stripped so they don't pollute the memoisation key:

```
rules:ctx_strip_planning(Context0, Context)
```

This removes `after/1` and `world_atom/1` — ordering and planning concerns that should not affect whether a proof is reusable.

20.4 Feature unification in detail

`feature_unification:unify/3` implements a **horizontal unification** algorithm inspired by Zeller's feature logic:

1. Normalise both terms (`{}` → `[]`).
2. Walk both lists. For each `Feature:Value` pair in list A, check if list B has the same `Feature`.
3. If both sides have `Feature`, merge values via `val/3` (or `val_hook/3` for domain-specific merge).
4. If only one side has `Feature`, include it in the result.
5. Plain terms are matched by identity; duplicates are dropped.

20.4.1 Value merge rules

V1	V2	Result	Semantics
`{L1}`	`{L2}`	`{Intersection}`	Set intersection (must be non-empty)
`[L1]`	`[L2]`	`[Union]`	Sorted union (fails on contradictions)
atom `V`	`{L}`	`{V}` if `V ∈ L`	Singleton intersection
`V`	`V`	`V`	Identity

20.4.2 Domain-specific hooks (`val_hook/3`)

Feature	Hook in	Merge behaviour
`build_with_use`	`use.pl`	`use_state(En1,Dis1)` ⍰ `use_state(En2,Dis2)` = union of enable/disable sets; **fails** if a flag appears in both enable and disable
`cn_domain`	`version.pl`	`version_domain` meet (intersection of version bounds); `none` is identity

20.5 `self/1` — parent provenance

The `self/1` tag identifies **which ebuild introduced this dependency**. It is critical for:

- **USE evaluation**: `use:effective_use_in_context/3` looks up the USE model of the ebuild in `self/1` to evaluate USE conditionals.
- **Blocker source**: `candidate:make_blocker_constraint/5` uses `self/1` to determine who is blocking whom.
- **Parent narrowing**: `candidate:maybe_learn_parent_narrowing/4` uses `self/1` to learn that the parent version should be excluded when a child dependency cannot be satisfied.
- **REQUIRED_USE**: `query:with_required_use_validate/3` annotates REQUIRED_USE terms with `:validate?{[self(...)]}` so the prover knows the ebuild context.

20.5.1 Invariant: at most one `self/1`

Without bounding, `self/1` would stack along dependency chains:

```
[self(A), self(B), self(C), ...]  ← unbounded growth
```

The system prevents this at two levels:

1. **`dependency:ctx_set_self/3`** replaces any existing `self/1` when setting a new parent.
2. **Feature term unification** (`ctx_union_raw/3`) strips all `self/1` from the old context and keeps only one from the new context.

20.6 `build_with_use` — bracketed USE requirements

When a dependency atom carries USE requirements (e.g. `dev-lang/python[ssl,threads]`), they are recorded as:

```
build_with_use:use_state([ssl, threads], [])
```

The enable list contains flags that must be ON; the disable list contains flags that must be OFF.

20.6.1 Per-edge, not inherited

Each dependency edge computes its own `build_with_use` from the dep atom. The parent's `build_with_use` is **removed** before computing the child's:

```
dependency:process_build_with_use(MergedUse, ContextDep, NewContext, ...)
```

This prevents a grandparent's USE requirements from leaking to grandchildren.

20.6.2 Merge semantics

When feature term unification merges two contexts with `build_with_use`, the `val_hook` in `use.pl` takes the **union** of enable sets and the **union** of disable sets. If a flag appears in both enable and disable, the merge **fails** (contradiction), forcing the prover to backtrack.

20.6.3 Post dependencies

On PDEPEND edges, `build_with_use` is dropped because PDEPEND dependencies are resolved at runtime, not build time, so build-time USE constraints do not apply.

20.7 Constraints vs contexts

The proof system uses two complementary mechanisms for carrying information, and it is easy to confuse them. **Contexts** are local: each literal in the proof carries its own context list (`?{...}`), recording where it came from, what USE flags were requested, and how it should be ordered. **Constraints** are global: they live in a shared ConstraintsAVL that spans the entire proof and track cross-cutting invariants like "only one version of this package may be selected" or "this package is blocked by another."

The table below summarises the key differences:

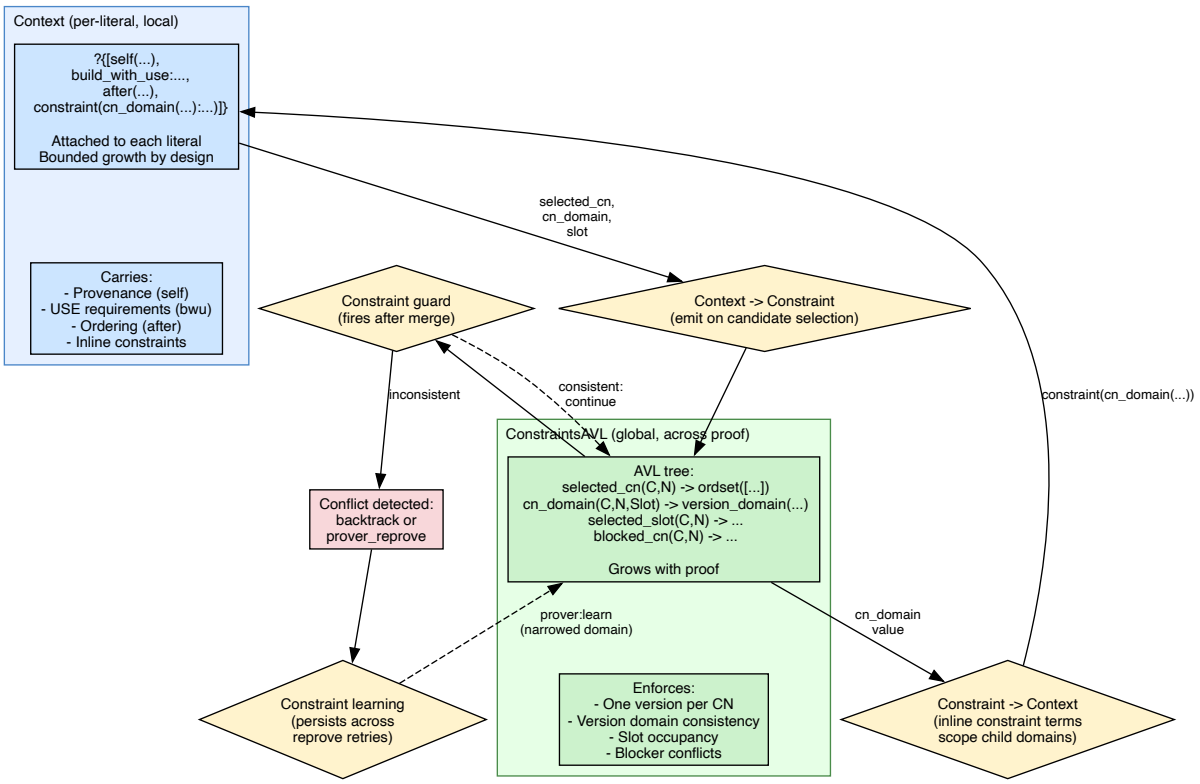


Figure 53 — Context vs constraint interaction

Aspect	Context	Constraint
Scope	Per-literal (local)	Global (across proof)
Storage	List attached to `?{...}`	AVL in ConstraintsAVL
Growth	Bounded by design	Grows with proof
Purpose	Provenance, ordering, USE	Version selection, slot locks, blockers

20.7.1 How they interact

Although contexts and constraints have different scopes, they are not isolated — information flows between them in both directions.

From context to constraint. When the rules layer selects a candidate version for a dependency, it emits constraint terms into the global ConstraintsAVL. For example, selecting `dev-libs/openssl-3.1.4` produces a `selected_cn(dev-libs, openssl)` constraint and a `cn_domain` constraint recording the version domain. These global constraints ensure that if another dependency path also needs `dev-libs/openssl`, the prover will detect any conflict.

From constraint to context. Sometimes a parent dependency wants to narrow the version domain for a child before candidate selection even begins. It does this by placing an inline constraint term like `constraint(cn_domain(C,N):{Domain})` directly in the context list. When the child's rule fires, it reads this term and applies the domain restriction.

Constraint guards. After each new constraint is merged into the global store, `rules:constraint_guard/2` fires to check consistency. The guard verifies that version domains are compatible with selected candidates, that each slot has at most one selected version, and that no selected package is blocked by another.

Constraint learning. When a guard detects an inconsistency, it can record a narrowed domain via `prover:learn/3`. This learned constraint persists across reprove retries, preventing the prover from repeating the same dead-end choice (see Chapter 9 and Chapter 10).

20.8 Ordering: `after` vs `after_only`

Both create ordering edges in the plan, but they differ in propagation:

Marker	Propagates to child deps?	Use case
`after(Lit)`	Yes	Build deps: the package and all its deps must come after `Lit`
`after_only(Lit)`	No	Runtime deps: only this package (not its deps) must come after `Lit`

20.8.1 Extraction

```
rules:ctx_take_after_with_mode(Context, After, AfterForDeps, ContextRest)
```

- If `after(X)` → `After = X`, `AfterForDeps = X` (propagate).
- If `after_only(X)` → `After = X`, `AfterForDeps = none` (don't propagate).
- If neither → both `none`.

20.9 Example: full context evolution

The following example traces how context evolves as the prover walks from a user target (`sys-apps/portage`) through two levels of dependencies. The diagram shows the key context tags at each step.

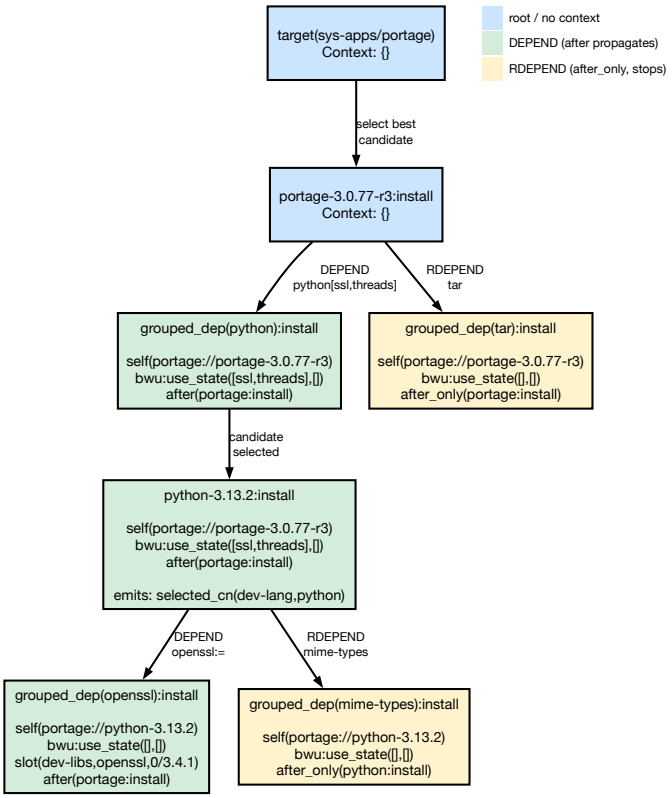


Figure 54 — Context evolution through a dependency chain

20.9.1 Step 1 — Target resolution

The user runs `emerge sys-apps/portage`. The target rule selects the best visible candidate (`portage-3.0.77-r3`). At this point the context is empty — there is no parent, no USE requirement, and no ordering constraint.

20.9.2 Step 2 — Expanding portage's dependencies

The install rule for portage expands its DEPEND and RDEPEND. Each dependency atom gets its own context, built from three operations:

- **`add_self_to_dep_contexts`** adds `self(portage://portage-3.0.77-r3)` to record that portage is the parent.
- **`process_build_with_use`** translates bracketed USE flags from the atom. For `dev-lang/python[ssl,threads]`, this produces `build_with_use:use_state([ssl,threads],[])`. For `app-arch/tar` (no brackets), the USE state is empty.

- `ctx_add_after` adds an ordering constraint. DEPEND atoms get `after(portage:install)` (propagates to children). RDEPEND atoms get `after_only(portage:install)` (does not propagate).

20.9.3 Step 3 — Resolving python

The candidate `python-3.13.2` is selected. A `selected_cn` constraint is emitted into the global ConstraintsAVL (not the context). The context itself is passed down unchanged from the grouped dependency.

20.9.4 Step 4 — Expanding python's dependencies

Python's own dependencies get fresh contexts. Notice how each tag is rebuilt at this level:

- `self` now points to `python-3.13.2`, not to portage. The `self` tag always records the immediate parent, never accumulates.
- `build_with_use` is replaced based on the new atom. `dev-libs/openssl:=` has no bracketed flags, so the USE state becomes empty.
- `after` is propagated from the parent (portage's install constraint travels down through DEPEND edges).
- **Slot lock** — the `:=` operator on `dev-libs/openssl:=` adds a `slot(dev-libs,openssl,0/3.4.1)` tag to the context, recording the sub-slot for rebuild tracking.
- `after_only` — the RDEPEND on `app-misc/mime-types` gets `after_only(python:install)`, which will not propagate to mime-types's own children.

20.9.5 Key observations

- `self/1` always points to the immediate parent, never accumulates along the chain.
- `build_with_use` is replaced at each edge based on the dependency atom's bracketed flags.
- `after/1` from DEPEND propagates down the tree; `after_only/1` from RDEPEND does not.
- **Slot locks** (`:=`) add `slot/3` entries to the context.
- **Constraint emissions** (e.g. `selected_cn`) go into the global ConstraintsAVL, not into the context.

20.10 Design rationale

20.10.1 Why feature unification?

Traditional dependency solvers use flat constraint lists or SAT clauses. portage-ng uses feature-term unification because:

1. **Composability**: Contexts from different proof branches merge naturally at join points without ad-hoc conflict resolution.
2. **Bounded growth**: The `self/1` stripping in feature term unification and the per-edge `build_with_use` replacement prevent unbounded context growth along dependency chains.
3. **Domain extensibility**: New context tags can be added without changing the merge infrastructure — just add a `val_hook/3` clause if domain-specific merge is needed.

4. **Conflict detection**: The merge fails (backtracks) on contradictions (e.g. a flag in both enable and disable), providing natural constraint propagation.

20.10.2 Why separate contexts and constraints?

Contexts are **local** (per-literal, scoped to a proof branch) while constraints are **global** (shared across the entire proof). This separation allows:

- **Contexts** to carry provenance information that should not leak across unrelated proof branches.
- **Constraints** to enforce global invariants (e.g. only one version of a package can be selected) that must hold across the entire proof.
- **Constraint learning** to persist across reprove retries, narrowing the search space incrementally.

Chapter 21

Dependency Resolver Comparison

21.1 Architecture Overview

All three resolvers solve the same problem: given a set of requested packages, figure out which concrete versions to install and in what order. Where they differ is in how they handle **conflicts** — situations where the first choice turns out to be wrong.

Each subsection below describes the resolver's strategy and illustrates its conflict-resolution loop.

21.1.1 Portage (Python)

Portage takes the most straightforward approach. It builds a dependency graph by walking every dependency and picking the newest stable candidate for each. If two packages end up claiming the same slot, Portage detects the conflict after the graph is already built.

Its recovery strategy is blunt: mask the conflicting package so it won't be picked again, throw away the entire graph, and rebuild it from scratch. Each retry adds one more mask. The masks accumulate across retries, but no other information carries over — the graph starts clean every time. Portage allows up to 20 retries by default (configurable with `--backtrack=N`).

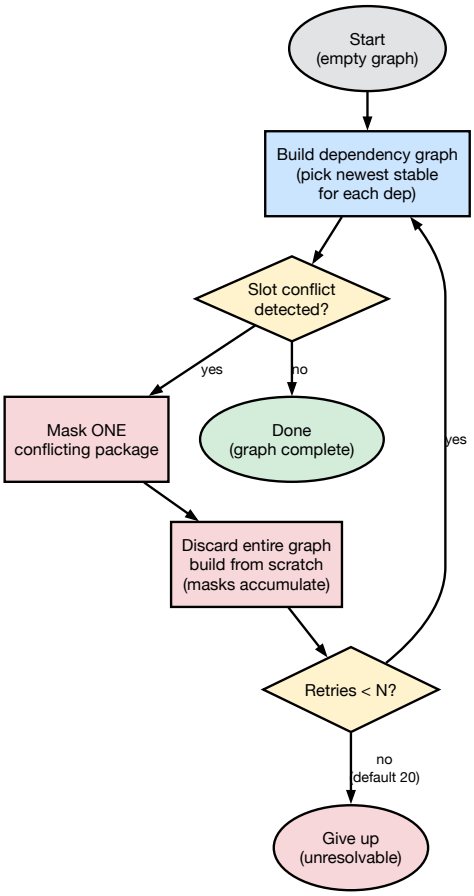


Figure 55 — Portage conflict-resolution loop

Because each retry rebuilds everything, this approach is the slowest of the three. Complex dependency tangles — like the OCaml Jane Street ecosystem — can require more than a dozen retries before Portage finds a consistent graph.

21.1.2 Paludis (C++)

Paludis is smarter about what it remembers. Instead of masking wrong candidates, it identifies the **right** one. When a new constraint conflicts with an earlier decision, Paludis evaluates all accumulated constraints for that package simultaneously and determines which candidate satisfies them all.

It then records a *preload* — an instruction that says "use this specific candidate next time." The resolver is discarded and a fresh one is created, but the preloads travel with it. This means the next attempt starts with positive guidance rather than just a list of things to avoid.

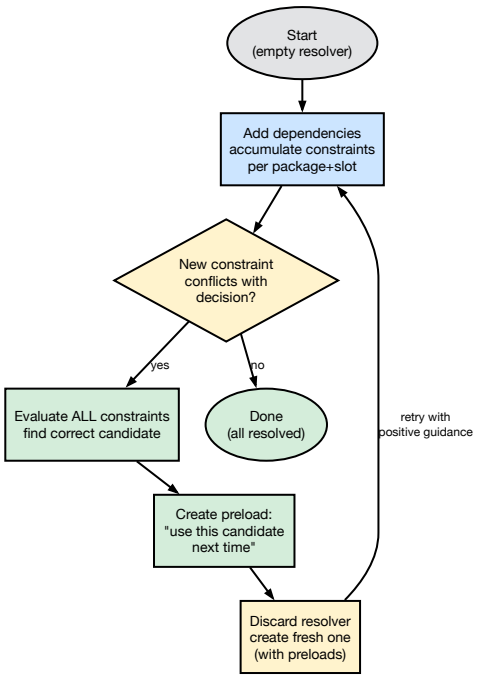


Figure 56 — Paludis conflict-resolution loop

Because Paludis carries forward the right answer instead of just rejecting the wrong one, it typically needs fewer restarts than Portage. However, each restart still creates a brand-new resolver, so the dependency walk itself is repeated.

21.1.3 portage-ng (SWI-Prolog)

portage-ng avoids the restart-from-scratch pattern altogether. It uses a depth-first proof search: each dependency becomes a proof obligation, and selecting a candidate adds constraints to a global store. Constraint guards monitor the store and fire immediately when a conflict appears.

When a guard fires, three things happen in sequence:

1. The conflicting domain is **learned** — the version set for that package is narrowed to exclude impossible choices.
2. The current candidate is **rejected** so it won't be tried again.
3. Only the affected **subtree is retried**, with the learned domain already in place to guide candidate selection.

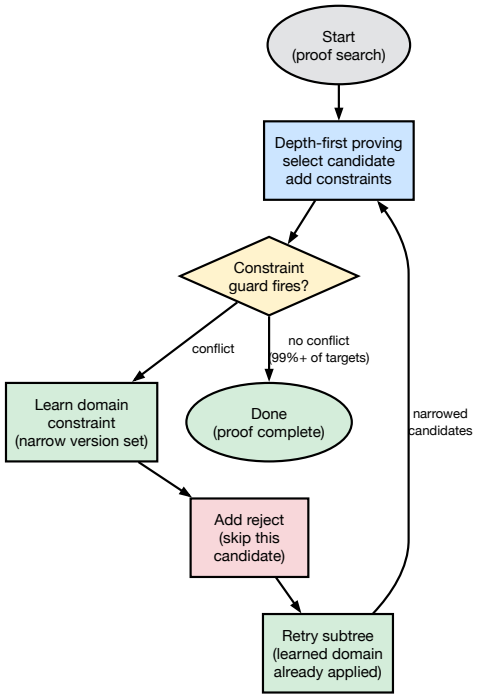


Figure 57 — portage-ng conflict-resolution loop

For the vast majority of packages (over 99%), no conflict arises at all and the proof completes in a single pass. When conflicts do occur, the combination of learned domains (positive guidance) and rejects (negative filtering) resolves them without rebuilding the entire proof tree. This makes portage-ng the fastest of the three resolvers.

21.2 Comparison Table

Aspect	Portage	Paludis	portage-ng
Language	Python	C++	SWI-Prolog
Conflict detection	Post-hoc (after graph built)	Incremental (on constraint add)	Incremental (constraint guard)
What carries across retries	Masks (negative)	Preloads (positive)	Learned domains (positive) + Rejects (negative)
Fresh state each retry?	Yes (new depgraph)	Yes (new Resolver)	Partial (reject set accumulates, learned store accumulates)
Finding the right candidate	Brute force (mask+retry)	`_try_to_find_decision_for` with ALL constraints	Domain narrowing (Zeller) + priority resolution (Vermeir)
Performance	Slowest (full rebuild)	Fast (targeted restarts)	Fastest (single-pass for most targets)
Package-specific code	None	None	None

21.3 Academic Foundations

21.3.1 Zeller & Snelting: Feature Logic (ESEC 1995, TOSEM 1997)

"Handling Version Sets through Feature Logic" (ESEC 1995, LNCS 989) and its expanded journal version "Unified Versioning Through Feature Logic" (TOSEM 1997, Vol. 6 No. 4) — version sets are identified by feature terms and configured by incrementally narrowing the set until each component resolves to a single version. portage-ng's `version_domain` with `domain_meet` (intersection) is essentially Zeller's feature term narrowing. The learned constraint store implements Zeller's feature implication propagation: constraints discovered in one proof attempt propagate to narrow version sets in the next attempt.

21.3.2 Vermeir & Van Nieuwenborgh: Ordered Logic Programs (JELIA 2002)

"Preferred Answer Sets for Ordered Logic Programs" — when rules conflict, a partial order determines which yields. portage-ng's `find_adjustable_origin` implements this: when a domain is inconsistent (two bounds that can't be simultaneously satisfied), the bound from the "adjustable" origin (the package that already has a learned constraint) is dropped, and the origin is narrowed further.

21.3.3 CDCL / PubGrub / SAT-based approaches

Modern package resolvers (libsolv, Resolvo, PubGrub) encode version constraints as boolean satisfiability problems. portage-ng's approach is different: it uses proof search with domain narrowing rather than SAT encoding. The learned constraint store is analogous to CDCL's learned clauses, but expressed as version domains rather than boolean clauses.

Chapter 22

Dependency Ordering

When a package manager installs several packages at once, the order in which they are merged matters. A compiler must be installed before the packages that need it for building; a shared library must be present before the programs that link against it can run. The Gentoo Package Manager Specification (PMS) defines five dependency types, each with different ordering strength. Portage, Paludis, and portage-ng all interpret these types, but they differ in how strictly they enforce ordering — especially when cycles make a perfect order impossible.

22.1 Dependency types

The PMS (Chapter 8) groups dependencies into five classes based on *when* the dependency must be available.

- **DEPEND / BDEPEND** — build-time dependencies. These must be installed and usable before `pkg_setup` runs and throughout the `src_*` build phases. BDEPEND (introduced in EAPI 7) targets the build host (CBUILD), while DEPEND targets the target host (CHOST). Both create **hard ordering constraints**: the dependency must be merged before the dependent can be built.
- **RDEPEND** — runtime dependencies. These must be installed before the package is "treated as usable." This is a **soft ordering constraint**: ideally satisfied before the dependent is merged, but the constraint can be relaxed when cycles exist.
- **PDEPEND** — post-dependencies. These only need to be installed "before the package manager finishes the batch." This is the **weakest constraint** — there is no requirement that the post-dependency is merged before the dependent.
- **IDEPEND** — install-time dependencies (EAPI 8+). Needed during `pkg_preinst` and `pkg_postinst`. Treated similarly to runtime dependencies for ordering purposes.

22.2 Phase functions and dependency availability

Different ebuild phases have access to different dependency classes:

Build phases (`src_unpack` through `src_install`)
: DEPEND, BDEPEND

Install phases (`pkg_preinst`, `pkg_postinst`, `pkg_prerm`, `pkg_postrm`)
: RDEPEND, IDEPEND

Configuration (`pkg_config`)
RDEPEND, PDEPEND

22.3 Portage's approach

Portage builds a single dependency graph where every package is a node and every dependency creates a directed edge. Each edge carries a *priority* that records how hard the ordering constraint is:

Dependency	Priority	Breakable?
DEPEND / BDEPEND	buildtime (highest)	Never
RDEPEND with `:=` slot op	runtime_slot_op	Only when cross-compiling
RDEPEND	runtime	Yes, for cycles
PDEPEND	runtime_post (lowest)	Yes, first to break

22.3.1 Progressive relaxation

When Portage runs its topological sort and cannot find a leaf node (a package with no unsatisfied dependencies), it progressively drops weaker edges until a leaf appears. The relaxation proceeds in four passes:

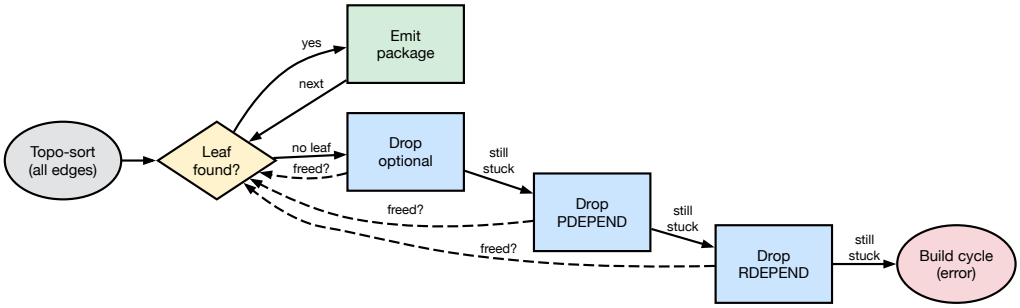


Figure 58 — Portage progressive edge relaxation

1. **All edges respected** — standard topological sort.
2. **Drop optional edges** — removes edges for optional dependencies.
3. **Drop PDEPEND edges** — the weakest real dependency type is discarded.
4. **Drop RDEPEND edges** — runtime edges are relaxed, freeing most remaining cycles.

If no leaf is found even after dropping RDEPEND edges, the remaining cycle involves only build-time dependencies. This is a hard error — Portage cannot resolve it and reports the cycle to the user.

22.3.2 What this means in practice

In the common (acyclic) case, Portage merges all dependencies — including RDEPEND — before the packages that need them. When cycles exist, PDEPEND edges are broken first, then RDEPEND edges. Build-time edges are never broken.

22.4 Paludis's approach

Paludis takes a different view. It builds a Node-Adjacency Graph (NAG) where edges carry two boolean flags: `build()` for build-time dependencies and `run()` for runtime dependencies. Notably, **PDEPEND creates no edge at all**. The Paludis source comments explain the reasoning: most post-dependencies already depend on the thing requiring them anyway, so adding a backwards edge would just create unnecessary cycles.

22.4.1 SCC-based cycle handling

Rather than relaxing edges progressively, Paludis uses Tarjan's algorithm to find strongly connected components (SCCs) and then classifies each one:

- **Single-node SCC** — no cycle, scheduled directly.
- **Runtime-only SCC** (no build edges) — ordered arbitrarily. Paludis treats runtime-only cycles as having no ordering significance at all.
- **Build-dep SCC** — Paludis tries to break the cycle by removing edges whose dependencies are already satisfied (`build_all_met` or `run_all_met`). If the SCC is still cyclic after that, it is marked as unorderable.

The key insight from Paludis is stronger than Portage's progressive relaxation: runtime-only cycles are not just *breakable* — they are *free to order however is convenient*.

22.5 portage-ng's approach

portage-ng models dependencies as a two-phase proof tree. Each package has an `:install` action (building) and a `:run` action (being usable). The different dependency types map naturally onto this structure.

22.5.1 Intra-group dependency ordering

Before proving the dependencies within a single package, `candidate:dep_priority/2` sorts them by constraint tightness. Tightly constrained dependencies are proved first so that their `selected_cn` locks early, preventing greedy conflicts where an unconstrained sibling picks a version that later clashes. The priority ladder (lower = proved first): tight upper bound (1) → tilde (4) → wildcard (8) → unconstrained (999). Within each tier, slot specificity further refines the order. See Chapter 11 for details.

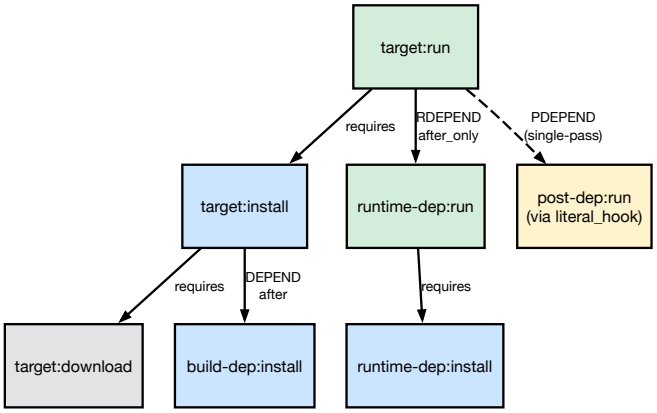


Figure 59 — portage-ng two-phase dependency model

- **DEPEND / BDEPEND** create edges to `:install` actions with `after()` context tags that propagate down the dependency chain. These are hard ordering constraints.
- **RDEPEND** create edges to `:run` actions with `after_only()` context tags that stop at the immediate children. This makes them naturally softer.
- **PDEPEND** are handled by `literal_hook` in a single pass during proof search, without creating explicit ordering edges.

When cycles appear, the wave planner produces an acyclic plan for the majority of the graph. The remaining cyclic portion goes to the scheduler, which uses Kosaraju's algorithm to find SCCs. Runtime-only SCCs are treated as freely orderable (matching Paludis's insight), while build-dep SCCs require special handling.

22.6 How the three approaches compare

The diagram below traces how each PMS dependency type is implemented across the three resolvers. Blue indicates hard (build-time) constraints, green indicates soft (runtime) constraints, and yellow indicates the weakest (post-dependency) constraints.

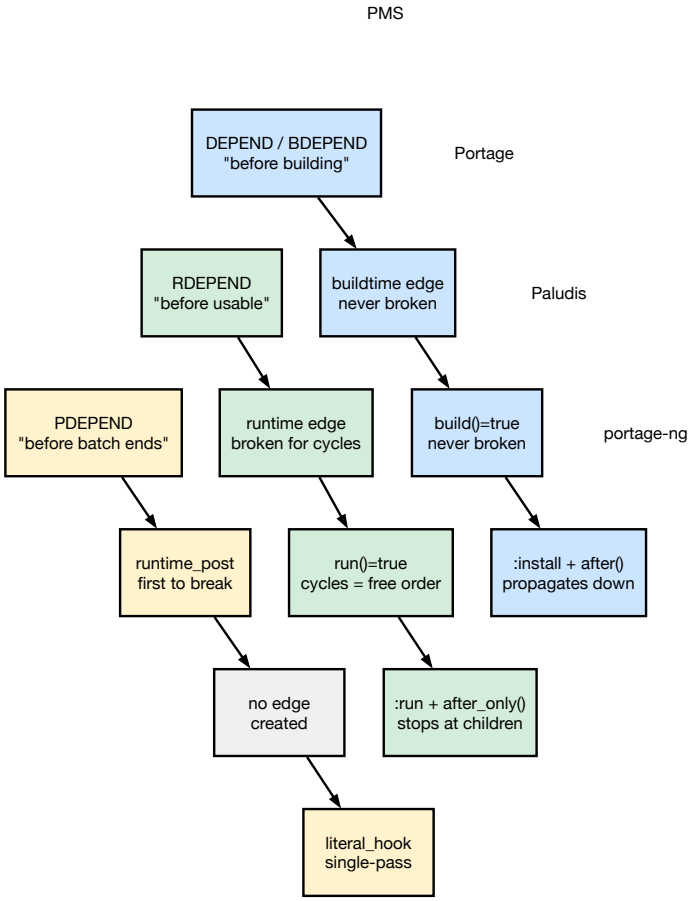


Figure 60 — Dependency type mapping across resolvers

Aspect	Portage	Paludis	portage-ng
DEPEND/BDEPEND	Hard edge, never broken	Hard edge, never broken	`:install` + `after()`, hard
RDEPEND	Soft edge, broken for cycles	Soft edge, cycles freely ordered	`:run` + `after_only()`, soft
PDEPEND	Weak edge, first to break	No edge at all	`literal_hook`, no edge
Cycle strategy	Progressive relaxation	SCC classification	Wave plan + SCC scheduling
Build-time cycles	Error / merge group	Relax met edges, then error	SCC merge-set

22.7 Annex: overlay test cases

portage-ng ships with a synthetic overlay (`Repository/Overlay/`) containing 80 test cases that exercise the resolver in isolation. Each test uses tiny packages with carefully crafted dependency relationships, making it easy to see exactly how the resolvers behave. The five cases

below illustrate the ordering concepts discussed in this chapter. For each case we show the dependency graph, a short description, and the captured output from both Portage (`emerge -vp`) and portage-ng (`--pretend`).

22.7.1 Annex A — Basic ordering (test01)

Four packages with a clean dependency chain: `web` depends on `app`, `db`, and `os`; `app` depends on `db` and `os`; `db` depends on `os`. No cycles, no special dependency types.

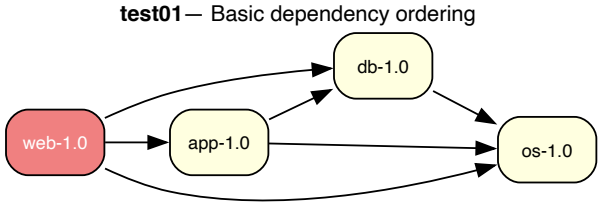


Figure 61 — test01 — Basic dependency ordering

Both resolvers produce the same order: `os` first (no dependencies), then `db` and `app` (whose dependencies are now satisfied), and finally `web`. portage-ng additionally shows the two-phase `:install` / `:run` structure and groups downloads into a parallel first step.

Portage output:

```
[ebuild  N     ] test01/os-1.0::overlay   0 KiB
[ebuild  N     ] test01/db-1.0::overlay   0 KiB
[ebuild  N     ] test01/app-1.0::overlay  0 KiB
[ebuild  N     ] test01/web-1.0::overlay  0 KiB

Total: 4 packages (4 new)
```

portage-ng output:

```
  step  1 | download  overlay://test01/web-1.0
          | download  overlay://test01/os-1.0
          | download  overlay://test01/db-1.0
          | download  overlay://test01/app-1.0

  step  2 | install   overlay://test01/os-1.0
  step  3 | run       overlay://test01/os-1.0
  step  4 | install   overlay://test01/db-1.0
  step  5 | run       overlay://test01/db-1.0
  step  6 | install   overlay://test01/app-1.0
  step  7 | run       overlay://test01/app-1.0
  step  8 | install   overlay://test01/web-1.0
  step  9 | run       overlay://test01/web-1.0

Total: 12 actions (4 downloads, 4 installs, 4 runs)
```

22.7.2 Annex B — Transitive RDEPEND (test50)

`app` has a compile-time dependency on `foo`, and `foo` has a runtime dependency on `bar`. The question is whether `bar` — a transitive runtime dependency of a build dependency — appears in the merge plan.

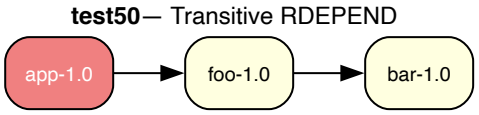


Figure 62 — test50 — Transitive RDEPEND

Both resolvers correctly include all three packages. The ordering is `bar` first (so `foo` can run), then `foo` (so `app` can build), then `app`.

Portage output:

```
[ebuild  N     ] test50/bar-1.0::overlay  0 KiB
[ebuild  N     ] test50/foo-1.0::overlay  0 KiB
[ebuild  N     ] test50/app-1.0::overlay  0 KiB

Total: 3 packages (3 new)
```

portage-ng output:

```
  step  1 | download  overlay://test50/foo-1.0
          | download  overlay://test50/bar-1.0
          | download  overlay://test50/app-1.0

  step  2 | install   overlay://test50/bar-1.0
  step  3 | run       overlay://test50/bar-1.0
  step  4 | install   overlay://test50/foo-1.0
  step  5 | install   overlay://test50/app-1.0
  step  6 | run       overlay://test50/app-1.0

Total: 8 actions (3 downloads, 3 installs, 2 runs)
```

22.7.3 Annex C — Runtime cycle (test07)

Same four-package graph as Annex A, but `os` adds a **runtime** dependency back on `web`, creating a cycle. The back-edge is an RDEPEND, so both resolvers treat the cycle as benign.

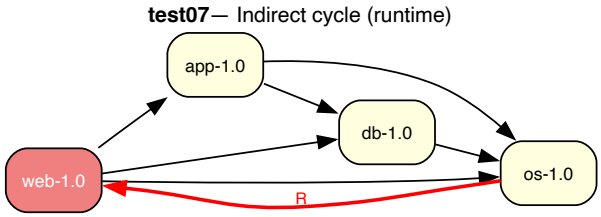


Figure 63 — test07 — Indirect cycle (runtime)

Portage reports the cycle as a warning but still produces a valid merge list (using 1 backtrack). portage-ng classifies it as benign and produces a clean plan with no assumptions — notice that `web`, `app`, and `db` are installed in parallel in step 3 because the runtime cycle makes their relative order irrelevant.

Portage output:

```
[ebuild  N     ] test07/web-1.0::overlay  0 KiB
[ebuild  N     ] test07/app-1.0::overlay  0 KiB
[ebuild  N     ] test07/db-1.0::overlay   0 KiB
[ebuild  N     ] test07/os-1.0::overlay   0 KiB

Total: 4 packages (4 new)

 * Error: circular dependencies:
(test07/os-1.0::overlay) depends on
 (test07/web-1.0::overlay) (runtime)
  (test07/os-1.0::overlay) (buildtime)
```

portage-ng output:

```
  step  1 | download  overlay://test07/web-1.0
          | download  overlay://test07/os-1.0
          | download  overlay://test07/db-1.0
          | download  overlay://test07/app-1.0

  step  2 | install   overlay://test07/os-1.0
  step  3 | install   overlay://test07/web-1.0
          | install   overlay://test07/app-1.0
          | install   overlay://test07/db-1.0
  step  4 | run       overlay://test07/web-1.0
          | run       overlay://test07/app-1.0
          | run       overlay://test07/db-1.0
  step  5 | run       overlay://test07/os-1.0

Total: 12 actions (4 downloads, 4 installs, 4 runs)
```

22.7.4 Annex D — PDEPEND (test66)

`app` depends on `lib` (compile-time), and `lib` declares `plugin` as a PDEPEND. The plugin should be resolved, but it does not need to be installed before `lib`.

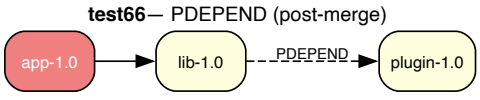


Figure 64 — test66 — PDEPEND (post-merge)

Portage merges `plugin` first (it has no hard dependencies), then `lib`, then `app`. portage-ng installs `lib` and `plugin` in parallel in step 2, since PDEPEND creates no ordering constraint between them. The plugin's `:run` action comes last, after the main target.

Portage output:

```
[ebuild  N     ] test66/plugin-1.0::overlay  0 KiB
[ebuild  N     ] test66/lib-1.0::overlay     0 KiB
[ebuild  N     ] test66/app-1.0::overlay     0 KiB

Total: 3 packages (3 new)
```

portage-ng output:

```
  step  1 | download  overlay://test66/plugin-1.0
          | download  overlay://test66/lib-1.0
          | download  overlay://test66/app-1.0

  step  2 | install   overlay://test66/lib-1.0
          | install   overlay://test66/plugin-1.0
  step  3 | run       overlay://test66/lib-1.0
  step  4 | install   overlay://test66/app-1.0
  step  5 | run       overlay://test66/app-1.0
  step  6 | run       overlay://test66/plugin-1.0

Total: 9 actions (3 downloads, 3 installs, 3 runs)
```

22.7.5 Annex E — PDEPEND cycle (test79)

`server` has an RDEPEND on `client`, and `client` has a PDEPEND back on `server`. This creates a cycle, but since PDEPEND creates no ordering edge, the cycle is naturally broken.

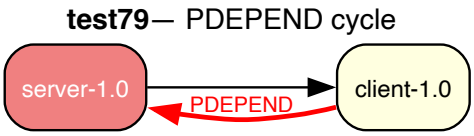


Figure 65 — test79 — PDEPEND cycle

Portage handles this cleanly — the PDEPEND obligation is already satisfied because `server` was merged as part of the same batch. portage-ng's proof obligation mechanism resolves the PDEPEND in a single pass with no assumptions needed.

Portage output:

```
[ebuild  N     ] test79/client-1.0::overlay  0 KiB
[ebuild  N     ] test79/server-1.0::overlay  0 KiB

Total: 2 packages (2 new)
```

portage-ng output:

```
  step  1 | download  overlay://test79/server-1.0
          | download  overlay://test79/client-1.0

  step  2 | install   overlay://test79/client-1.0
  step  3 | run       overlay://test79/client-1.0
```

```
  step  4 | install   overlay://test79/server-1.0
  step  5 | run       overlay://test79/server-1.0

Total: 6 actions (2 downloads, 2 installs, 2 runs)
```

22.8 References

- PMS Chapter 8: https://projects.gentoo.org/pms/8/pms.html
- Portage source: `lib/_emerge/depgraph.py` (method `_serialize_tasks`)
- Portage priorities: `lib/_emerge/DepPriorityNormalRange.py`
- Paludis orderer: `paludis/resolver/orderer.cc`
- Paludis classifier: `paludis/resolver/labels_classifier.cc`
- Full overlay test suite: `Documentation/Tests/README.md` (80 test cases)

Chapter 23

Testing and Regression

portage-ng uses multiple testing strategies: PLUnit tests for unit logic, overlay regression tests for end-to-end scenario validation, and merge-vs-emerge comparison for correctness measurement against Portage.

23.1 PLUnit tests

Standard SWI-Prolog unit tests in `Source/Test/unittest.pl`:

```
make test
```

These test individual predicates in isolation — version comparison, domain operations, context merging, EAPI parsing, etc.

23.2 Overlay regression tests

The overlay test suite (`make test-overlay`) runs 80 curated scenarios against a test overlay in `Repository/Overlay/`. Each scenario has a specific dependency story and expected behavior.

23.2.1 Running

```
make test-overlay
```

Or from the interactive shell:

```
test:run(cases).
```

23.2.2 Test scenario anatomy

Each test under `Documentation/Tests/testNN/` contains:

- **`README.md`** — description of the dependency story and expected outcome
- **`testNN.svg`** — dependency graph visualization
- **Collapsible transcripts** — `emerge -vp` vs `portage-ng --pretend` output for comparison

23.2.3 Coverage areas

Area	Tests
Basic ordering / default version	01-02
Cycles (self, indirect, 3-way, PDEPEND)	03-08, 47, 61-64, 79
Missing dependencies	09-11
Keywords (stable vs unstable)	12
Version operators (`=`, `>=`, `~`, `<=`)	13, 55-56, 69-70, 80
USE conditionals	14-15
Choice groups (`^^`, `||`, `??`)	17-25
Blockers (strong/weak)	26-31, 60
REQUIRED_USE	32, 40
USE dependencies (`[flag]`, `[-flag]`, `=`)	33-39
Slots (`:*`, `:=`, sub-slot)	41-44
Conflicts (USE, slot, diamond)	45-46, 48-49, 51
USE merge (shared deps)	52-53
Virtuals	57-58
Installed / VDB operations	65, 73-77
PDEPEND	66, 79
BDEPEND / IDEPEND	67, 72
Multi-slot co-install	68
Fetch-only	71
Onlydeps	78

23.2.4 Failure testing

Tests 58, 59, and 60 are explicitly marked as expected failures (XFAIL) in the test matrix — known limitations that are documented but not yet fixed.

23.3 Merge vs emerge comparison

The primary correctness metric is comparison against Portage's `emerge` output across the entire Portage tree. The comparison harness now lives in the tinderbox-ng repository, which drives both engines through identical sessions and analyses the resulting plan logs.

23.3.1 Running a comparison

Per-target compare (planner only, fresh sessions on both sides):

```
tinderbox-ng compare www-servers/apache
```

Whole-tree matrix run plus aggregate analysis:

```
sudo tinderbox-ng compare-matrix
sudo tinderbox-ng analyze                        # latest matrix run
sudo tinderbox-ng analyze --run /srv/tinderbox-ng/reports/compare-matrix-<stamp>
```

`tinderbox-ng analyze` feeds each `portage-ng.plan.log` / `emerge.plan.log` pair through `share/tinderbox-ng/compare-merge-emerge.py` (inside the tinderbox-ng repo) and writes `analysis.json` + `analysis.txt` into the matrix run directory.

23.3.2 Metrics

The comparison produces several accuracy metrics:

Metric	Formula	Meaning
CN	`100 * inter_cn / union_cn`	Category/Name match (ignoring version)
CN+V	`100 * inter_cnv / union_cnv`	Category/Name+Version match
CN+V+U	`100 * inter_cnvu / union_cnvu`	Full match including USE flags
Order%	`100 * (pairs - inversions) / pairs`	Ordering concordance

Additional counts (from `emerge_ok` pairs only):

- `#blockers` — total blocker assumptions
- `#cycle breaks` — total prover cycle-break assumptions
- `#domain assumptions` — total domain assumptions

23.3.3 Targeted comparison

For a single package, use `--target-regex` on `tinderbox-ng analyze`:

```
sudo tinderbox-ng analyze --target-regex '^sys-apps/portage-3.0.77-r3$'
```

Or run a one-off per-target compare directly:

```
tinderbox-ng compare sys-apps/portage
```

23.4 md5-cache extractor regression

`md5cache_validate/0,1` (in `Source/Test/unittest.pl`) runs the standalone bash extractor at `Source/Domain/Gentoo/Ebuild/ebuild-depend.sh --batch` over every md5-cache entry in the configured Portage tree and diffs the produced metadata against the on-disk cache, key by key.

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --shell <<'PL'
load_files(portage('Source/Test/unittest'), [if(true)]).
md5cache_validate([limit(50), verbose(true)]).
halt.
PL
```

Options: `repo(Atom)` (default `portage`), `limit(N)` (0 = all), `verbose(Bool)`, `out(Path)` (writes a Prolog-term report).

23.5 Further reading

- Chapter 2: Installation and Quick Start — `make test` commands
- Chapter 24: Performance and Profiling — `prover:test_stats` for bulk testing
- Chapter 25: Contributing — development workflow with regression testing

Chapter 24

Performance and Profiling

portage-ng loads on the order of **32,000 ebuilds** into memory and reasons about their dependencies with **formal proof search**. That combination is easy to make slow: naive parsing, interpreted queries, imperative undo stacks, exponential backtracking, and repeated failed branches can each dominate runtime on their own. The design question is not "which single trick wins?" but **how we stack complementary strategies** so the whole pipeline stays responsive.

The answer is **the five pillars of portage-ng performance**: compiled knowledge (qcompiled cache), compile-time query expansion, persistent AVL structures for proof state, prescient proving that avoids redundant work, and incremental learning that narrows the search after failures. Together they explain why the tree can load with sub-second queries and why a full prove across all packages can finish in under a minute on a strong multi-core machine—while leaving room for profiling and targeted optimization.

This chapter walks those pillars in order, then covers **instrumentation** (the sampler), **bulk testing**, and a **performance comparison** between portage-ng and Portage.

24.1 Pillar 1: Compiled knowledge (qcompiled `.qlf` files)

The Portage tree is **not** parsed from scratch on every startup. During `--sync`, metadata is read and the knowledge base is written in a form that SWI-Prolog can **qcompile** into a binary load unit—`Knowledge/kb.qlf` (source facts live in `Knowledge/kb.raw`). The next time the application starts, it loads that **binary** representation instead of re-parsing large textual artifacts.

That is the **largest single speedup** in the system: startup drops from **tens of seconds** of parsing and assertion to **under a second** for the compiled cache, after which reasoning works directly over in-memory facts. Everything else in this chapter assumes that this first pillar is in place; without it, no amount of clever proving would feel fast enough.

24.2 Pillar 2: Goal expansion macros

High-level queries in the knowledge layer are written for clarity; at **compile time** they are rewritten into **direct cache access**, so the runtime path never pays for meta-interpretation over generic search.

`goal_expansion/2` in `Source/Knowledge/query.pl` performs this rewrite. For example, a search by repository, category, and package name expands straight to an ordered cache entry lookup:

```
user:goal_expansion(query:search(R, C, N), cache:ordered_entry(R, _, C, N, _)).
```

The expanded code calls the indexed predicate **directly**. SWI-Prolog's **first-argument indexing** on `cache:entry/5` (and related entry predicates) makes those lookups **O(1) amortized** in typical use: the prover's inner loop sees plain deterministic cache reads, not a slow interpretive layer.

For how the knowledge base and query surface fit together, see Chapter 6: Knowledge Base and Cache.

24.3 Pillar 3: Persistent AVL trees

Proof search maintains large associative structures—proof literals, models, constraints, triggers—using **`library(assoc)` AVL trees**. Lookups and updates are **O(log n)**; for about **32,000** entries that is on the order of **fifteen comparisons** per operation, which is cheap enough to live in the inner loop of dependency proving.

The deeper win is **persistence**: AVL trees in Prolog are **immutable structures** threaded through the search. **Backtracking** automatically restores the previous tree without hand-written save/restore stacks or explicit undo logs—the kind of machinery imperative resolvers often maintain by hand. That keeps the prover's control flow simple while remaining safe under deep choicepoints.

Practical caveat: Proof and Model AVLs still **grow with proof size**. Algorithms should avoid **full traversals** when a more local structure suffices; the Triggers AVL (see the next pillar) exists partly so reverse lookups do not devolve into scanning the entire proof tree. That trade-off shows up again in practice when proof trees grow large.

24.4 Pillar 4: Prescient proving (avoiding backtracking)

Naive proof search can exhibit **$O(2^n)$** behaviour in the worst case: each wrong choice is explored and then undone by backtracking. portage-ng pushes hard in the other direction by **merging proof context** when the same literal is encountered again with **refined constraints**—via mechanisms such as **feature term unification**—so the system does not blindly re-prove from scratch every time the dependency graph revisits a head under slightly different assumptions.

In practice, for most real packages, that style of **prescient** handling yields **O(n) amortized** proof steps rather than exponential churn. The **Triggers AVL** complements this: it supports **efficient identification of affected heads** when something downstream changes, instead of linear scans over the whole proof.

The sampler's **`ctx_union` sampling** (documented later in this chapter) exists precisely to spot **hot merge paths**—a sign that context merging is working harder than it should and that some literals may still be reproved more often than necessary.

24.5 Pillar 5: Incremental learning (avoiding repeated failures)

When a proof attempt fails, portage-ng does not always forget what went wrong. **Learned constraints** from failed branches can **persist across reprove retries**, **narrowing domains** so the same conflict is not hit twice the same way. Together with a **reject set** that records candidates already ruled out, the prover avoids thrashing on the same dead ends.

That closes the loop with Chapter 8: The Prover: reprove and learning are part of the same story as performance. If retries explode without narrowing behaviour improving, runtime suffers.

24.6 Sampler module

The sampler (`Source/Application/Performance/sampler.pl`) is the main place to **measure** whether the pillars above are behaving as intended in production-like runs.

24.6.1 Hook performance

```
sampler:phase_walltime(Phase, Goal)
```

Wraps `Goal` and records wall-clock time for the named `Phase`. Used by the pipeline to time each stage (prove, plan, schedule).

```
sampler:phase_record(Phase, Duration)
```

Records a phase timing for later retrieval.

24.6.2 Test statistics

```
prover:test_stats(Repository)
prover:test_stats_pkgs(Repository, PackageList)
```

Run the prover across all packages (or a specific list) in a repository and collect aggregate statistics:

- Total packages attempted
- Success rate (no assumptions)
- Cycle-break-only rate
- Domain assumption rate
- Average proof time

24.6.3 Feature term unification sampling

The sampler tracks feature term unification operations to identify hot paths in context merging. Excessive merges can indicate redundant re-proving.

24.7 Bulk testing workflow

The standard performance testing workflow uses the `--shell` here-doc pattern:

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --shell --timeout 60
<<'PL'
prover:test_stats(portage).
halt.
PL
```

For specific packages:

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --shell <<'PL'
prover:test_stats_pkgs(portage, ['kde-apps'-'kde-apps-meta']).
halt.
PL
```

24.8 Performance comparison: portage-ng vs Portage

The numbers below put the five pillars in perspective with measured data from a full Portage tree comparison. Both resolvers start from the same input: an identical Portage tree snapshot (roughly 32,000 ebuilds), the same VDB (installed package database), and the same `/etc/portage` configuration. The measurement is **dependency resolution time only** — how long it takes to produce a merge plan, not how long the actual builds take.

- **Portage** uses `emerge -vp <target>`, which runs the Python `depgraph.py` resolver with greedy selection and backtracking.
- **portage-ng** uses `--mode standalone --pretend <target>`, which runs the Prolog prover, planner, and scheduler.

24.8.1 Resolution speed (emerge_ok packages)

The comparison covers **31,331 packages** resolved by both tools. For the **22,781** packages where Portage itself reports a clean result (`emerge_ok`), portage-ng is faster in **100**% of cases:

	Portage	portage-ng	Speedup
Average	1,239 ms	32 ms	**38x**
Median	1,159 ms	6 ms	**193x**
95th percentile	1,679 ms	165 ms	**10x**
Maximum	11,165 ms	920 ms	**12x**
Cumulative (22,781 pkgs)	7.84 hours	12.3 minutes	**38x**

The median package resolves in **6 milliseconds** in portage-ng versus **1.16 seconds** in Portage — nearly a two-hundred-fold improvement. Even at the 95th percentile (complex packages with deep dependency chains), portage-ng finishes in 165 ms while Portage needs nearly 1.7 seconds. portage-ng is faster in every single `emerge_ok` pair — no exceptions.

24.8.2 Resolution speed (all packages)

Across all **31,331** packages (including those where Portage reports errors), portage-ng is faster in **99.7**% of cases:

	Portage	portage-ng	Speedup
Average	1,302 ms	71 ms	**18x**
Median	1,161 ms	11 ms	**106x**
95th percentile	2,069 ms	264 ms	**8x**
Maximum	11,165 ms	9,107 ms	**1.2x**
Cumulative (31,331 pkgs)	11.33 hours	37.2 minutes	**18x**

The 103 cases (0.3%) where Portage finishes faster are all packages where **Portage itself fails** (`emerge_notok`). In those cases Portage exits early with an error while portage-ng still performs a full resolution attempt. Among the `emerge_ok` population — the apples-to-apples comparison — portage-ng wins 100% of cases.

24.8.3 Why portage-ng is faster

The performance gap is not about language speed (Prolog vs Python). It comes from architectural differences that compound across thousands of packages:

Factor	Portage	portage-ng
Startup cost	Python interpreter + module imports per invocation	Qcompiled cache loads once, shared across all queries
Graph construction	Build full graph, then check for conflicts	Single-pass proof — no separate graph phase
Conflict recovery	Discard entire graph, rebuild from scratch	Retry only the affected subtree with learned constraints
Repeated queries	Each `emerge -vp` starts cold	In-memory facts persist; subsequent queries are instant
Parallelism	Sequential graph walk	Wave planner identifies parallel steps automatically

The largest single factor is the **qcompiled cache** (Pillar 1): once loaded, all 32,000 ebuilds are in memory as indexed Prolog facts, and queries hit first-argument indexing directly. Portage re-reads and re-parses metadata structures on each invocation.

The second factor is **single-pass proving** (Pillar 4): for over 99% of packages, portage-ng needs no backtracking at all. Portage's greedy approach works well for simple cases but scales poorly when conflicts require multiple backtracks — each of which rebuilds the entire dependency graph.

Chapter 25

Contributing

This chapter covers the development workflow, coding conventions, and testing practices for contributing to portage-ng.

25.1 Development workflow

1. **Start from clean committed state.** Always begin development with no uncommitted changes.
2. **Make changes** using the project wrapper for testing:

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --pretend <target>
```

3. **Run tests** to verify correctness:

```
make test            # PLUnit tests
make test-overlay    # Overlay regression tests
```

4. **Regenerate `.merge` files** by asking the maintainer to run `--graph` to produce updated `.merge` output for the graph directory.
5. **Run compare analysis** to detect regressions. The compare harness lives in the tinderbox-ng repository:

```
# Whole-tree matrix run (on the tinderbox host):
sudo tinderbox-ng compare-matrix
sudo tinderbox-ng analyze

# Or a quick per-target compare while iterating locally:
tinderbox-ng compare <category>/<package>
```

   `tinderbox-ng analyze` produces `analysis.json` + `analysis.txt` in the matrix run directory, replacing the old `compare-<date>-<hash>.json.gz` snapshots.
6. **Review the comparison table** for regressions in CN, CN+V, CN+V+U match percentages, ordering concordance, and assumption counts.
7. **Commit** when regression-free.

25.2 How to run

25.2.1 Dev wrapper

Always use the dev wrapper for testing — never run ad-hoc `swipl -g "..."` snippets, as they miss required operator definitions, libraries, and module load order:

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --pretend <target>
./Source/Application/Wrapper/portage-ng-dev --mode standalone --shell
```

25.2.2 Scripted sessions (here-doc pattern)

For reproducible, non-interactive debugging:

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --shell --timeout 60
<<'PL'
prover:test_stats(portage).
halt.
PL
```

25.2.3 CI mode

For automated checks:

```
./Source/Application/Wrapper/portage-ng-dev --mode standalone --ci --pretend
<target>
echo $?  # 0 = no assumptions, 1 = cycle breaks, 2 = domain assumptions
```

Always include `--pretend` to avoid mutating local state.

25.3 Source file documentation style

Every `.pl` source file follows a strict layout. Use `Source/Application/System/bonjour.pl` as the canonical reference.

25.3.1 File header

```
/*
  Author:   Pieter Van den Abeele
  E-mail:   pvdabeel@mac.com
  Copyright (c) 2005-2026, Pieter Van den Abeele

  Distributed under the terms of the LICENSE file in the root directory of this
  project.
*/
```

25.3.2 Module documentation (PlDoc)

```
/** <module> MODULE_NAME_UPPERCASE
Short one-line description.

Optional longer description.
*/
```

Module name in the `<module>` tag is UPPERCASE.

25.3.3 Module declaration

```
:- module(modulename, []).
```

25.3.4 Chapter header (one per file)

```
% ============================================================================
% MODULE_NAME_UPPERCASE declarations
% ============================================================================
```

Exactly one `=====` chapter per file, immediately after `:- module`.

25.3.5 Section headers

```
% ----------------------------------------------------------------------------
% Section title
% ----------------------------------------------------------------------------
```

All subsequent sections use `-----` dashes.

25.3.6 Predicate documentation

```
%! module:predicate_name(+Arg1, -Arg2)
%
% Short description of what the predicate does.

module:predicate_name(Arg1, Arg2) :-
  body.
```

25.3.7 Spacing rules

Element	Blank lines after
File header `*/`	1
PlDoc module comment `*/`	1
`:- module(...)` declaration	1
`=====` chapter header	1
`-----` section header	1
Predicate doc + last clause	2
Between clauses of same predicate	0
End of file	0 (no trailing blank line)

25.4 Naming conventions

- Source filenames must NOT contain hyphens (`-`) or underscores (`_`). Use concatenated lowercase words: `knowledgebase.pl`, not `knowledge_base.pl`.
- Exception: `portage-ng.pl` (project entry point).
- Prolog module names follow the same rule: `:- module(gentoo, [])`.
- Subdirectory names under `Source/` may use CamelCase: `Application/`, `Domain/`, `Config/`, `Pipeline/`.

25.5 Comment guidelines

Do not add comments that just narrate what the code does. Comments should only explain non-obvious intent, trade-offs, or constraints. Avoid:

```
% Get the version     ← redundant
version:get(V).
```

Prefer:

```
% Suffix rank maps PMS suffix ordering to integers for compare/3
suffix_rank('_alpha', 1).
```

25.6 Compare tooling

Regression tooling is hosted in two places:

- **Merge-vs-emerge plan comparison** — driven by tinderbox-ng via `tinderbox-ng compare` / `tinderbox-ng compare-matrix` / `tinderbox-ng analyze`. The underlying Python script lives at `share/tinderbox-ng/compare-merge-emerge.py` in that repository and is invoked automat-

ically by `tinderbox-ng analyze`. Outputs are `analysis.json` + `analysis.txt` in the matrix run directory.
- **md5-cache extractor regression** — `md5cache_validate/0,1` in `Source/Test/unittest.pl` (re-extracts metadata via `Source/Domain/Gentoo/Ebuild/ebuild-depend.sh --batch` and diffs the result key by key against the on-disk md5-cache).

Do not create ad-hoc compare scripts outside these two locations.

25.7 Further reading

- Chapter 23: Testing and Regression — testing methodology
- Chapter 24: Performance and Profiling — performance testing
- Chapter 2: Installation and Quick Start — build and run instructions

Chapter 26

Closing Thoughts

This book opened with a question that has quietly shaped every chapter since: as software systems grow more complex, can the tools that manage them keep up?

portage-ng's answer is to treat package management not as a logistics problem — downloading and unpacking files — but as a **reasoning problem**. Dependencies become proof obligations. Configuration choices become constraints. Conflicts become learning opportunities that narrow the search space for the next attempt. The result is a system that can walk tens of thousands of packages, resolve their interdependencies, and produce a buildable plan — often in a single pass.

26.1 What we covered

The book traced this idea from concept to implementation:

- **Part I** set the stage: the growing complexity of source-based distributions, the limits of imperative solvers, and how Prolog's backtracking and unification provide a natural fit for dependency reasoning.
- **Part II** unpacked the architecture: how the knowledge base stores and indexes package metadata; how the EAPI grammar parses dependency specifications; how the prover searches for consistent models; how assumptions, constraint learning, and version domains handle conflicts; how rules encode Gentoo's domain logic; and how the planner and scheduler turn a proof into a concrete build order.
- **Part III** covered the features built on top of that foundation: the command-line interface, build execution, semantic search with LLM integration, distributed proving across clusters, and upstream bug tracking.
- **Part IV** explored the theoretical underpinnings: contextual logic programming, feature unification, and the comparison with other resolvers — showing how portage-ng's approach relates to Portage's progressive relaxation, Paludis's constraint accumulation, and academic work on feature logic and ordered logic programs.
- **Part V** described the practical side of development: testing strategies, performance profiling, and contribution guidelines.

26.2 Design principles worth remembering

A few recurring themes run through the design and are worth calling out explicitly:

- **Declarative over imperative.** The prover does not maintain mutable state that must be carefully unwound on failure. AVL trees are persistent; backtracking is automatic; learned constraints accumulate naturally. This makes the system easier to reason about and extend.
- **Single-pass where possible, learning where not.** Most packages resolve in one pass. When conflicts arise, the system learns from them — narrowing version domains, recording rejects — so the next attempt is better informed. This is fundamentally different from starting over with a blank slate on each retry.
- **Separation of concerns.** The prover knows nothing about Gentoo. The rules layer knows nothing about proof search. The planner knows nothing about dependency types. Each layer has a clean interface, and domain-specific knowledge stays in domain-specific modules.
- **Transparency.** Assumptions are not silent failures — they are classified, explained, and reported. The explainer can trace any package's presence in the plan back through the proof to the original dependency that required it. When something goes wrong, the system tells you why.

26.3 Looking ahead

portage-ng is a living project. Several directions remain open for exploration:

- **Broader platform support.** The reasoning engine is not tied to Gentoo — any system that can express its dependencies as structured rules could use the same prover and planner.
- **Richer learning.** The current constraint learning mechanism handles version domains and parent narrowing. More sophisticated strategies — learning across multiple proof runs, or sharing learned constraints between cluster workers — could further reduce proving time.
- **Tighter LLM integration.** The explainer already bridges proof traces and natural language. Future work could let users ask higher-level questions ("why is my build slow?", "what changed since last week?") and receive answers grounded in the formal proof structure.
- **Binary package support.** As Gentoo's binary package infrastructure matures, portage-ng could reason about mixed source/binary strategies — deciding which packages to build from source and which to install from pre-built archives.

26.4 Thank you

If you have read this far, you have a thorough understanding of how portage-ng works and why it was built this way. Whether you are using it to manage a Gentoo system, studying it as an example of applied logic programming, or contributing to its development — thank you for your interest, and welcome to the project.